

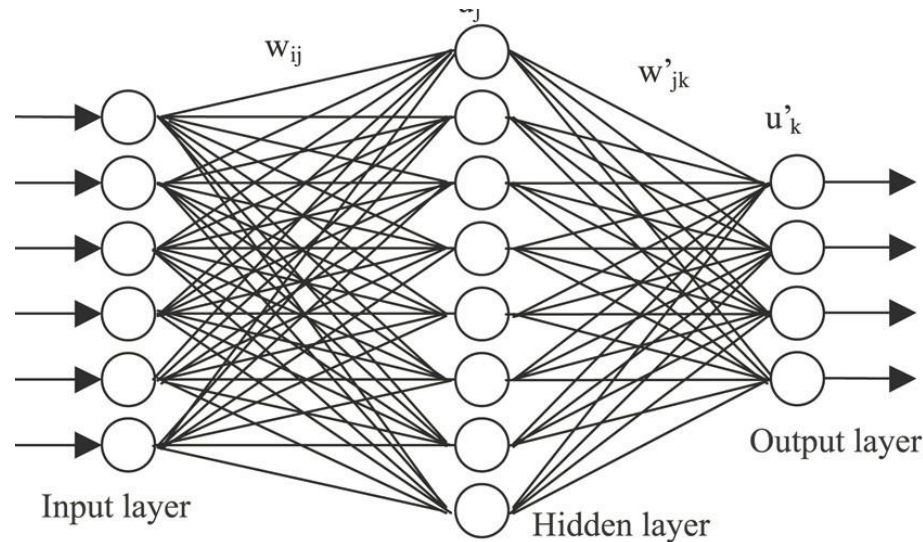
Neural Networks

Learning the network

Topics for the day

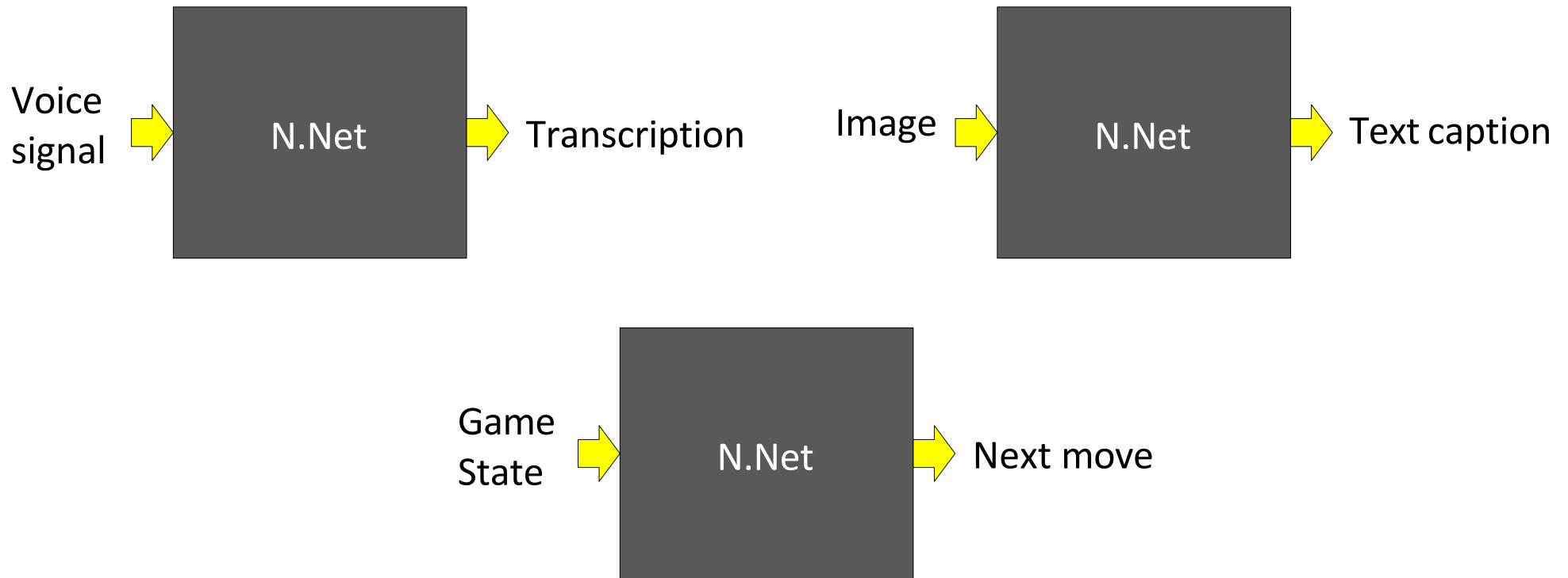
- The problem of learning
- The perceptron rule for learning individual perceptrons
 - And its inapplicability to multi-layer perceptrons
- Learning through Empirical Risk Minimization
- Intro to function optimization and gradient descent
- Back Propagation

Recap



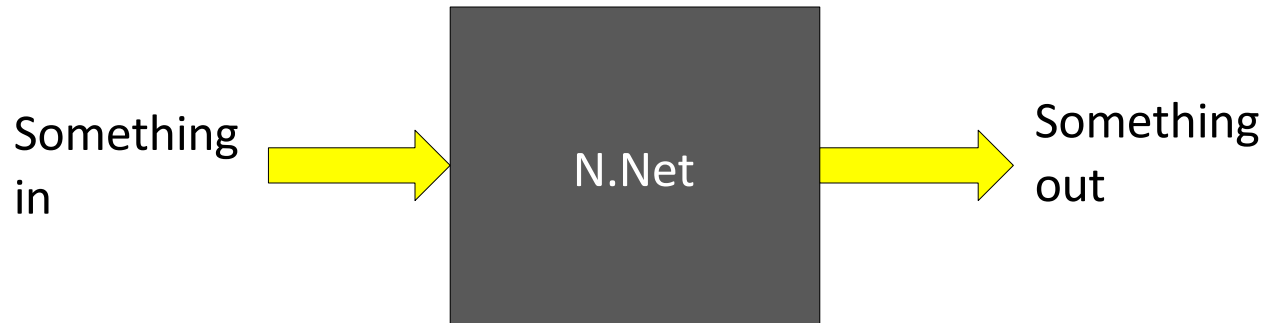
- **Neural networks are universal function approximators**
 - Can model any Boolean function
 - Can model any classification boundary
 - Can model any continuous valued function
- *Provided the network satisfies minimal architecture constraints*
 - Networks with fewer than the required number of parameters can be very poor approximators

These boxes are functions



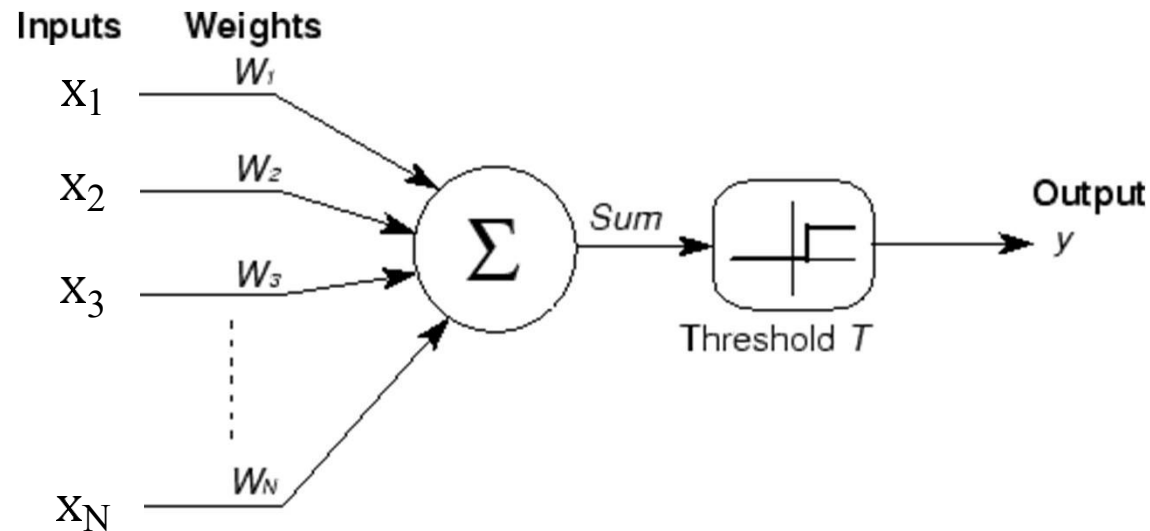
- Take an input
- Produce an output
- Can be modeled by a neural network!

Questions



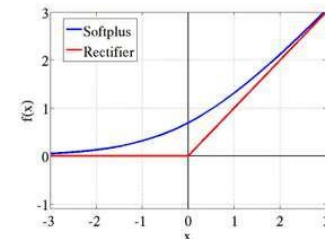
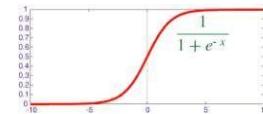
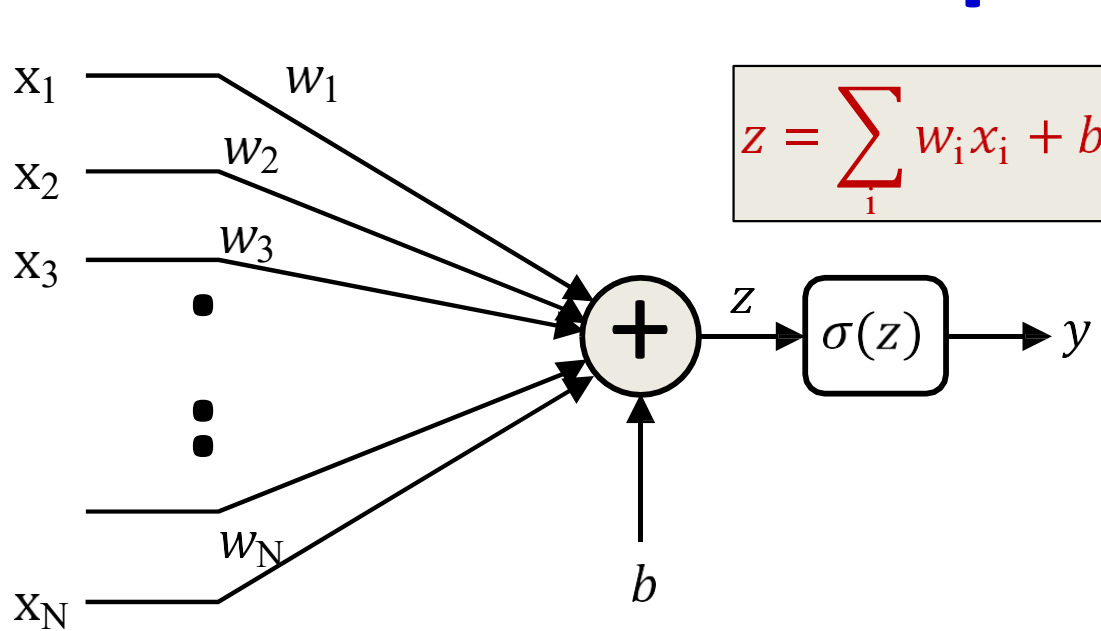
- Preliminaries:
 - How do we represent the input?
 - How do we represent the output?
- How do we compose the network that performs the requisite function?

The original perceptron



- Simple threshold unit
 - Unit comprises a set of weights and a threshold

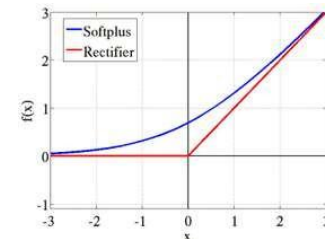
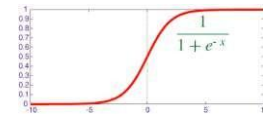
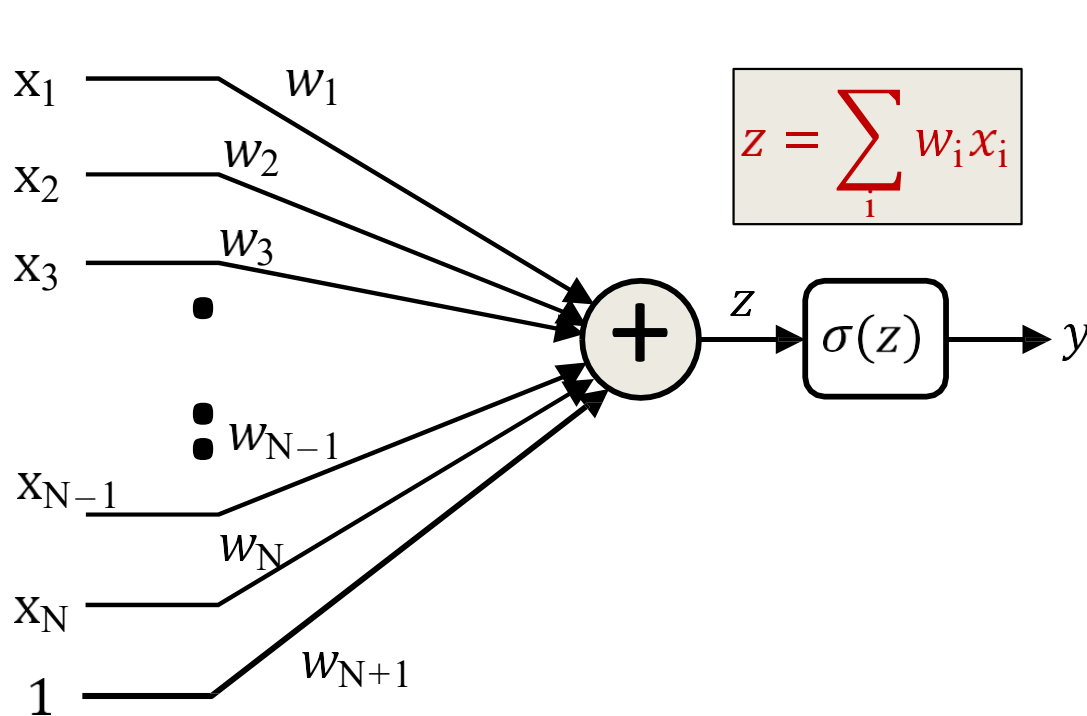
Preliminaries: The units in the network – the perceptron



Activation functions $\sigma(z)$

- Perceptron
 - General setting, inputs are real-valued
 - A *bias* b representing a threshold to trigger the perceptron
 - Activation functions are not necessarily threshold functions
- The parameters of the perceptron (which determine how it behaves) are its weights and bias

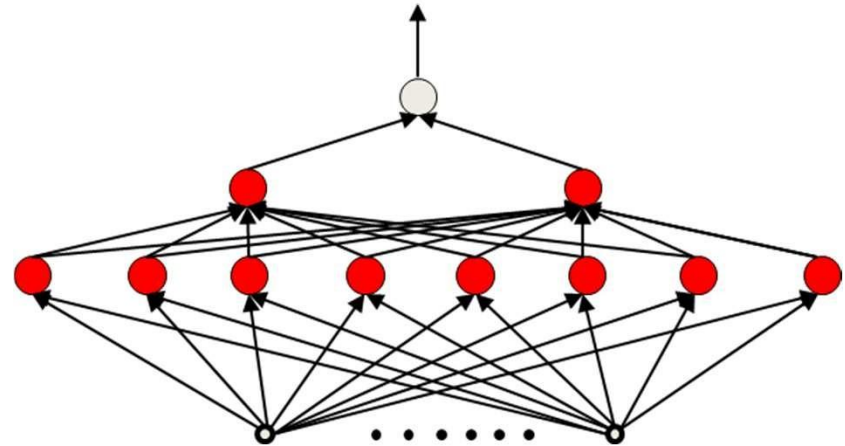
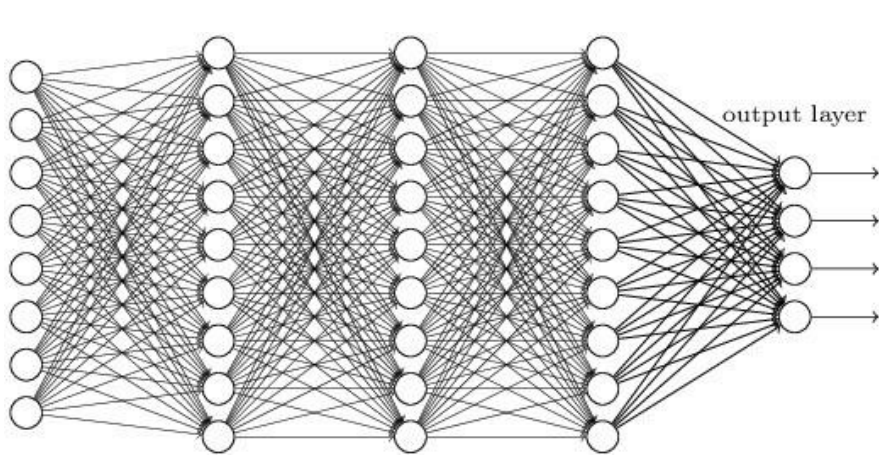
Preliminaries: Redrawing the neuron



Activation functions $\sigma(z)$

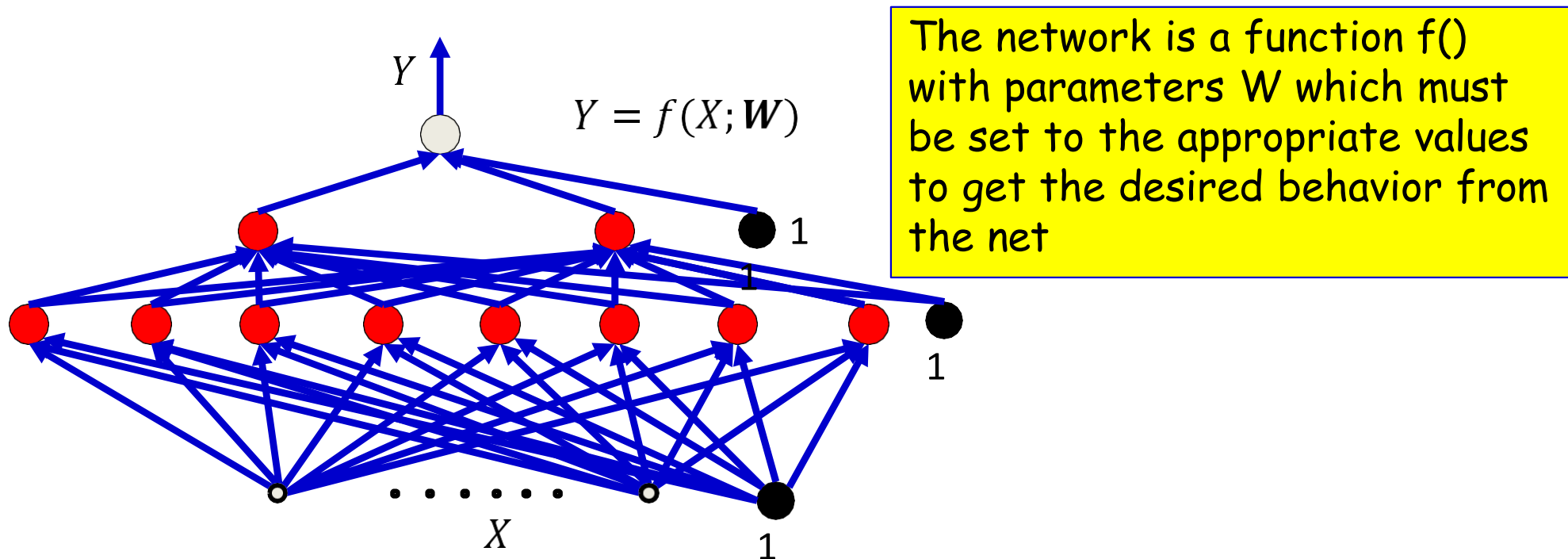
- The bias can also be viewed as the weight of another input component that is always set to 1
 - If the bias is not explicitly mentioned, we will implicitly be assuming that every perceptron has an additional input that is always fixed at 1

First: the structure of the network



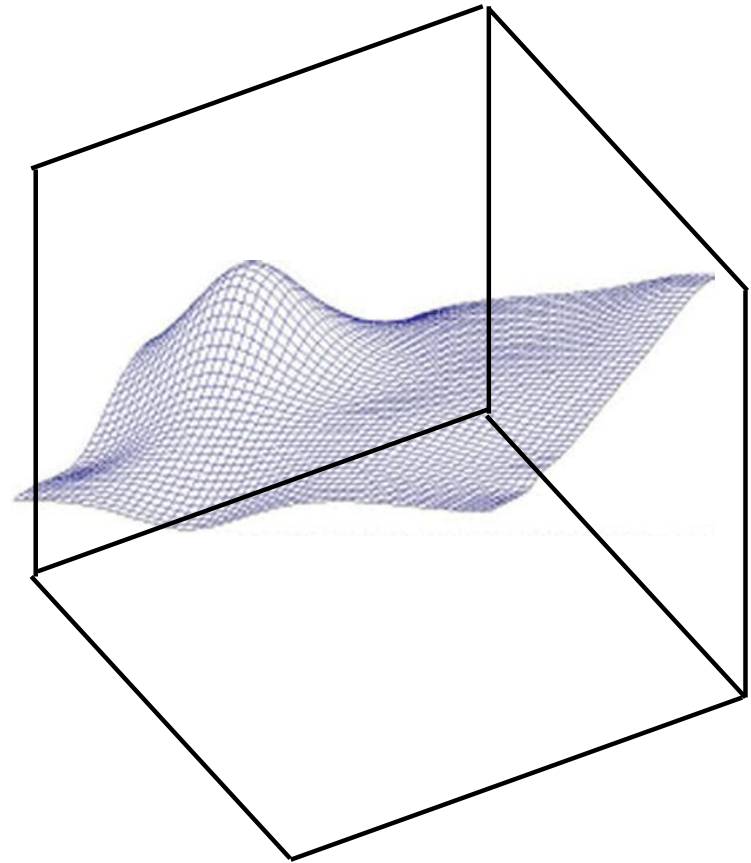
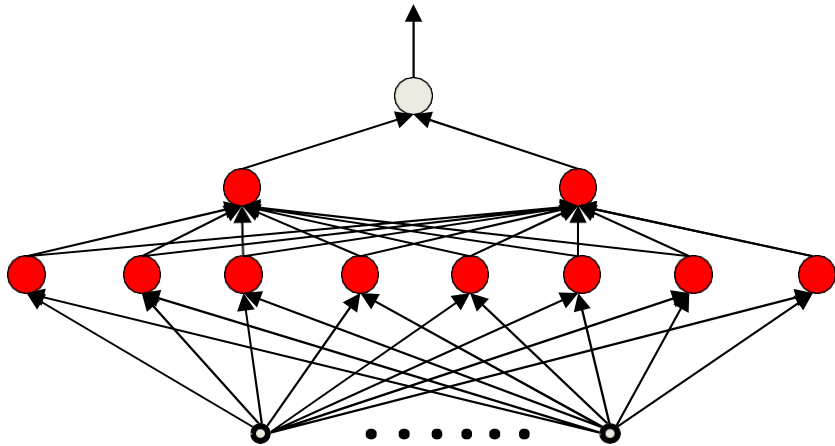
- We will assume a *feed-forward* network
 - No loops: Neuron outputs do not feed back to their inputs directly or indirectly
 - Loopy networks are a future topic
- **Part of the design of a network: The architecture**
 - How many layers/neurons, which neuron connects to which and how, etc.
- For now, assume the architecture of the network is capable of representing the needed function

What we learn: The parameters of the network



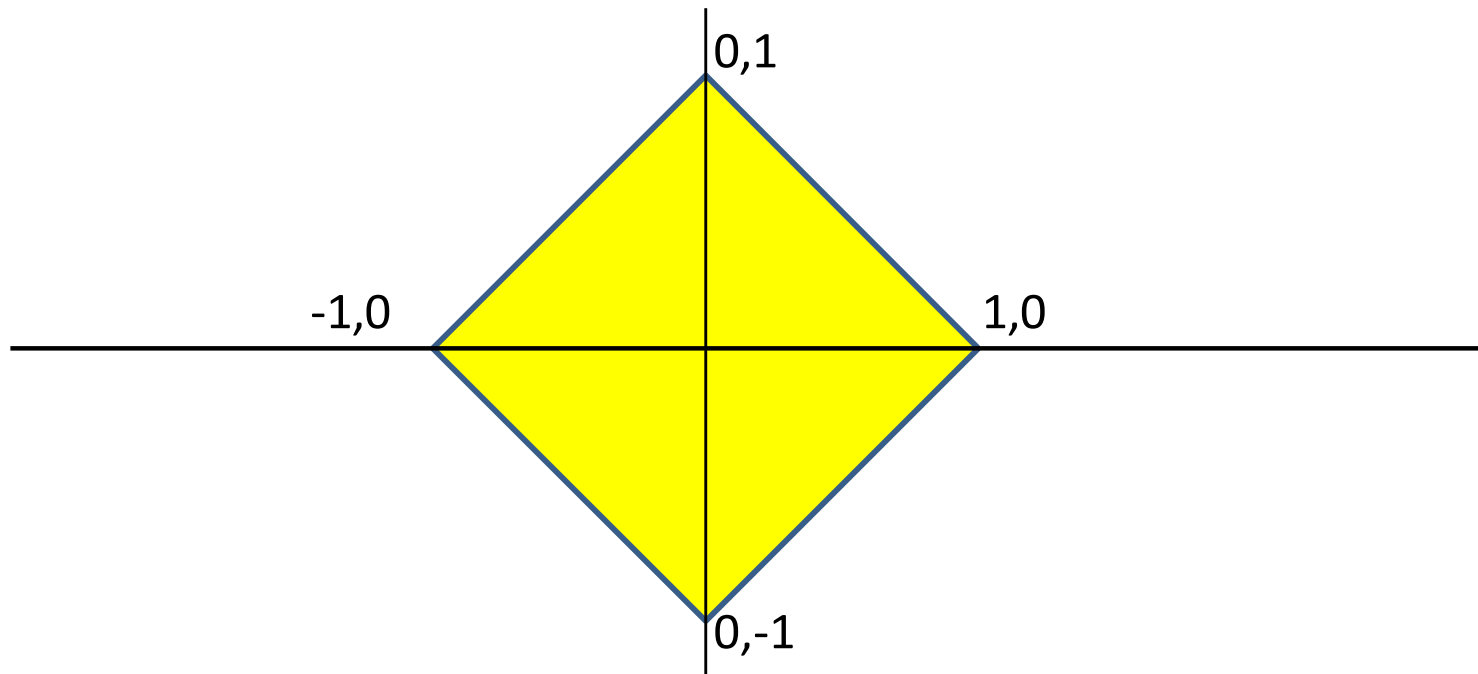
- **Given:** the architecture of the network
- **The parameters of the network:** The weights and biases
 - The weights associated with the blue arrows in the picture
- ***Learning the network* :** Determining the values of these parameters such that the network computes the desired function

The MLP *can* represent anything



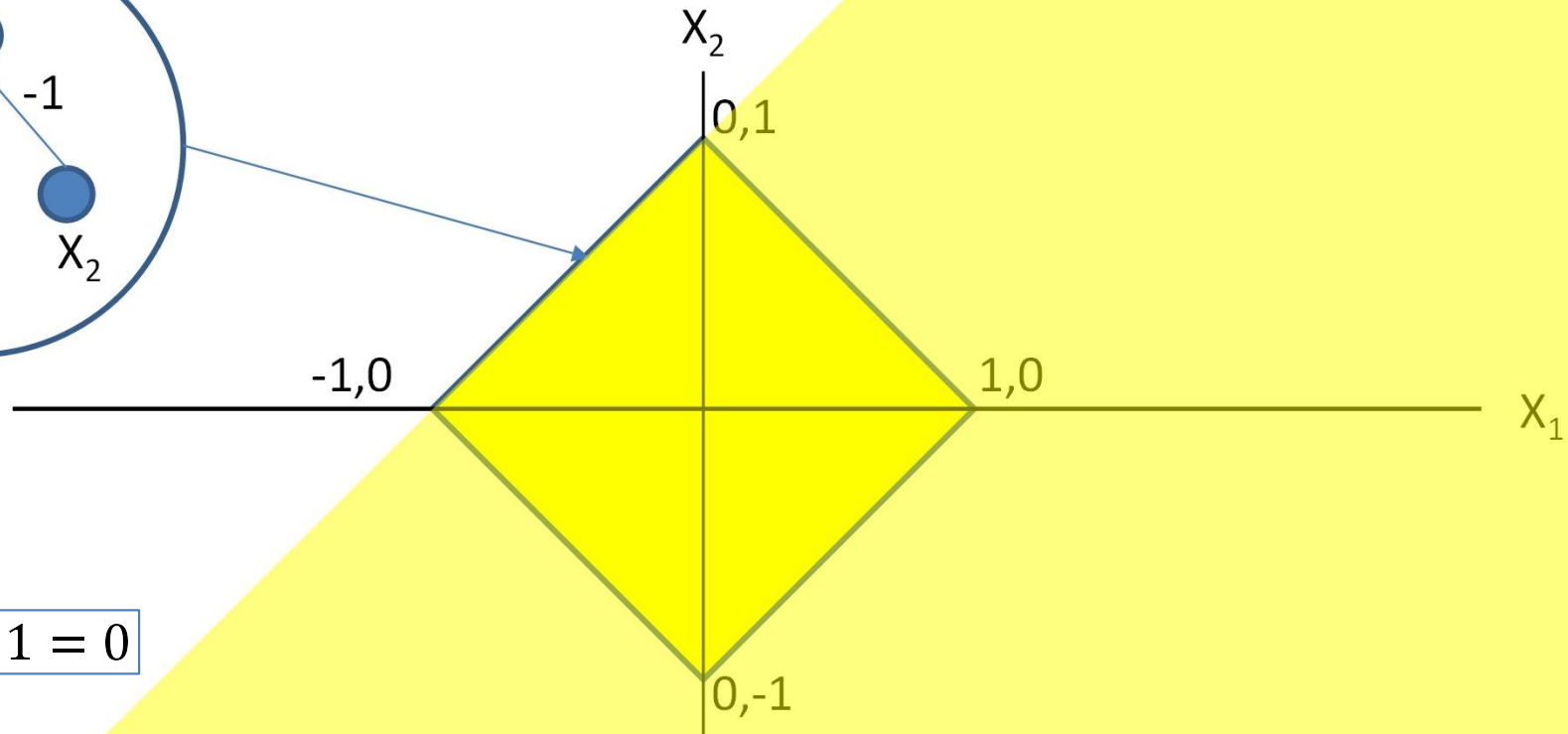
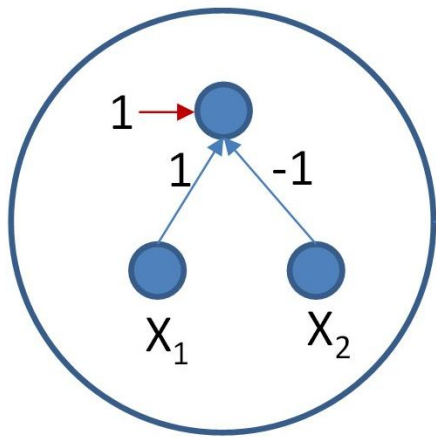
- The MLP *can be constructed* to represent anything
- But *how* do we construct it?

Option 1: Construct by hand



- Given a function, *handcraft* a network to satisfy it
- E.g.: Build an MLP to classify this decision boundary

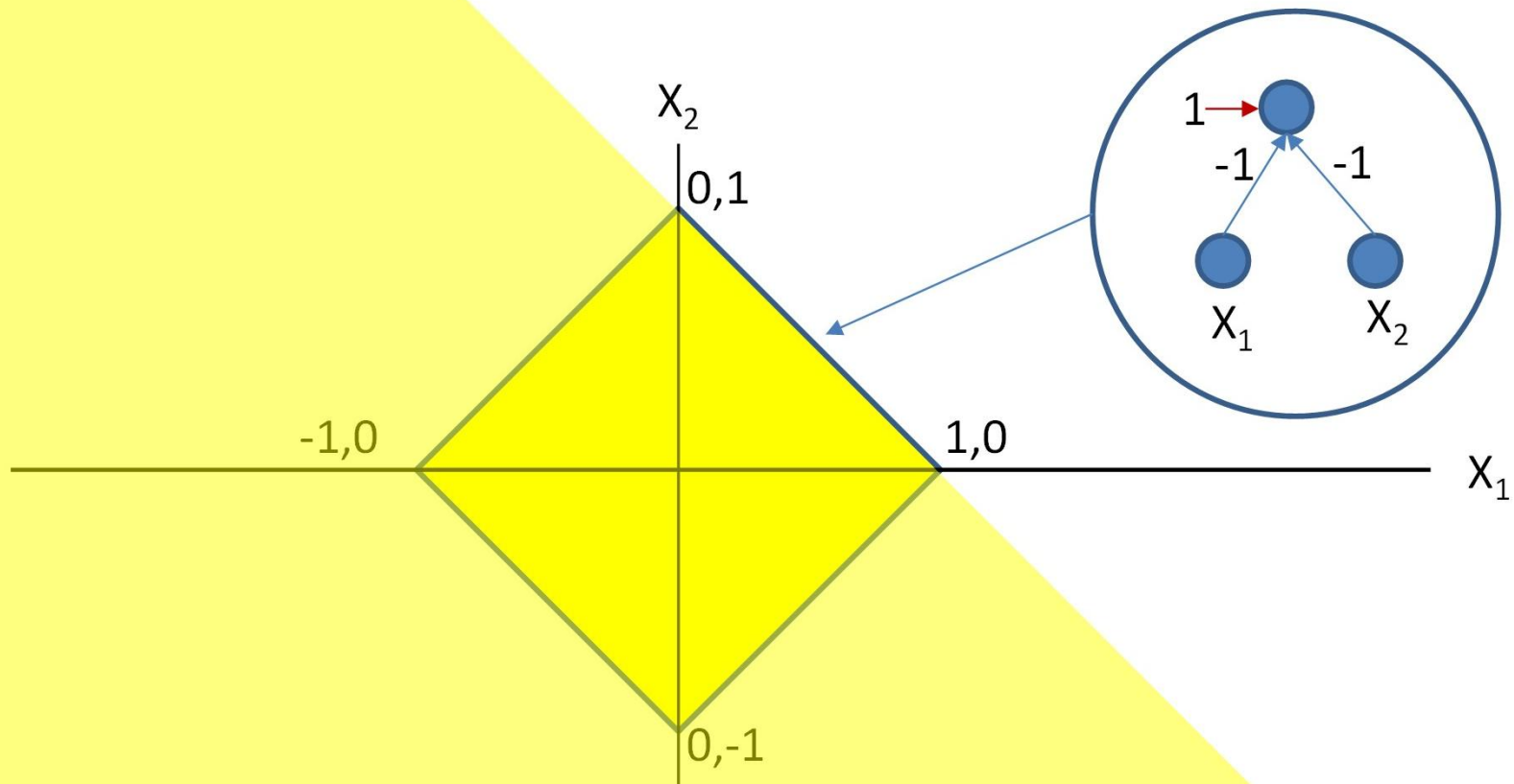
Option 1: Construct by hand



$$x_1 - x_2 + 1 = 0$$

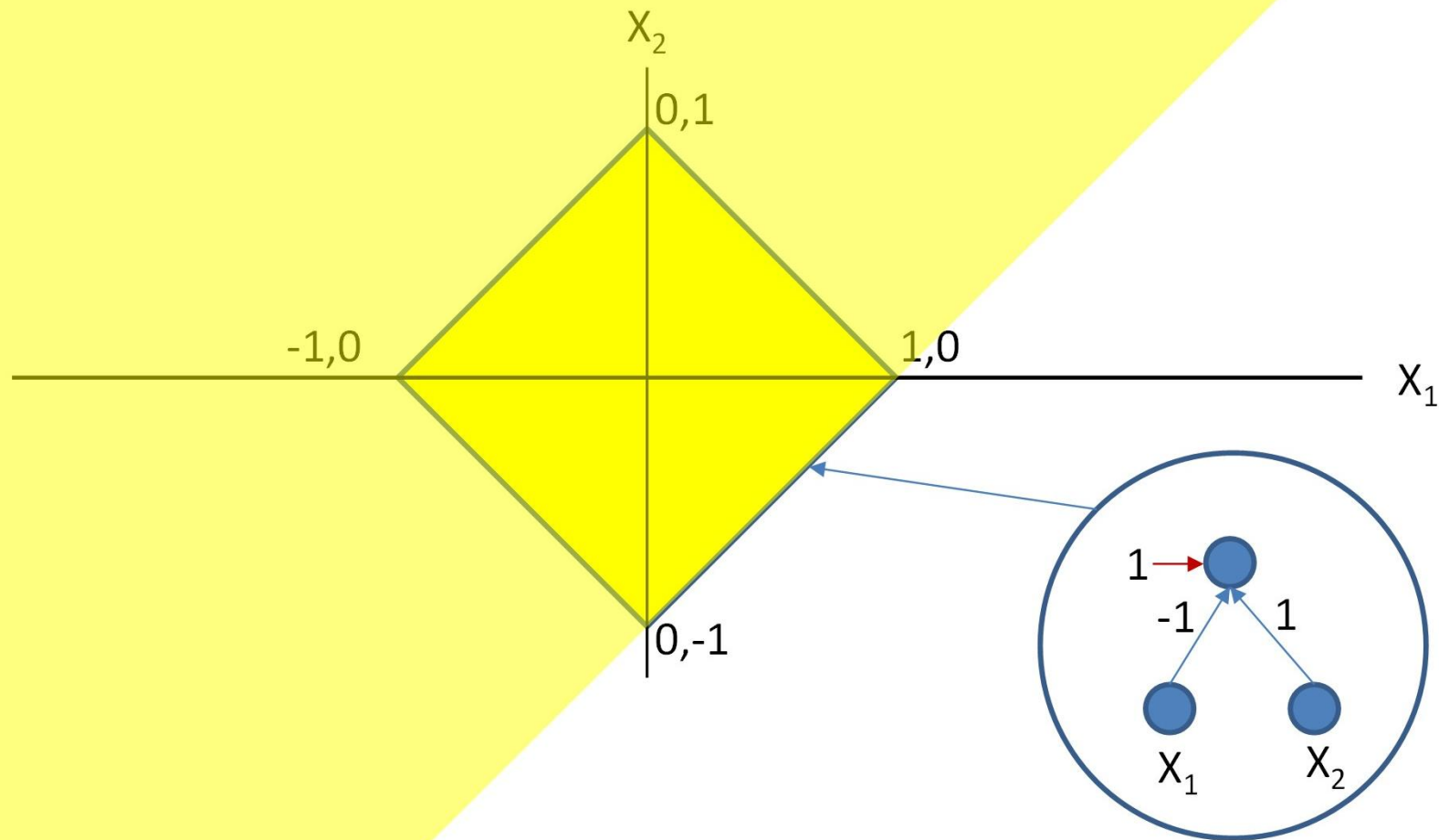
Assuming simple perceptrons:
output = 1 if $\sum_i w_i x_i + b_i \geq 0$, else 0

Option 1: Construct by hand



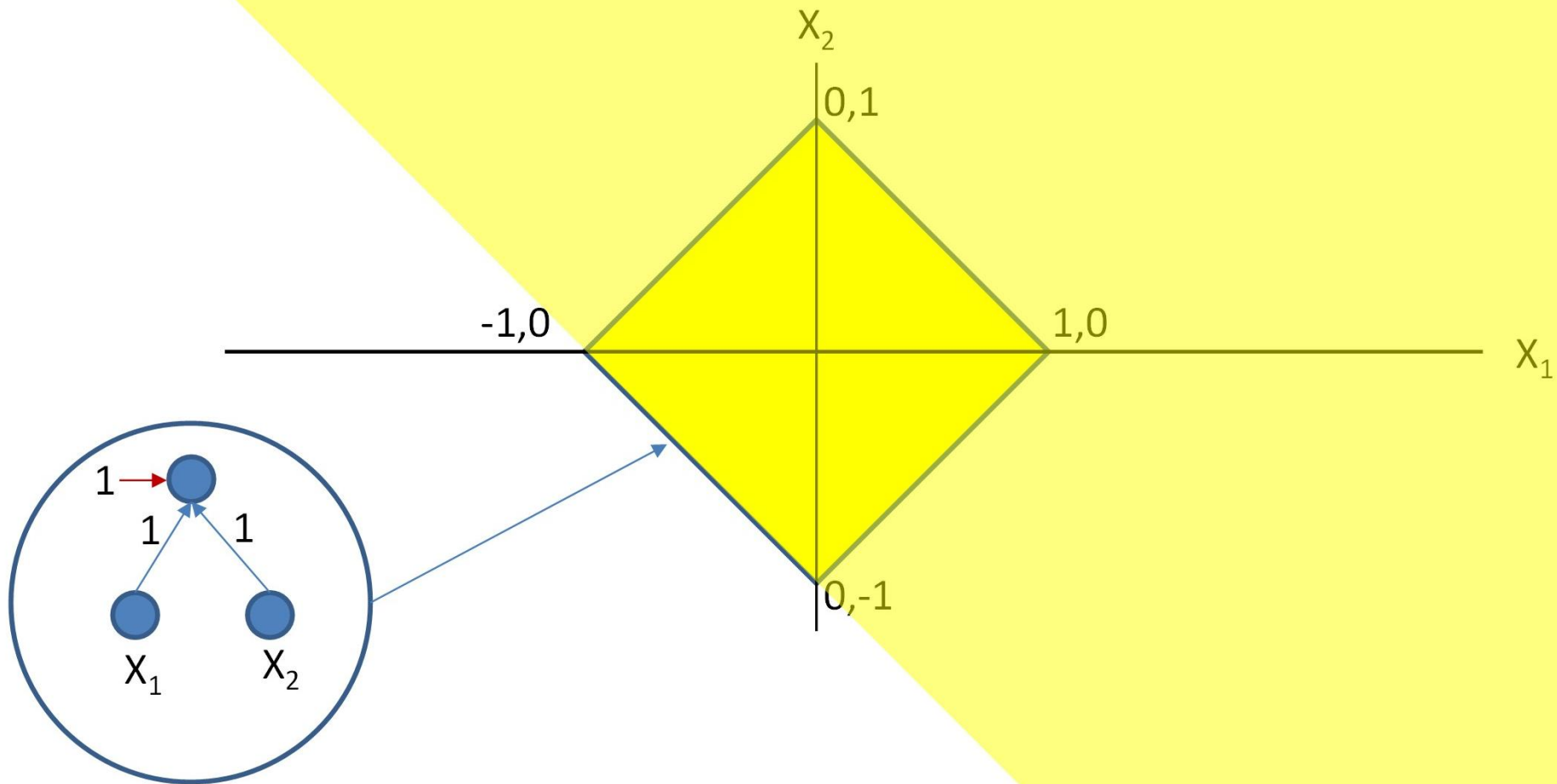
Assuming simple perceptrons:
output = 1 if $\sum_i w_i x_i + b_i \geq 0$, else 0

Option 1: Construct by hand



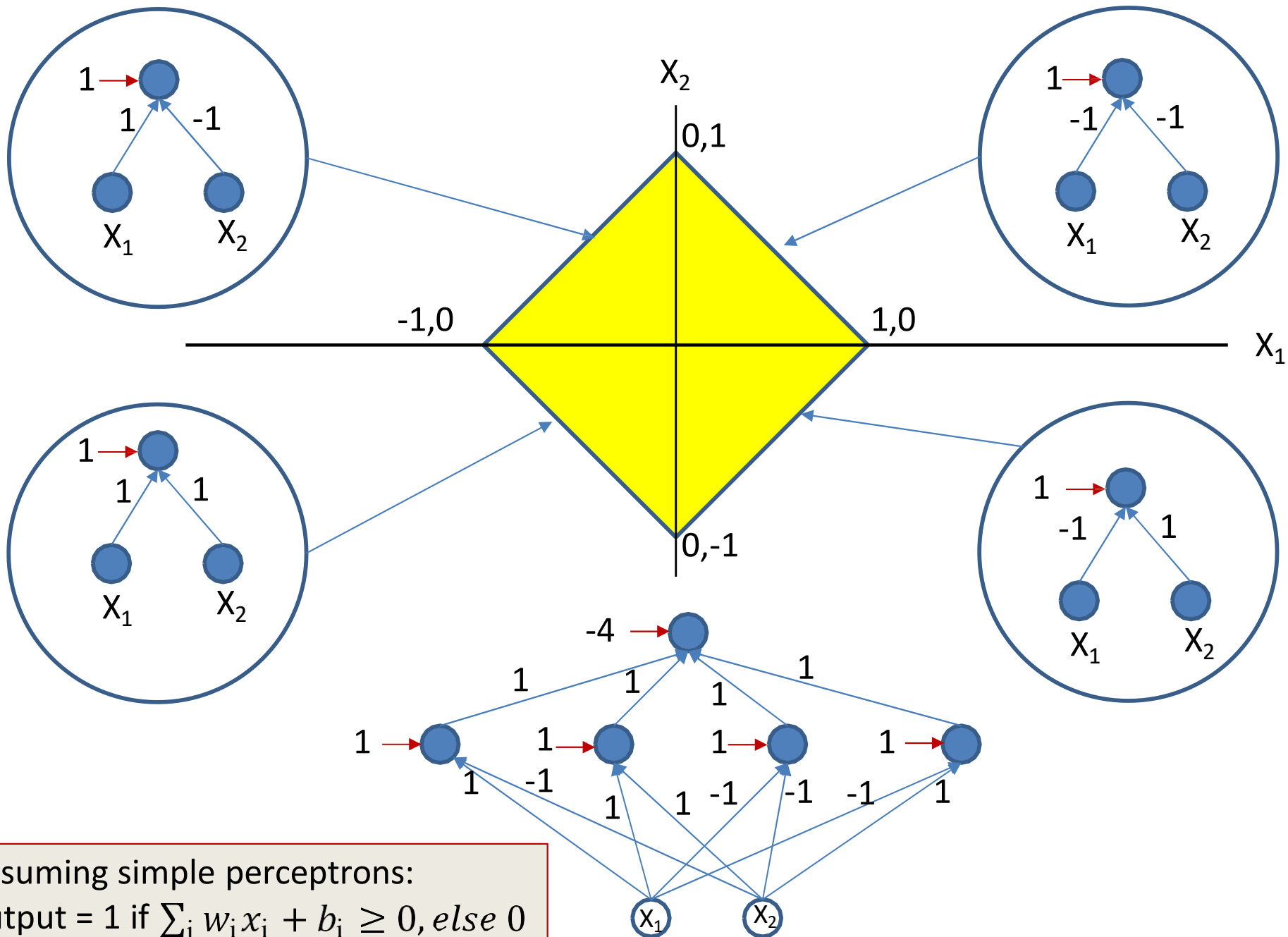
Assuming simple perceptrons:
output = 1 if $\sum_i w_i x_i + b_i \geq 0$, else 0

Option 1: Construct by hand



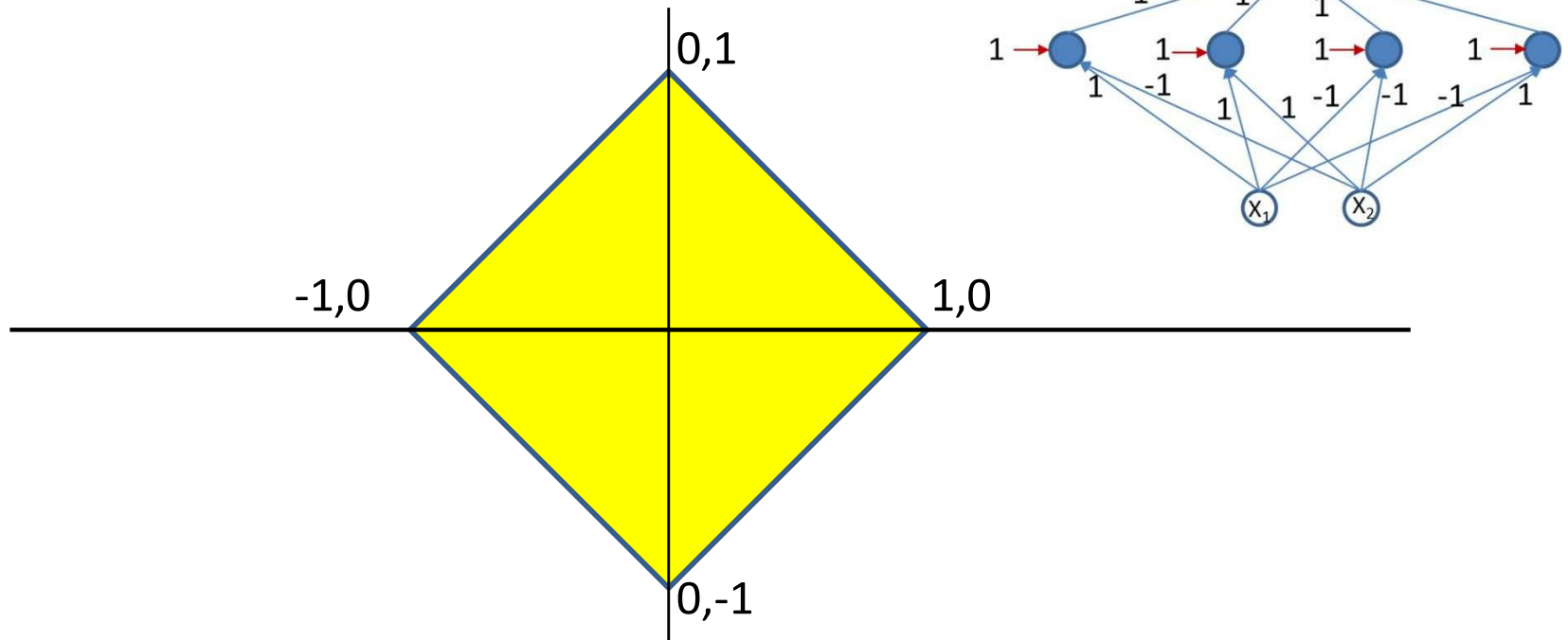
Assuming simple perceptrons:
output = 1 if $\sum_i w_i x_i + b_i \geq 0$, else 0

Option 1: Construct by hand



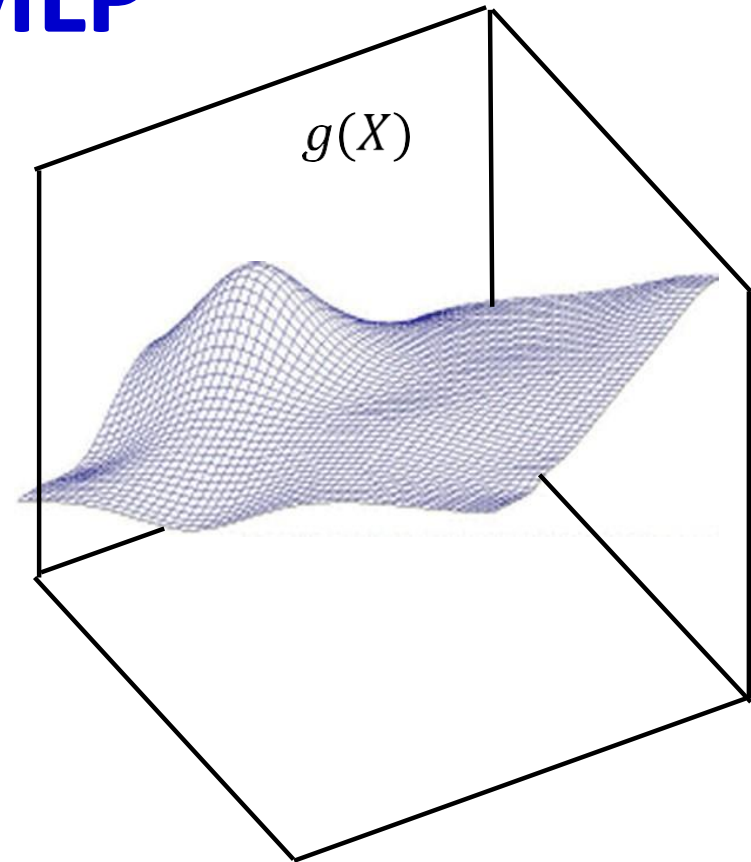
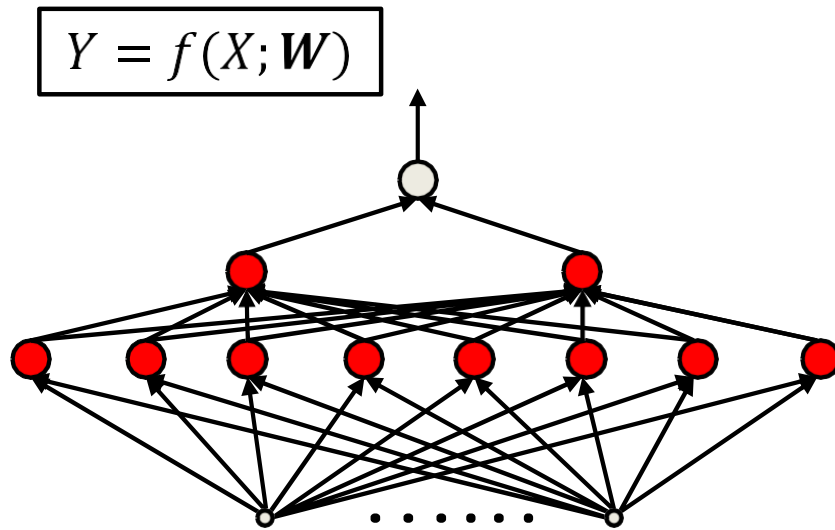
Assuming simple perceptrons:
 output = 1 if $\sum_i w_i x_i + b_i \geq 0$, else 0

Option 1: Construct by hand



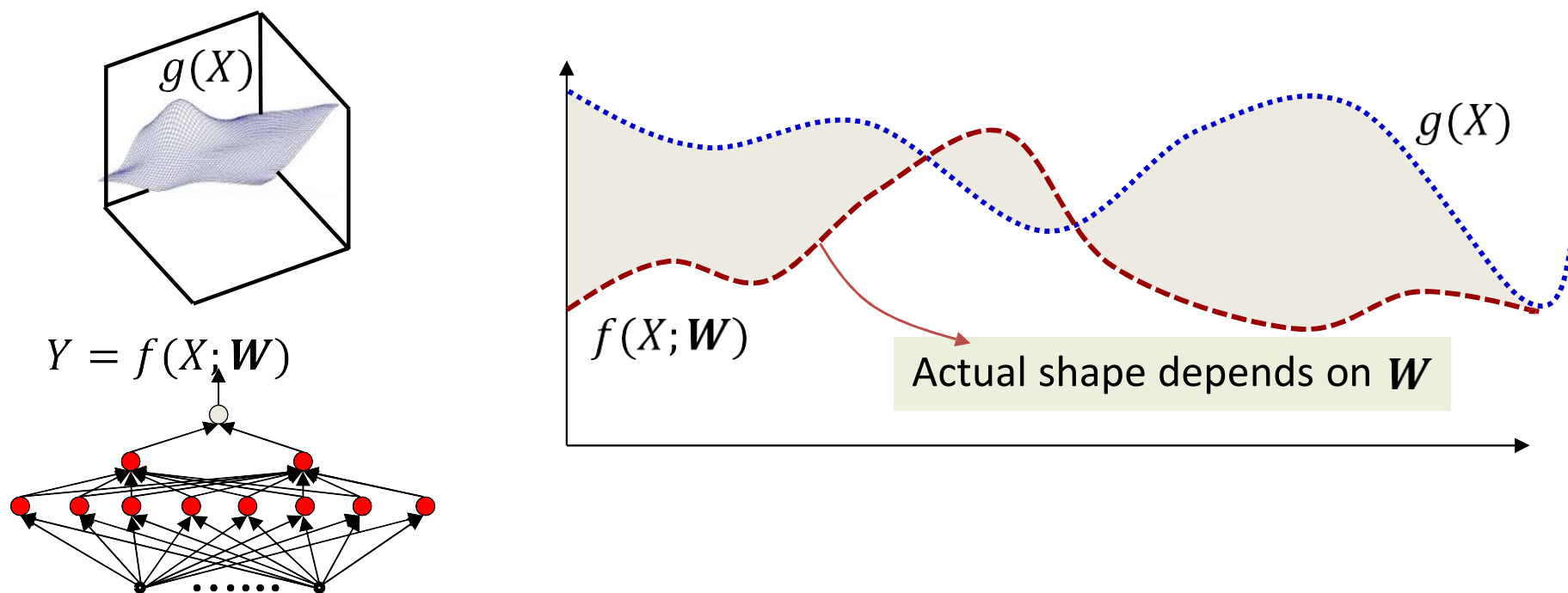
- Given a function, *handcraft* a network to satisfy it
- E.g.: Build an MLP to classify this decision boundary
- Not possible for all but the simplest problems..

Option 2: Automatic estimation of an MLP



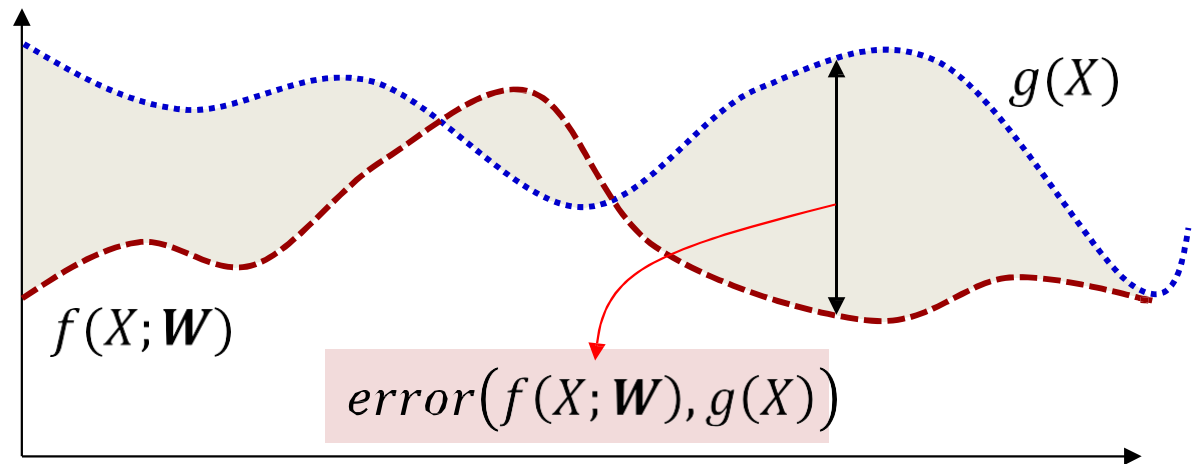
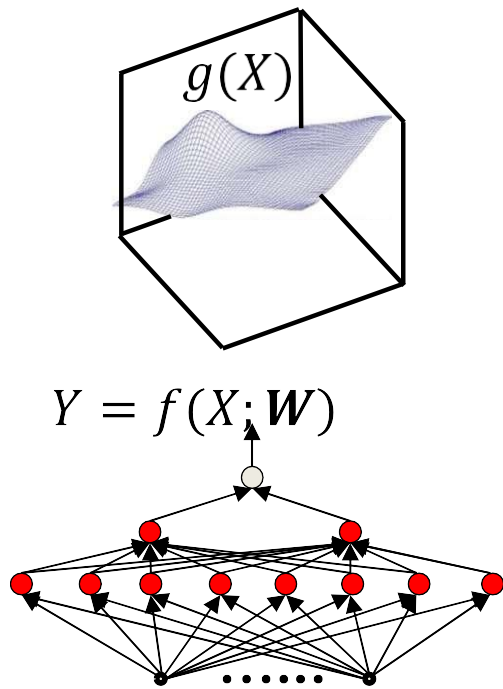
- More generally, *given* the function $g(X)$ to model, we can *derive* the parameters of the network to model it, through computation

How to learn a network?



- Solution: Estimate parameters to minimize the error between the target function $g(X)$ and the network function $f(X; W)$
 - Find the parameter W that minimizes the shaded area

How to learn a network?



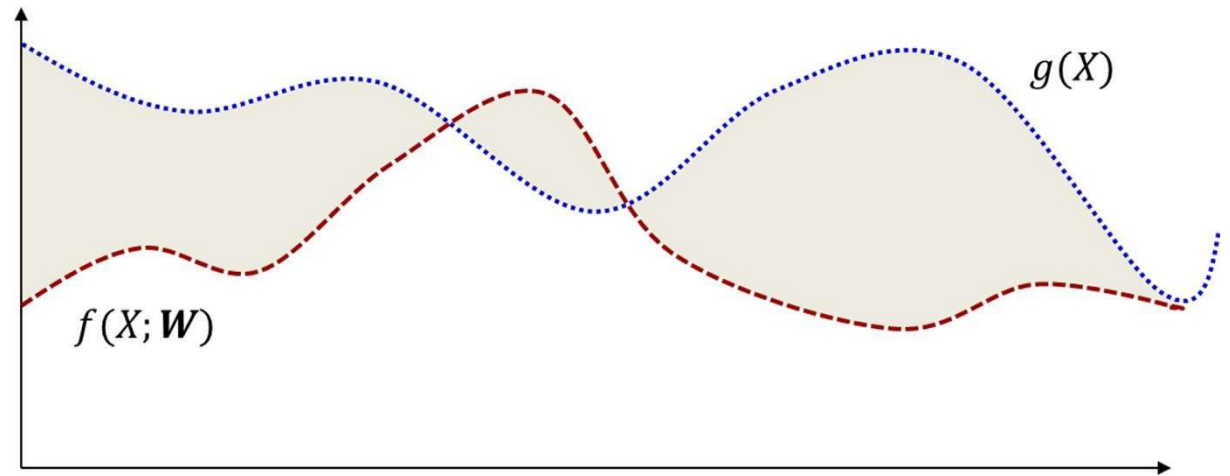
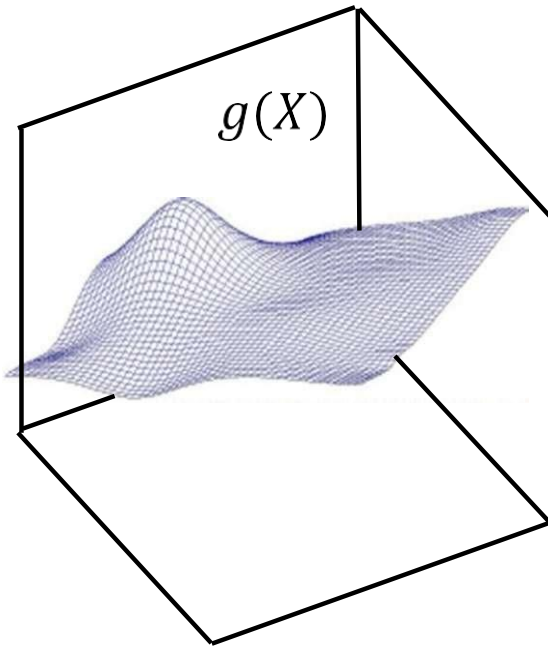
- The shaded area

$$totalerr(W) = \int_{-\infty}^{\infty} error(f(X; W), g(X)) dX$$

- The optimal W

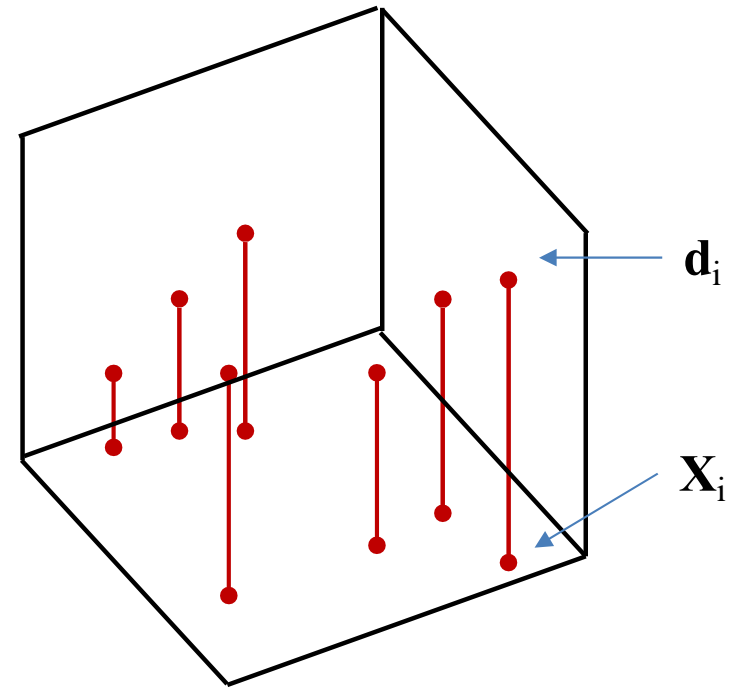
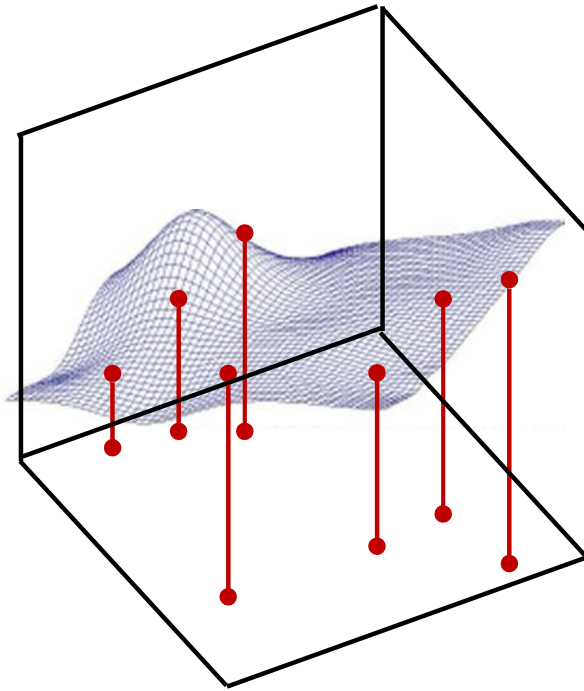
$$\widehat{W} = \operatorname{argmin}_W totalerr(W)$$

Problem: $g(X)$ is unknown



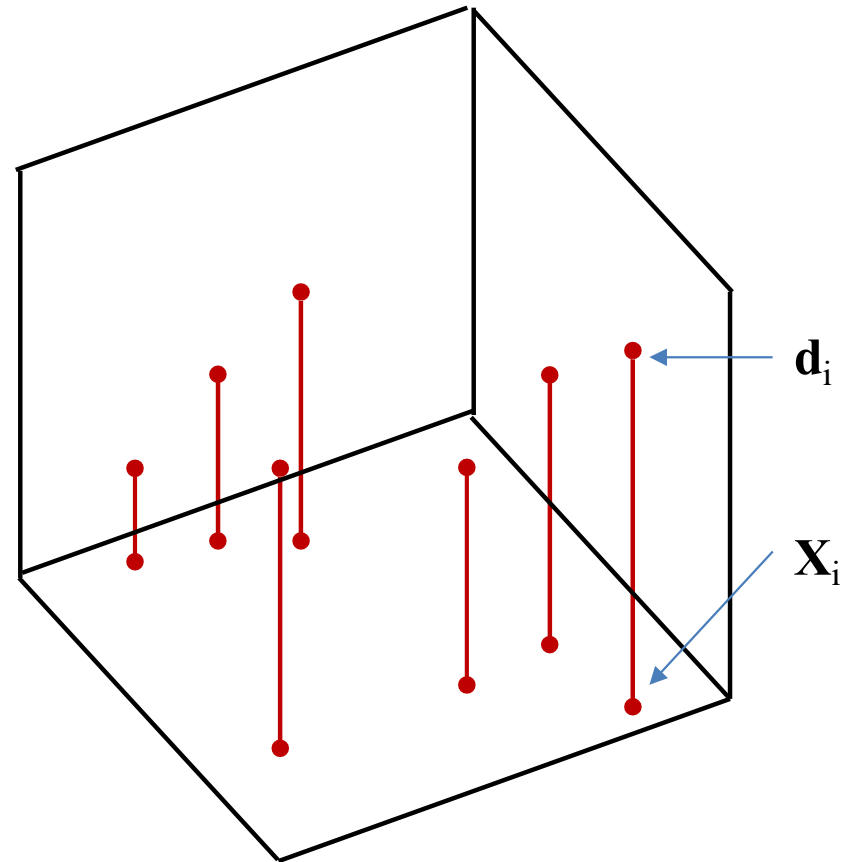
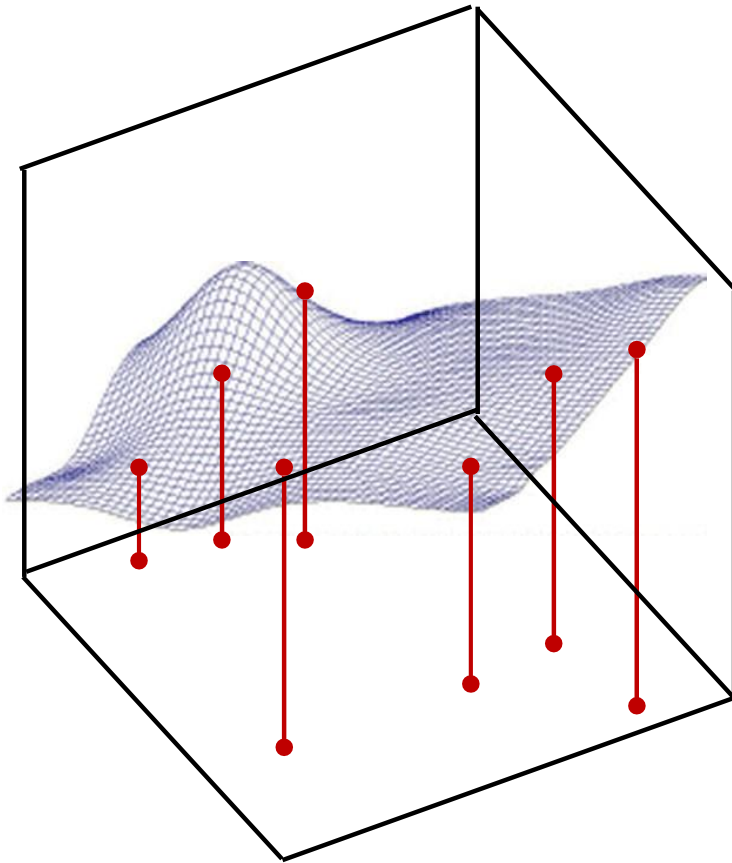
- Function $g(X)$ must be fully specified in order to compute
$$\int_{-\infty}^{\infty} \text{error}(f(X; \mathbf{W}), g(X)) dX$$
 - Known *everywhere*, i.e. for *every* input X
- **In practice we will not have such specification**

Sampling the function



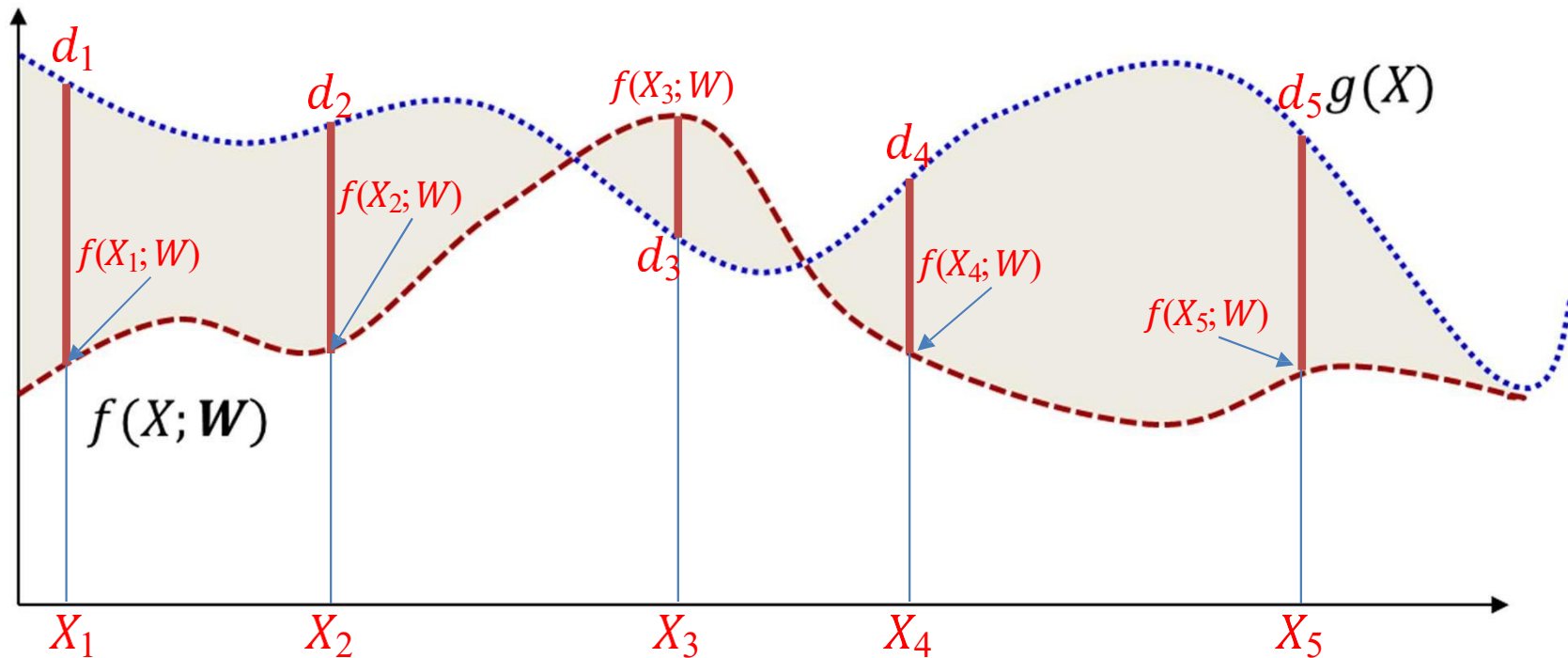
- *Sample $g(X)$*
 - Basically, get input-output pairs for a number of samples of input X_i
 - Many samples (X_i, d_i) , where $d_i = g(X_i) + \text{noise}$
- Very easy to do in most problems: just gather training data
 - E.g. set of images and their class labels
 - E.g. speech recordings and their transcription

Drawing samples



- We must *learn* the *entire* function from these few examples
 - The “training” samples

The *Empirical* error



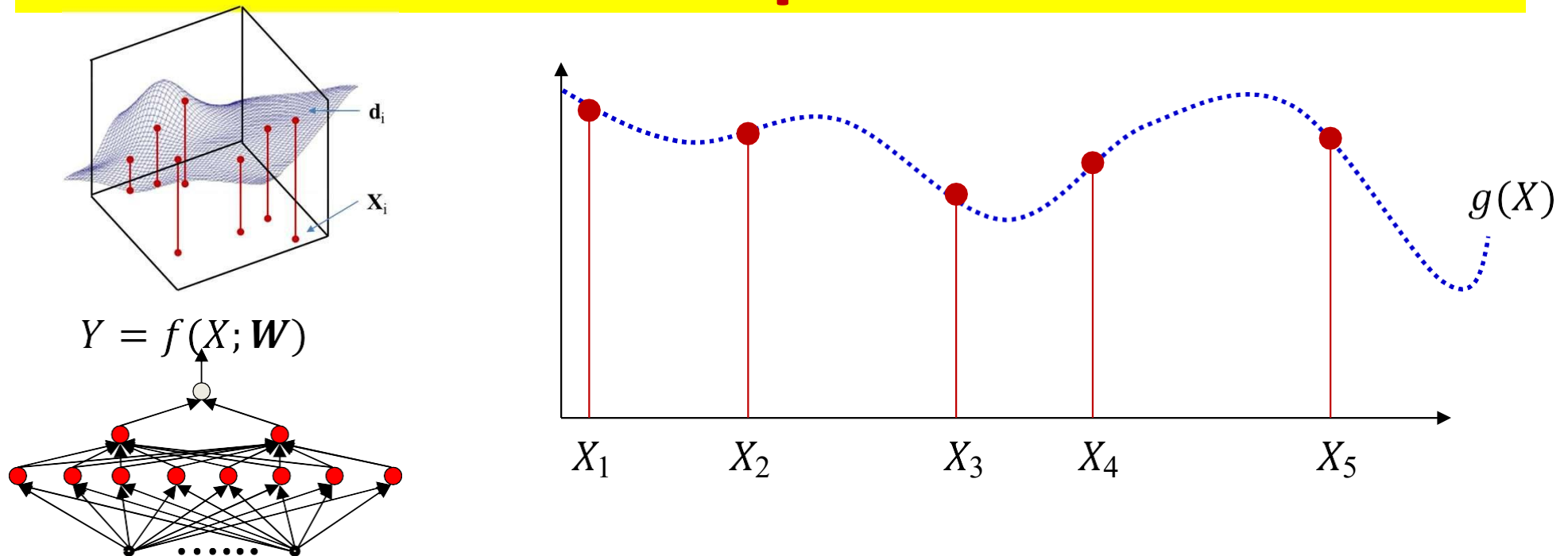
- The *empirical estimate* of the error is the *average* error over the training samples

$$\text{EmpiricalError}(\mathbf{W}) = \frac{1}{N} \sum_{i=1}^N \text{error}(f(X_i; \mathbf{W}), d_i)$$

- Estimate network parameters to minimize this average error instead

$$\widehat{\mathbf{W}} = \underset{\mathbf{W}}{\operatorname{argmin}} \text{EmpiricalError}(\mathbf{W})$$

Learning the function from training samples



- Aim: Find the network parameters that “fit” the training points exactly
 $W: EmpiricalError(W) = 0$
 - Assuming network architecture is sufficient for such a fit
 - Assuming unique output d at any X
- And hopefully the resulting function is also correct where we *don't* have training samples

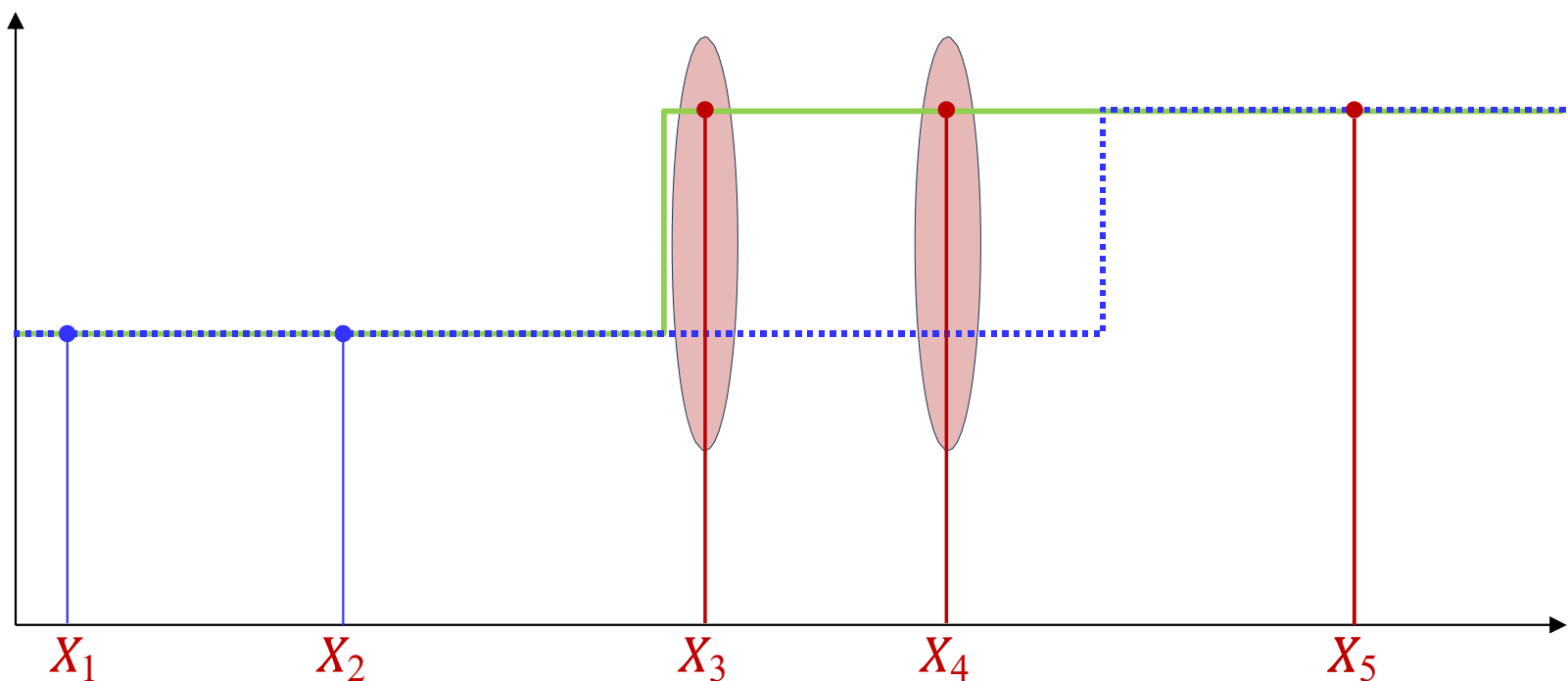
Story so far

- “Learning” a neural network == determining the parameters of the network (weights and biases) required for it to model a desired function
 - The network must have sufficient capacity to model the function
- Ideally, we would like to optimize the network to represent the desired function everywhere
- However, this requires knowledge of the function everywhere
- Instead, we draw “input-output” *training* instances from the function and estimate network parameters to “fit” the input-output relation at these instances
 - And hope it fits the function elsewhere as well

Let's begin with a simple task

- Learning a *classifier*
 - Simpler than regressions
- This was among the earliest problems addressed using MLPs
- Specifically, consider *binary* classification
 - Generalizes to multi-class

The *Empirical Classification* error

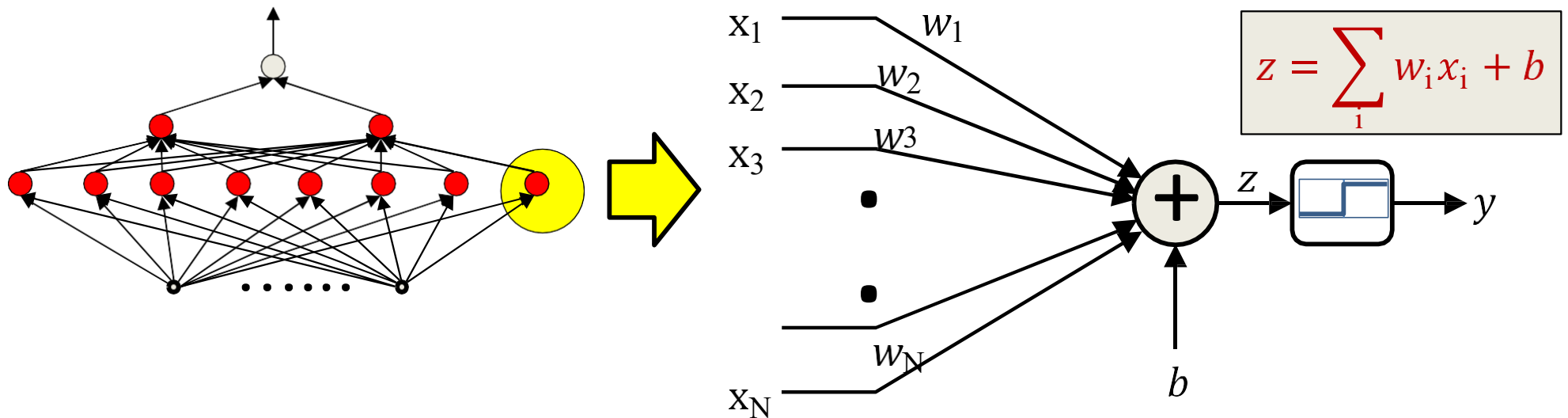


- The obvious error metric in a classifier is binary
 - The classifier is either right (error=0) or wrong (error=1)
 - Either $f(X; \mathbf{W}) = d$, or $f(X; \mathbf{W}) \neq d$

$$\text{EmpiricalError}(\mathbf{W}) = \frac{1}{N} \sum_{i=1}^N 1(f(X_i; \mathbf{W}) \neq d_i)$$

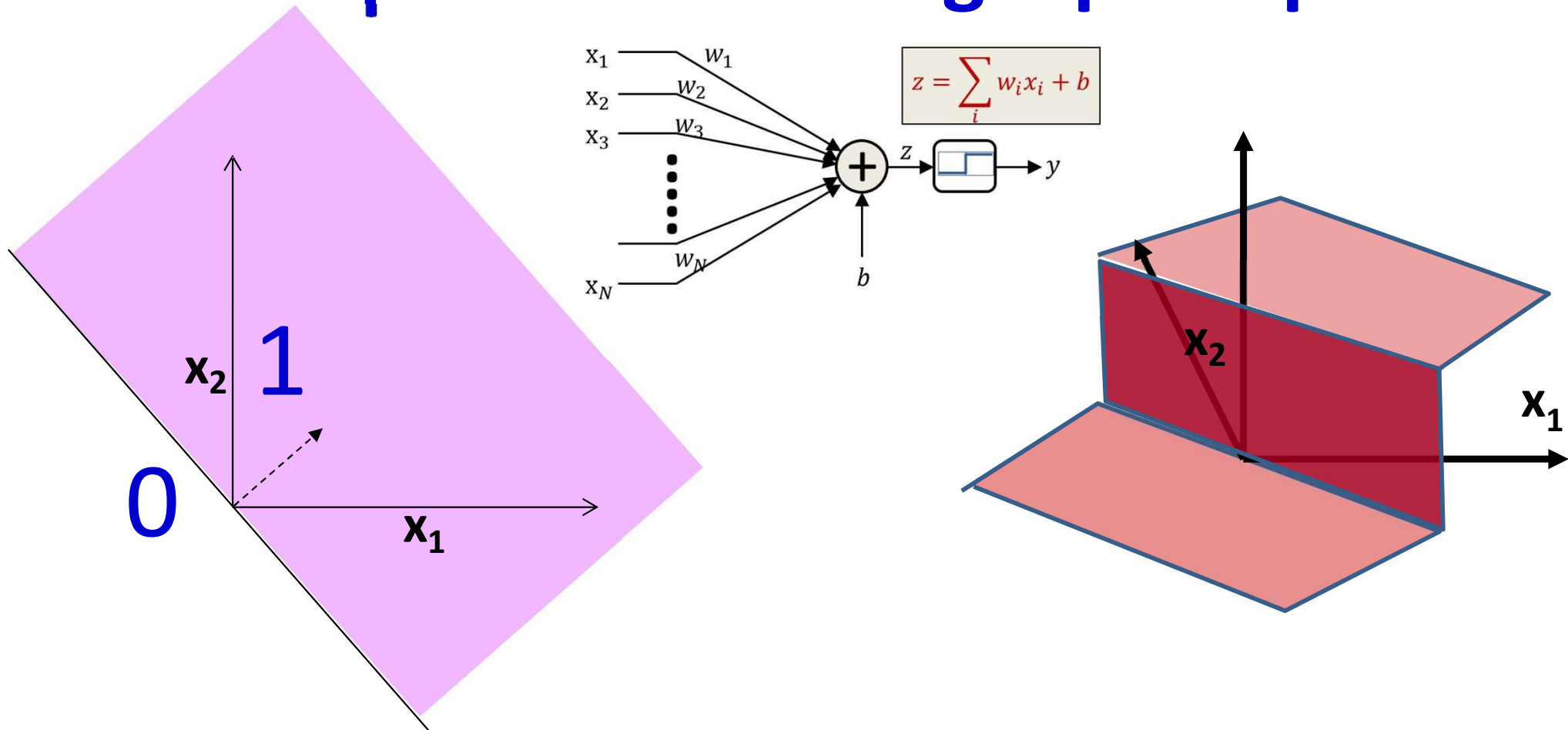
- **Learning the classifier:** Minimizing the count of misclassifications

History: The original MLP



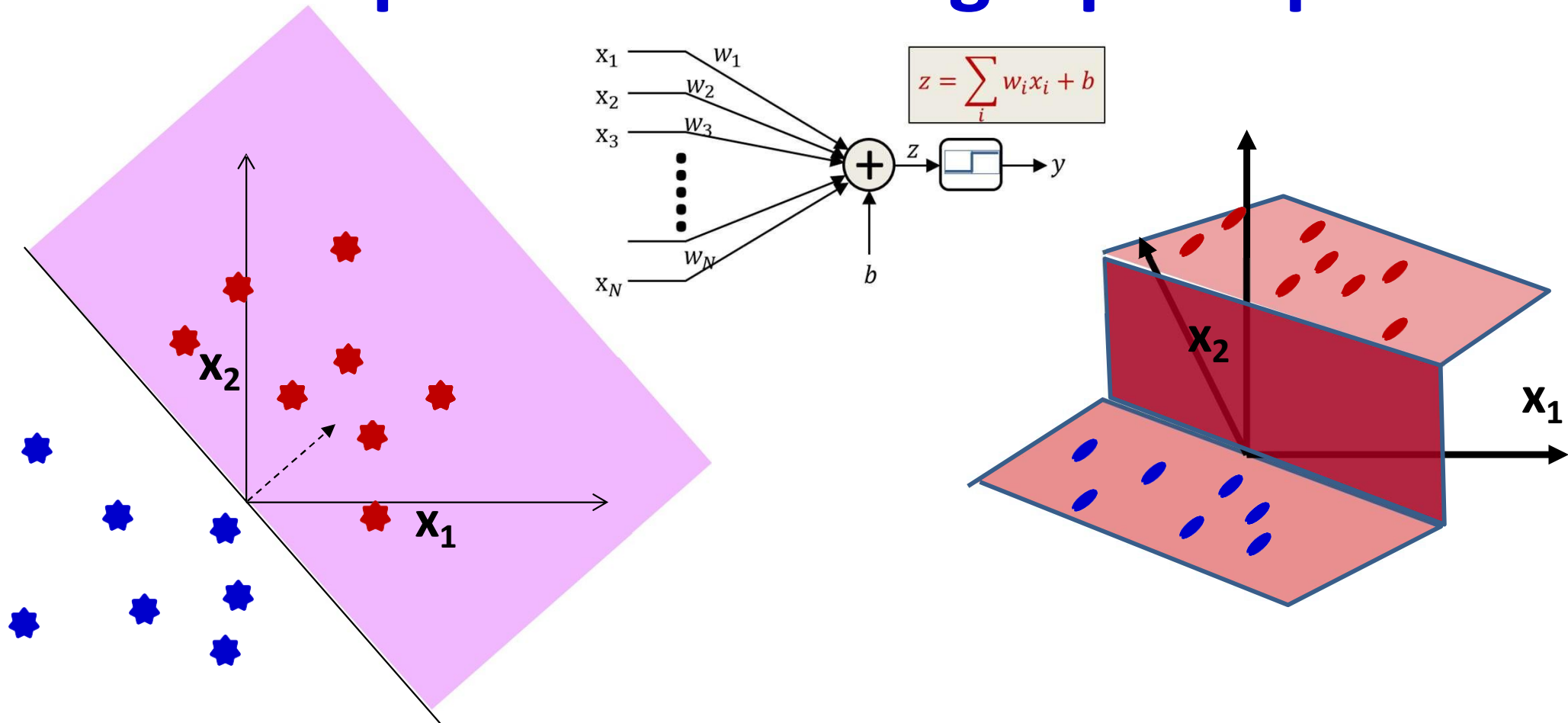
- The original MLP as proposed by Rosenblatt: a network of threshold units
 - But how do you train it?
 - Given only “training” instances of input-output pairs

The simplest MLP: a single perceptron



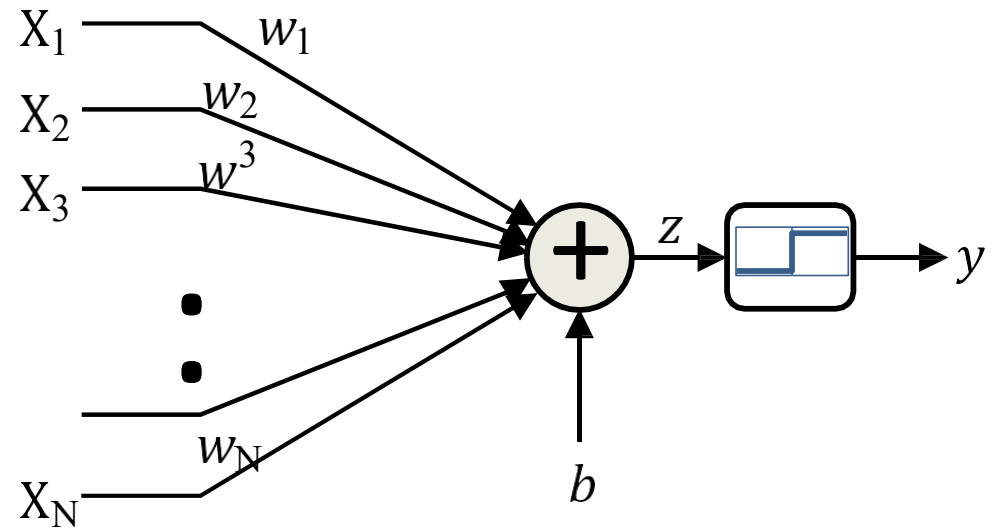
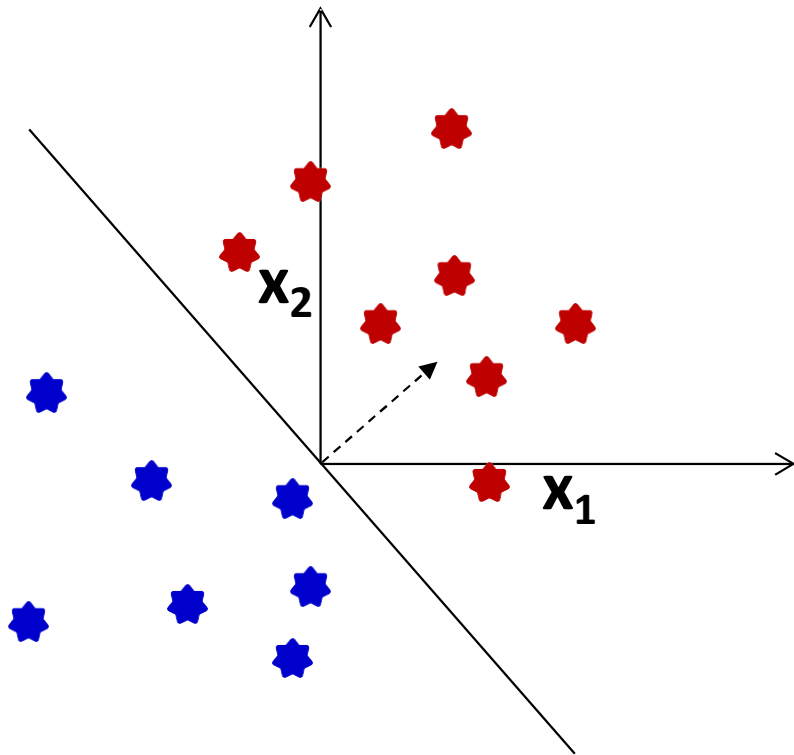
- Learn this function
 - A step function across a hyperplane

The simplest MLP: a single perceptron



- Learn this function
 - A step function across a hyperplane
 - Given only samples from it

Learning the perceptron



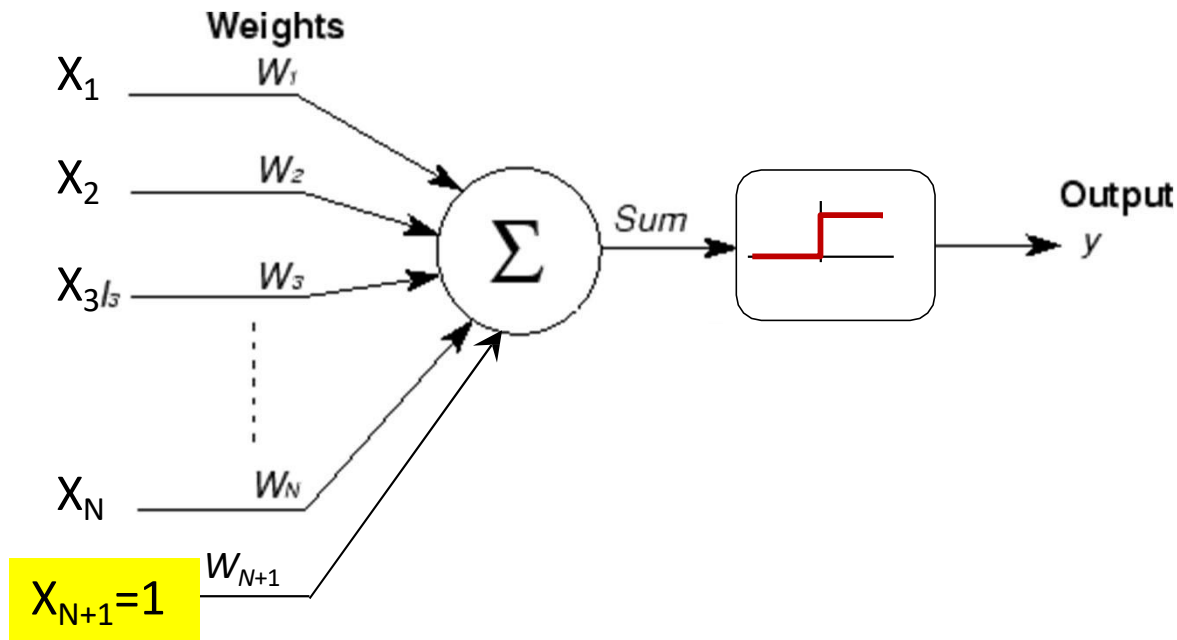
- Given a number of input output pairs, learn the weights and bias

$$y = \begin{cases} 1 & \text{if } \sum_{i=1}^N w_i X_i + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\text{Boundary: } \sum_{i=1}^N w_i X_i + b = 0$$

- Learn $W = [w_1 \dots w_N]^T$ and b , given several (X, y) pairs
 - $X = [X_1 \dots X_N]^T$

Restating the perceptron



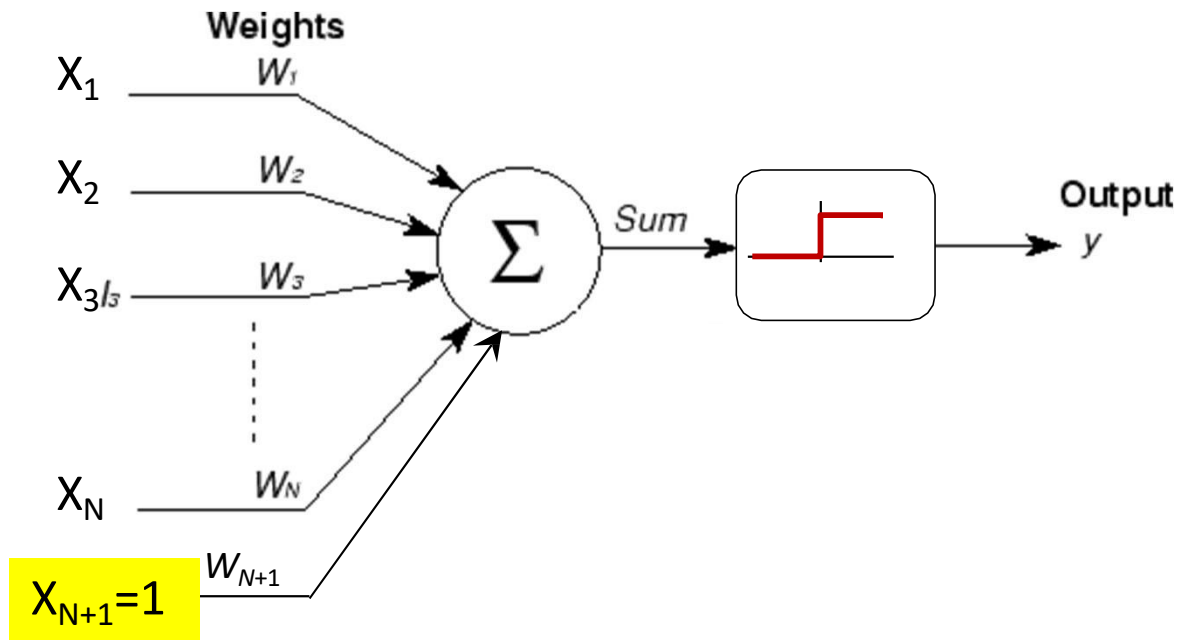
- Restating the perceptron equation by adding another dimension to X

$$y = \begin{cases} 1 & \text{if } \sum_{i=1}^{N+1} w_i X_i \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

where $X_{N+1} = 1$

- Note that the boundary $\sum_{i=1}^{N+1} w_i X_i = 0$ is now a hyperplane through origin

Restating the perceptron



- Restating the perceptron equation by adding another dimension to X

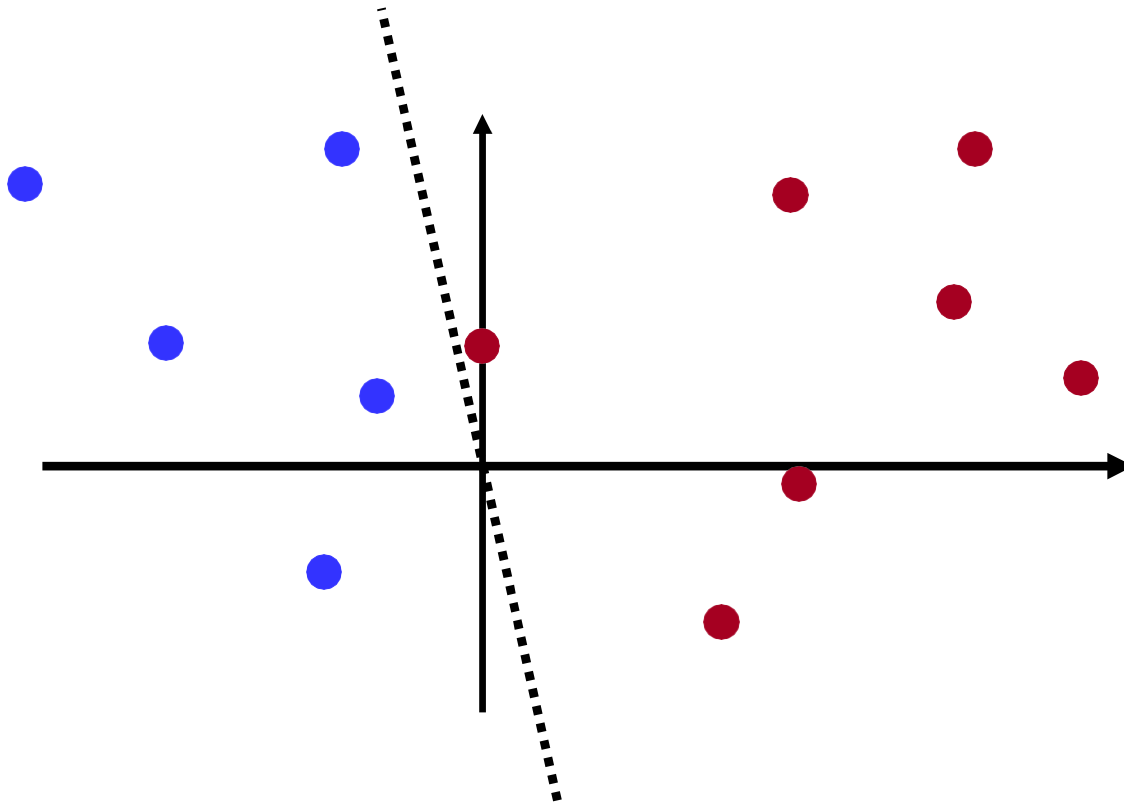
$$y = \begin{cases} 1 & \text{if } \sum_{i=1}^{N+1} w_i X_i \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

where $X_{N+1} = 1$

How did we make a hyperplane that did not pass through the origin suddenly go through the origin by adding a dimension?

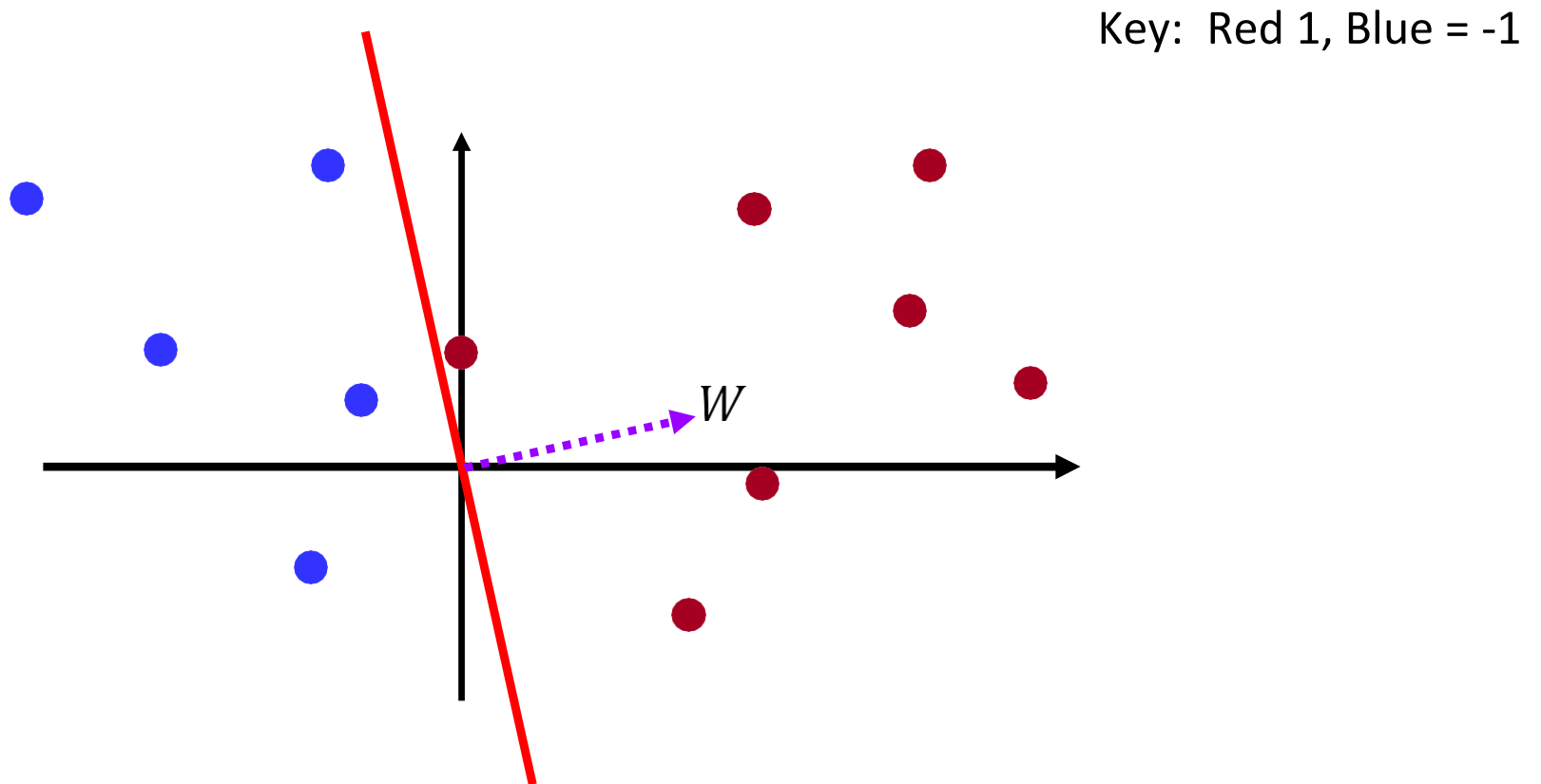
- Note that the boundary $\sum_{i=1}^{N+1} w_i X_i = 0$ is now a hyperplane through origin

The Perceptron Problem



- Find the hyperplane $\sum_{i=1}^{N+1} w_i X_i = 0$ that perfectly separates the two groups of points

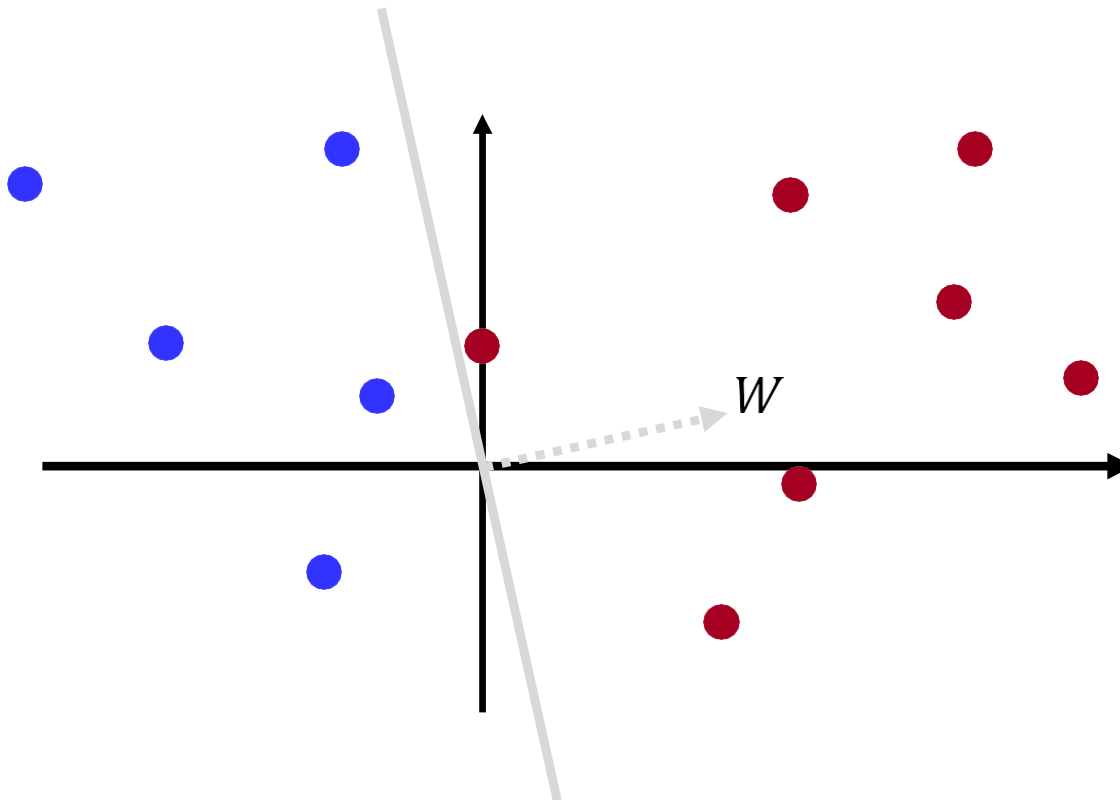
The Perceptron Problem



- Find the hyperplane $\sum_{i=1}^{N+1} w_i X_i = 0$ that perfectly separates the two groups of points
 - Let vector $W = [w_1, w_2, \dots, w_{N+1}]^T$ and vector $X = [x_1, x_2, \dots, x_N, 1]^T$
 - $\sum_{i=1}^{N+1} w_i X_i = W^T X$ is an inner product
 - $W^T X = 0$ is the hyperplane comprising all X s orthogonal to vector W
 - Learning the perceptron = finding the weight vector W for the separating hyperplane
 - W points in the direction of the positive class

The Perceptron Problem

Key: Red 1, Blue = -1



- Learning the perceptron: Find the weights vector W such that the plane described by $W^T X = 0$ perfectly separates the classes
 - $W^T X$ is positive ($W^T X > 0$) for all red dots
 - The angle between W and positive-class vectors is less than 90
 - $W^T X$ is negative ($W^T X < 0$) for all blue dots
 - The angle between W and negative-class vectors is greater than 90

The *online* perceptron solution

- The popular solution, originally proposed by Rosenblatt is an *online* algorithm
 - The famous “perceptron” algorithm
- Initializes W and incrementally updates it each time we encounter an instance that is incorrectly classified
 - Guaranteed to find the correct solution for linearly separable data

Perceptron Algorithm: Summary

- Cycle through the training instances
- Only update W on misclassified instances
- If instance misclassified:
 - If instance is positive class (positive misclassified as negative)

$$W = W + X_i$$

- If instance is negative class (negative misclassified as positive)

$$W = W - X_i$$

Perceptron Learning Algorithm

- Given N training instances $(X_1, y_1), (X_2, y_2), \dots, (X_N, y_N)$
 - $y_i = +1$ or -1

Using a +1/-1 representation
for classes to simplify
notation

- Initialize W
- Cycle through the training instances:
- do

– For $i = 1 \dots N$

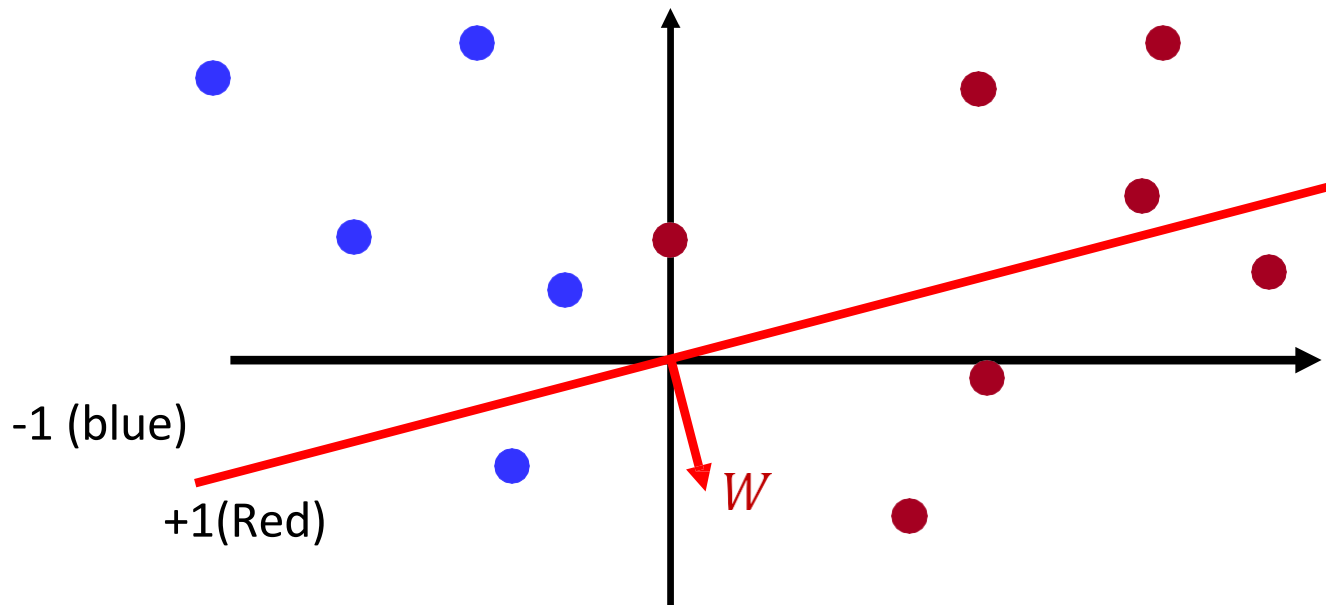
$$O(X_i) = \text{sign}(W^T X_i)$$

- If $O(X_i) \neq y_i$

$$W = W + y_i X_i$$

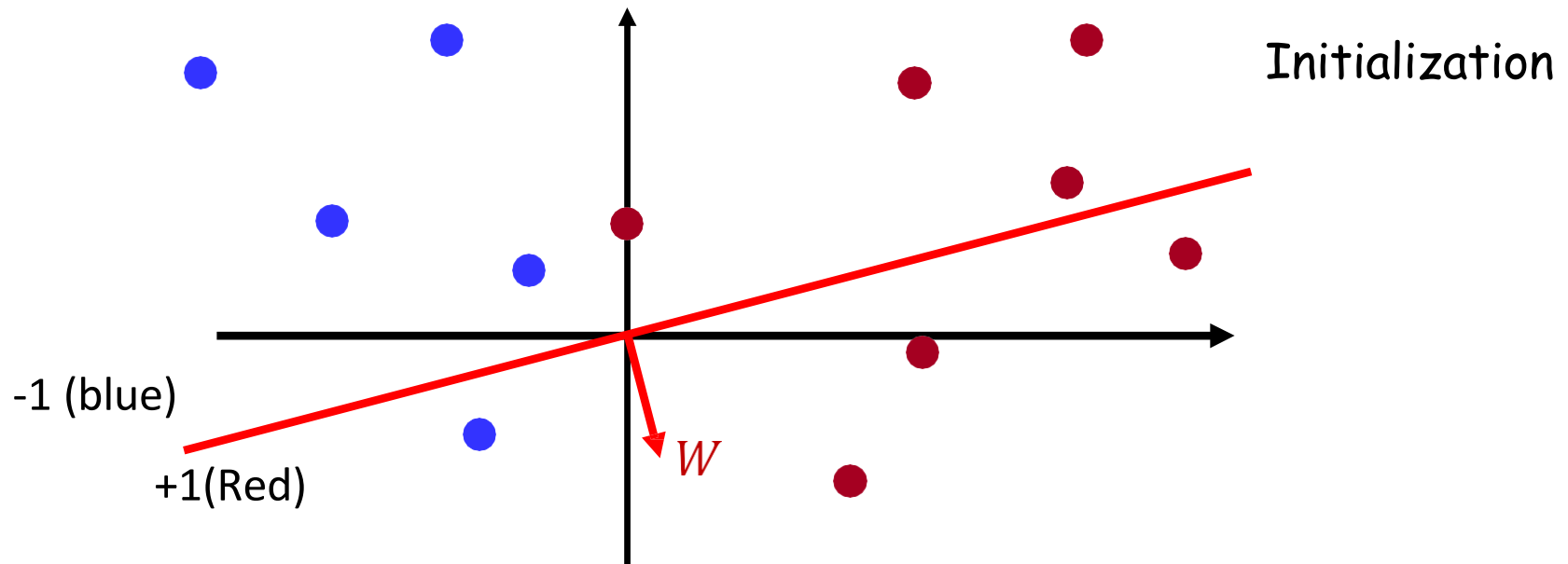
- until no more classification errors

A Simple Method: The Perceptron Algorithm

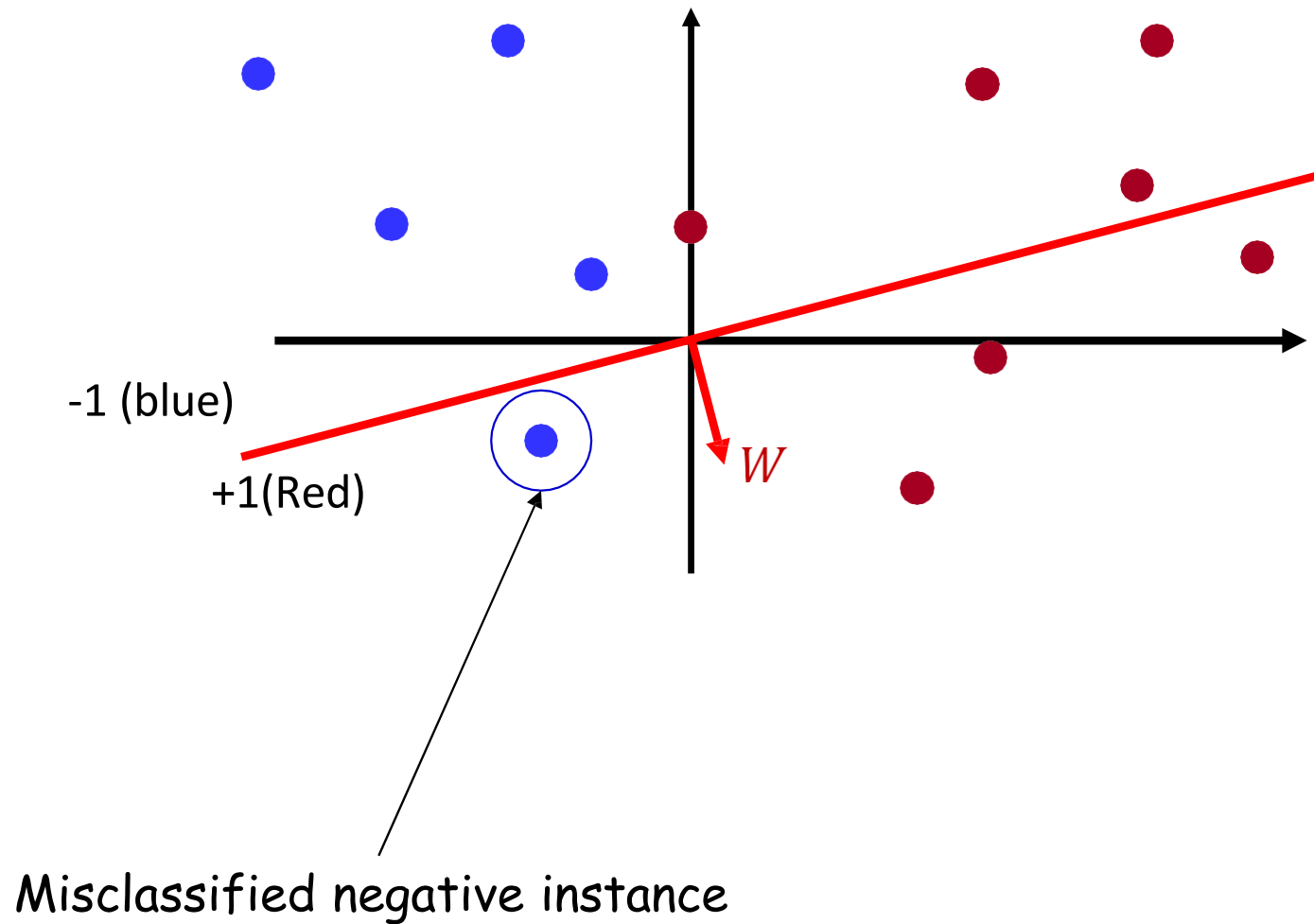


- **Initialize:** Randomly initialize the hyperplane
 - I.e. randomly initialize the normal vector W
- **Classification rule** $\text{sign}(W^T X)$
 - Vectors on the same side of the hyperplane as W will be assigned +1 class, and those on the other side will be assigned -1
- The random initial plane will make mistakes

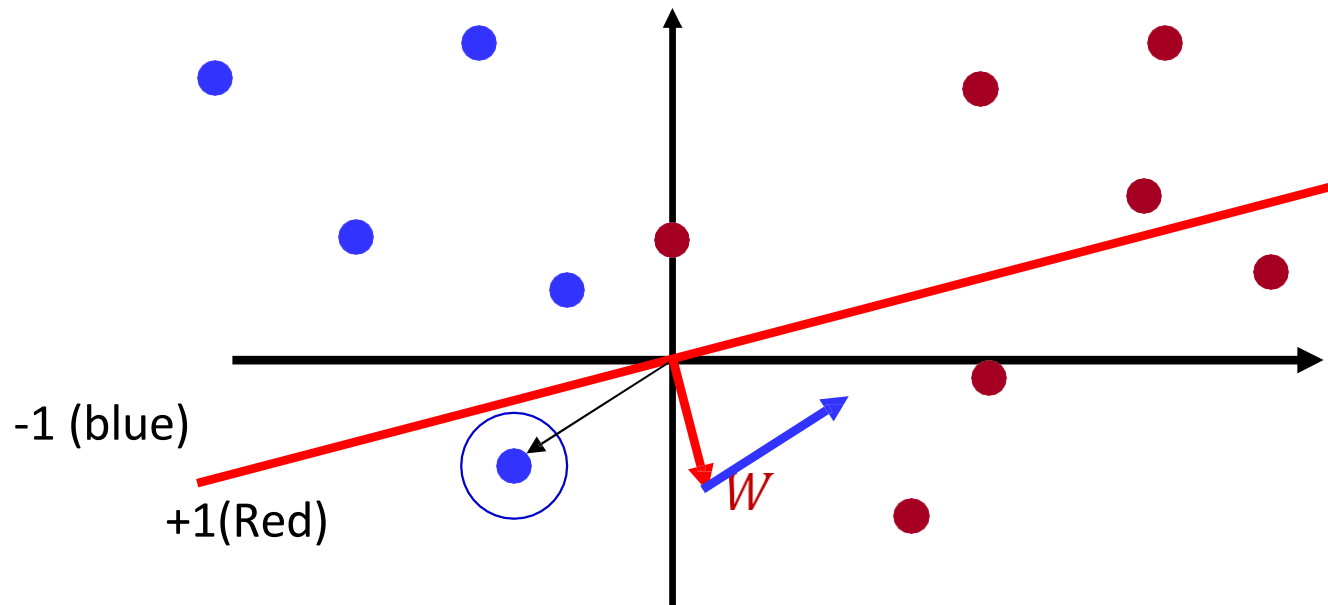
Perceptron Algorithm



Perceptron Algorithm

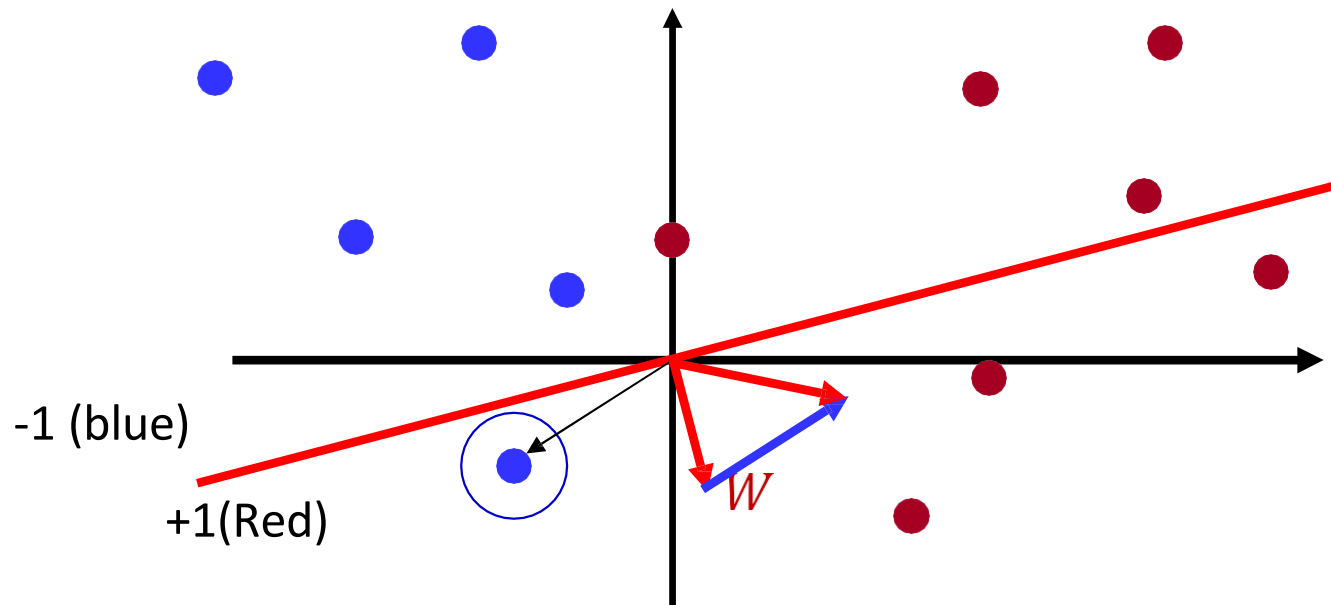


Perceptron Algorithm



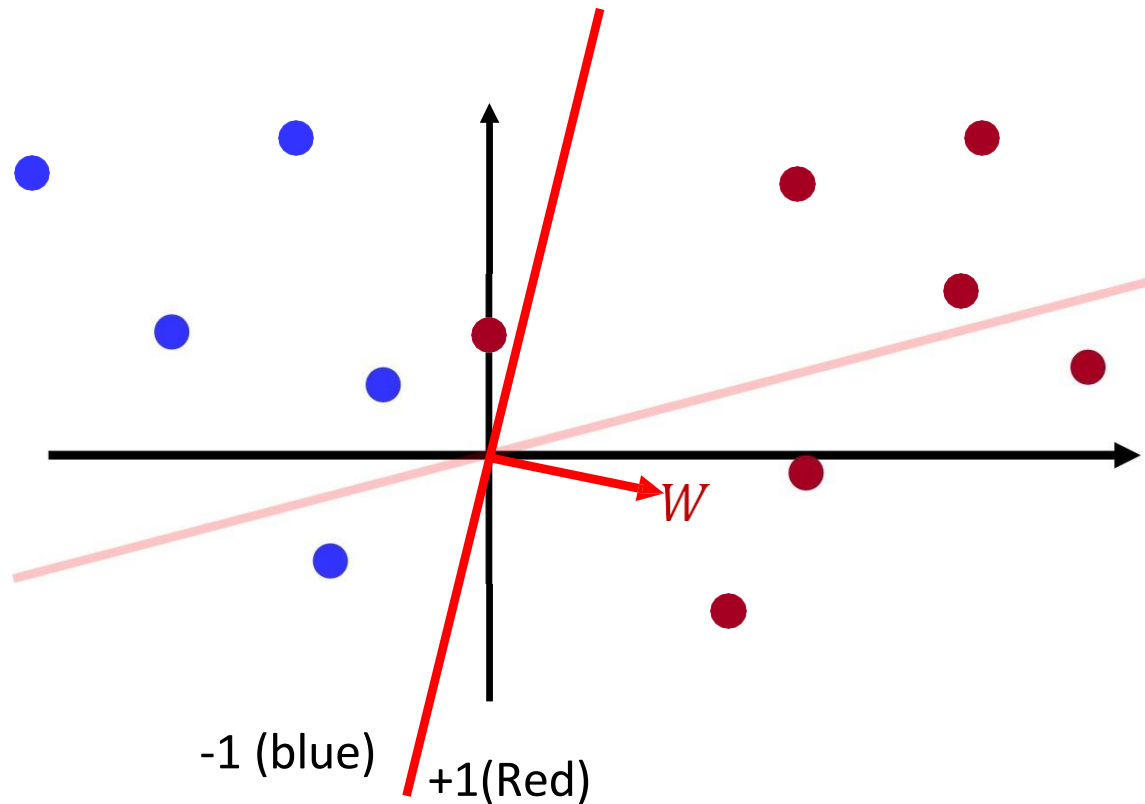
Misclassified *negative* instance, *subtract* it from W

Perceptron Algorithm



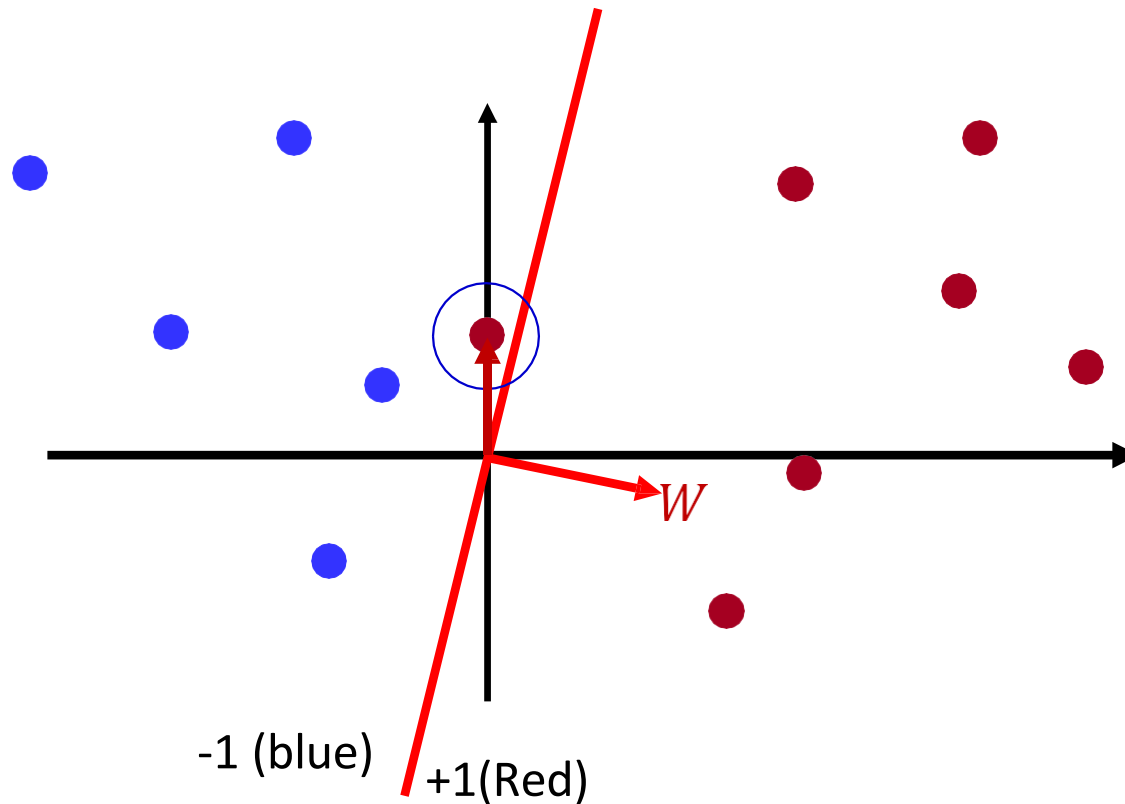
The new weight

Perceptron Algorithm



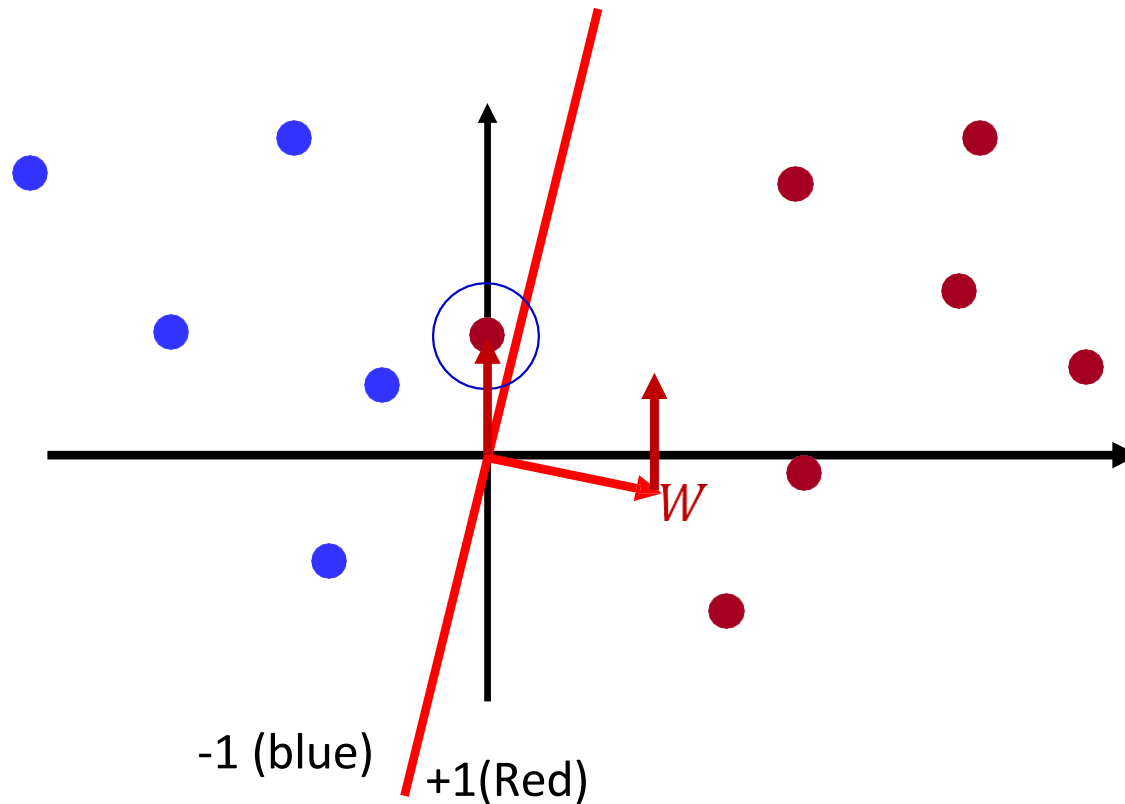
The new weight (and boundary)

Perceptron Algorithm



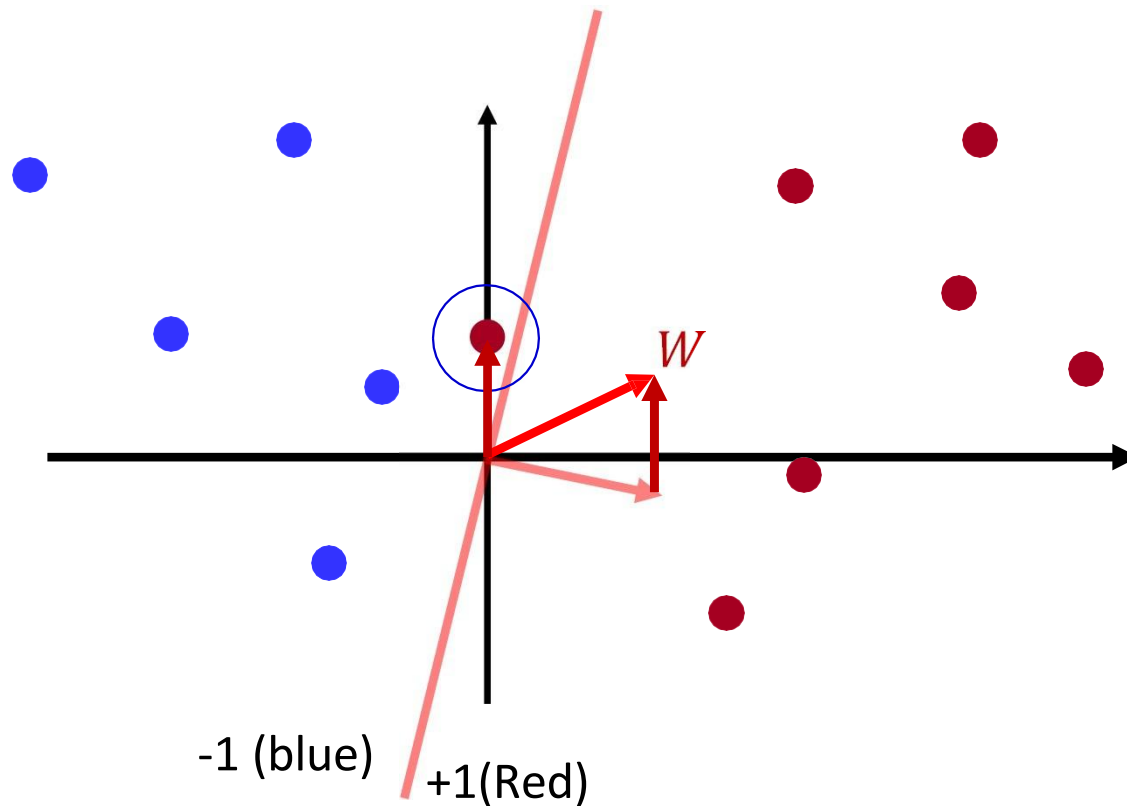
Misclassified *positive* instance

Perceptron Algorithm



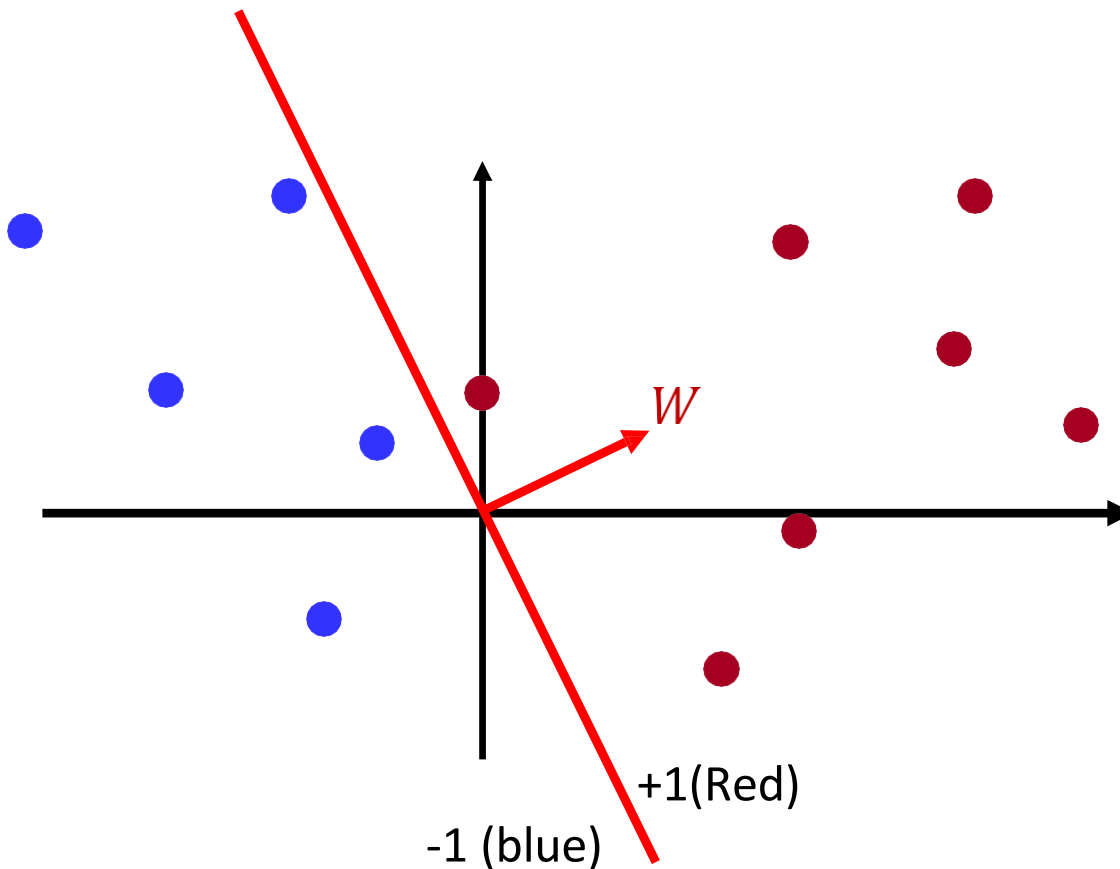
Misclassified *positive* instance, *add* it to W

Perceptron Algorithm



The new weight vector

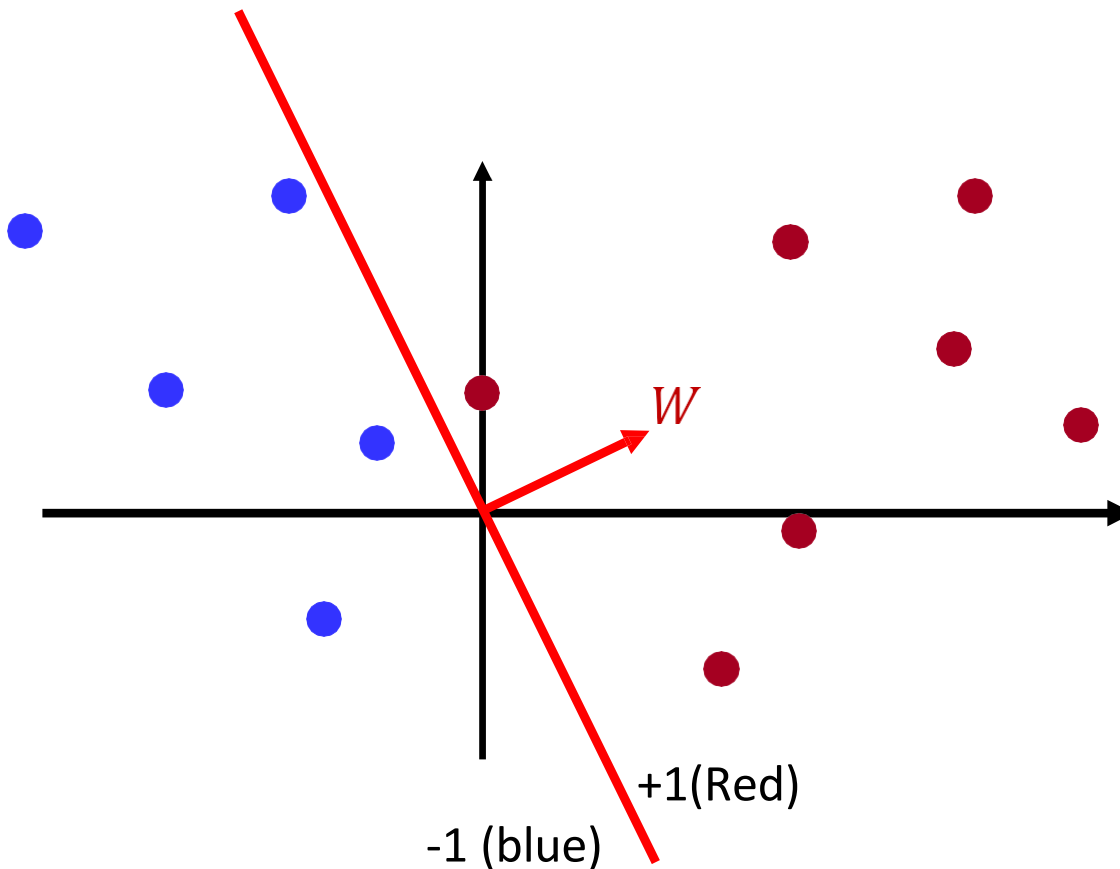
Perceptron Algorithm



The new decision boundary

Perfect classification, no more updates, we are done

Perceptron Algorithm



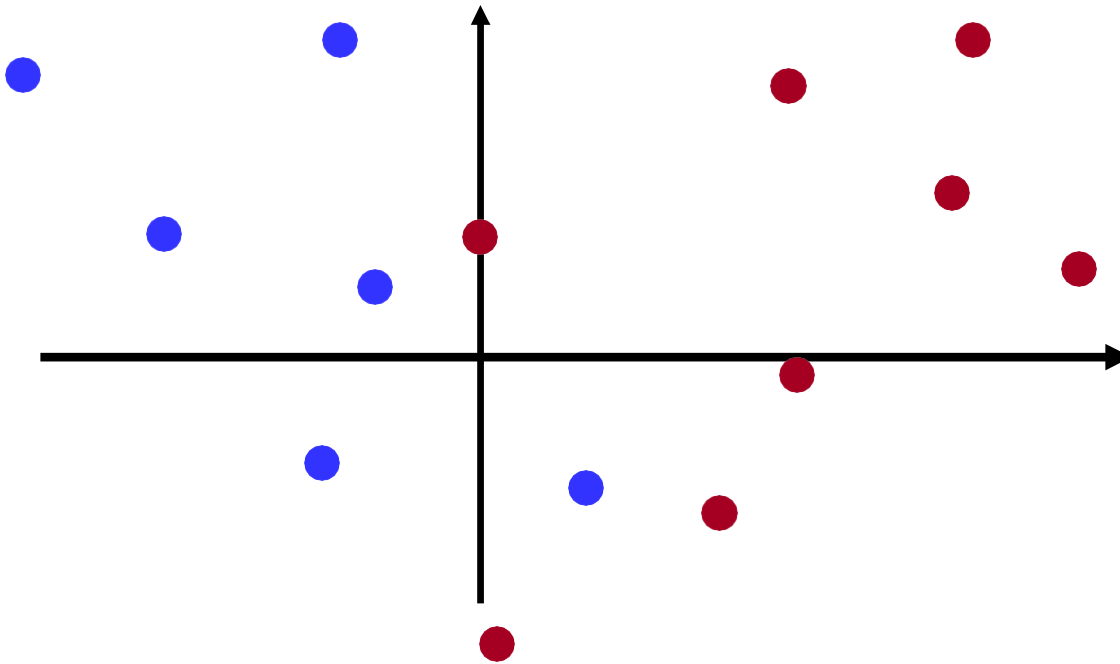
The new decision boundary

Perfect classification, no more updates, we are done

If the classes are linearly separable, guaranteed to converge in a finite number of steps

The Perceptron Solution: when classes are not linearly separable

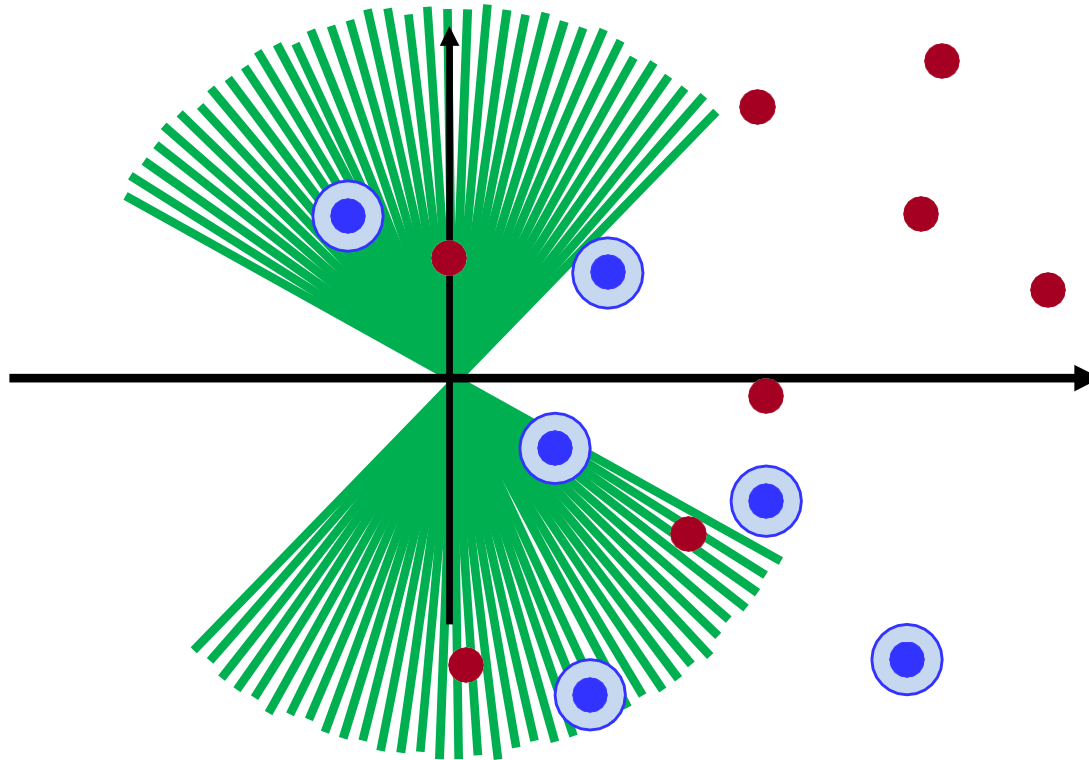
Key: Red 1, Blue = -1



- When classes are not linearly separable, not possible to find a separating hyperplane
 - No “support” plane for reflected data
 - Some points will always lie on the other side
- Model does not support perfect classification of this data
- **Perceptron algorithm will never converge**

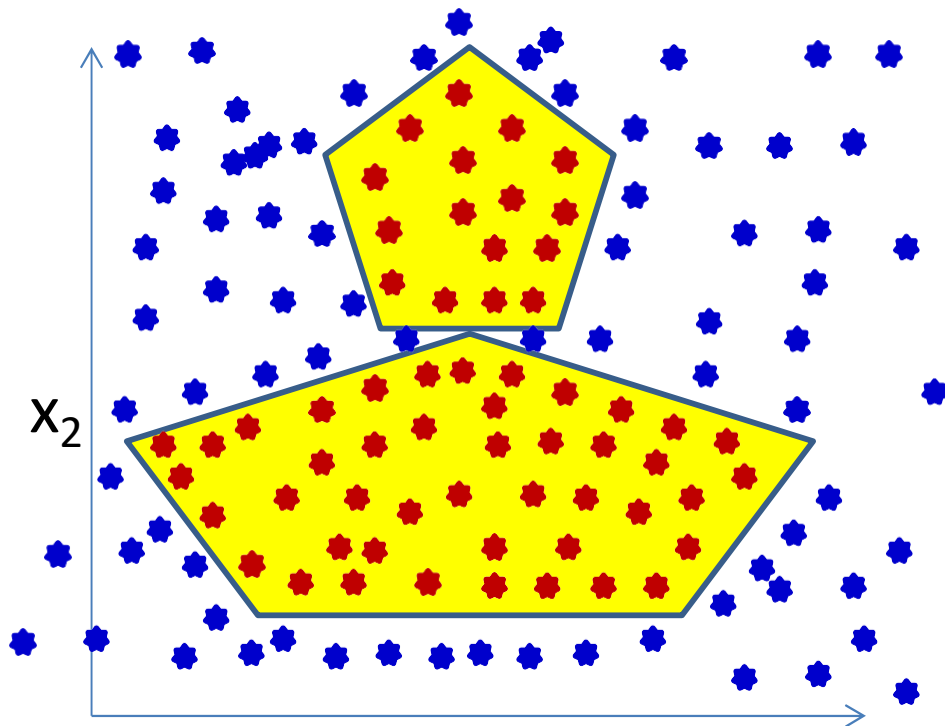
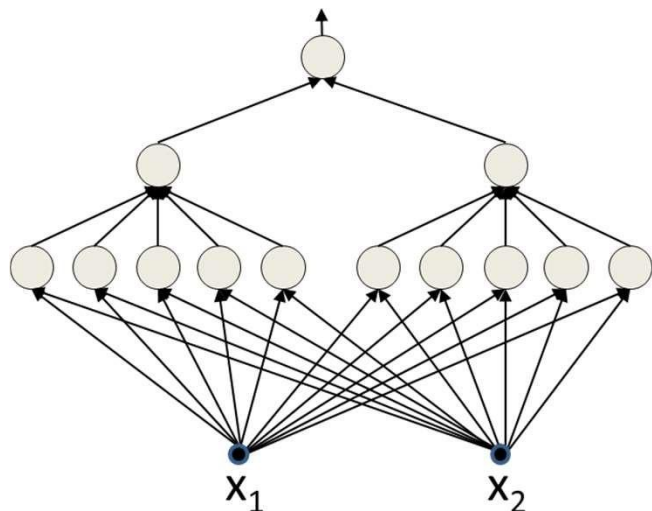
The Perceptron Solution: when classes are not linearly separable

Key: Red 1, Blue = -1



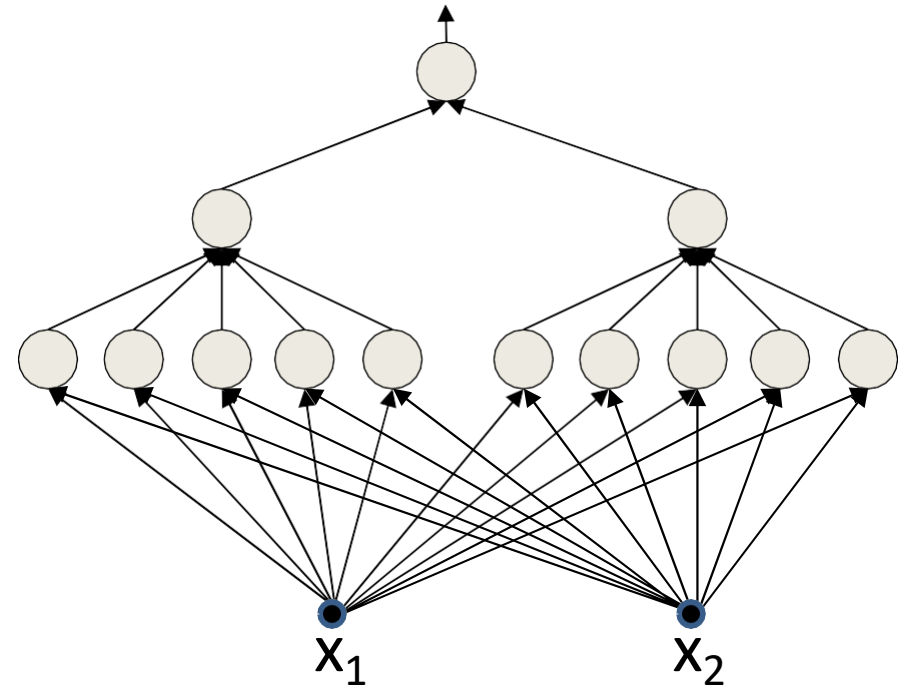
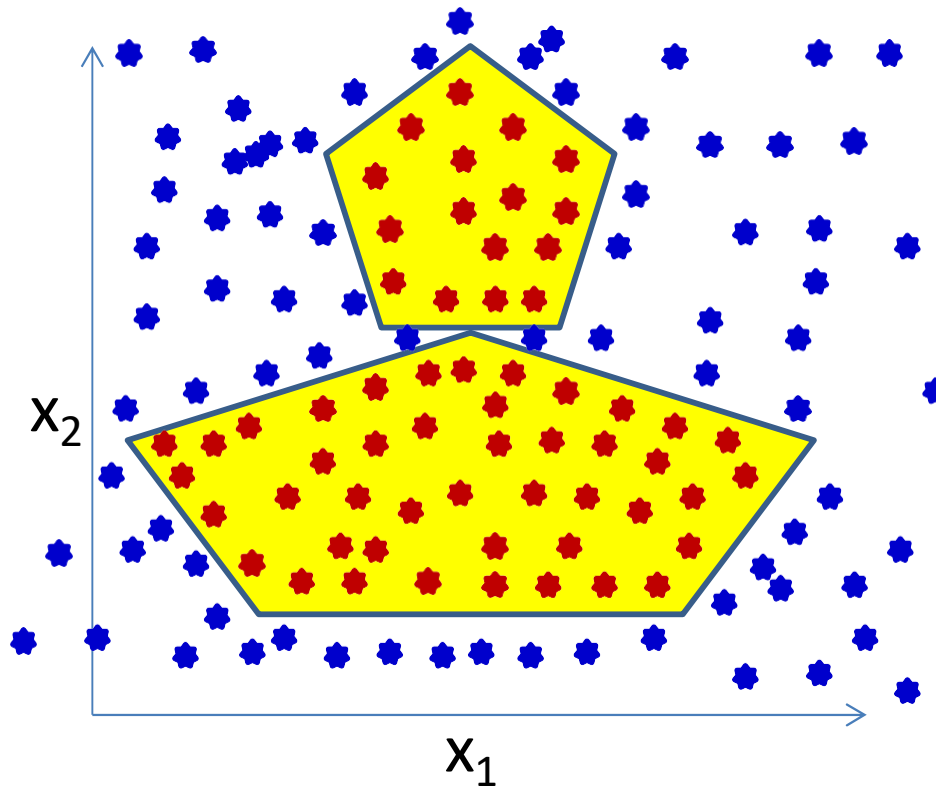
- When classes are not linearly separable, not possible to find a separating hyperplane
 - No “support” plane for reflected data
 - Some points will always lie on the other side
- Model does not support perfect classification of this data

History: A more complex problem



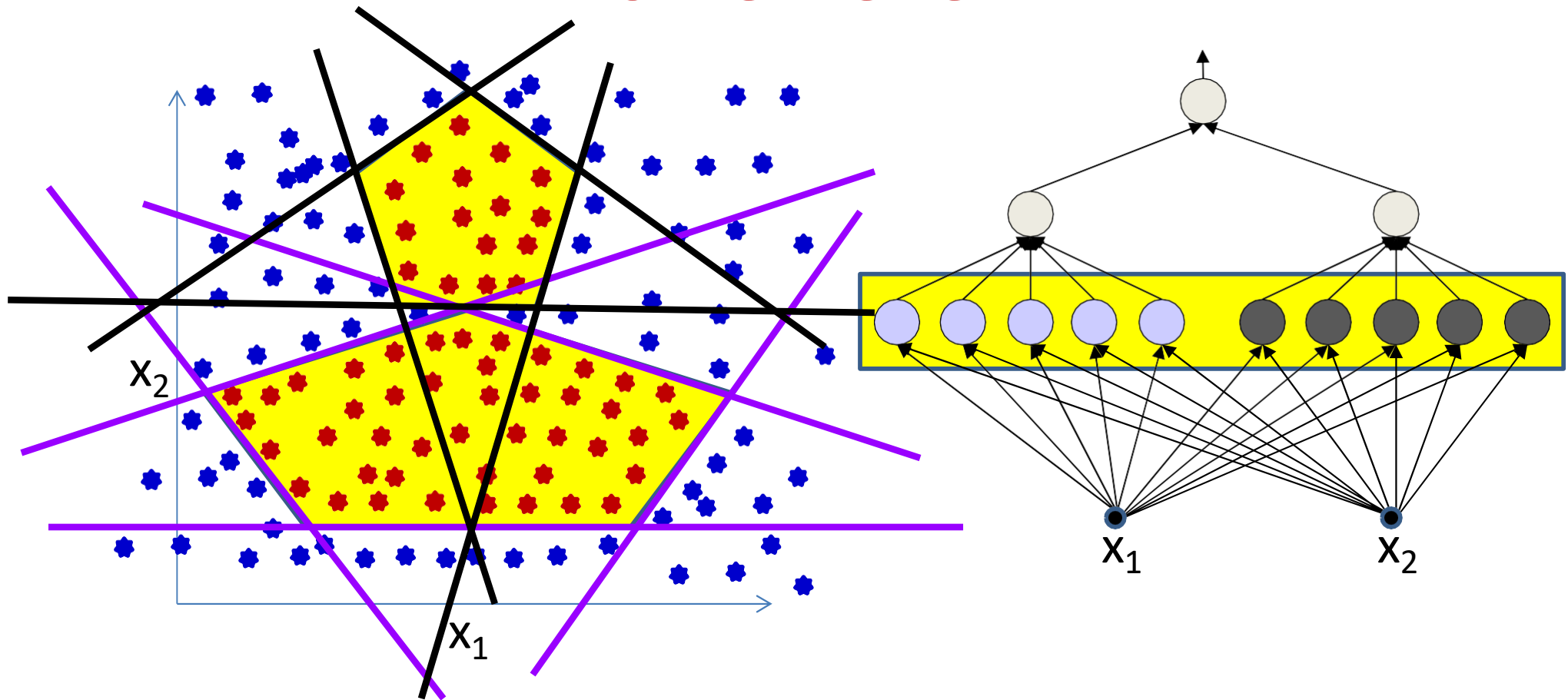
- Learn an *MLP* for this function
 - 1 in the yellow regions, 0 outside
- Using just the samples
- We know this can be perfectly represented using an MLP

More complex decision boundaries



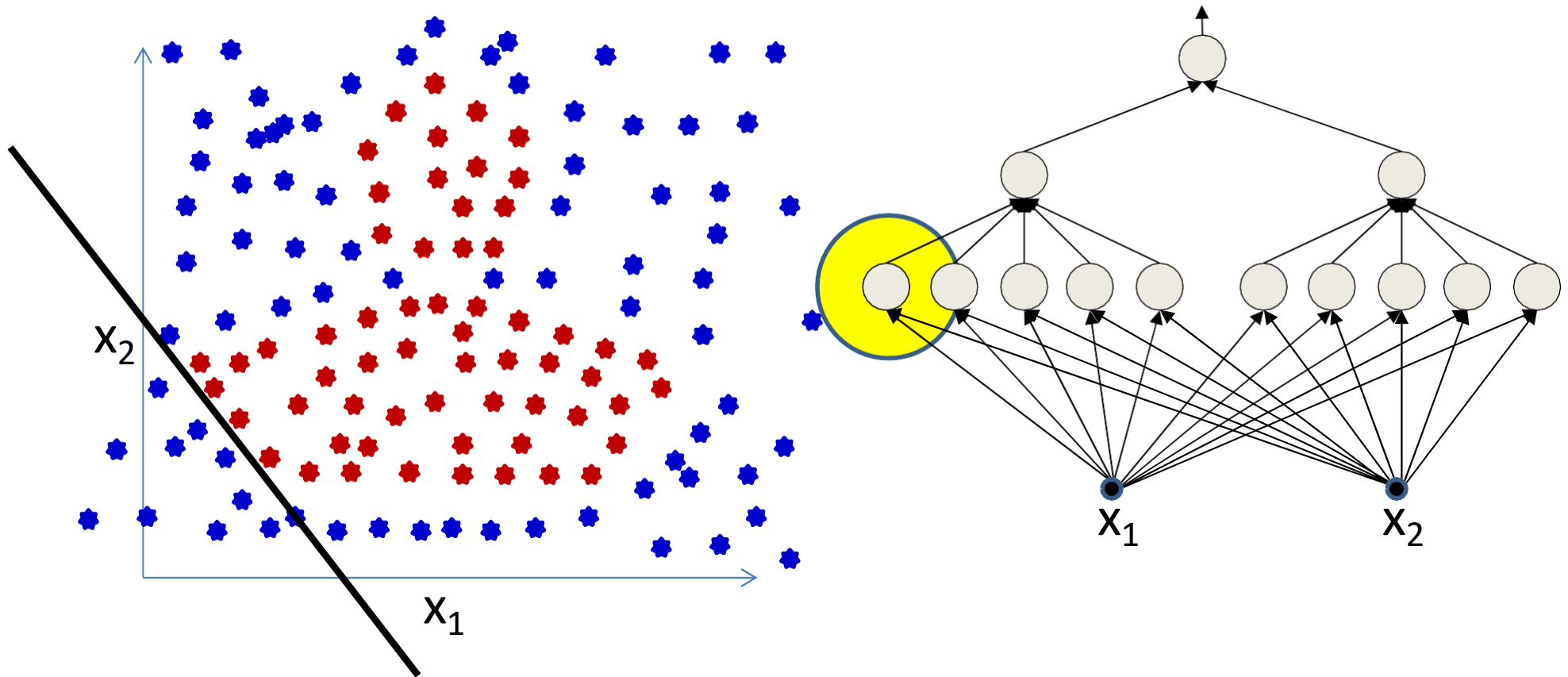
- Even using the perfect architecture...
- ... can we use perceptron learning rules to learn this classification function?

The pattern to be learned at the lower level



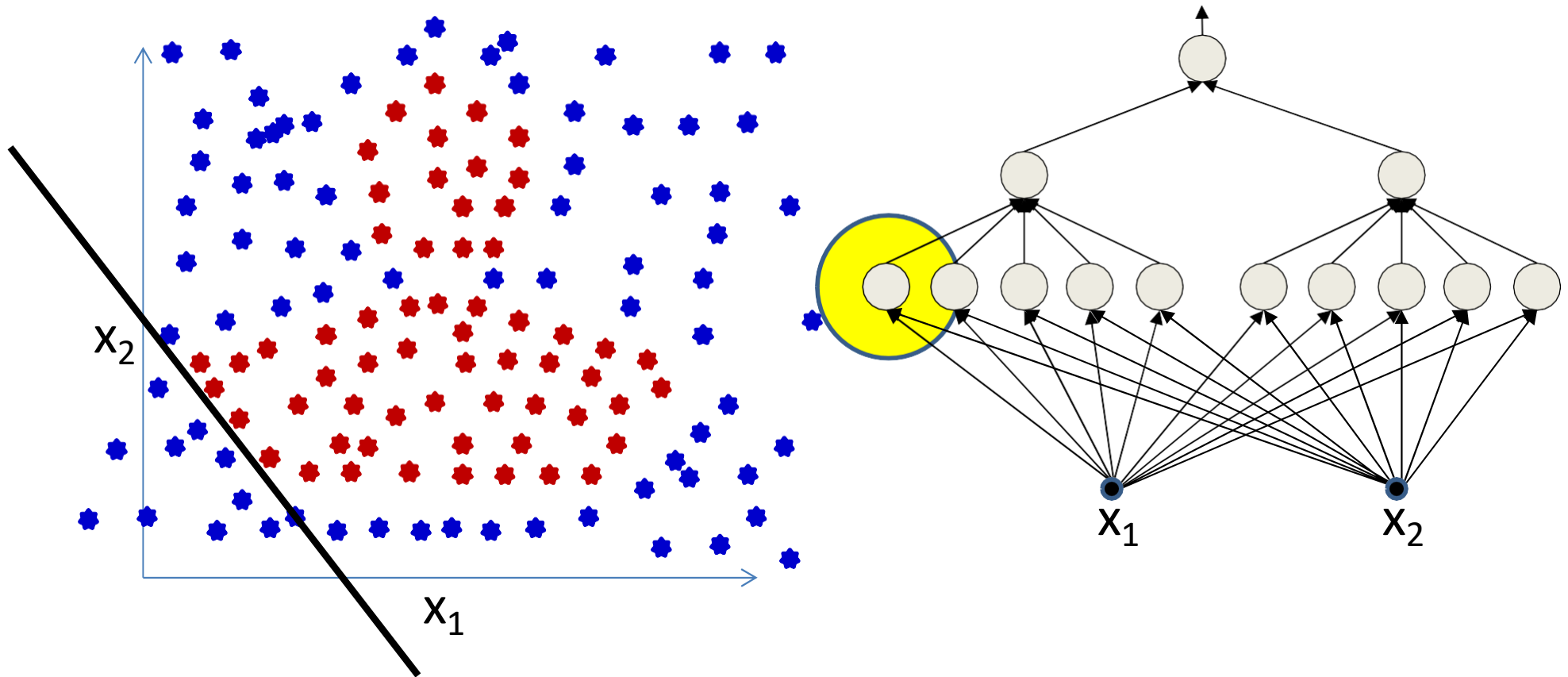
- The lower-level neurons are linear classifiers
 - They require linearly separated labels to be learned
 - The actually provided labels are not linearly separated

The pattern to be learned at the lower level



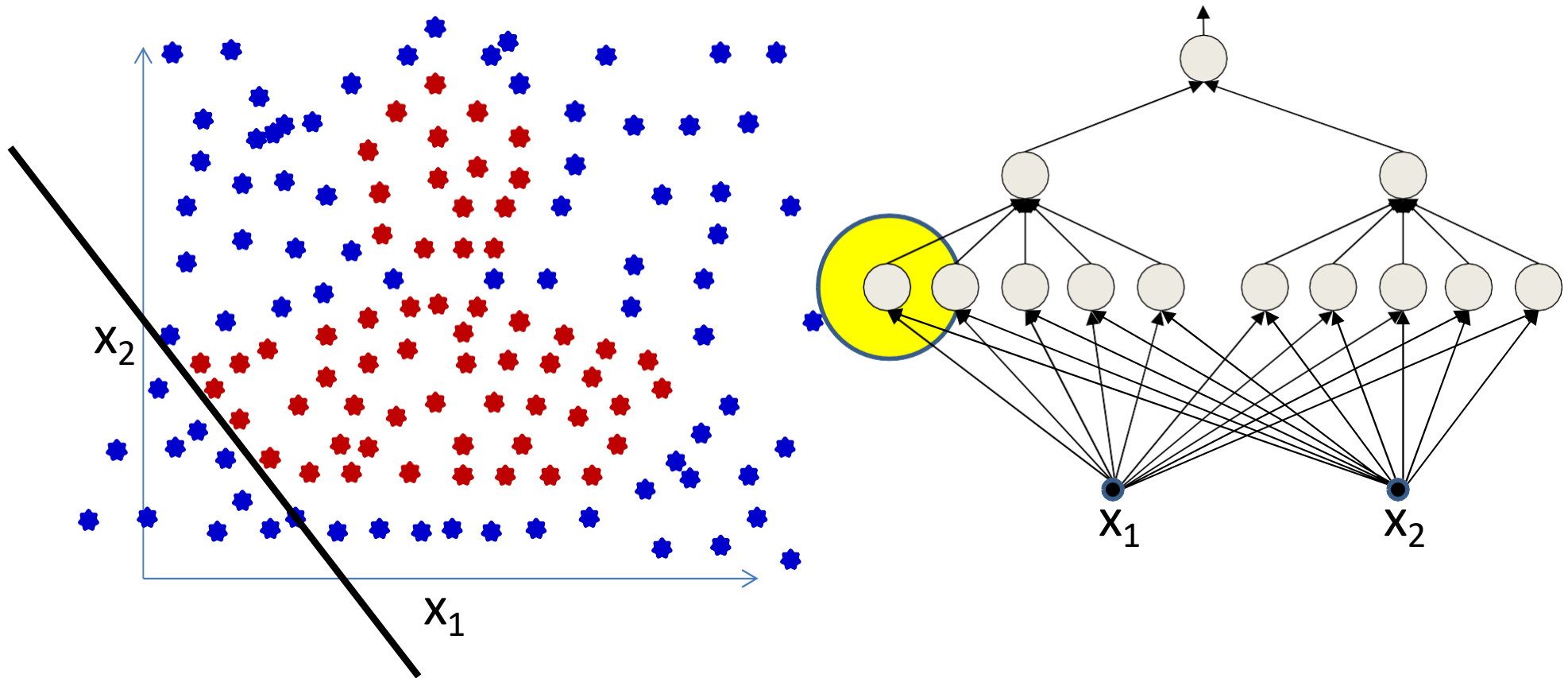
- Consider a single linear classifier that must be learned from the training data
 - Can it be learned from this data?

The pattern to be learned at the lower level



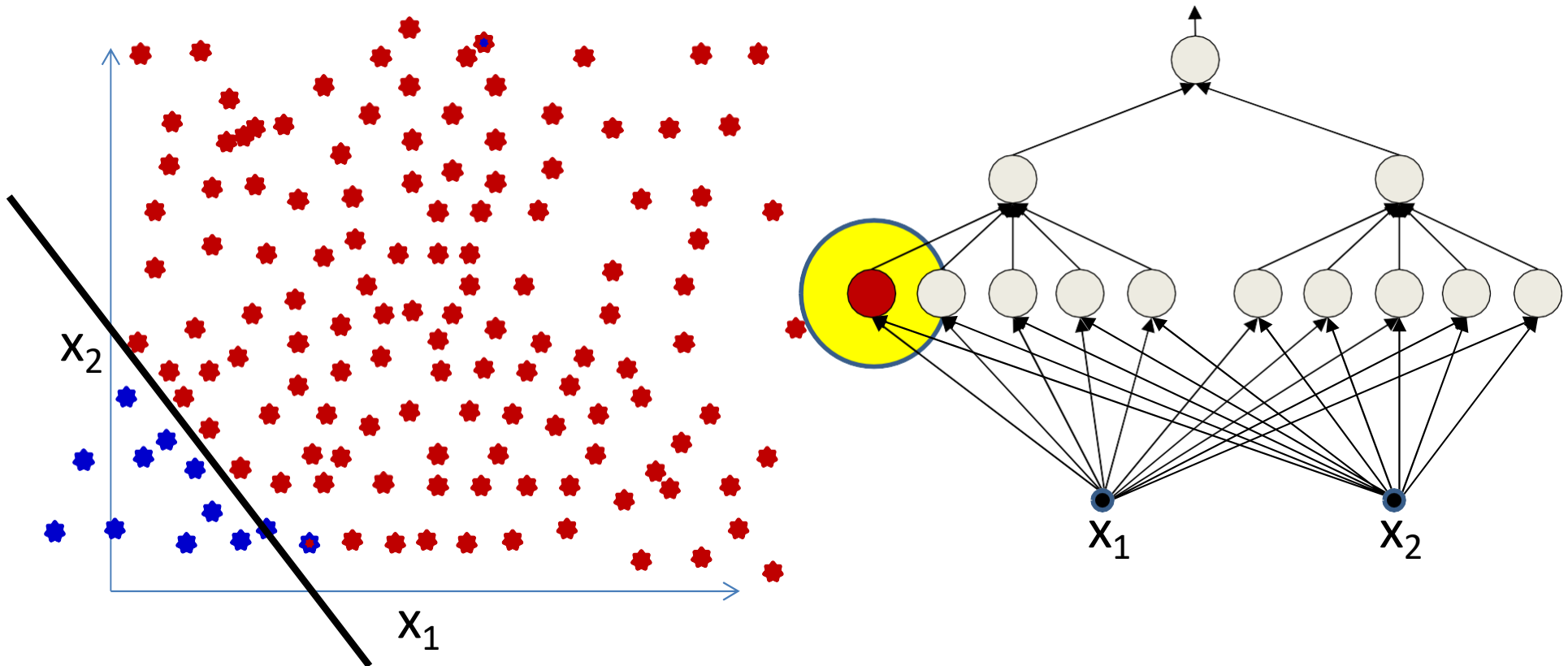
- Consider a single linear classifier that must be learned from the training data
 - Can it be learned from this data?
 - Will require relabelling some instances to learn this line
 - Which ones?

The pattern to be learned at the lower level



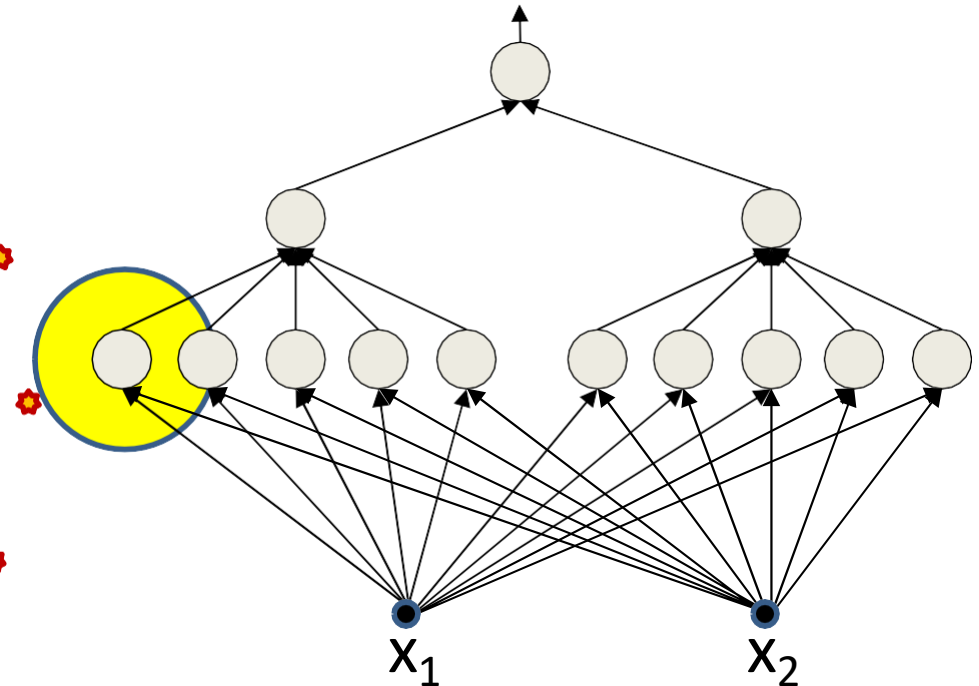
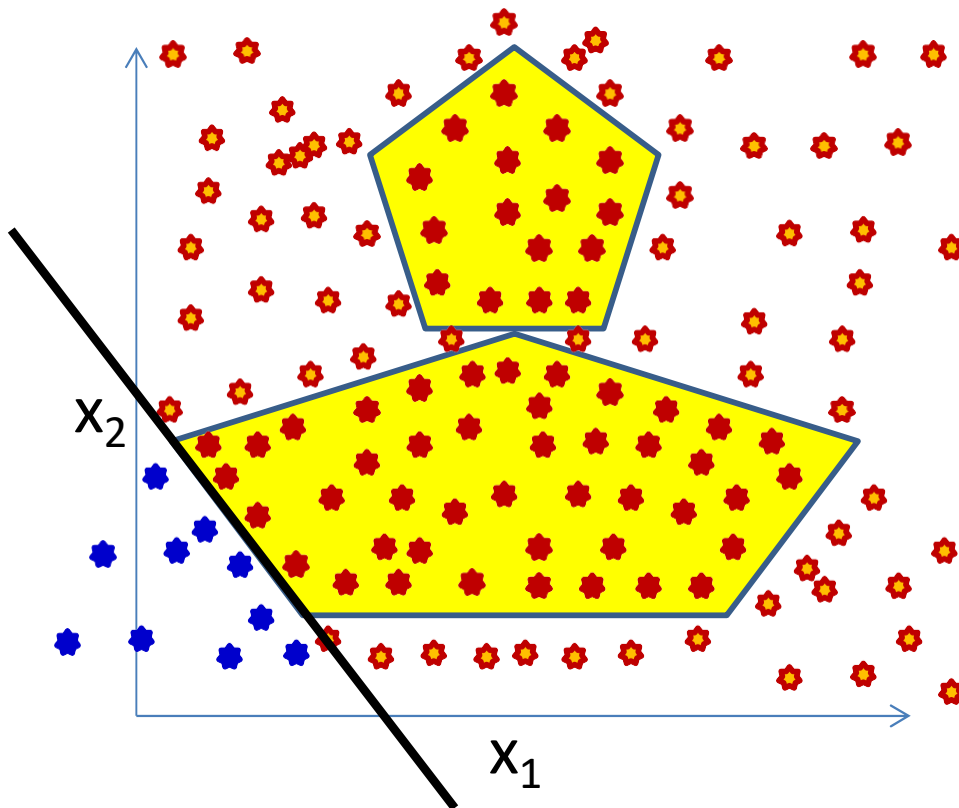
- Consider a single linear classifier that must be learned from the training data
 - Can it be learned from this data?

The pattern to be learned at the lower level



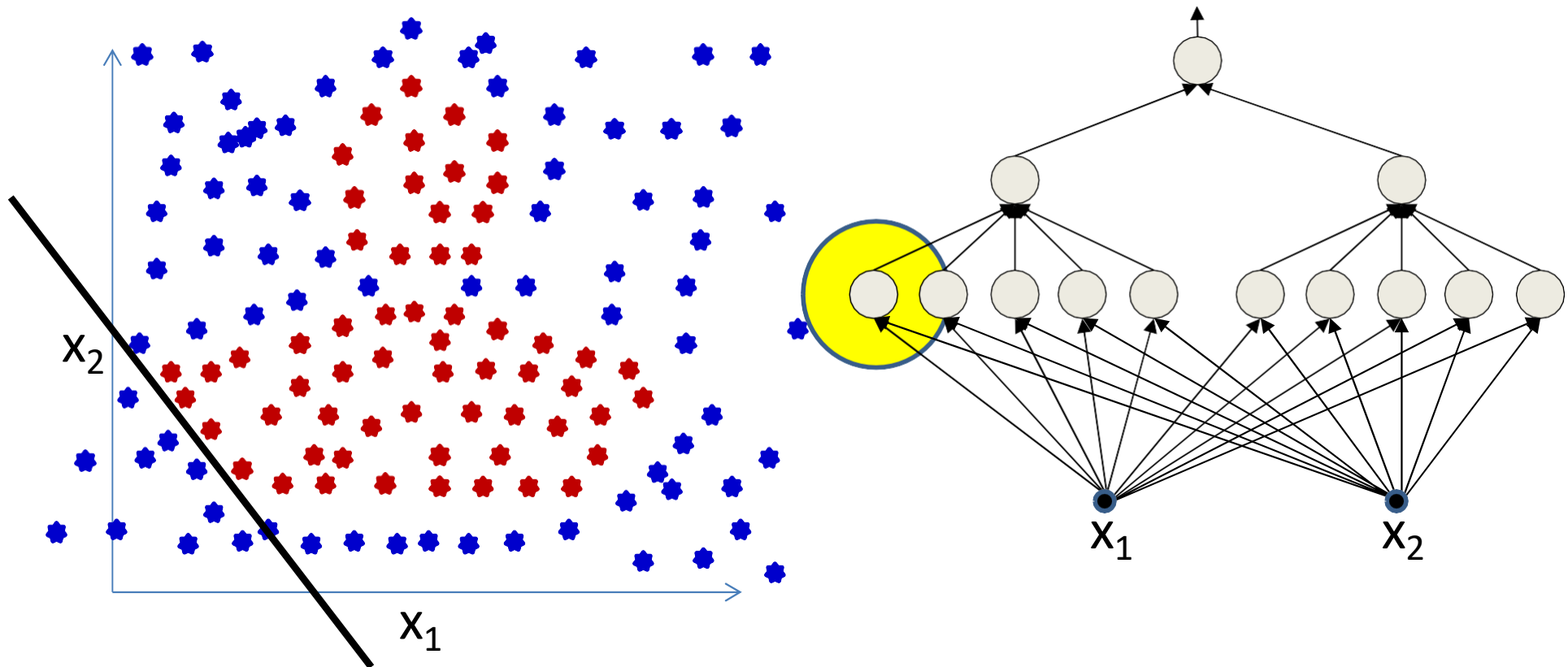
- Consider a single linear classifier that must be learned from the training data
 - Can it be learned from this data?
 - **The individual classifier actually requires the kind of labelling shown here**
 - Which is *not* given!!

The pattern to be learned at the lower level



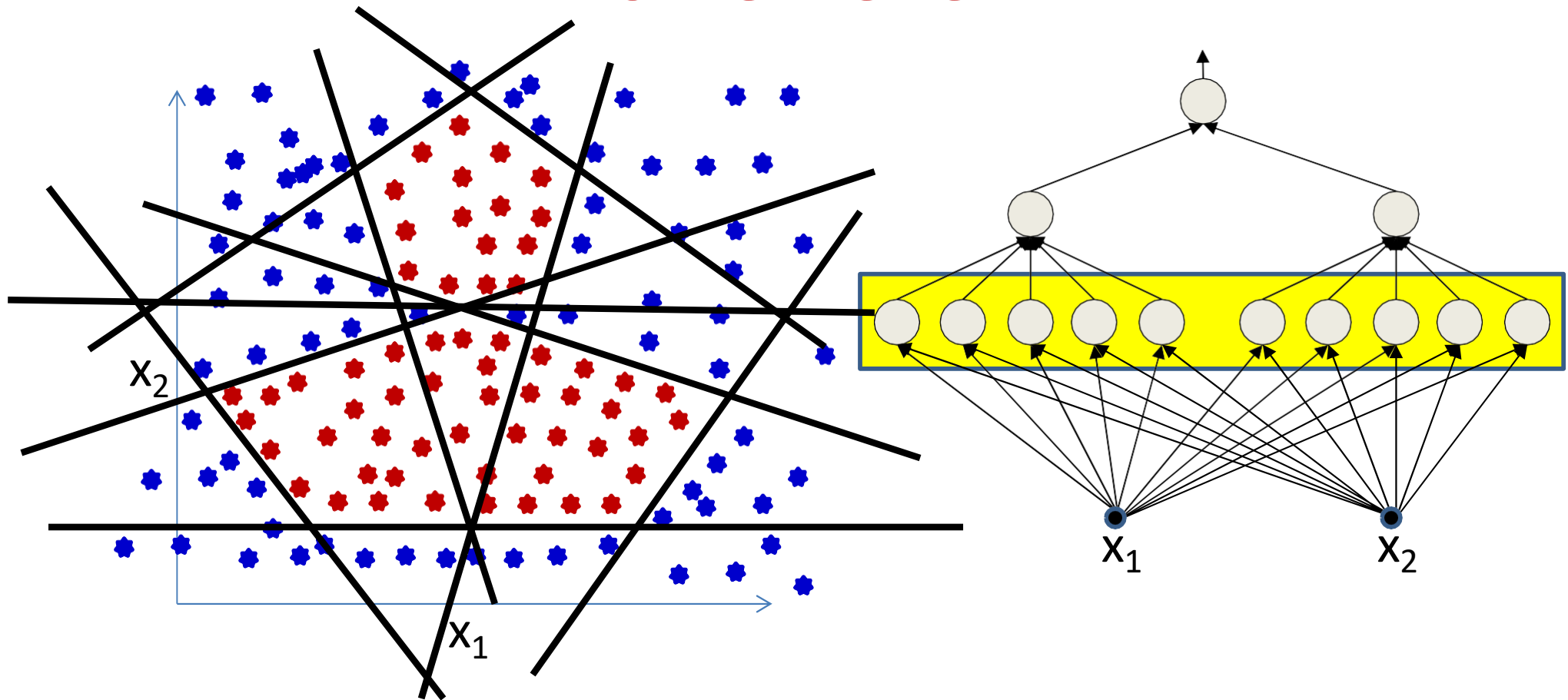
- The lower-level neurons are linear classifiers
 - They require linearly separated labels to be learned
 - The actually-provided labels are not linearly separated
 - *Challenge: Must also learn the labels for the lowest units!*

The pattern to be learned at the lower level



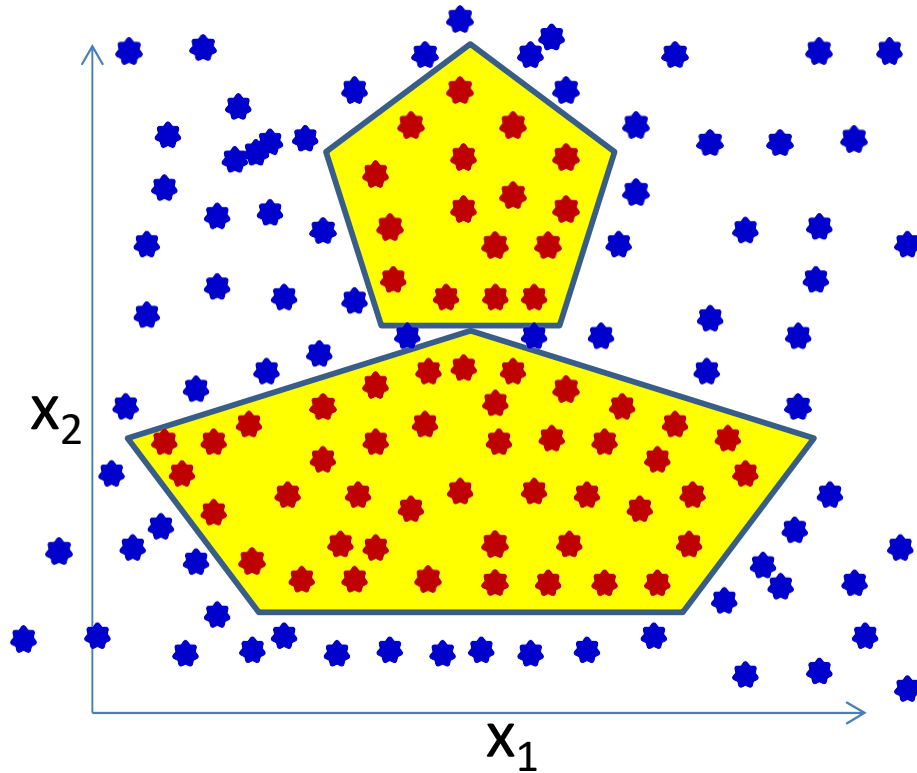
- For a single line:
 - Try out *every possible way of relabeling the blue dots such that we can learn a line that keeps all the red dots on one side!*

The pattern to be learned at the lower level



- This must be done for *each* of the lines (perceptrons)
- Such that, when all of them are combined by the higher-level perceptrons we get the desired pattern
 - Basically an exponential search over inputs

Individual neurons represent one of the lines that compose the figure (linear classifiers)



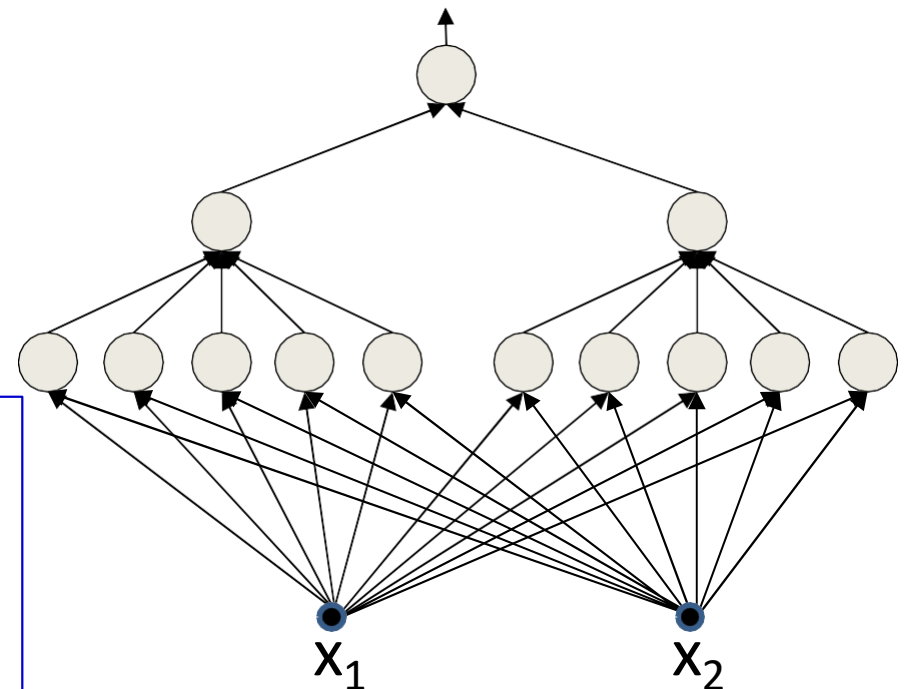
Must know the desired output of every neuron for *every* training instance, in order to learn this neuron

The outputs should be such that the neuron individually has a linearly separable task

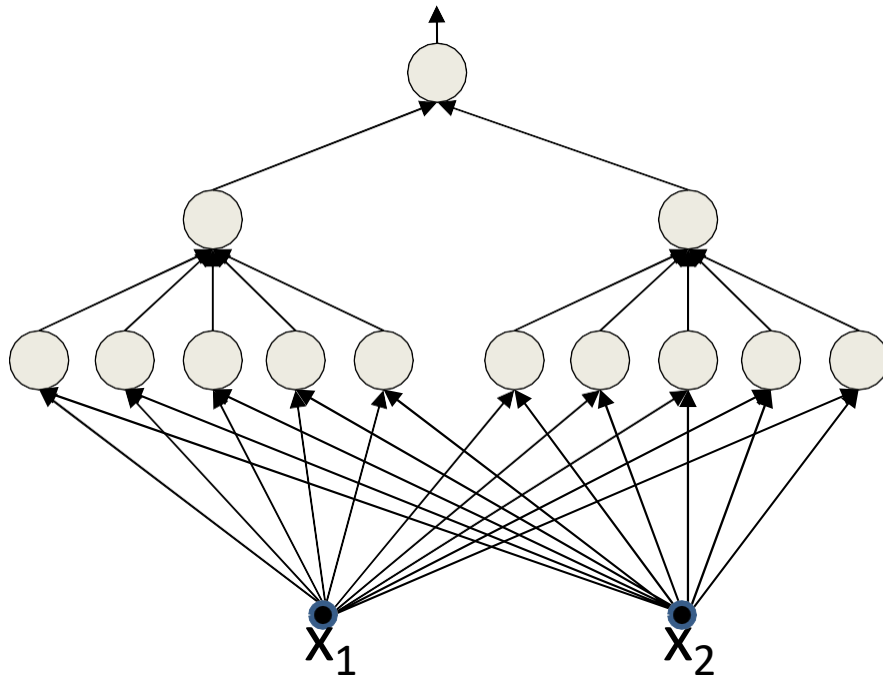
The linear separators must combine to form the desired boundary

This must be done for *every* neuron

Getting any of them wrong will result in incorrect output!



Learning a *multilayer* perceptron



Training data only specifies
input and output of network

Intermediate outputs (outputs
of individual neurons) are not specified

- Training this network using the perceptron rule is a combinatorial optimization problem
- We don't know the outputs of the individual intermediate neurons in the network for any training input
- **Must also determine the correct output for *each* neuron for *every* training instance**
- **At least exponential (in inputs) time complexity!!!!!!**

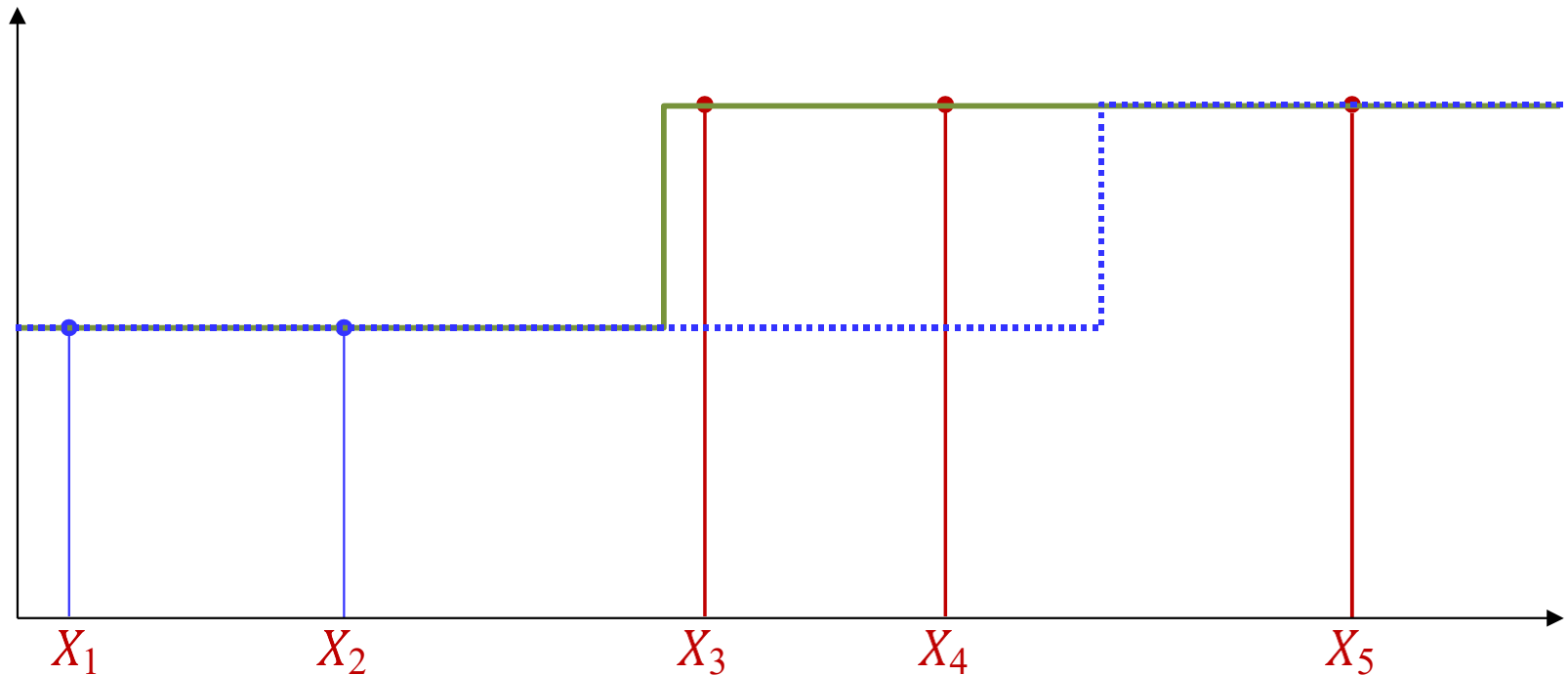
Greedy algorithms: Adaline and Madaline

- Perceptron learning rules cannot directly be used to learn an MLP
 - Exponential complexity of assigning intermediate labels
 - Even worse when classes are not actually separable
- Can we use a *greedy* algorithm instead?
 - Adaline / Madaline

History..

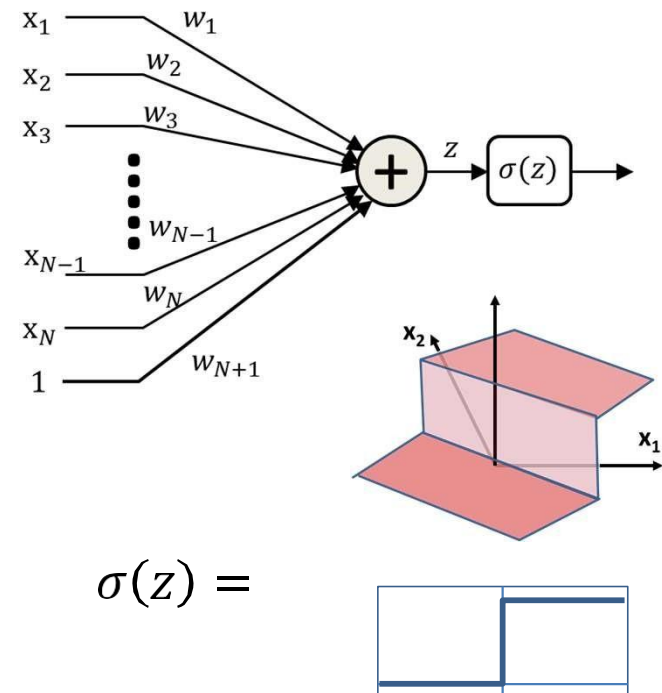
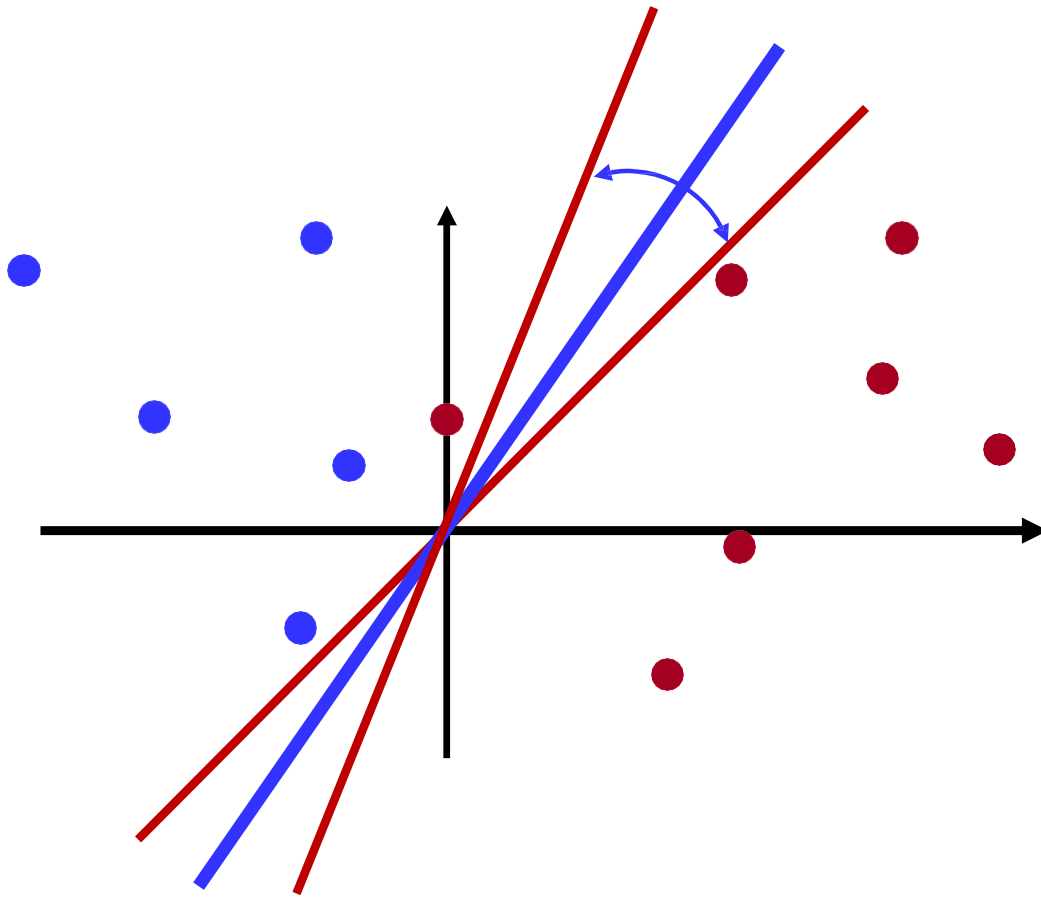
- The realization that training an entire MLP was a combinatorial optimization problem stalled development of neural networks for well over a decade!

The problem



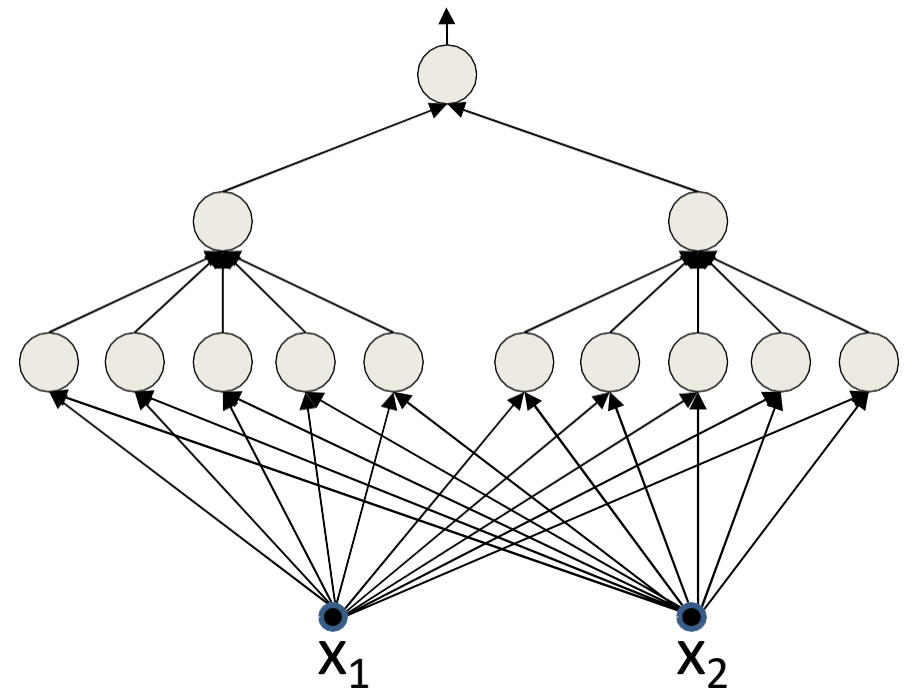
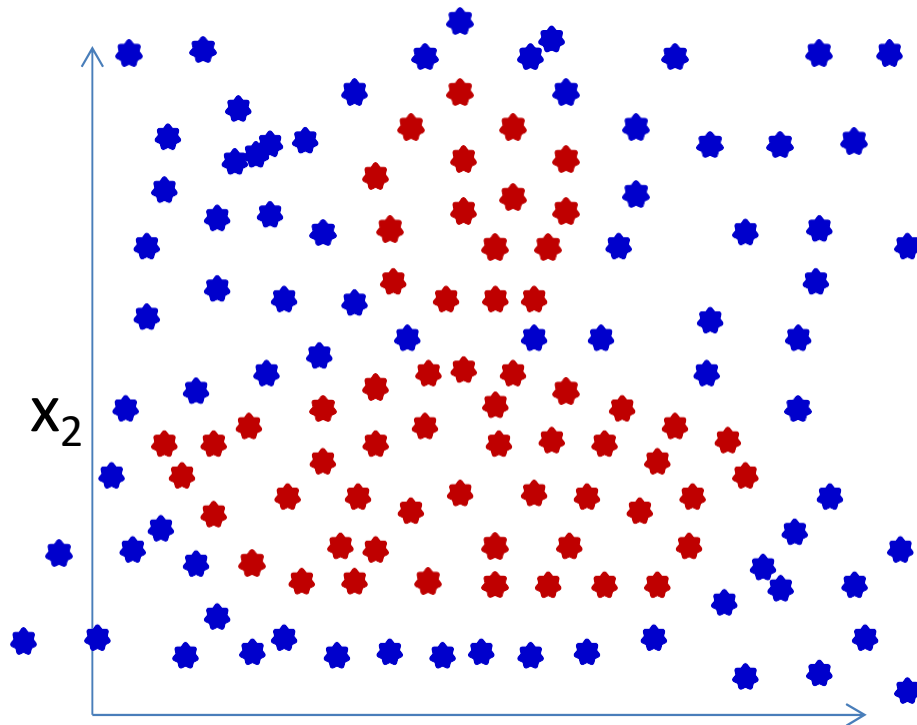
- Our binary error metric is not useful
 - To improve the classifier, we must move the blue dotted line left
 - But if we move it only slightly, moving it either right or left results in no change in error

Why this problem?



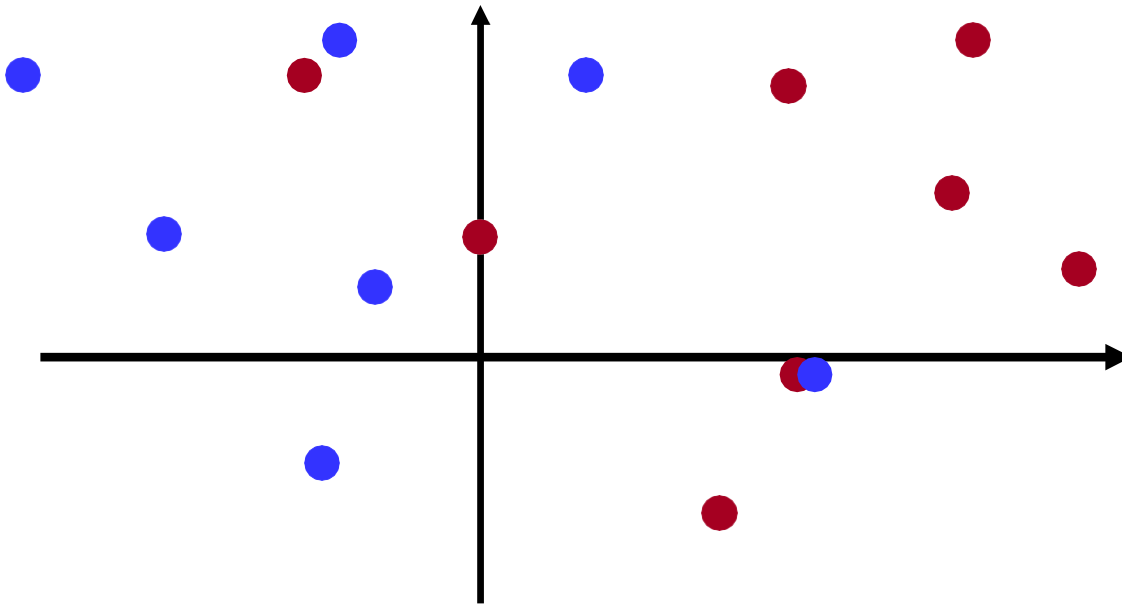
- The perceptron is a flat function with zero derivative everywhere, except at 0 where it is non-differentiable
 - You can vary the weights a *lot* without changing the error
 - There is no indication of which direction to change the weights to reduce error

This only compounds on larger problems



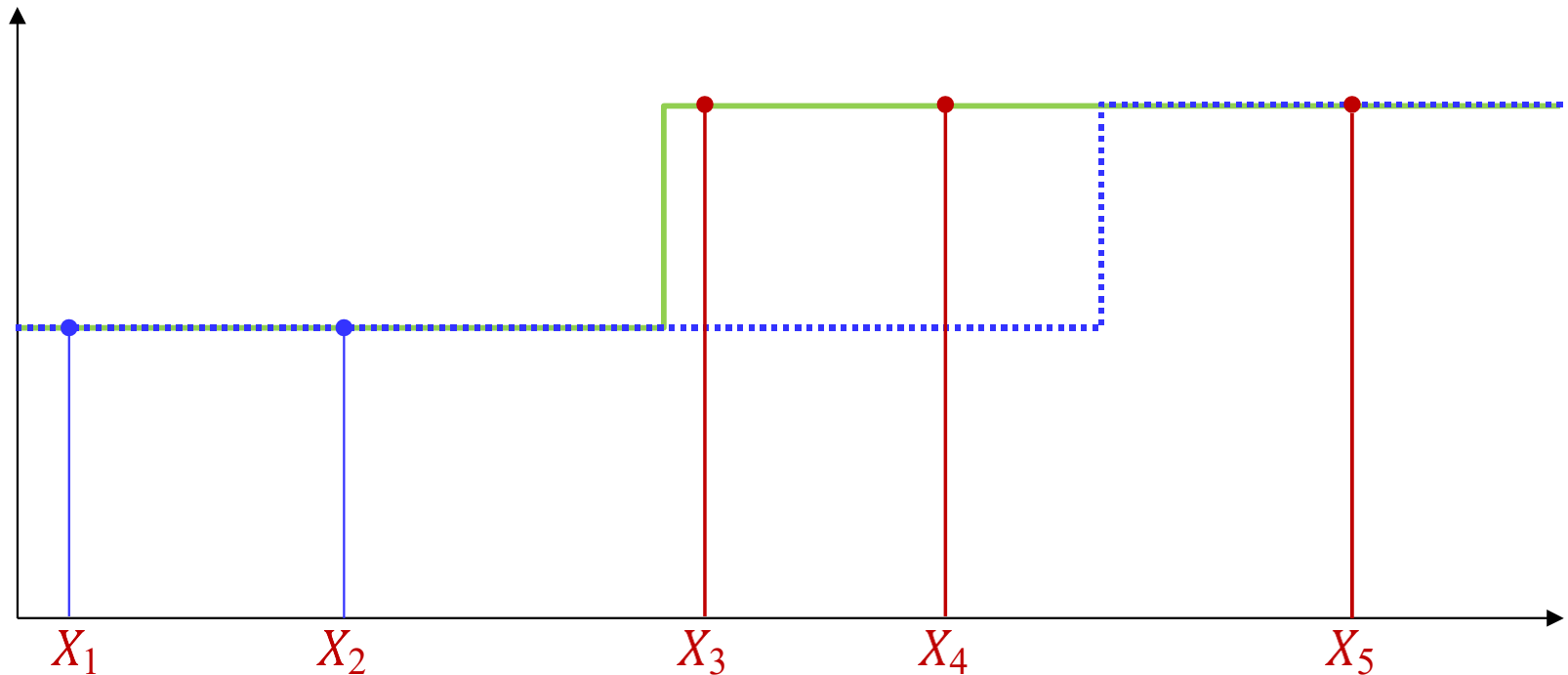
- Individual neurons' weights can change significantly without changing overall error
- The simple MLP is a flat, non-differentiable function
 - Actually, a function with 0 derivative nearly everywhere, and no derivatives at the boundaries

A second problem: What we *actually* model



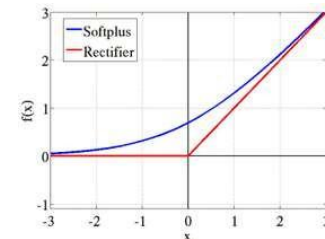
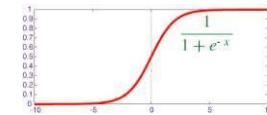
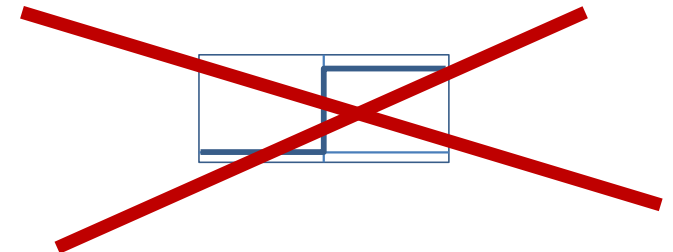
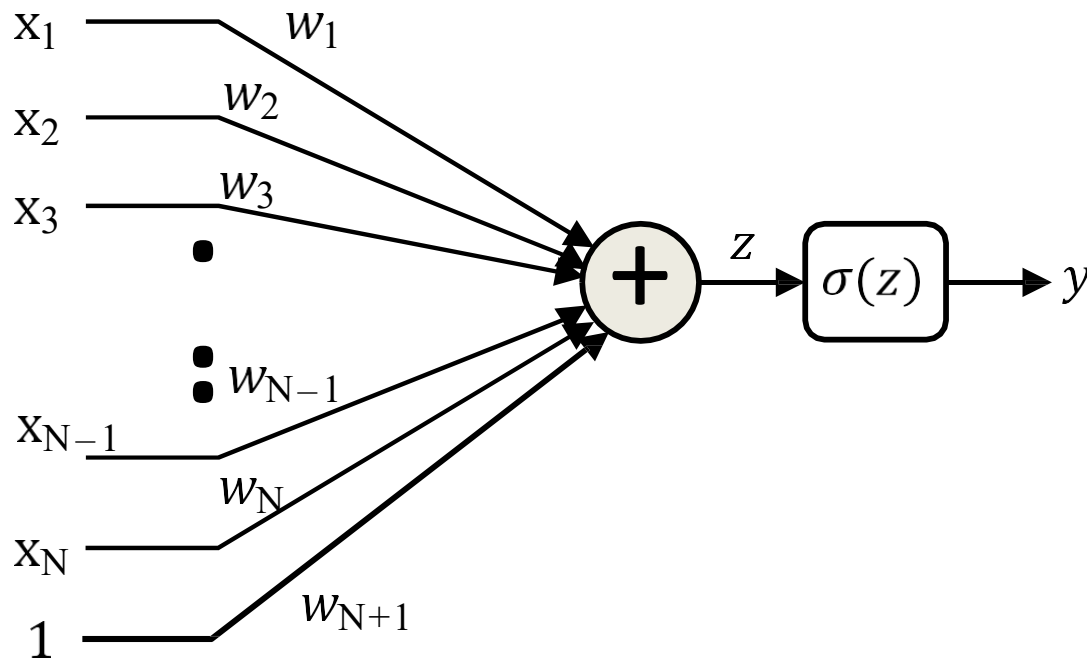
- Real-life data are rarely clean
 - Not linearly separable
 - Rosenblatt's perceptron learning rule wouldn't work in the first place

The solution



- Change our way of computing the mismatch such that modifying the classifier slightly lets us know if we are going the right way or not
 - This requires changing both, our activation functions, and the manner in which we evaluate the mismatch between the classifier output and the target output
 - Our mismatch function will now not actually count errors, but a *proxy* for it

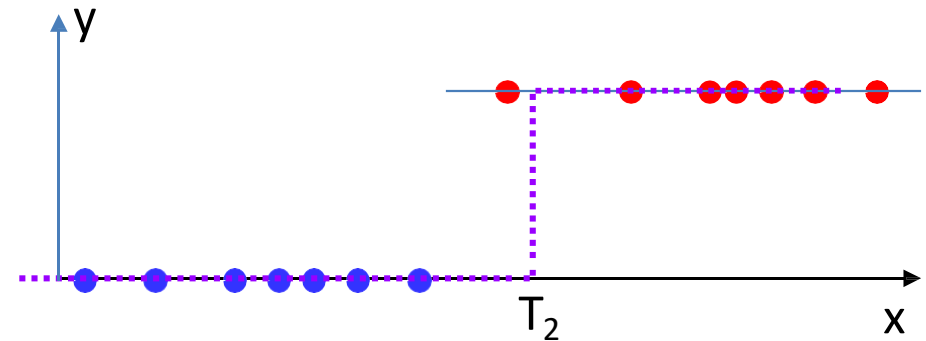
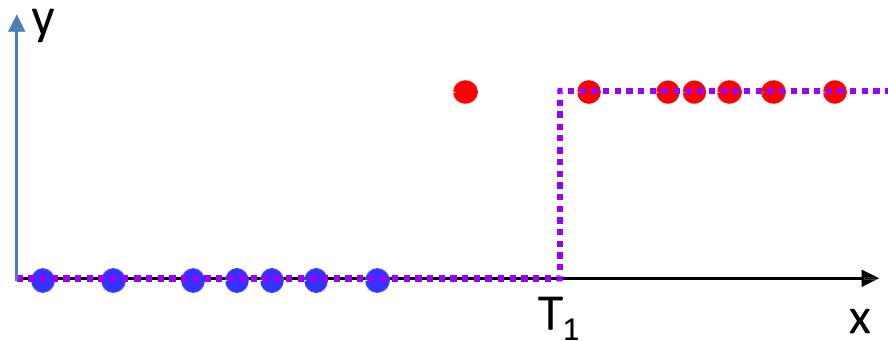
Solution: Differentiable activation



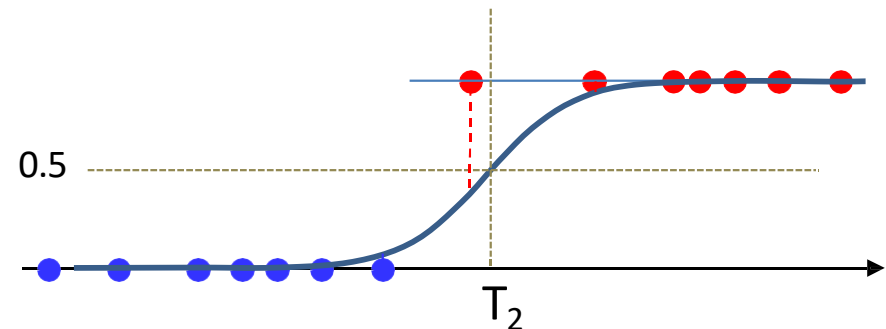
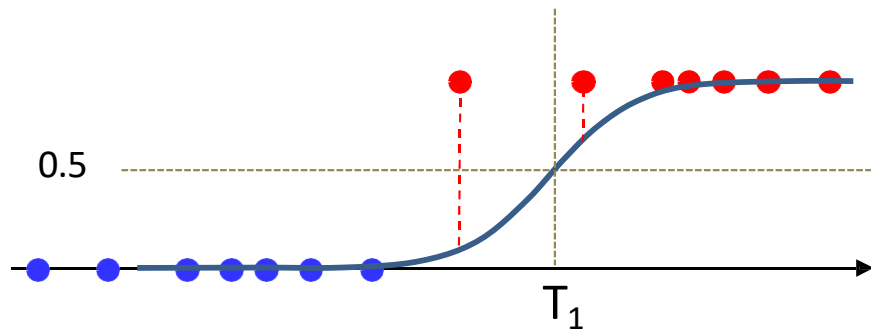
Activation functions $\sigma(z)$

- Let's make the neuron differentiable, *with non-zero derivatives over much of the input space*
 - Small changes in weight can result in non-negligible changes in output
 - This enables us to estimate the parameters using gradient descent techniques..

Differentiable Mismatch function

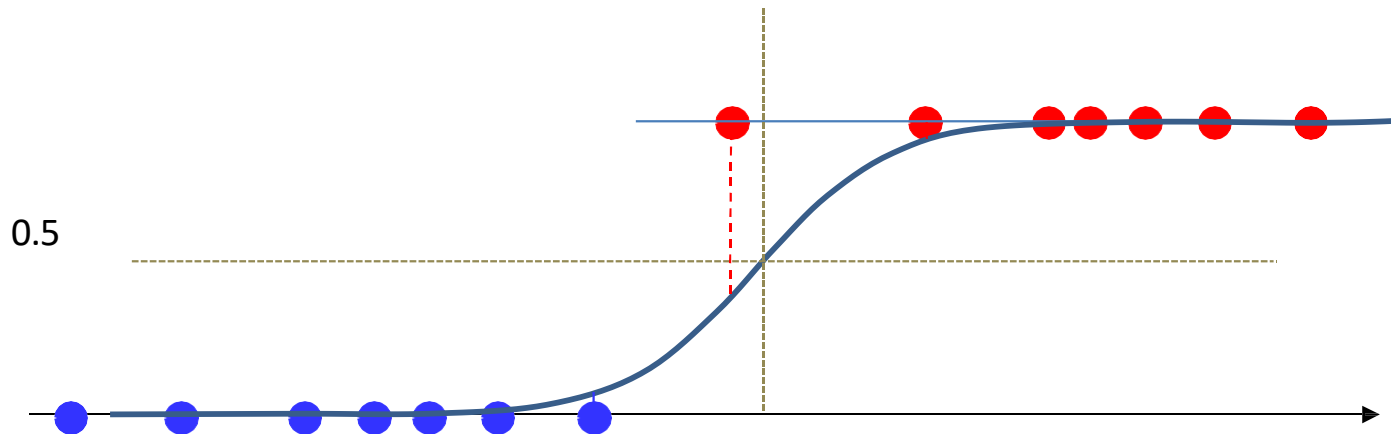


- Threshold activation: shifting the threshold from T_1 to T_2 does not change classification error
 - Does not indicate if moving the threshold left was good or not



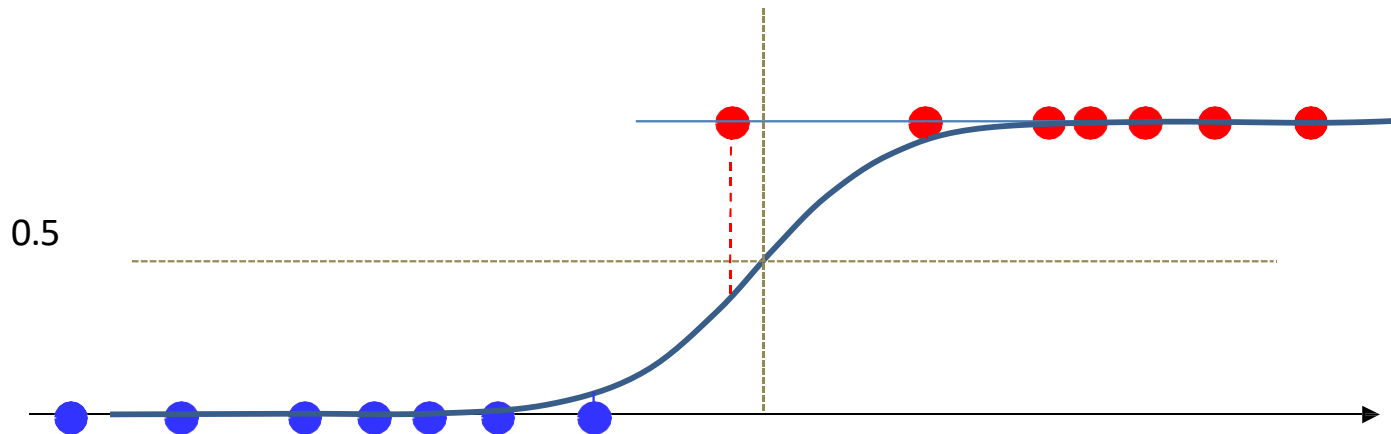
- Smooth, continuously varying activation: Classification based on whether the output is greater than 0.5 or less
 - Quantify *how much* the output differs from the desired target value (0 or 1)
 - Moving the function left or right changes this quantity, even if the classification error itself doesn't change

The two key requirements for learnability



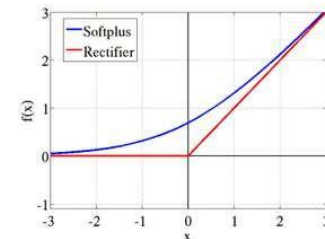
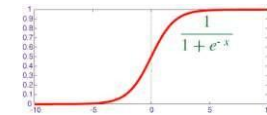
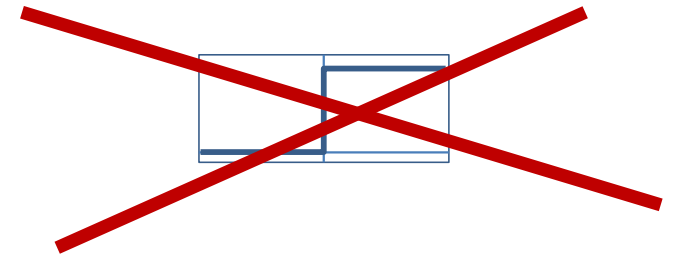
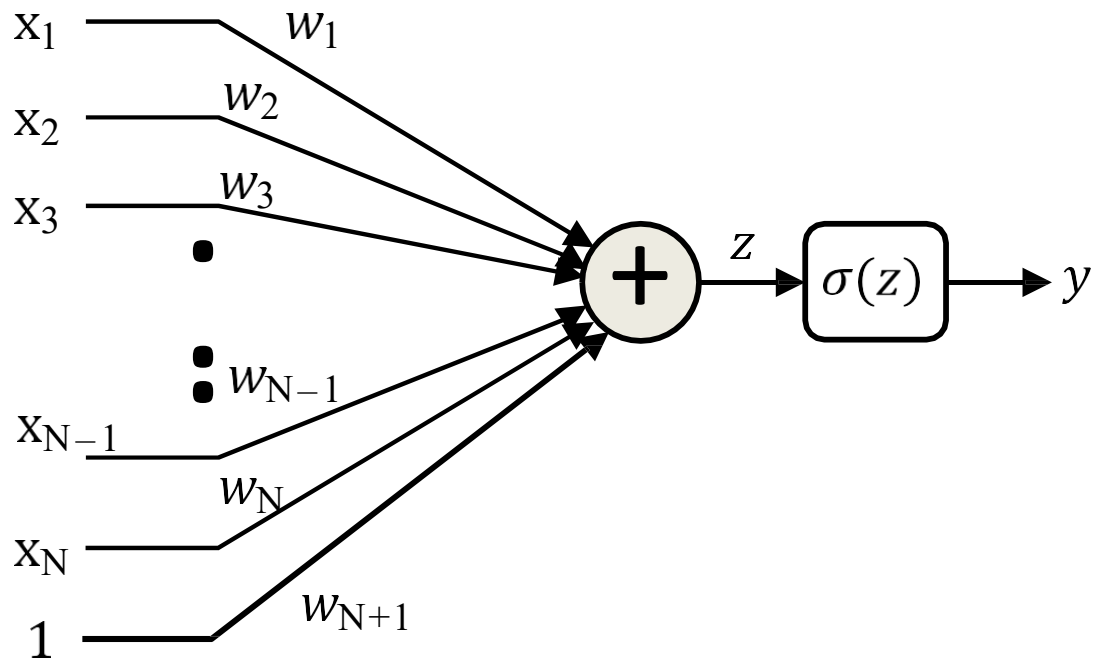
- Continuously varying activation
 - Differentiable
- Continuously varying error function
 - Also differentiable

The two key requirements for learnability



- Continuously varying activation
 - Differentiable
- Continuously varying error function
 - Also differentiable

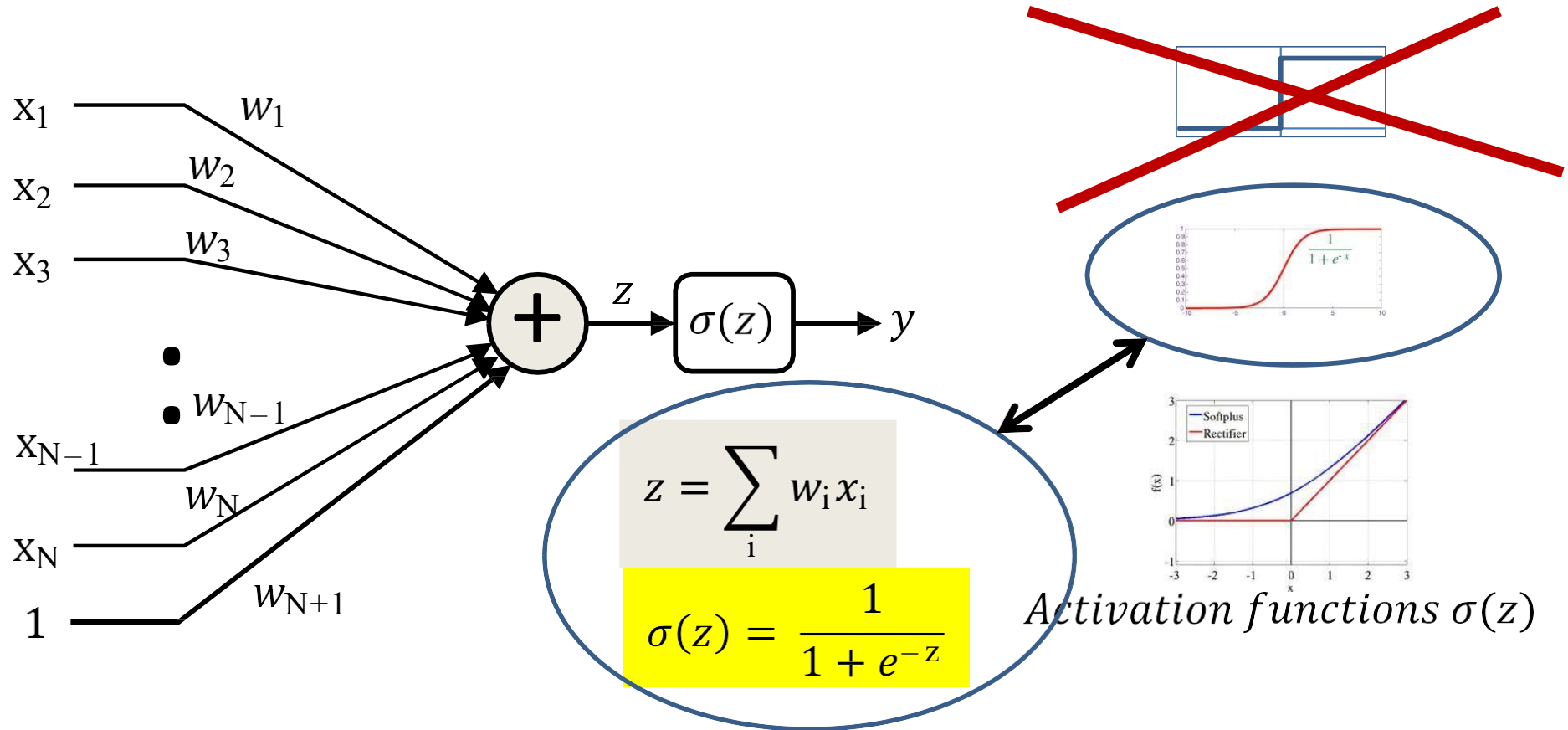
Continuous Activations



Activation functions $\sigma(z)$

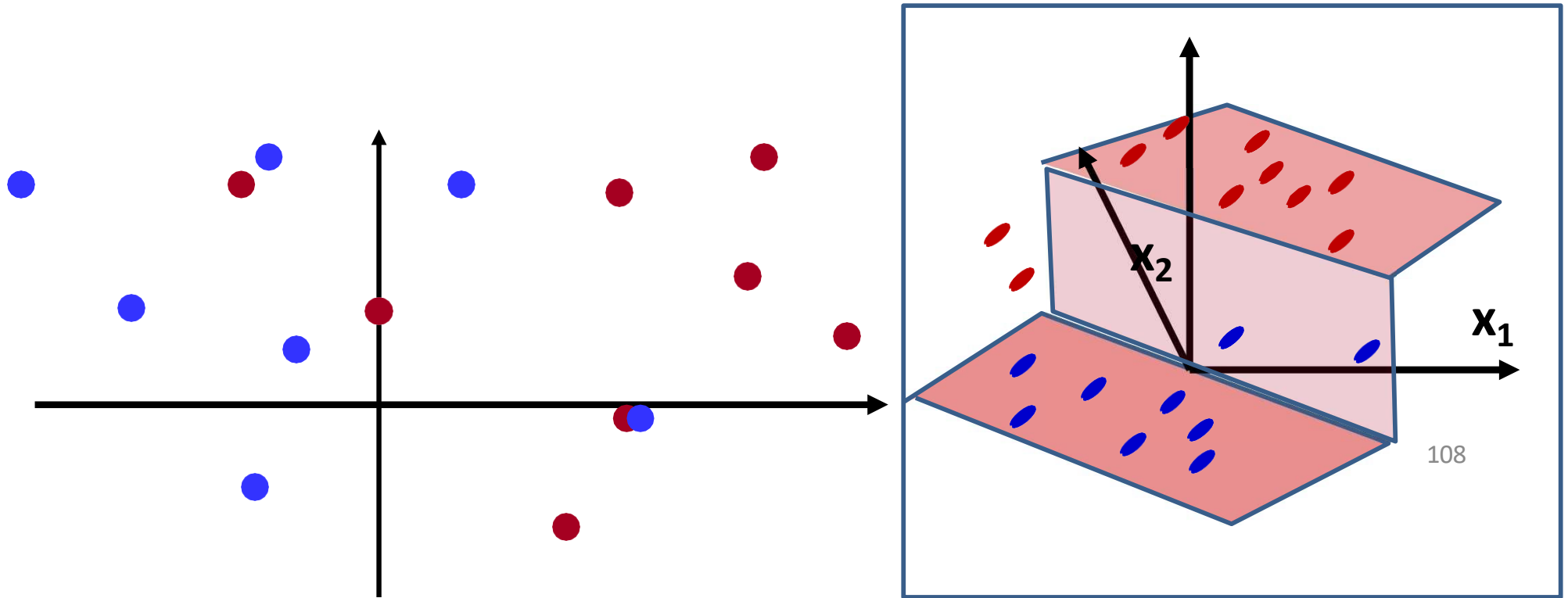
- Replace the threshold activation with continuous graded activations
 - E.g. RELU, softplus, sigmoid etc.
- The activations are *differentiable* almost everywhere
 - Have derivatives almost everywhere
 - And have “subderivatives” at non-differentiable corners
 - Bounds on the derivative that can substitute for derivatives in our setting

The sigmoid activation is special



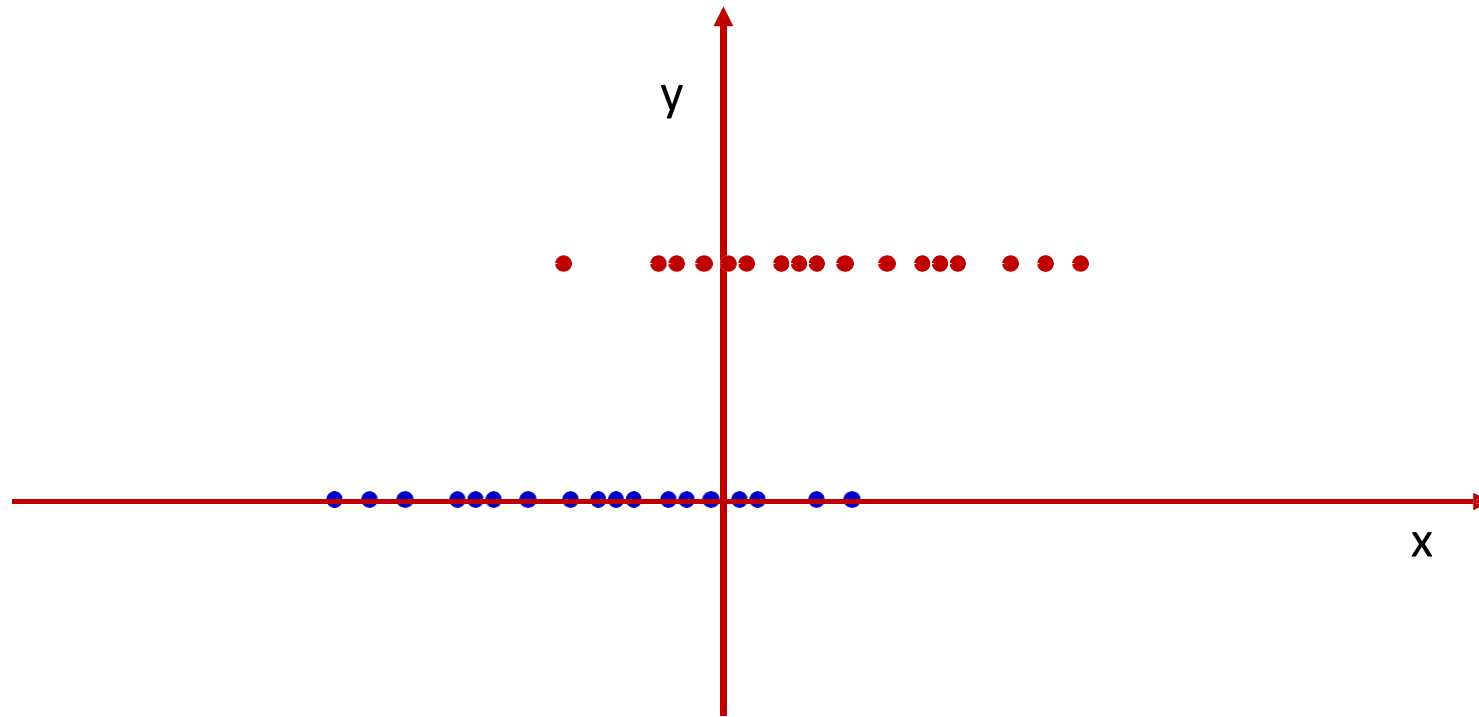
- This particular one has a nice interpretation
- It can be interpreted as $P(y = 1|x)$

Non-linearly separable data



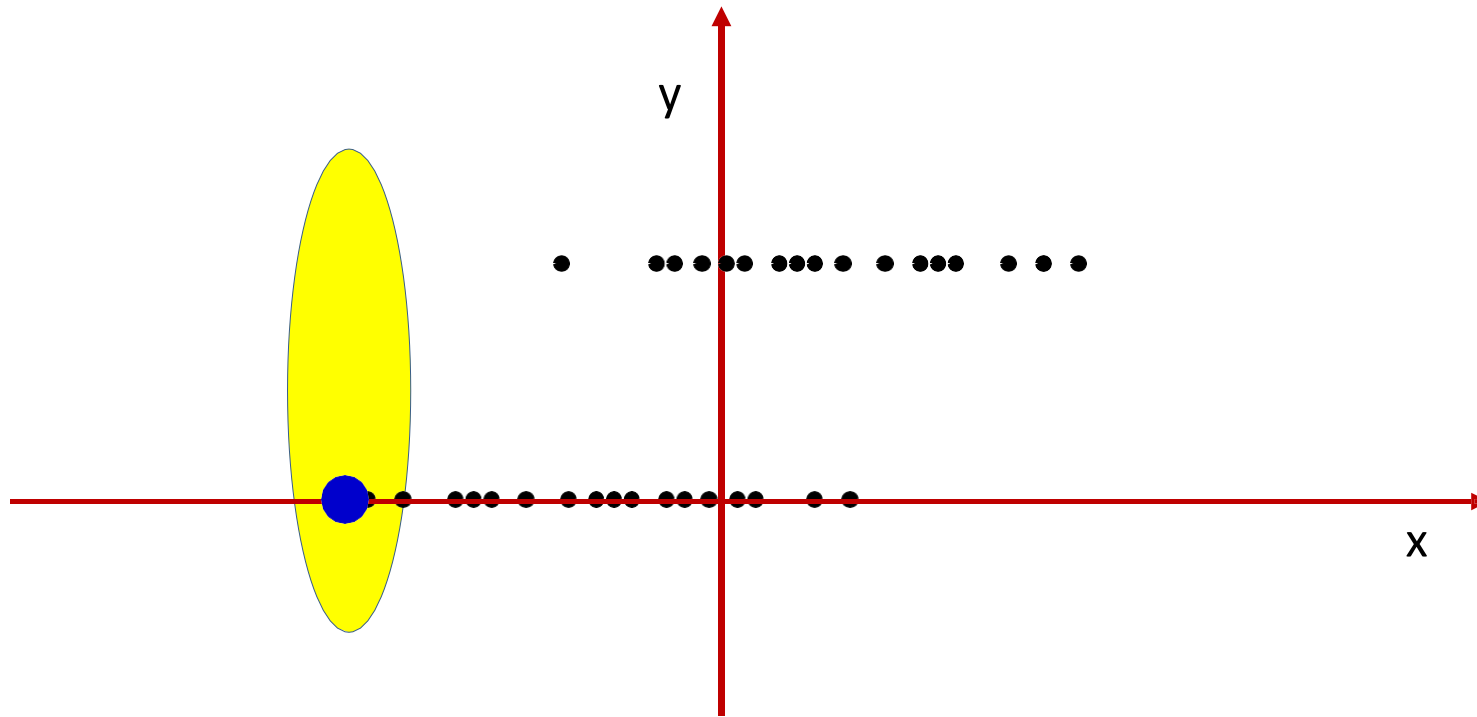
- Two-dimensional example
 - Blue dots (on the floor) on the “red” side
 - Red dots (suspended at $Y=1$) on the “blue” side
 - No line will cleanly separate the two colors

Non-linearly separable data: 1-D example



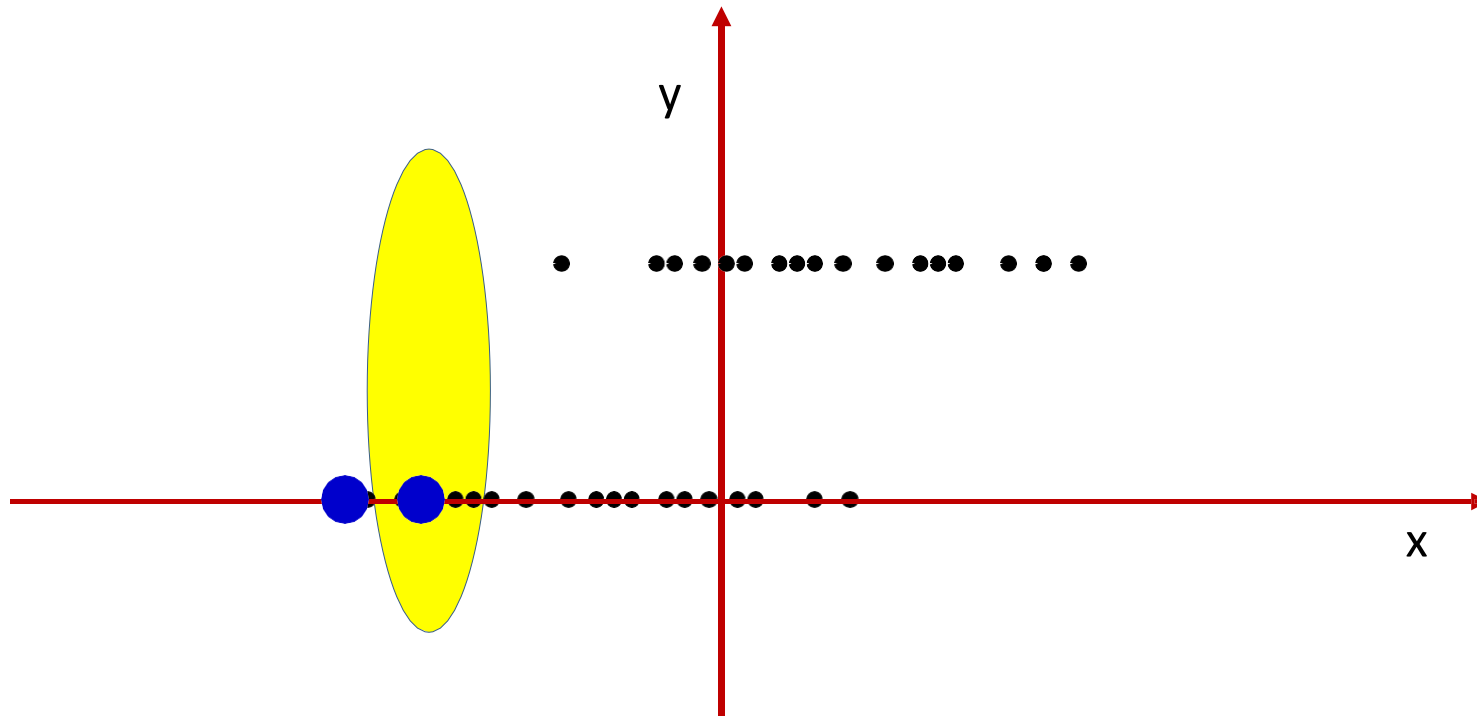
- One-dimensional example for visualization
 - All (red) dots at $Y=1$ represent instances of class $Y=1$
 - All (blue) dots at $Y=0$ are from class $Y=0$
 - The data are not linearly separable
 - In this 1-D example, a linear separator is a threshold
 - No threshold will cleanly separate red and blue dots

The *probability* of $y=1$



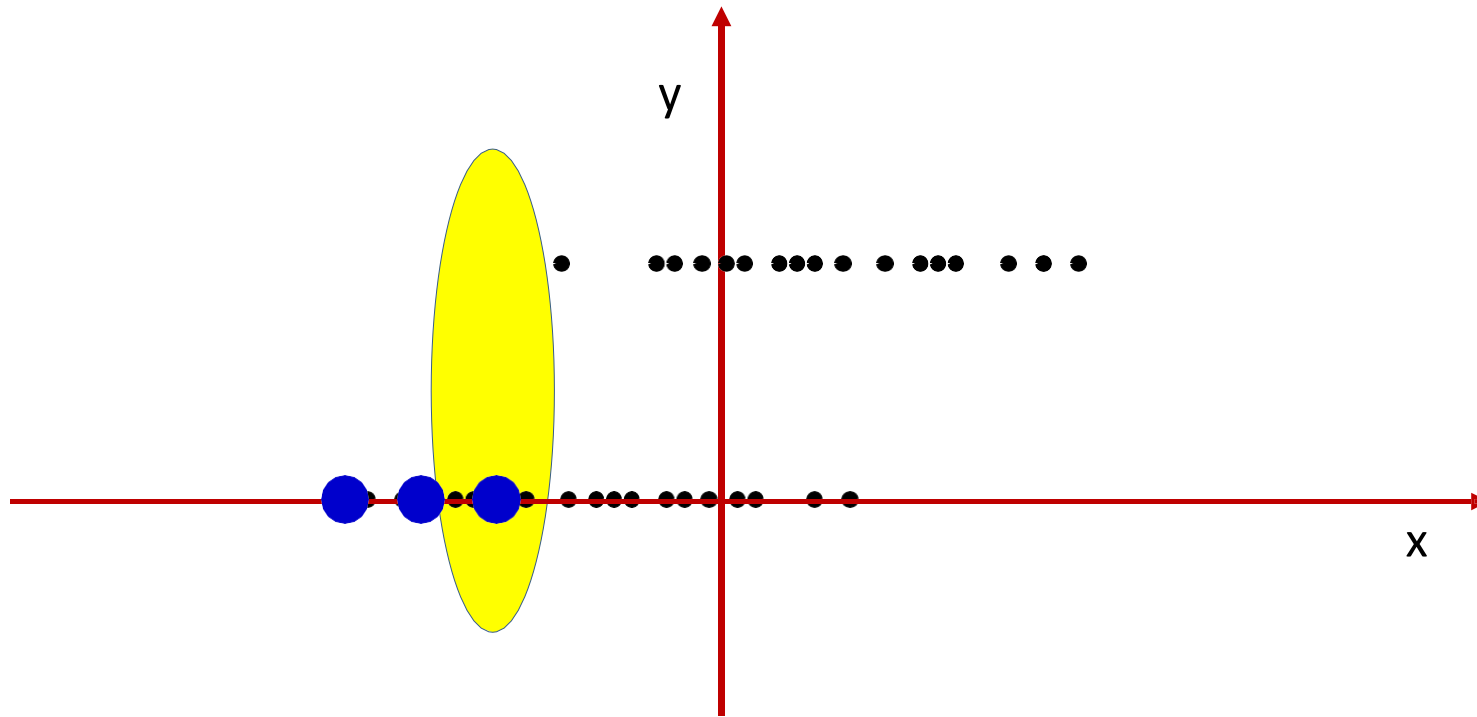
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of $Y=1$ at that point

The *probability* of $y=1$



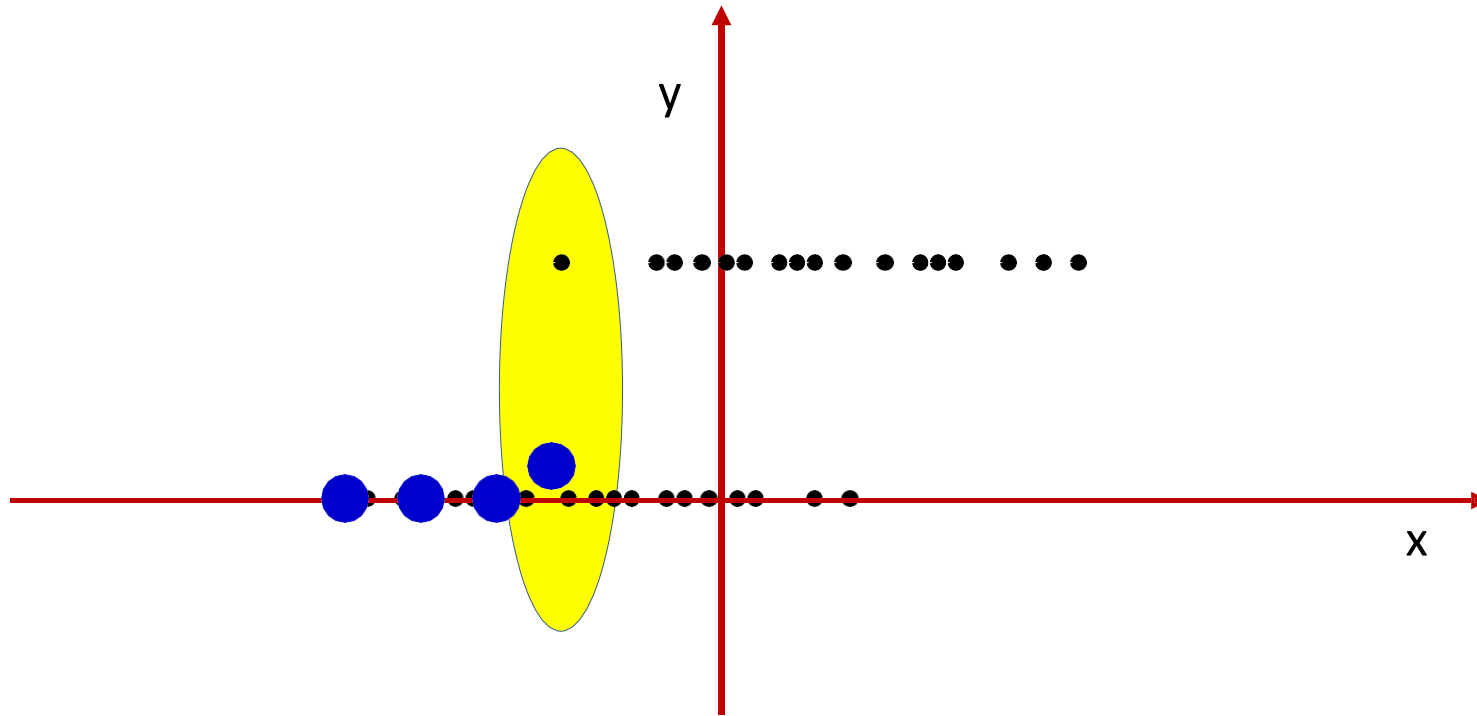
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



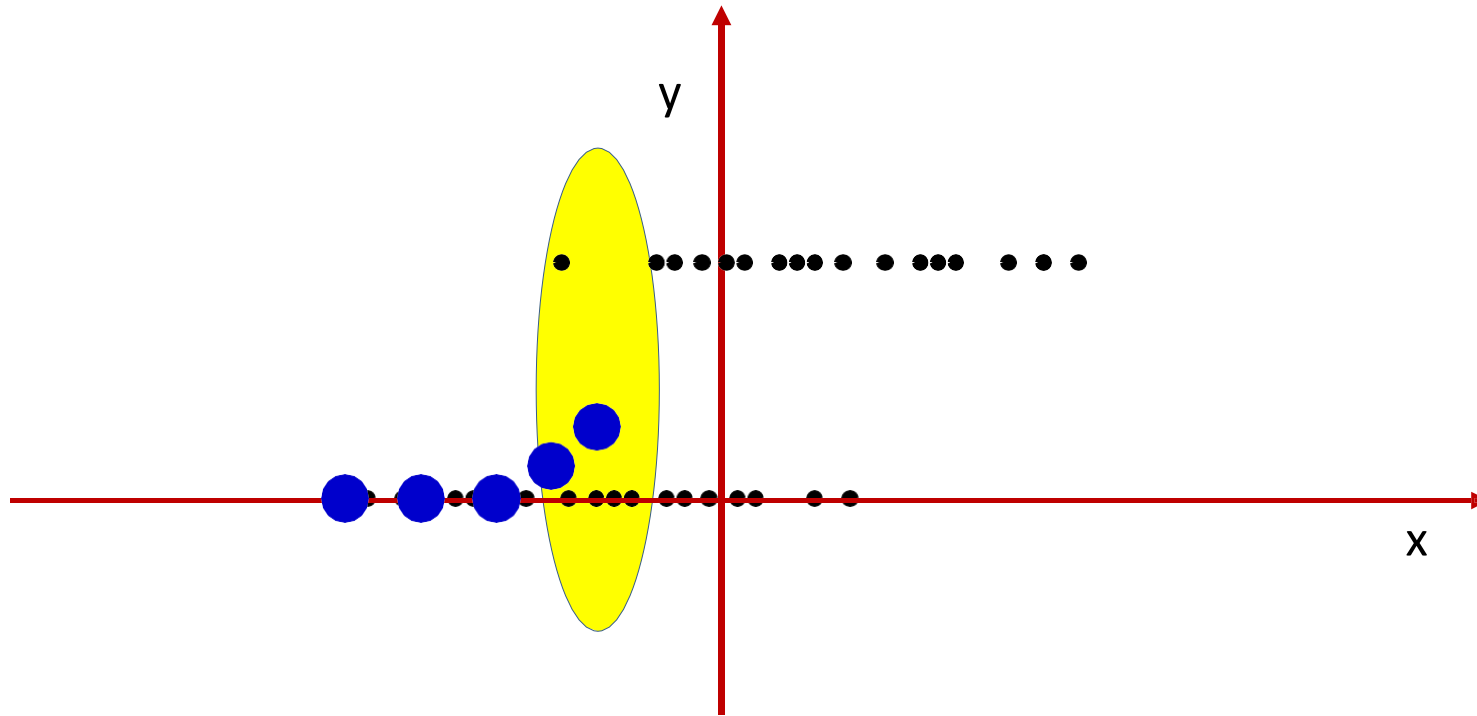
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



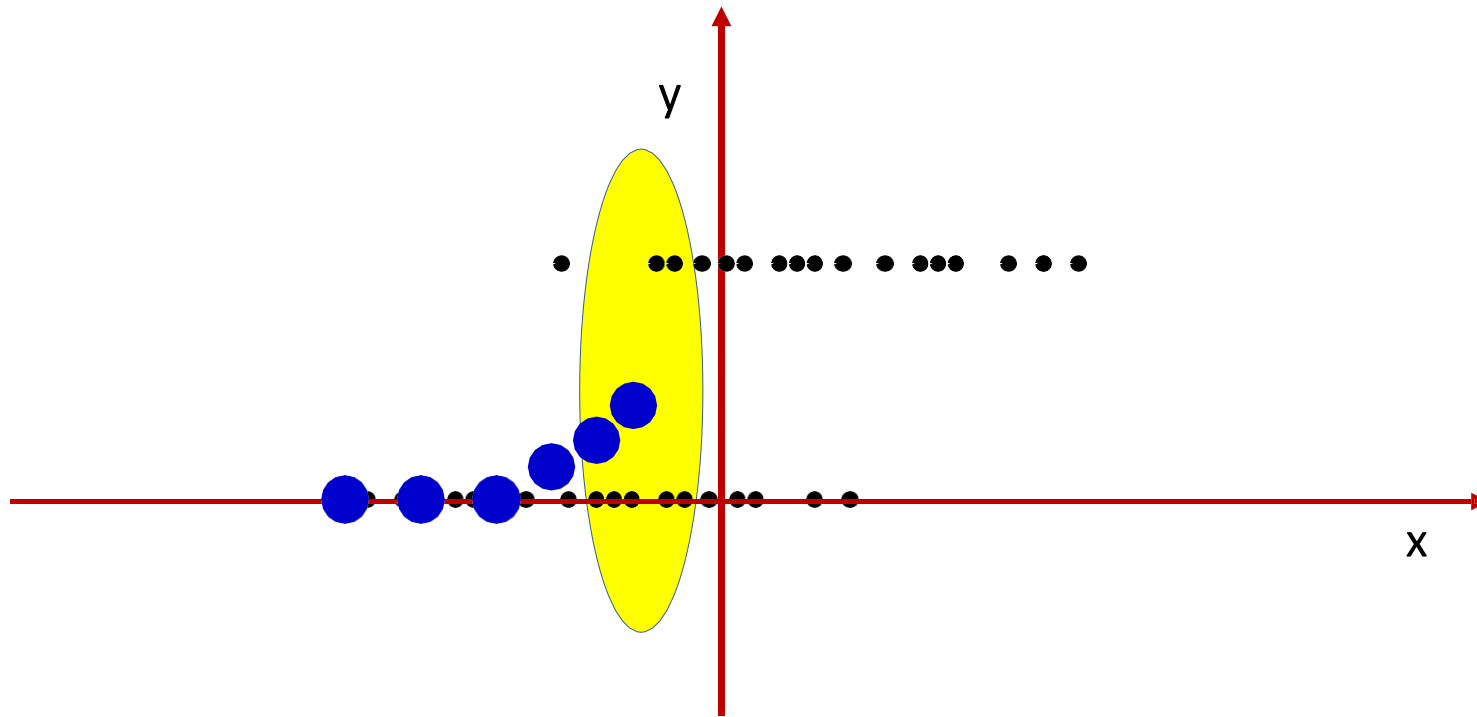
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



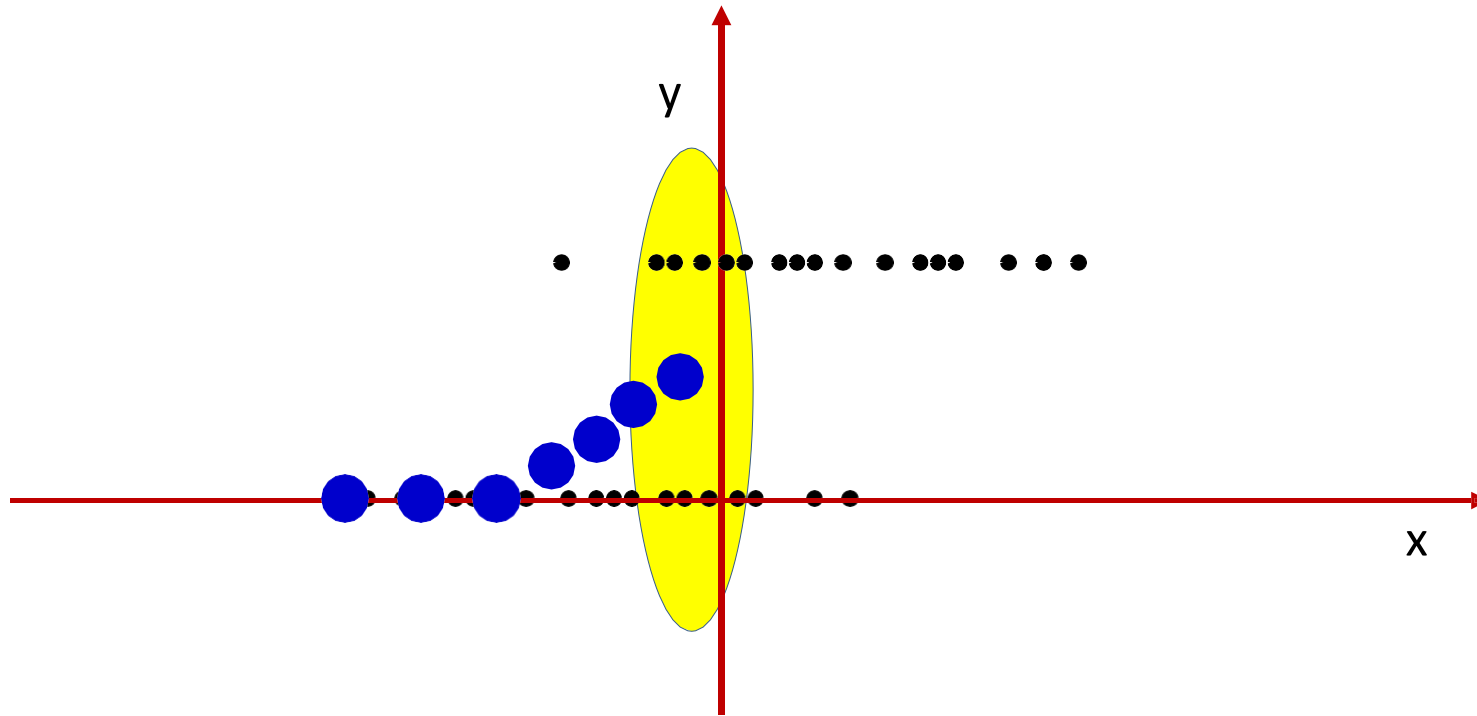
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



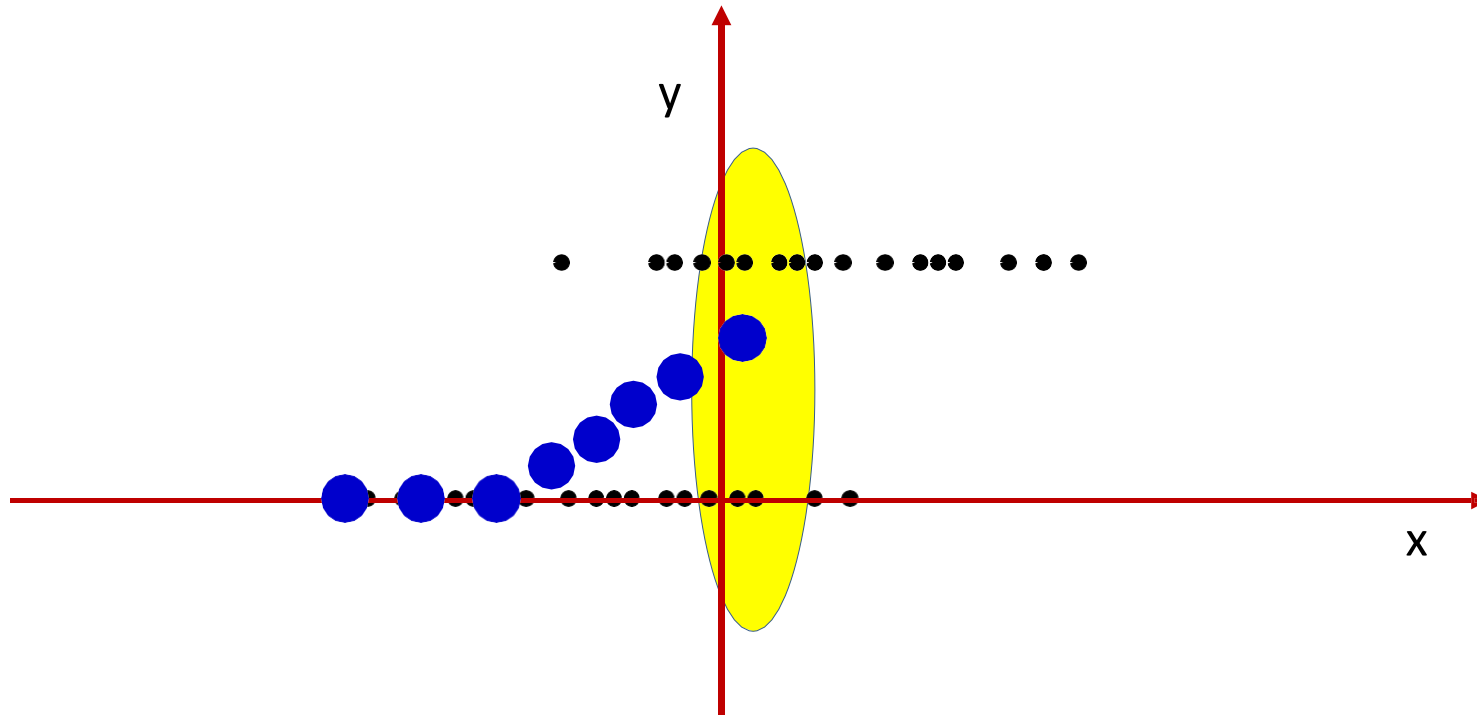
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



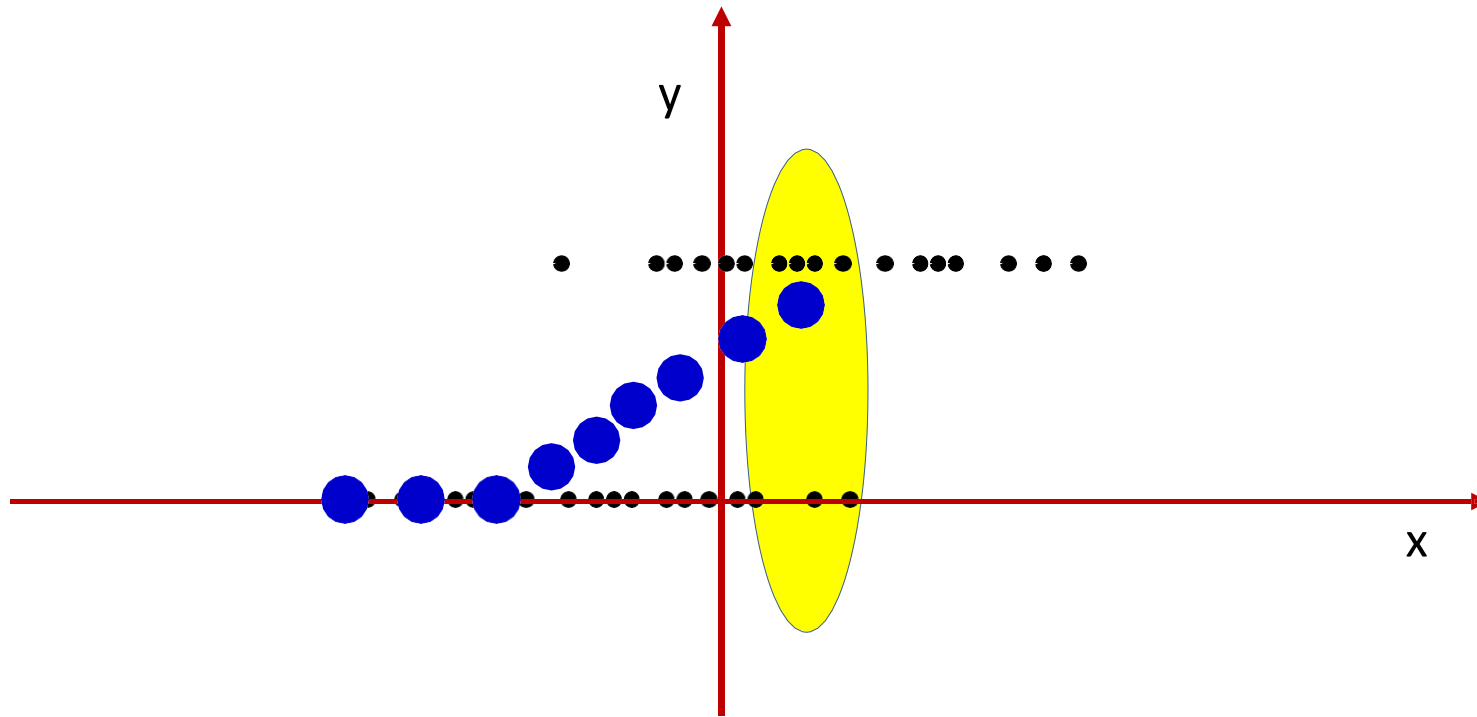
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



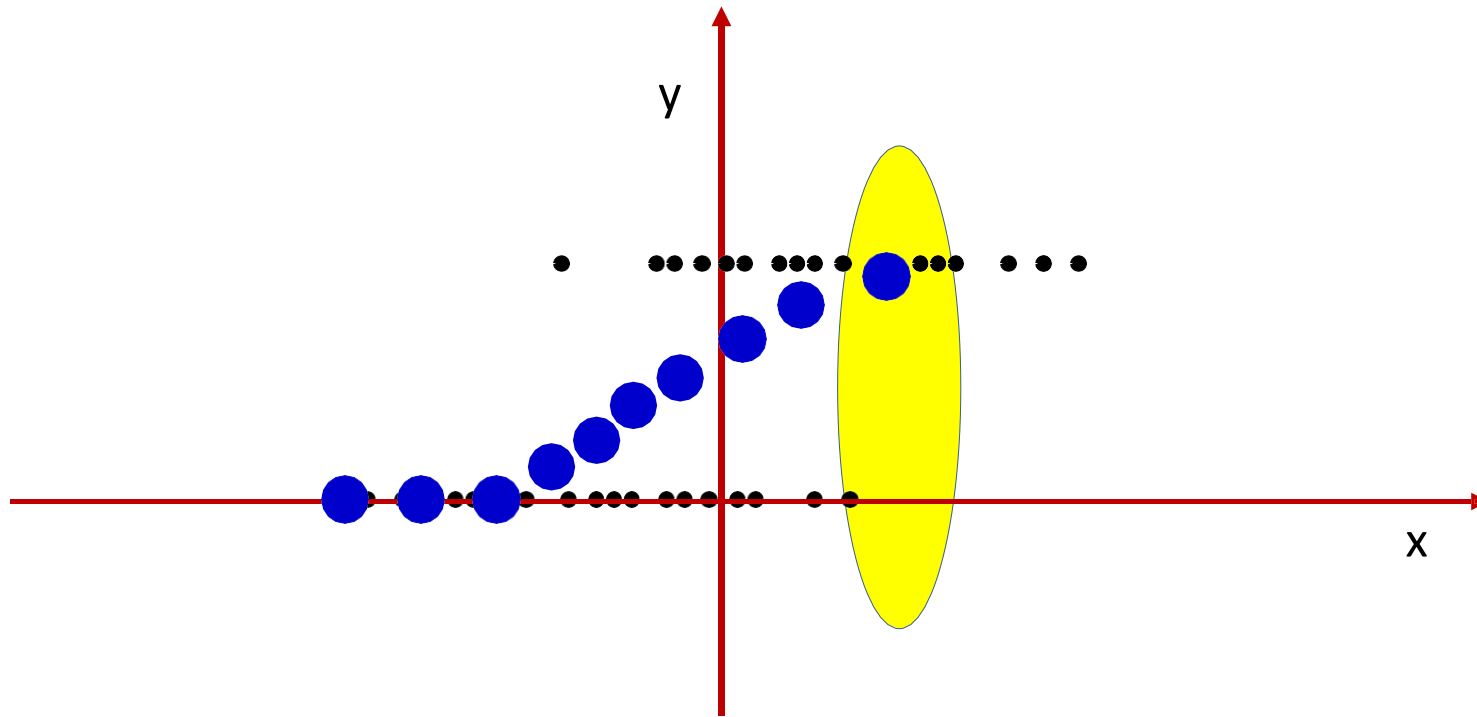
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



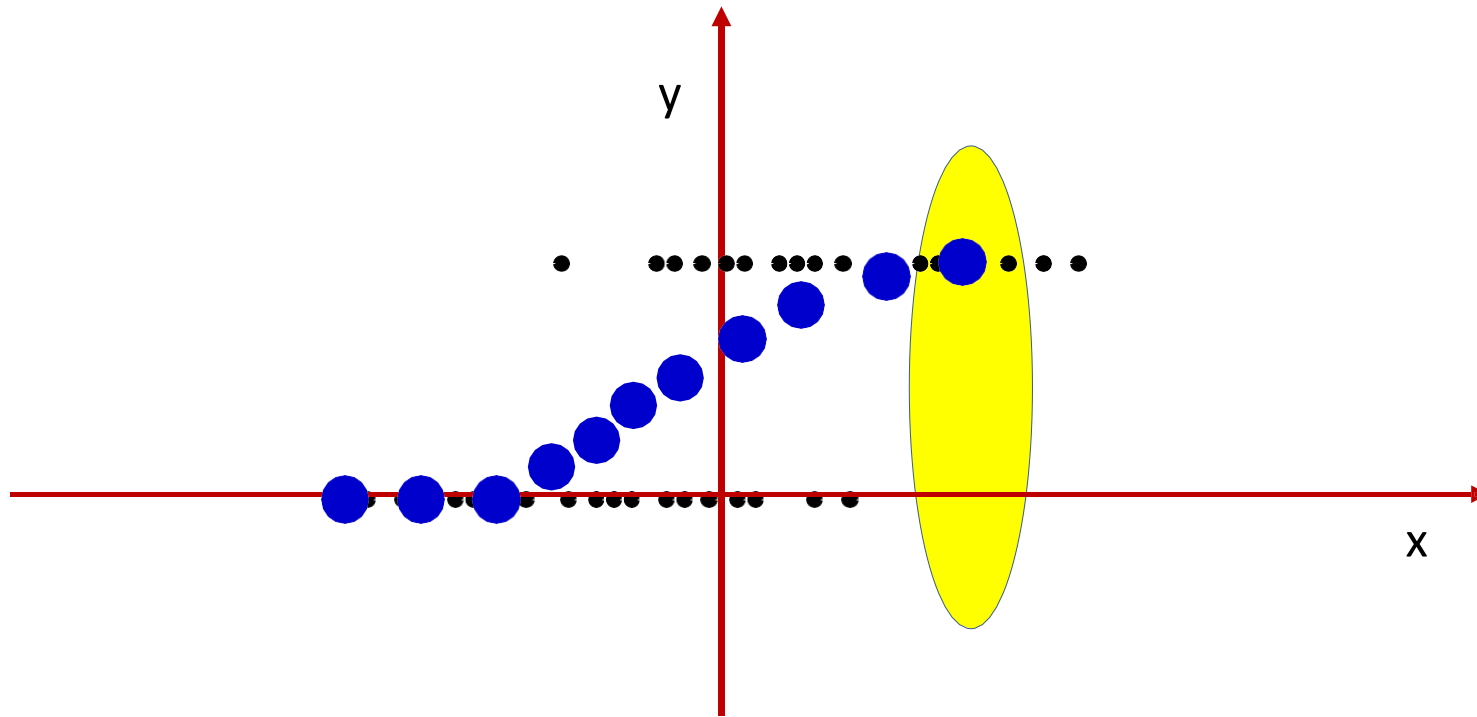
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



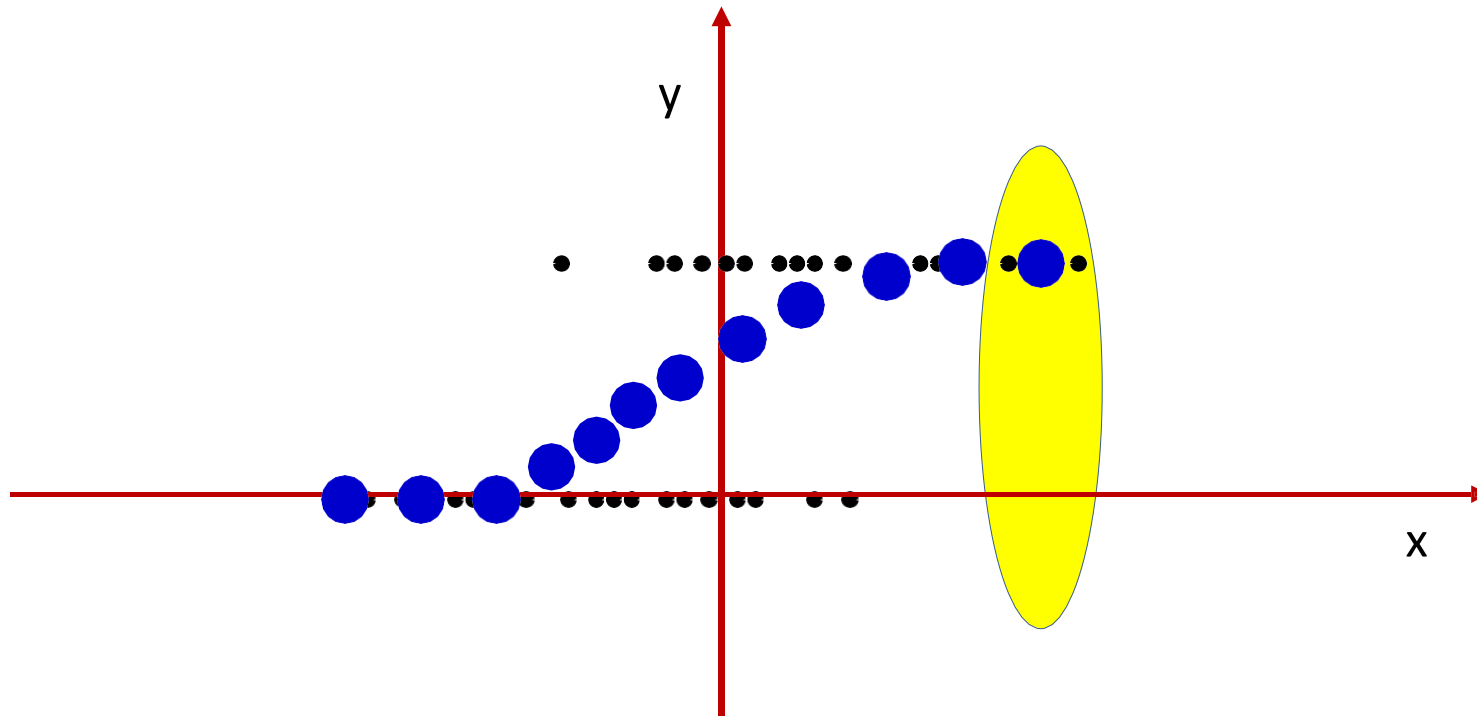
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



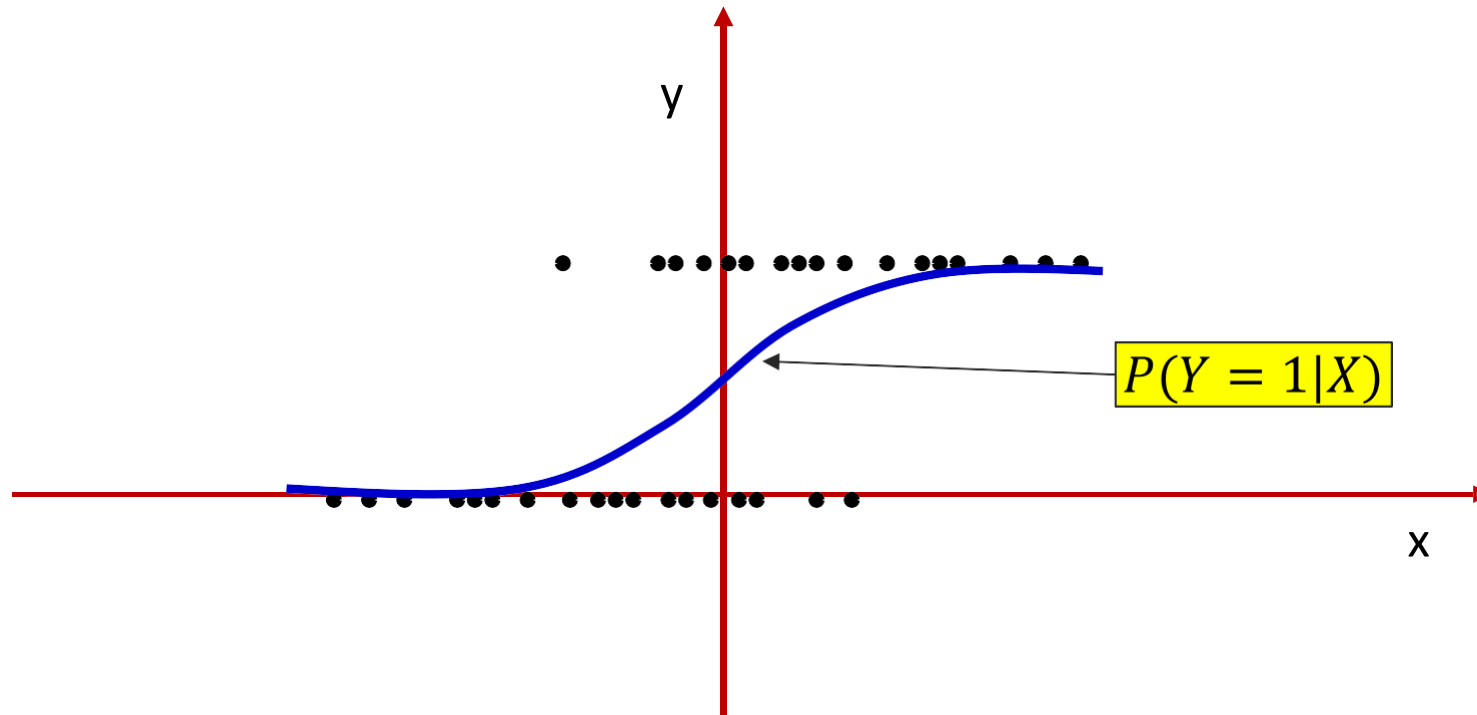
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



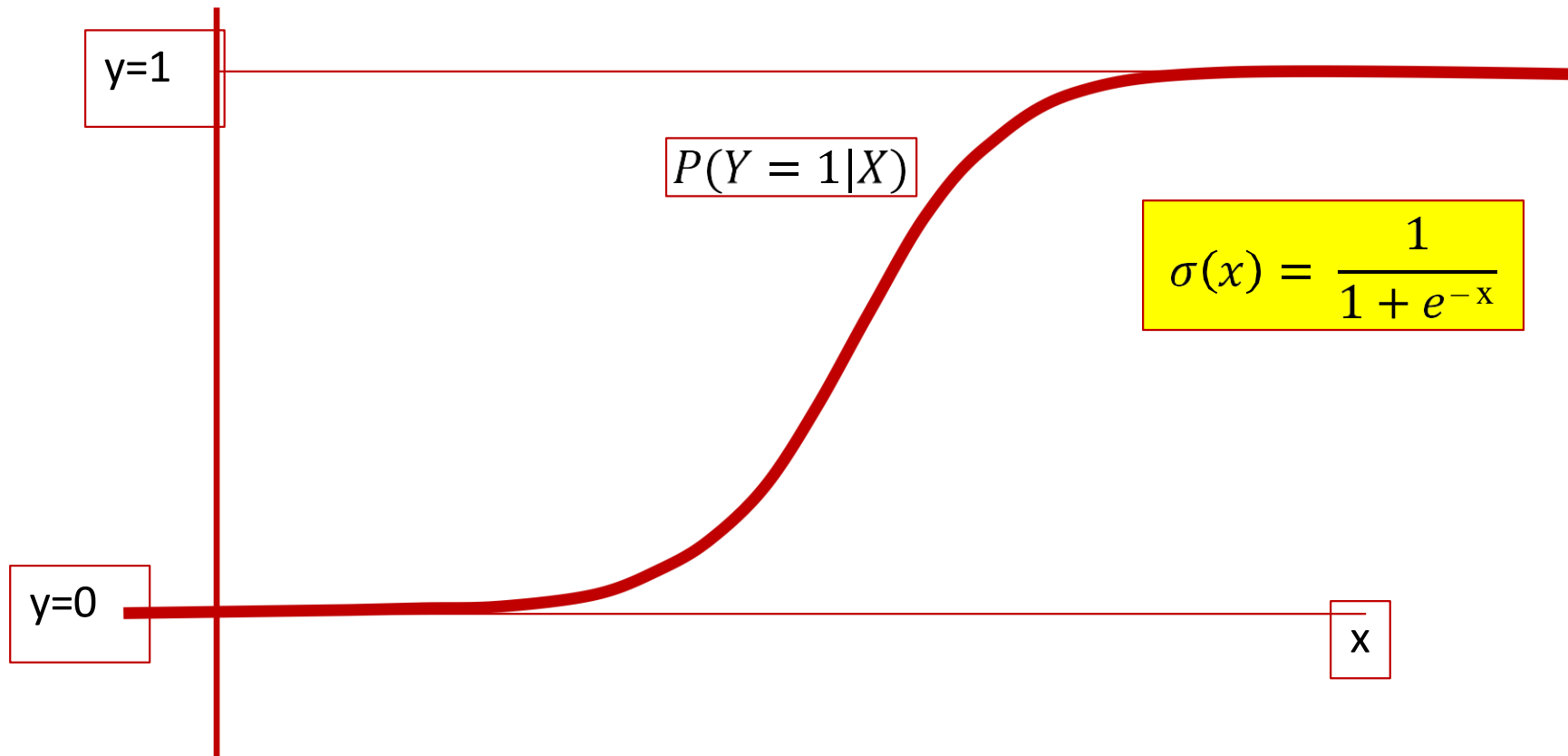
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

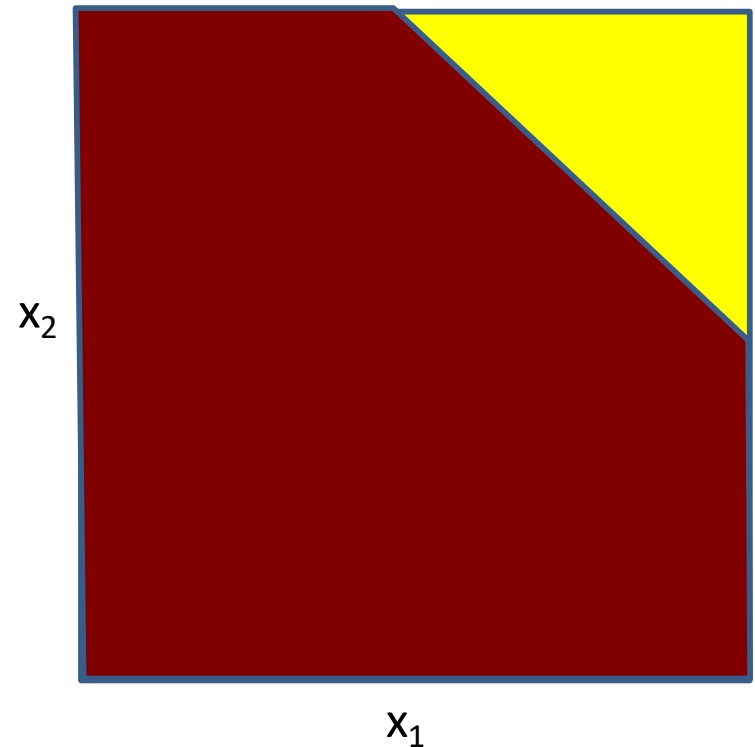
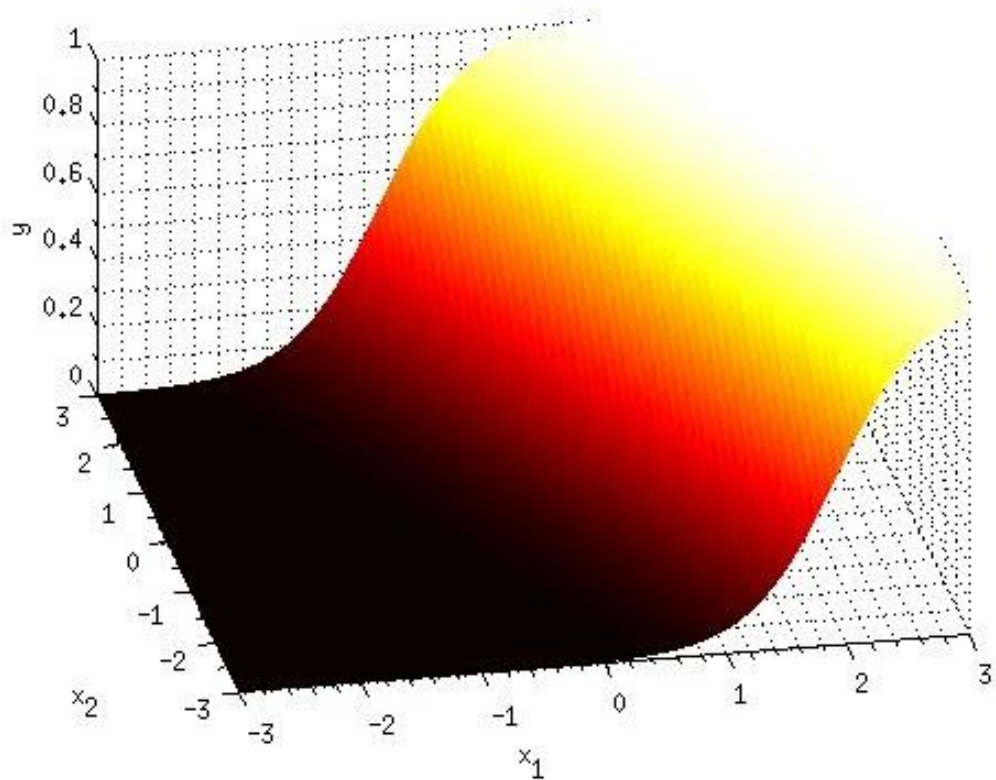
The logistic regression model



- Class 1 becomes increasingly probable going left to right
 - Very typical in many problems

Logistic regression

Decision: $y > 0.5$?

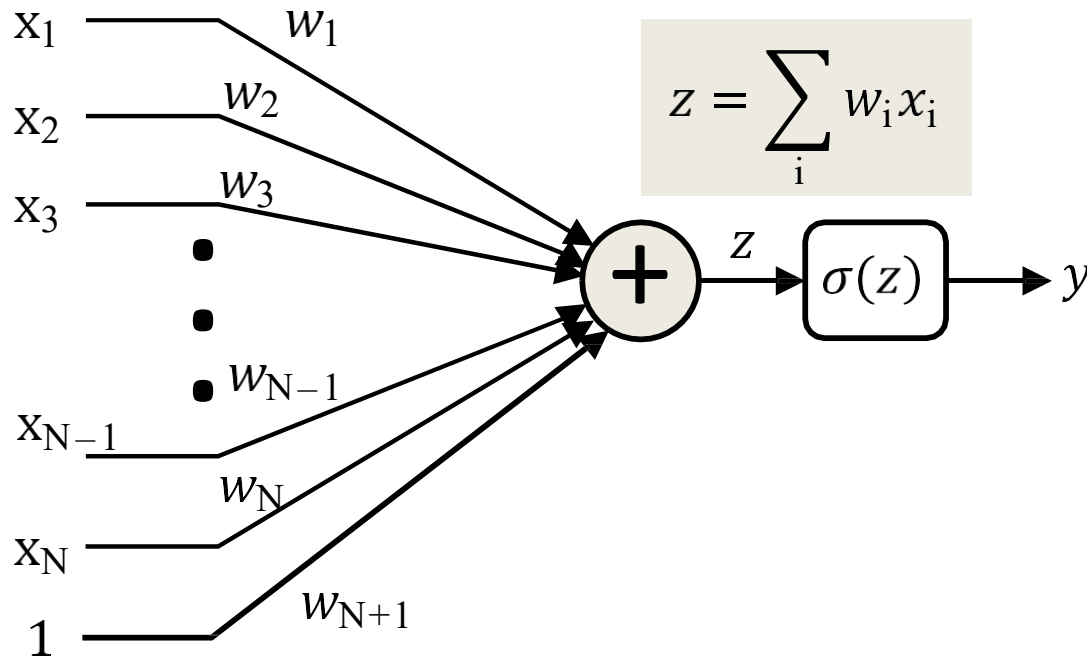


When X is a 2-D variable

$$P(Y = 1|X) = \frac{1}{1 + \exp(-\sum_i w_i x_i - b)}$$

- This is the perceptron with a sigmoid activation
 - It actually computes the *probability* that the input belongs to class 1

Perceptrons with differentiable activation functions



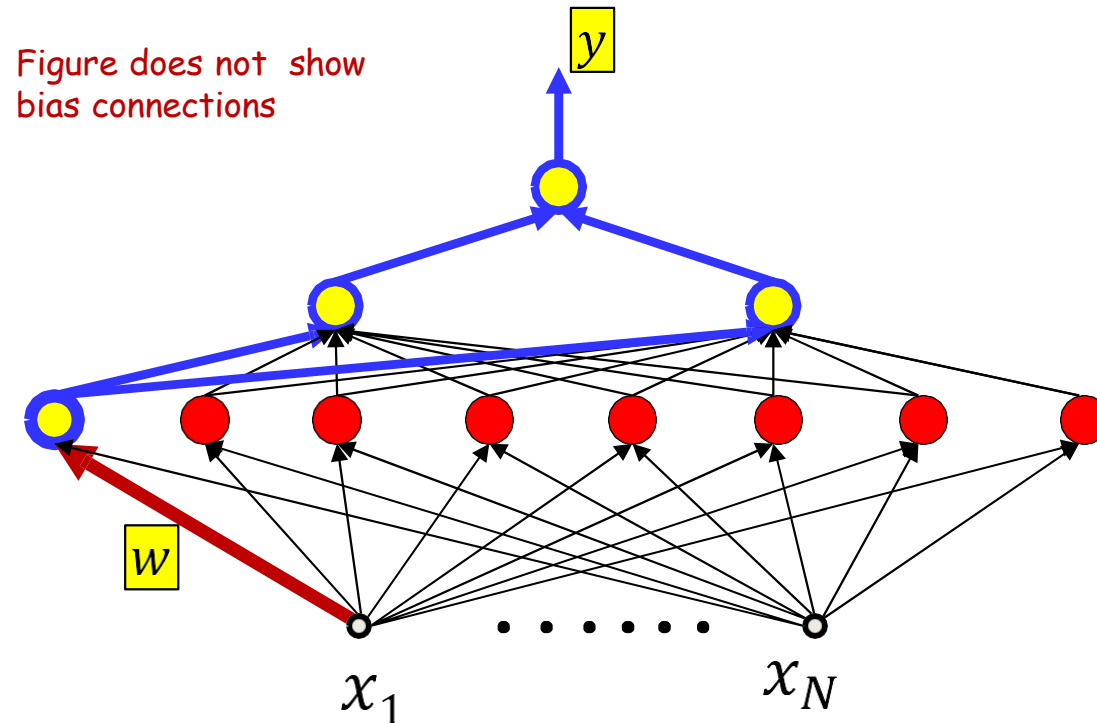
$$\frac{dy}{dz} = \sigma'(z)$$

$$\frac{dy}{dw_i} = \frac{dy}{dz} \frac{dz}{dw_i} = \sigma'(z) x_i$$

$$\frac{dy}{dx_i} = \frac{dy}{dz} \frac{dz}{dx_i} = \sigma'(z) w_i$$

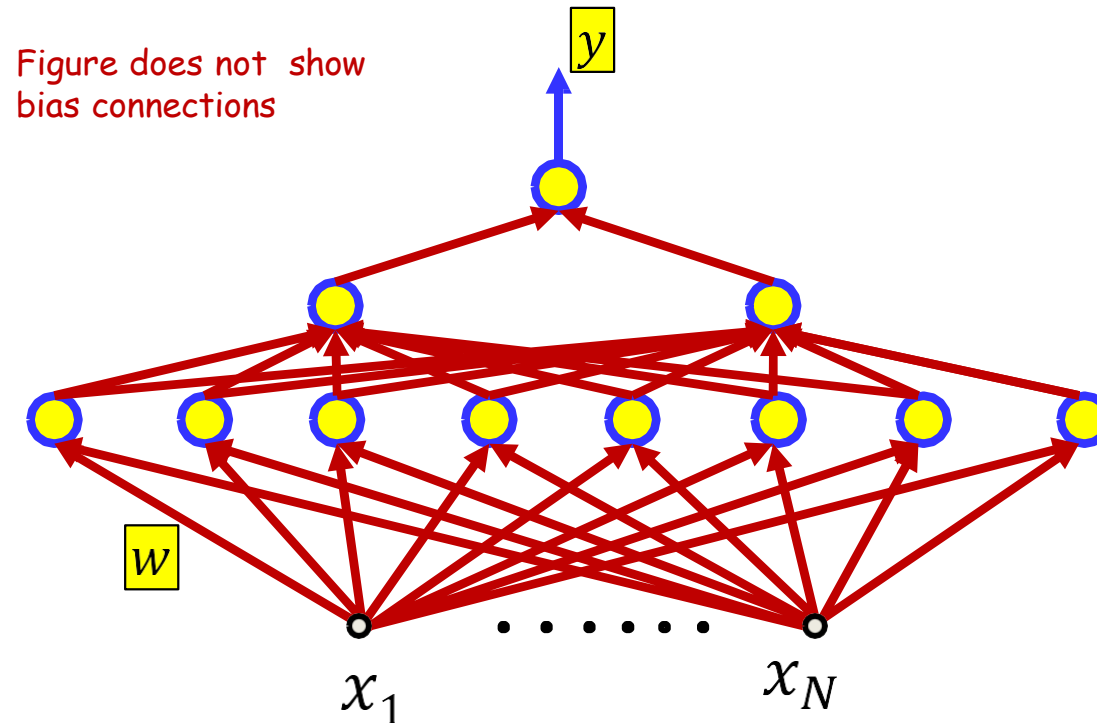
- $\sigma(z)$ is a differentiable function of z
 - $\frac{d\sigma(z)}{dz}$ is well-defined and finite for all z
- Using the chain rule, y is a differentiable function of both inputs x_i and weights w_i
- This means that we can compute the change in the output for *small* changes in either the input or the weights

Overall network is differentiable



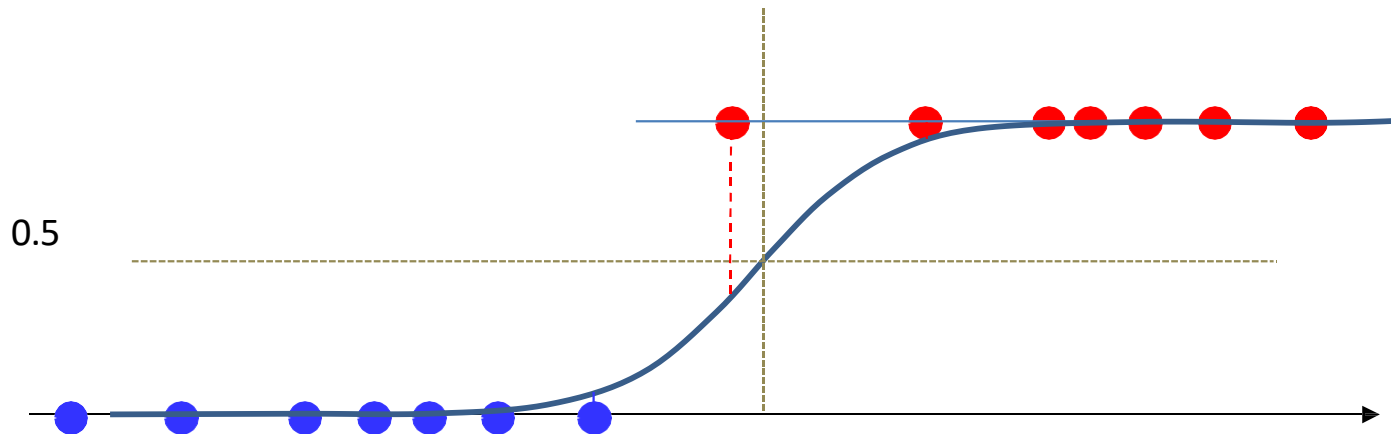
- Every individual perceptron is differentiable w.r.t its inputs and its weights (including “bias” weight)
 - Small changes in the parameters result in measurable changes in output
- Using the chain rule can compute how small perturbations of a parameter change the output of the network
 - The network output is differentiable with respect to the parameter

Overall function is differentiable



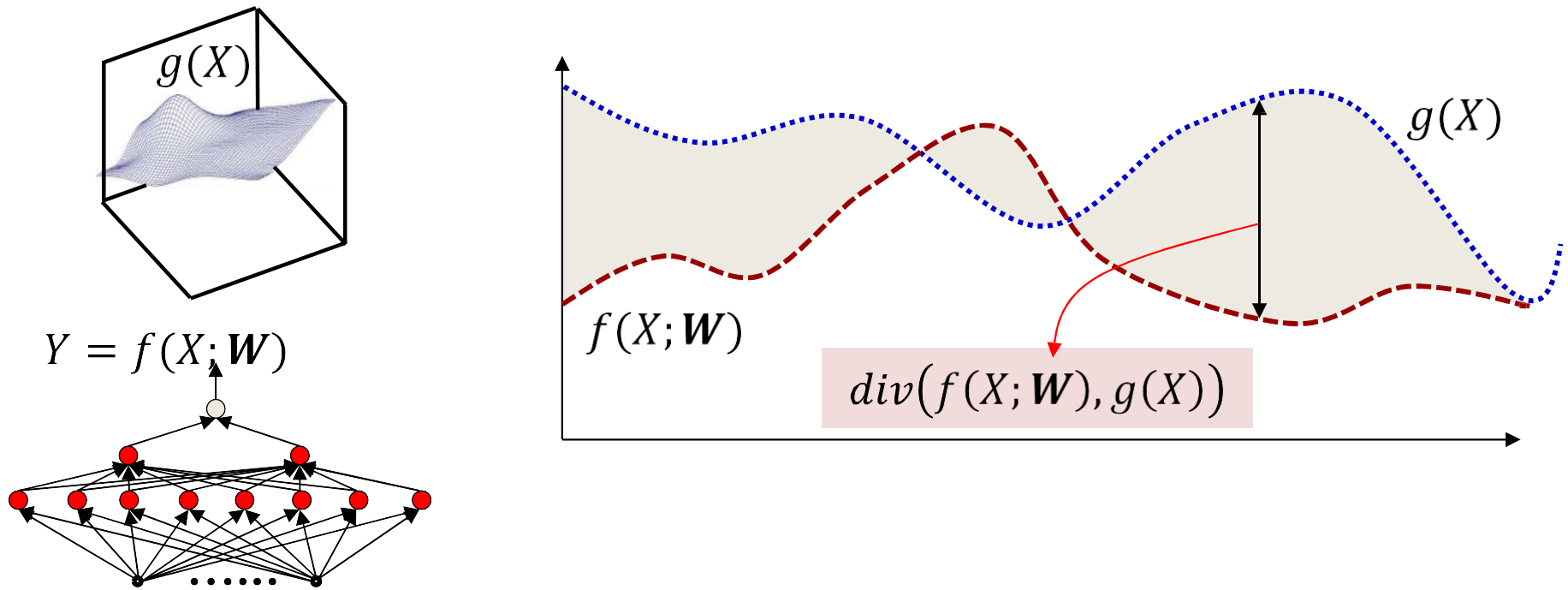
- By extension, the overall function is differentiable w.r.t every parameter in the network
 - We can compute how small changes in the parameters change the output
 - For non-threshold activations the derivative are finite and generally non-zero
- We will derive the actual derivatives using the chain rule later

The two key requirements for learnability



- Continuously varying activation
 - Differentiable
- Continuously varying error function
 - Also differentiable

The error function



- Define a *divergence* function $div(f(X; W), g(X))$ with the following properties

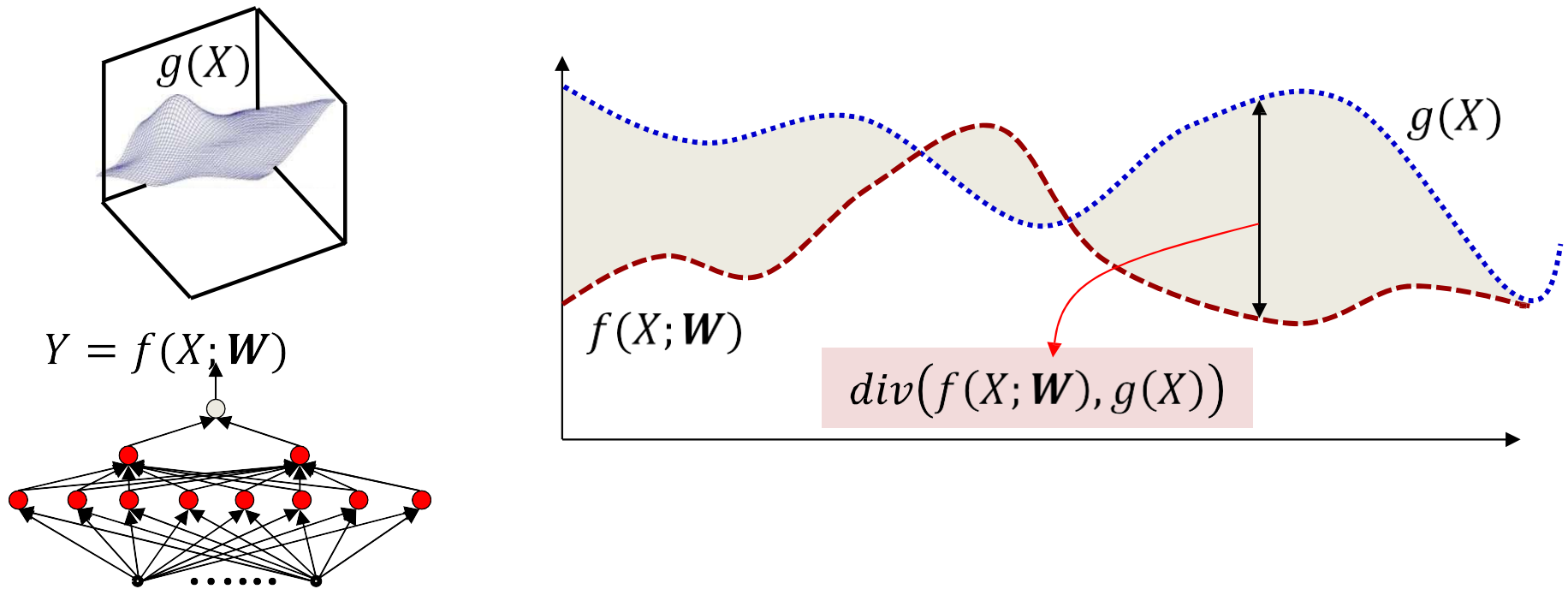
$$div(f(X; W), g(X)) = 0 \text{ if } f(X; W) = g(X)$$

$$div(f(X; W), g(X)) > 0 \text{ if } f(X; W) \neq g(X)$$

$div(f(X; W), g(X))$ is differentiable w.r.t x

- The divergence function quantifies mismatch between the network output and target function
 - For classification, this is usually not the classification error but a proxy to it

How to learn a network

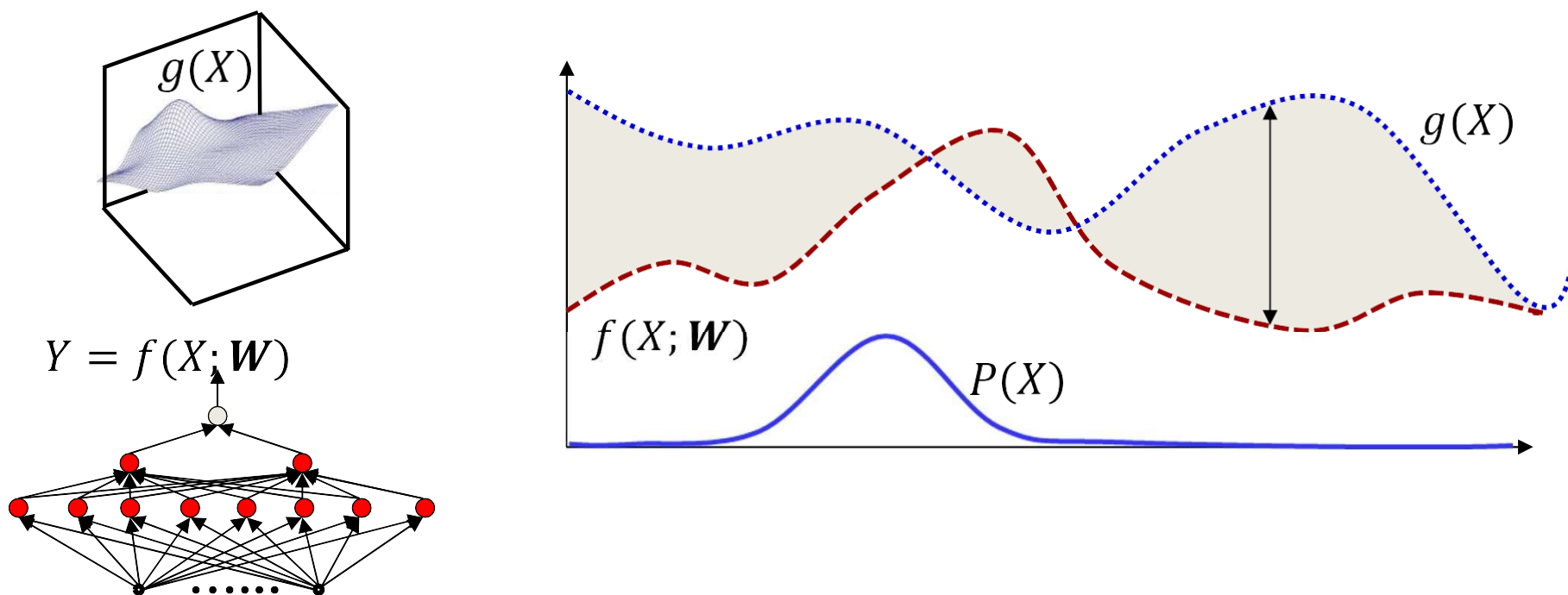


- When $f(X; W)$ has the capacity to exactly represent $g(X)$

$$\widehat{W} = \operatorname{argmin}_W \int_X div(f(X; W), g(X)) dX$$

- $div()$ is a divergence function that goes to zero when $f(X; W) = g(X)$

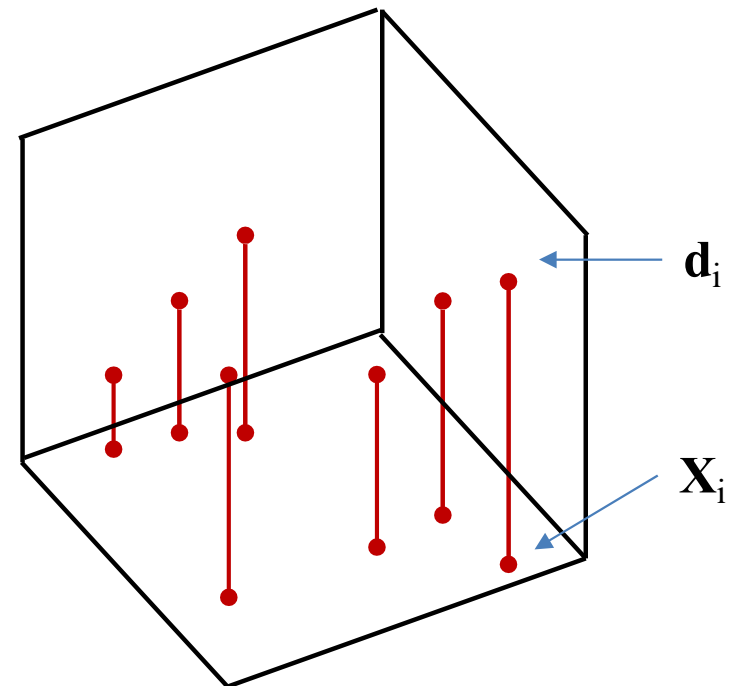
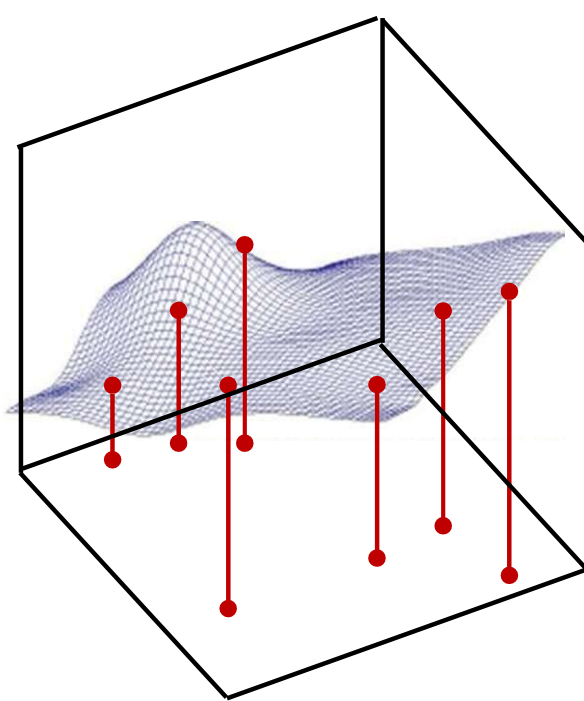
Minimizing *expected* divergence



- More generally, assuming X is a random variable

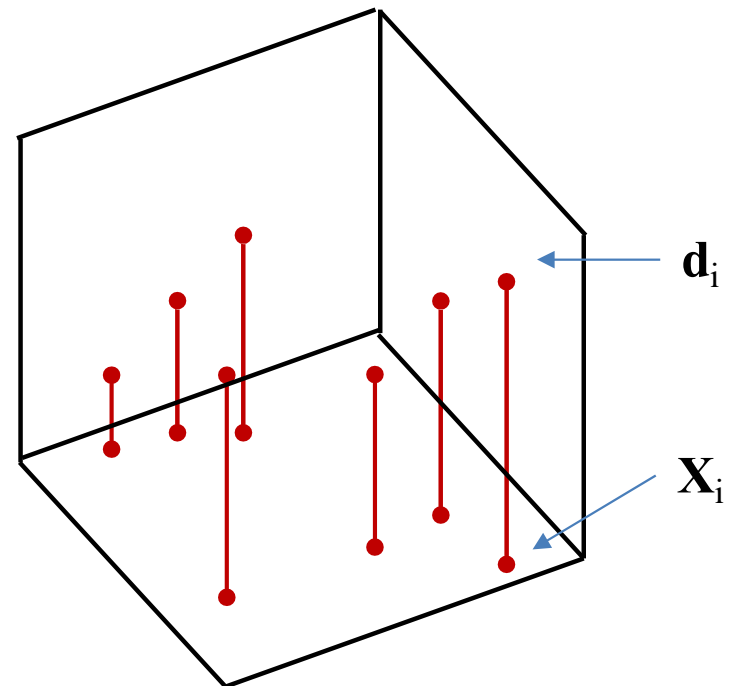
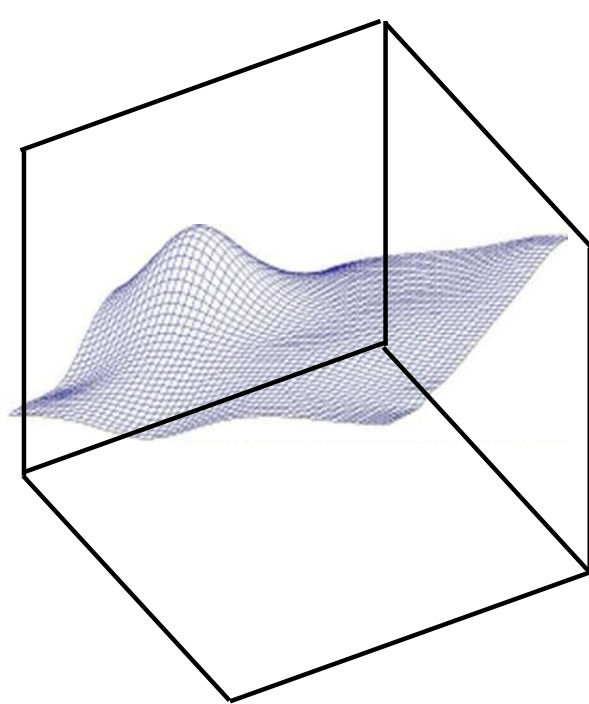
$$\begin{aligned}\widehat{W} &= \operatorname{argmin}_W \int_X \operatorname{div}(f(X; W), g(X)) P(X) dX \\ &= \operatorname{argmin}_W E[\operatorname{div}(f(X; W), g(X))]\end{aligned}$$

Recap: Sampling the function



- *We don't have $g(X)$ so sample $g(X)$*
 - Obtain input-output pairs for a number of samples of input X_i
 - Good sampling: the samples of X will be drawn from $P(X)$
- Estimate function from the samples

The *Empirical* risk



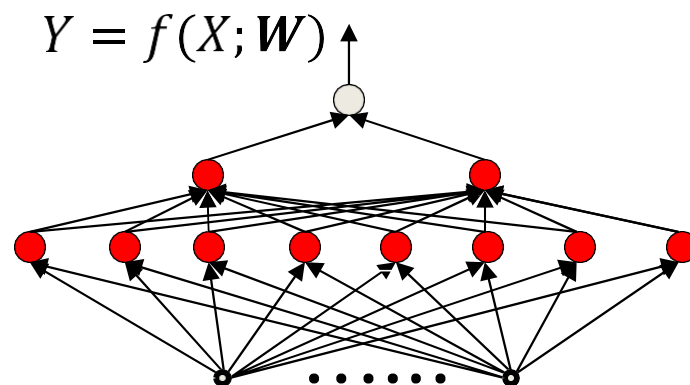
- The *expected* divergence (or risk) is the average divergence over the entire input space

$$E[\text{div}(f(X; W), g(X))] = \int_{\mathbf{X}} \text{div}(f(X; W), g(X)) P(X) dX$$

- The *empirical estimate* of the expected risk is the *average* divergence over the samples

$$E[\text{div}(f(X; W), g(X))] \approx \frac{1}{N} \sum_{i=1}^N \text{div}(f(\mathbf{x}_i; W), d_i)$$

Training the network: Empirical Risk Minimization



- Given a training set of input-output pairs $(X_1, d_1), (X_2, d_2), \dots, (X_N, d_N)$
 - Quantification of error on the i^{th} instance: $\text{div}(f(X_i; W), d_i)$
 - Empirical average divergence (**Empirical Risk**) on all training data:

$$\text{Loss}(W) = \frac{1}{N} \sum_i \text{div}(f(X_i; W), d_i)$$

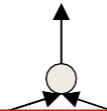
- Estimate the parameters to minimize the empirical estimate of expected divergence (**empirical risk**)

$$\widehat{W} = \underset{W}{\operatorname{argmin}} \text{Loss}(W)$$

- I.e. minimize the *empirical risk* over the drawn samples

Empirical Risk Minimization

$$Y = f(X; W)$$



Note : It's really a measure of error, but using standard terminology, we will call it a "Loss"

Note 2: The empirical risk $Loss(W)$ is only an empirical approximation to the true risk $E[div(f(X; W), g(X))]$ which is our *actual* minimization objective

Note 3: For a given training set the loss is only a function of W

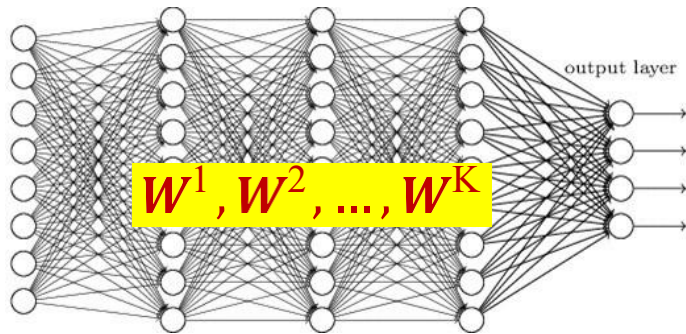
$$Loss(W) = \frac{1}{N} \sum_i div(f(X_i; W), d_i)$$

- Estimate the parameters to minimize the empirical estimate of expected error

$$\hat{W} = \underset{W}{\operatorname{argmin}} Loss(W)$$

- I.e. minimize the *empirical error* over the drawn samples

ERM for neural networks



Actual output of network:

$$Y_i = \text{net} \left(X_i; \left\{ w_{i,j}^k \forall i, j, k \right\} \right) \\ = \text{net}(X_i; W^1, W^2, \dots, W^K)$$

Desired output of network: d_i

Error on i-th training input: $\text{Div}(Y_i, d_i; W^1, W^2, \dots, W^K)$

Average training error(loss):

$$\text{Loss}(W^1, W^2, \dots, W^K) = \frac{1}{N} \sum_{i=1}^N \text{Div}(Y_i, d_i; W^1, W^2, \dots, W^K)$$

- What is the exact form of $\text{Div}()$? More on this later
- Optimize network parameters to minimize the total error over all training inputs

Problem Statement

- Given a training set of input-output pairs $(X_1, d_1), (X_2, d_2), \dots, (X_N, d_N)$
- Minimize the following function

$$Loss(W) = \frac{1}{N} \sum_i div(f(X_i; W), d_i)$$

w.r.t W

- This is problem of function minimization
 - An instance of optimization

Story so far

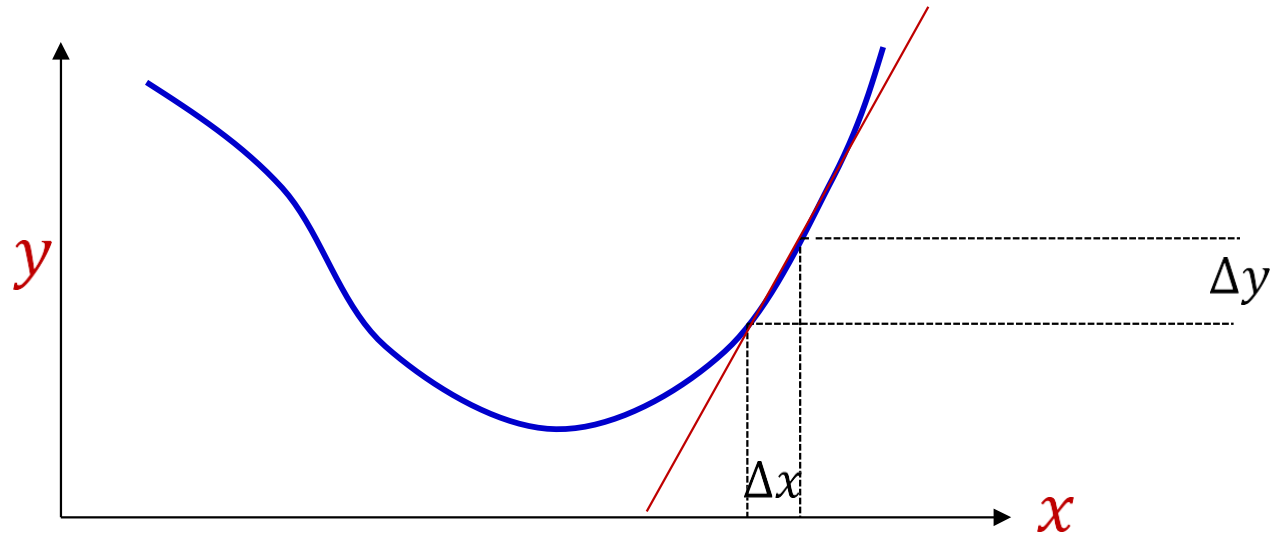
- We learn networks by “fitting” them to training instances drawn from a target function
- Learning networks of threshold-activation perceptrons requires solving a hard combinatorial-optimization problem
 - Because we cannot compute the influence of small changes to the parameters on the overall error
- Instead we use continuous activation functions with non-zero derivatives to enable us to estimate network parameters
 - This makes the output of the network differentiable w.r.t every parameter in the network
 - The *logistic* activation perceptron actually computes the *a posteriori* probability of the output given the input
- We define differentiable *divergence* between the output of the network and the desired output for the training instances
 - And a total error, which is the average divergence over all training instances
- We optimize network parameters to minimize this error
 - Empirical risk minimization
- This is an instance of function minimization

A quick intro to function optimization

with an initial discussion of derivatives

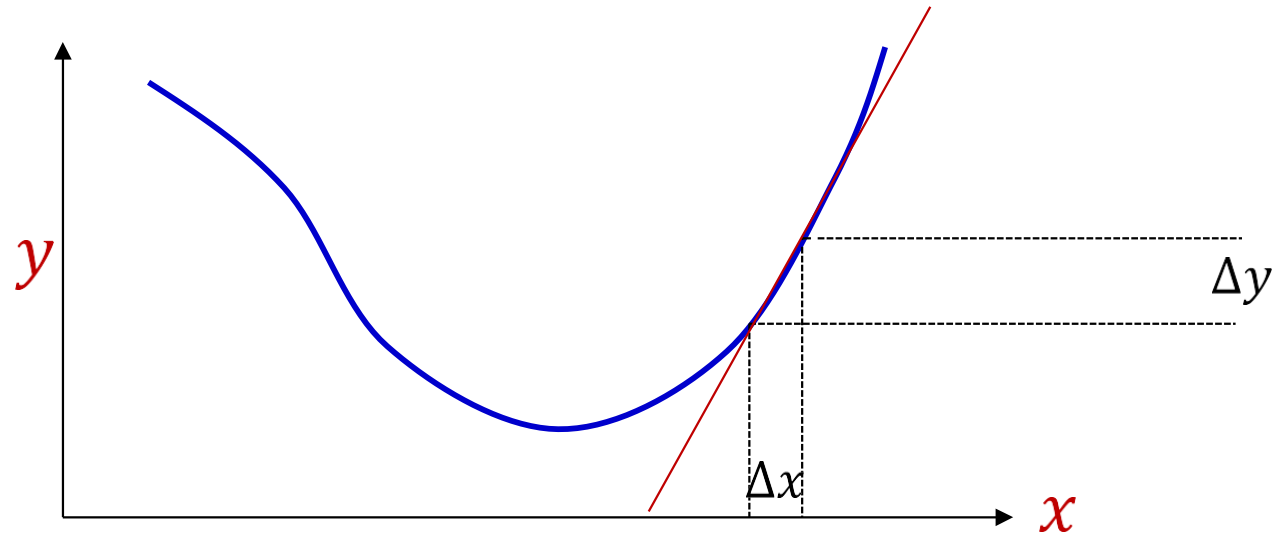


A brief note on derivatives..



- A derivative of a function at any point tells us how much a minute increment to the *argument* of the function will increment the *value* of the function
 - For any $y = f(x)$, expressed as a multiplier α to a tiny increment Δx to obtain the increments Δy to the output
$$\Delta y = \alpha \Delta x$$
 - Based on the fact that at a fine enough resolution, any smooth, continuous function is locally linear at any point

Scalar function of scalar argument



- When x and y are scalar

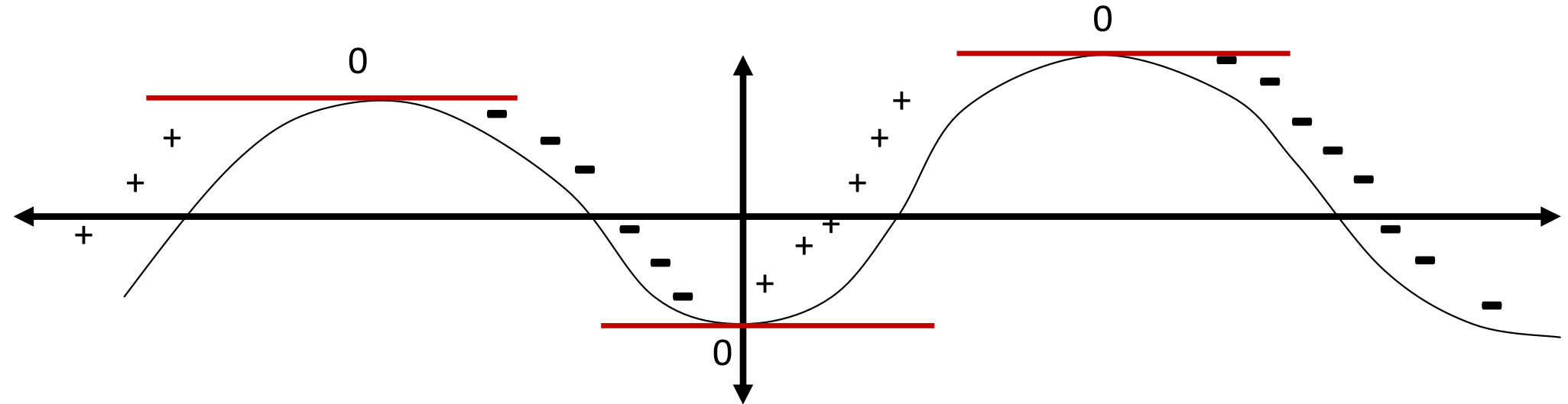
$$y = f(x)$$

- Derivative:

$$\Delta y = \alpha \Delta x$$

- Often represented as $\frac{dy}{dx}$
- Or alternately (and more reasonably) as $f'(x)$

Scalar function of scalar argument



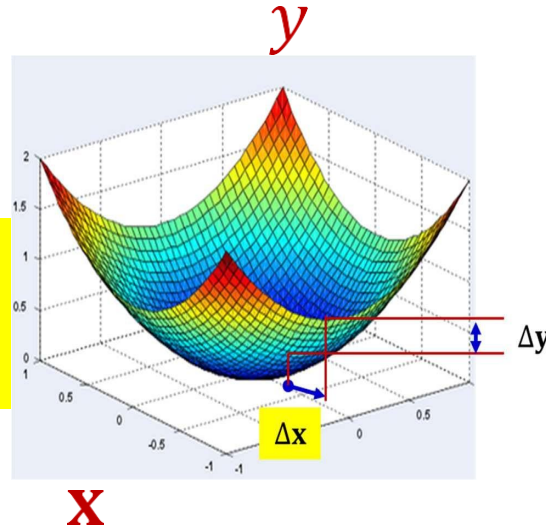
- Derivative $f'(x)$ is the *rate of change* of the function at x
 - How fast it increases with increasing x
 - The magnitude of $f'(x)$ gives you the steepness of the curve at x
 - Larger $|f'(x)| \rightarrow$ the function is increasing or decreasing more rapidly
- It will be positive where a small increase in x results in an *increase* of $f(x)$
 - Regions of positive slope
- It will be negative where a small increase in x results in a *decrease* of $f(x)$
 - Regions of negative slope
- It will be 0 where the function is locally flat (neither increasing nor decreasing)

Multivariate scalar function:

Scalar function of *vector* argument

$$y = f(\mathbf{x})$$

$\mathbf{x}$ is now a vector: $\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_D \end{bmatrix}$



Note: $\Delta \mathbf{x}$ is now also a vector

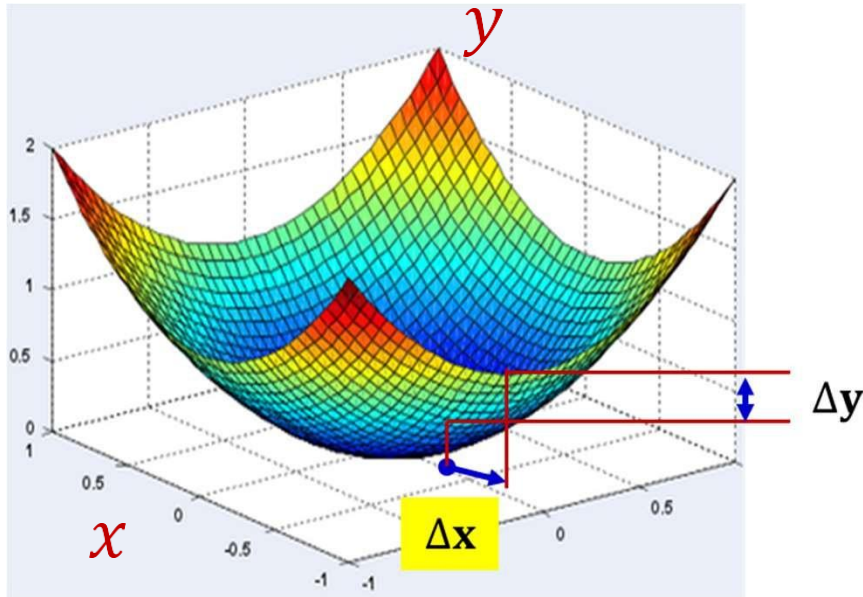
$$\Delta \mathbf{x} = \begin{bmatrix} \Delta x_1 \\ \vdots \\ \Delta x_D \end{bmatrix}$$

- Giving us that α is a row vector: $\alpha = [\alpha_1 \quad \cdots \quad \alpha_D]$
$$\Delta y = \alpha_1 \Delta x_1 + \alpha_2 \Delta x_2 + \cdots + \alpha_D \Delta x_D$$
- The *partial* derivative α_i gives us how y increments when *only* x_i is incremented
- Often represented as $\frac{\partial y}{\partial x_i}$

$$\Delta y = \frac{\partial y}{\partial x_1} \Delta x_1 + \frac{\partial y}{\partial x_2} \Delta x_2 + \cdots + \frac{\partial y}{\partial x_D} \Delta x_D$$

Multivariate scalar function:

Scalar function of *vector* argument



Note: $\Delta \mathbf{x}$ is now a vector

$$\Delta \mathbf{x} = \begin{bmatrix} \Delta x_1 \\ \vdots \\ \Delta x_D \end{bmatrix}$$

$$\Delta y = \nabla_{\mathbf{x}} y \Delta \mathbf{x}$$

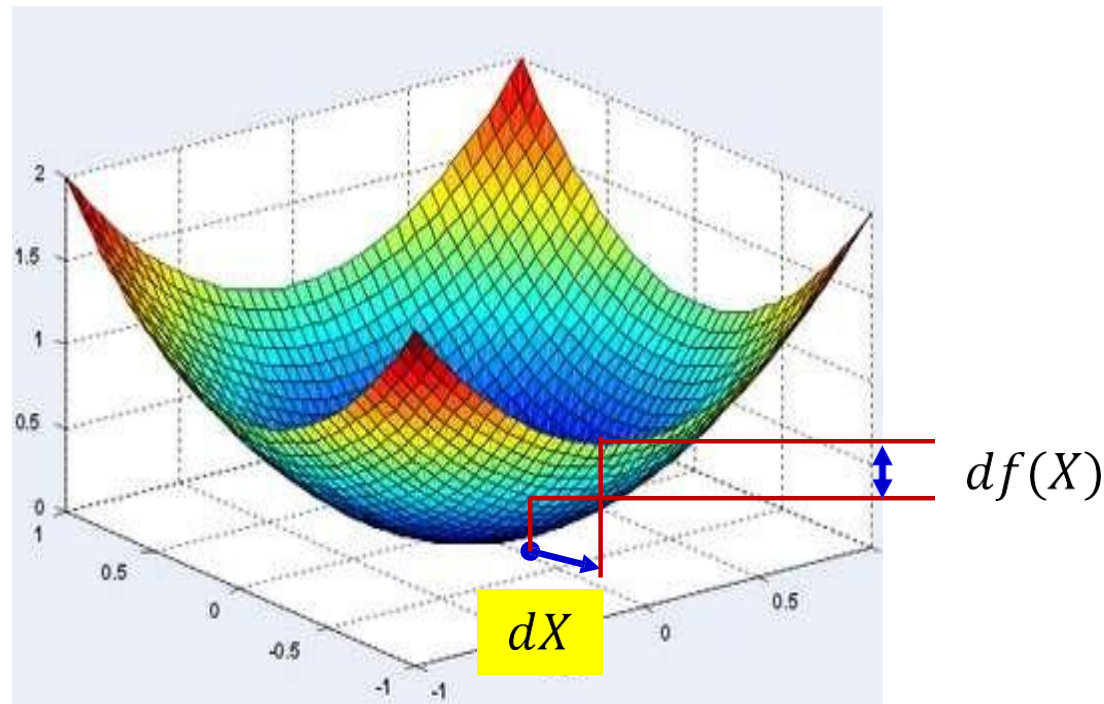
- Where

$$\nabla_{\mathbf{x}} y = \begin{bmatrix} \frac{\partial y}{\partial x_1} & \dots & \frac{\partial y}{\partial x_D} \end{bmatrix}$$

We will be using this symbol for vector and matrix derivatives

- You may be more familiar with the term “gradient” which is actually defined as the transpose of the derivative

Gradient of a scalar function of a vector



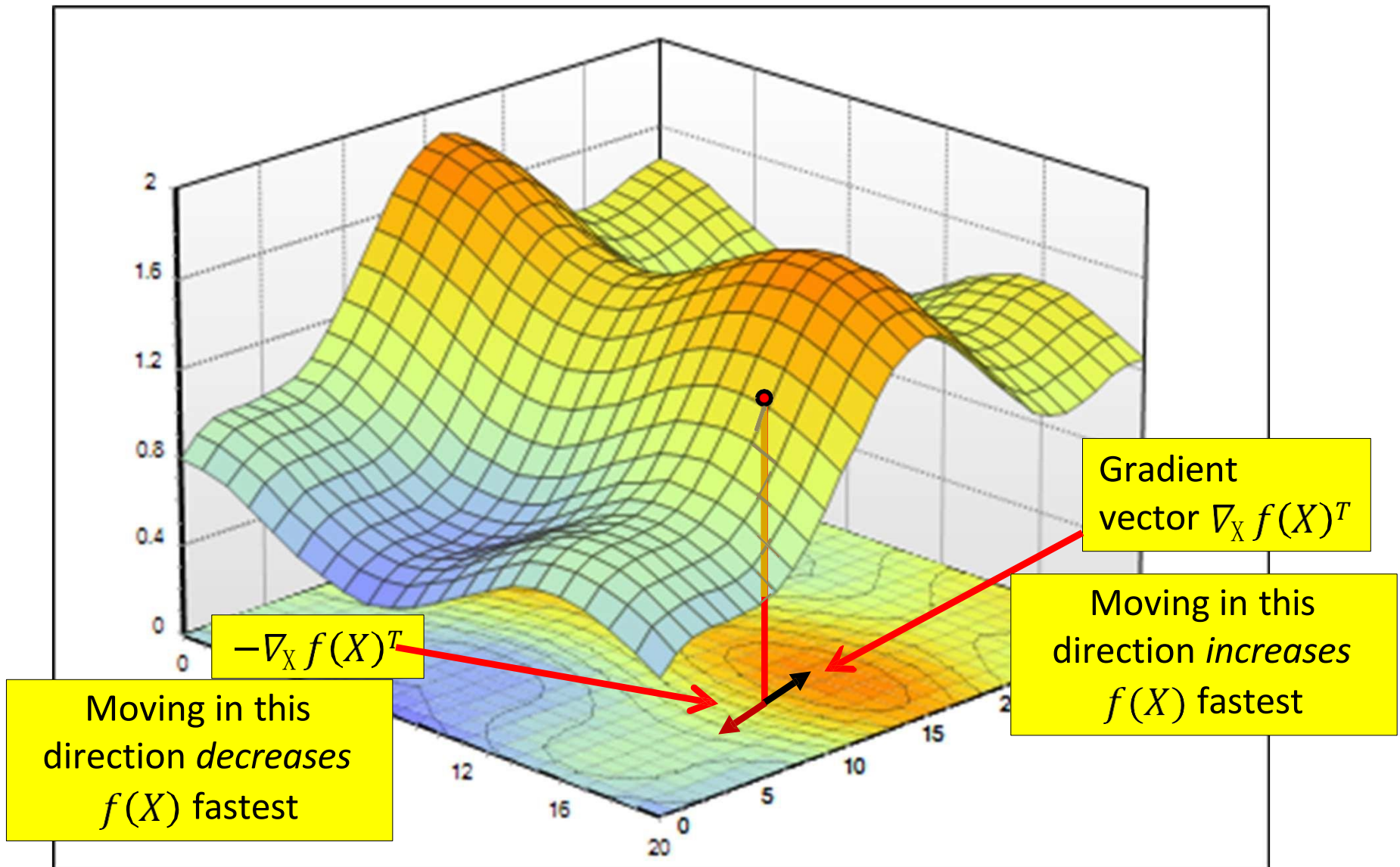
- The *derivative* $\nabla_X f(X)$ of a scalar function $f(X)$ of a multi-variate input X is a multiplicative factor that gives us the change in $f(X)$ for tiny variations in X

$$df(X) = \nabla_X f(X) dX$$

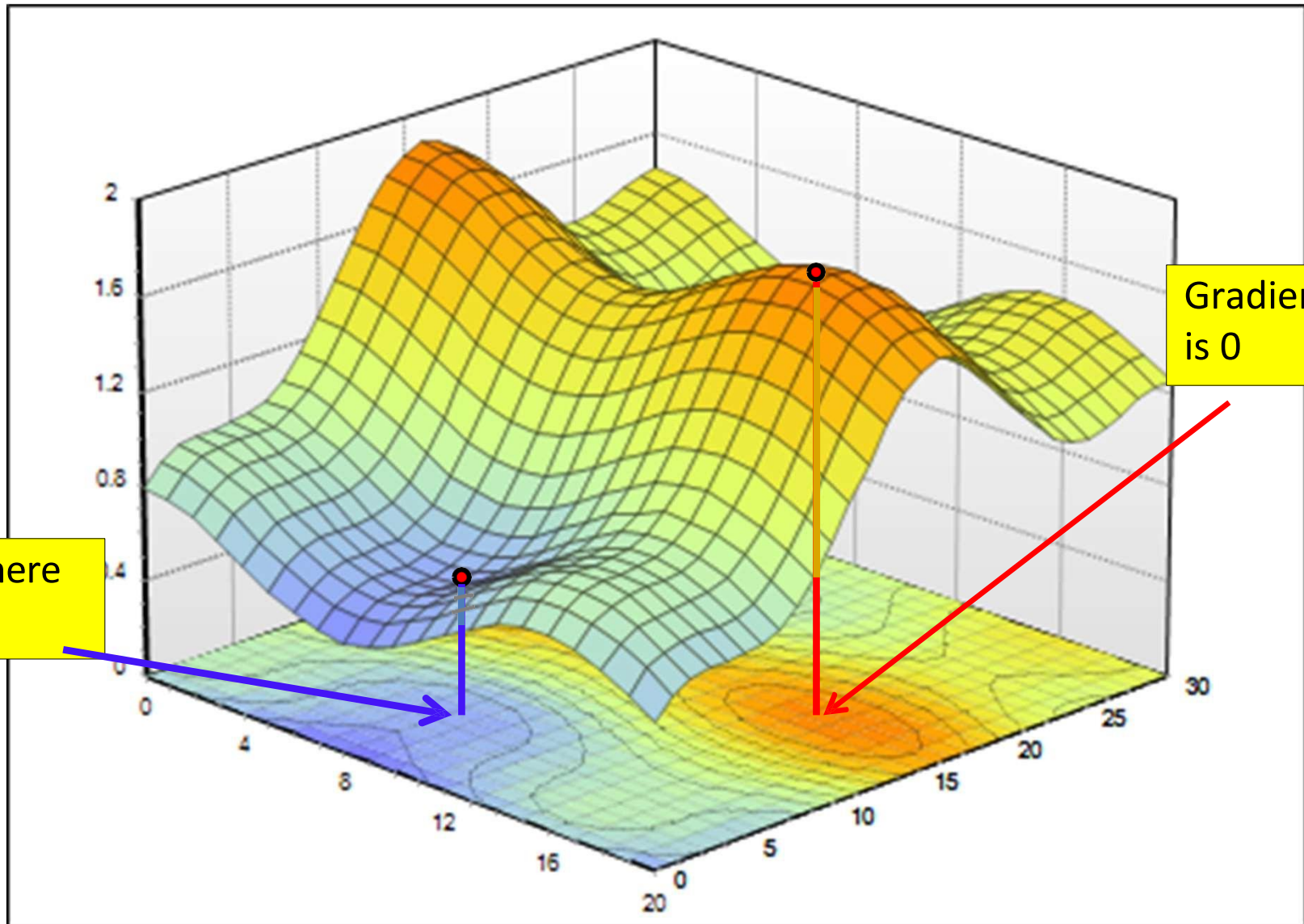
$$\nabla_X f(X) = \left[\frac{\partial f(X)}{\partial x_1} \quad \frac{\partial f(X)}{\partial x_2} \quad \dots \quad \frac{\partial f(X)}{\partial x_n} \right]$$

- The **gradient** is the transpose of the derivative $\nabla_X f(X)^T$
 - A column vector of the same dimensionality as X

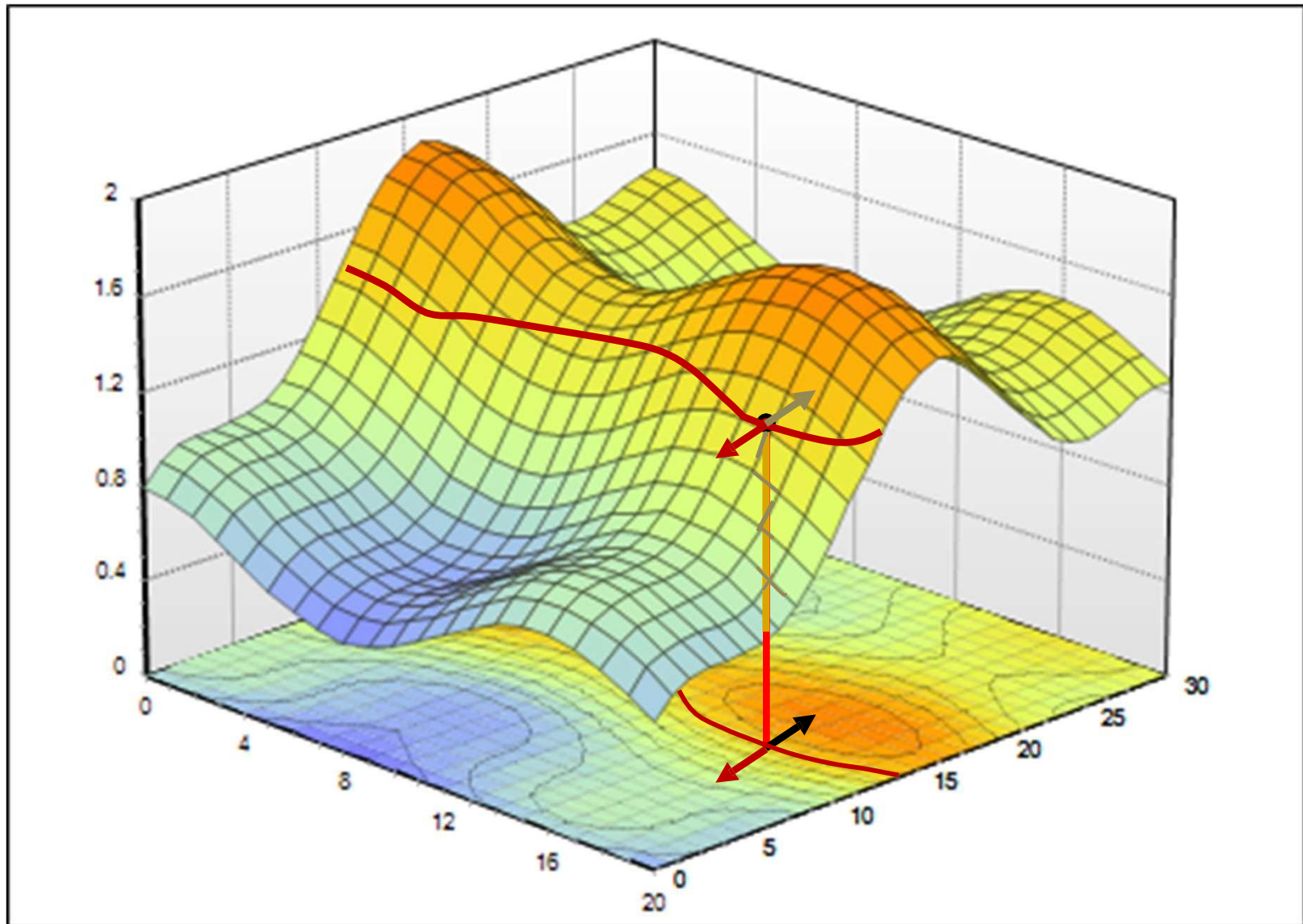
Gradient



Gradient



Properties of Gradient: 2



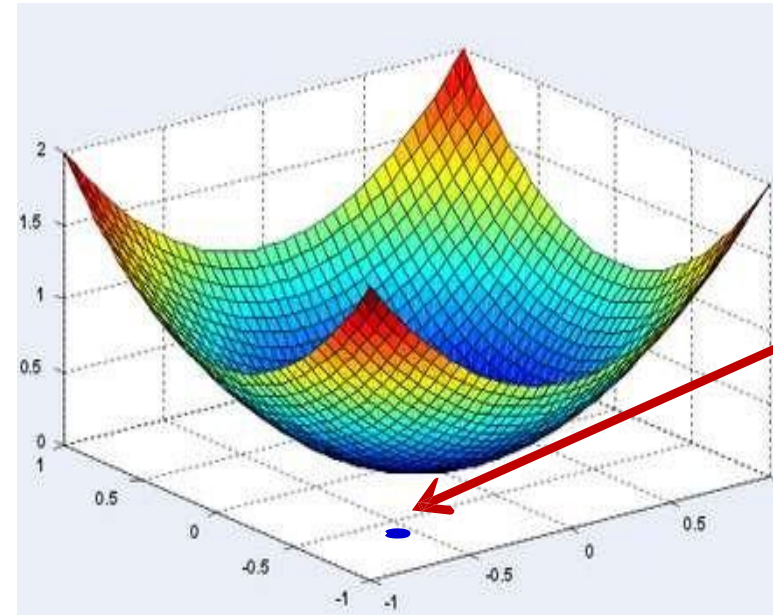
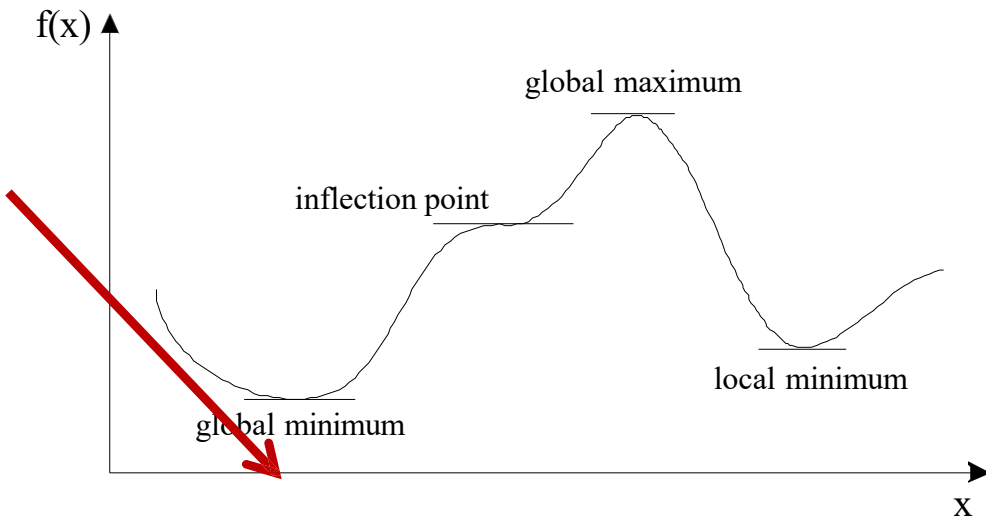
- The gradient vector $\nabla_x f(X)^T$ is perpendicular to the level curve

The Hessian

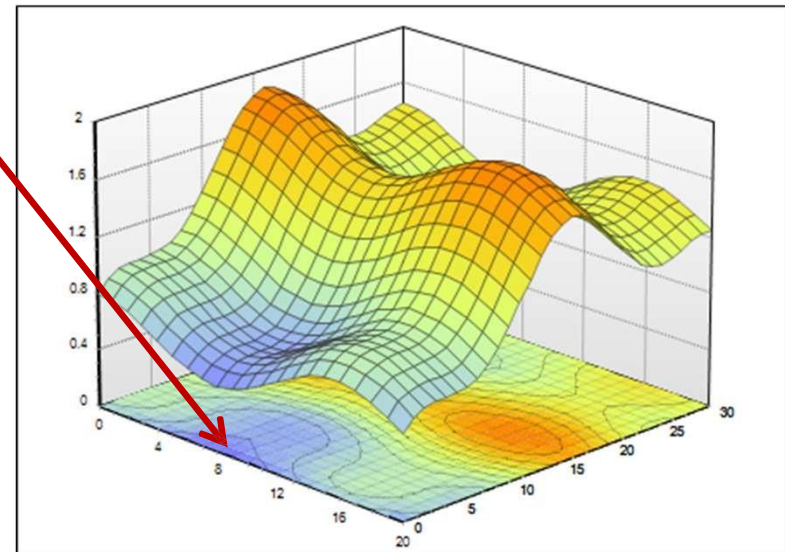
- The Hessian of a function $f(x_1, x_2, \dots, x_n)$ is given by the second derivative

$$\nabla_x^2 f(x_1, \dots, x_n) := \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdot & \cdot & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdot & \cdot & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdot & \cdot & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

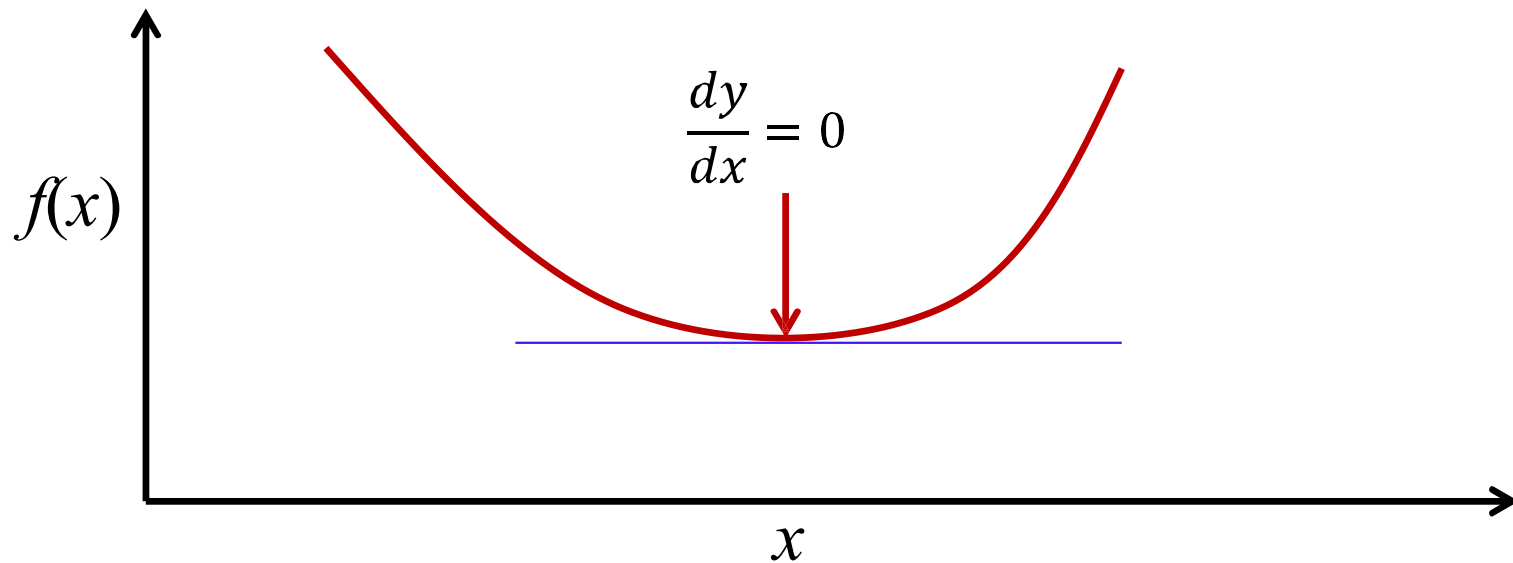
The problem of optimization



- General problem of optimization: Given a function $f(x)$ of some variable x ...
- Find the value of x where $f(x)$ is minimum



Finding the minimum of a function

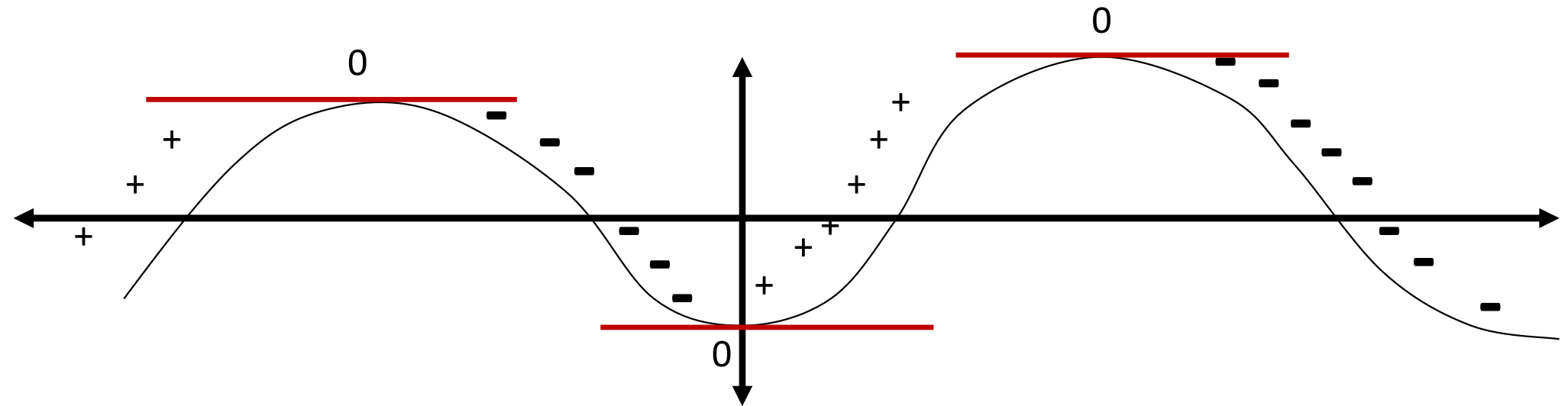


- Find the value x at which $f'(x) = 0$
 - Solve

$$\frac{df(x)}{dx} = 0$$

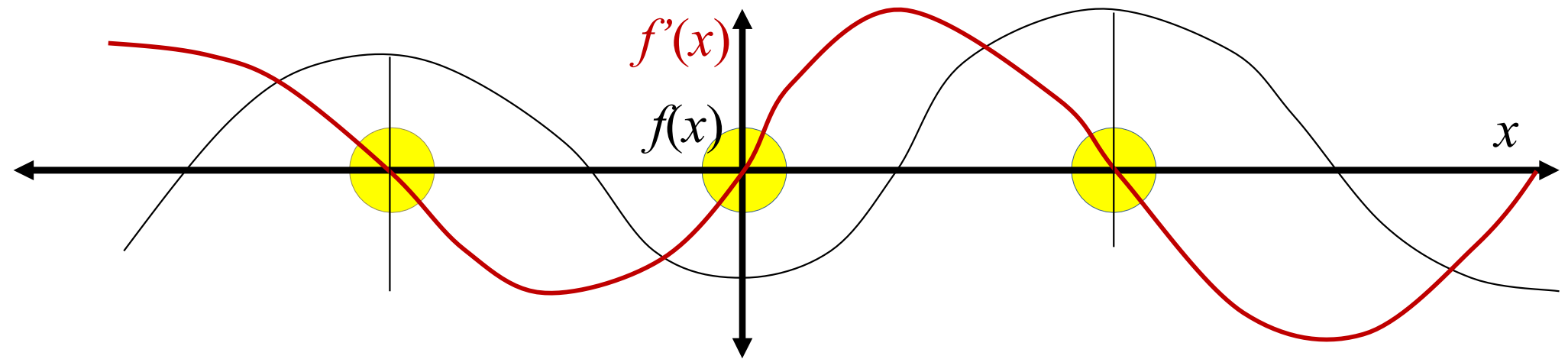
- The solution is a “turning point”
 - Derivatives go from positive to negative or vice versa at this point
- But is it a minimum?

Turning Points



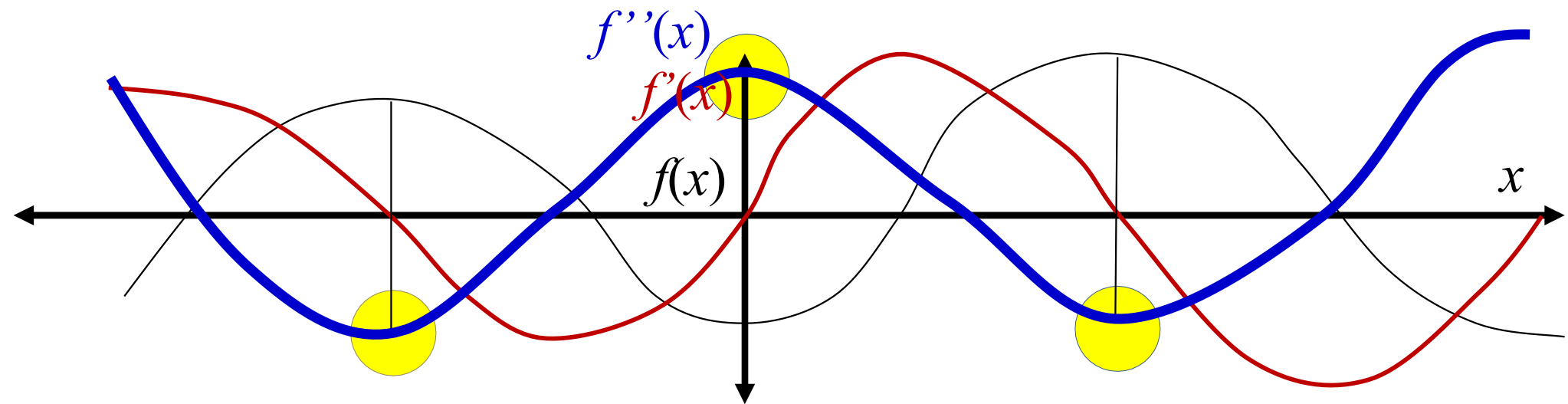
- Both *maxima* and *minima* have zero derivative
- Both are turning points

Derivatives of a curve



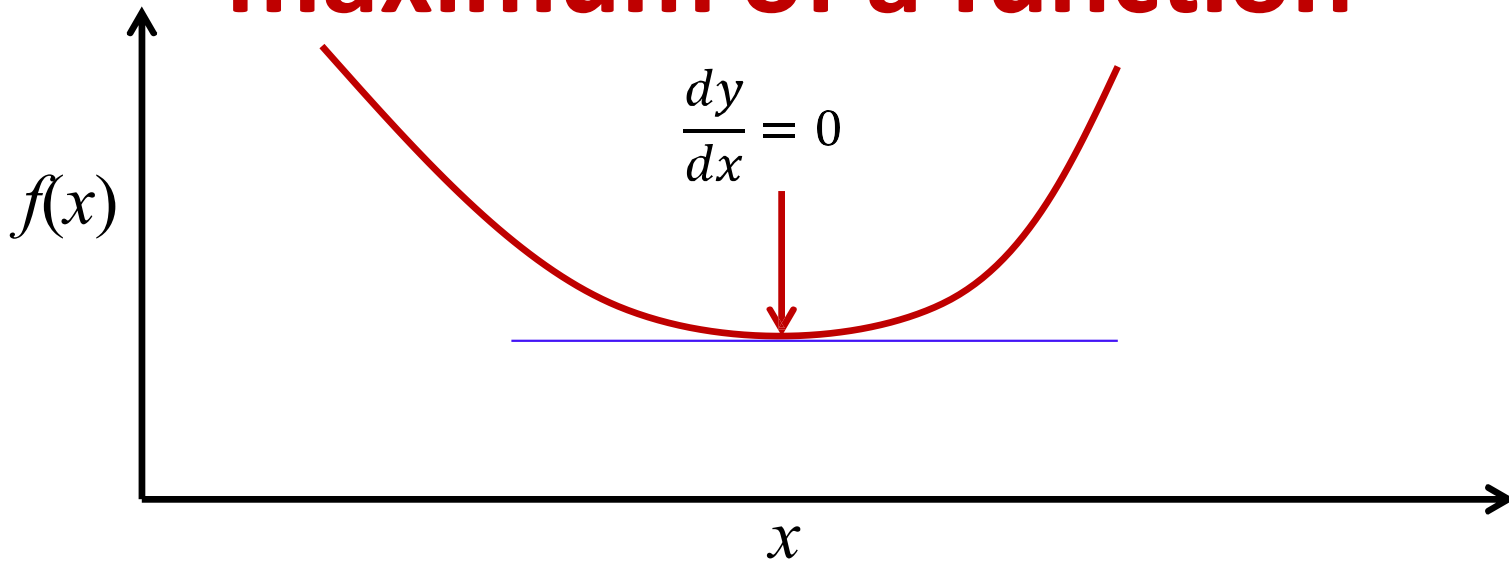
- Both *maxima* and *minima* are turning points
- Both *maxima* and *minima* have **zero derivative**

Derivative of the derivative of the curve



- Both *maxima* and *minima* are turning points
- Both *maxima* and *minima* have zero derivative
- The *second derivative* $f''(x)$ is –ve at maxima and +ve at minima!

Solution: Finding the minimum or maximum of a function



- Find the value x at which $f'(x) = 0$: Solve

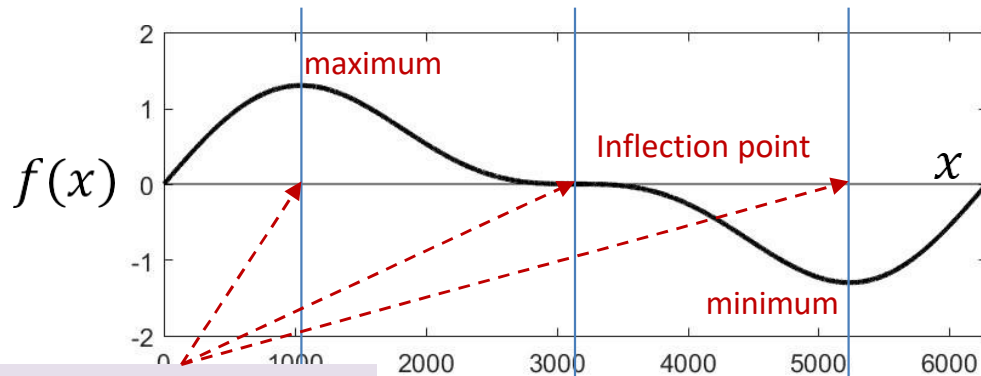
$$\frac{df(x)}{dx} = 0$$

- The solution x_{soln} is a **turning point**
- Check the double derivative at x_{soln} : compute

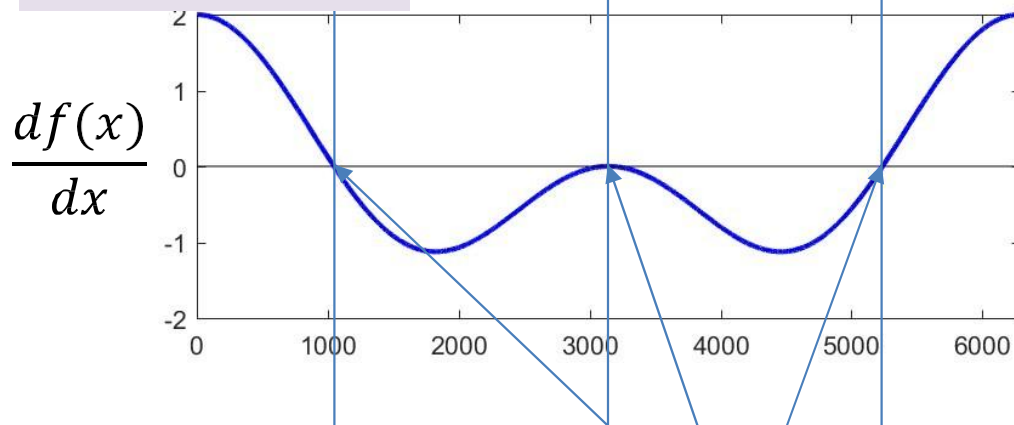
$$f''(x_{\text{soln}}) = \frac{df'(x_{\text{soln}})}{dx}$$

- If $f''(x_{\text{soln}})$ is positive x_{soln} is a minimum
 - If it is negative, it is a maximum

A note on derivatives of functions of single variable



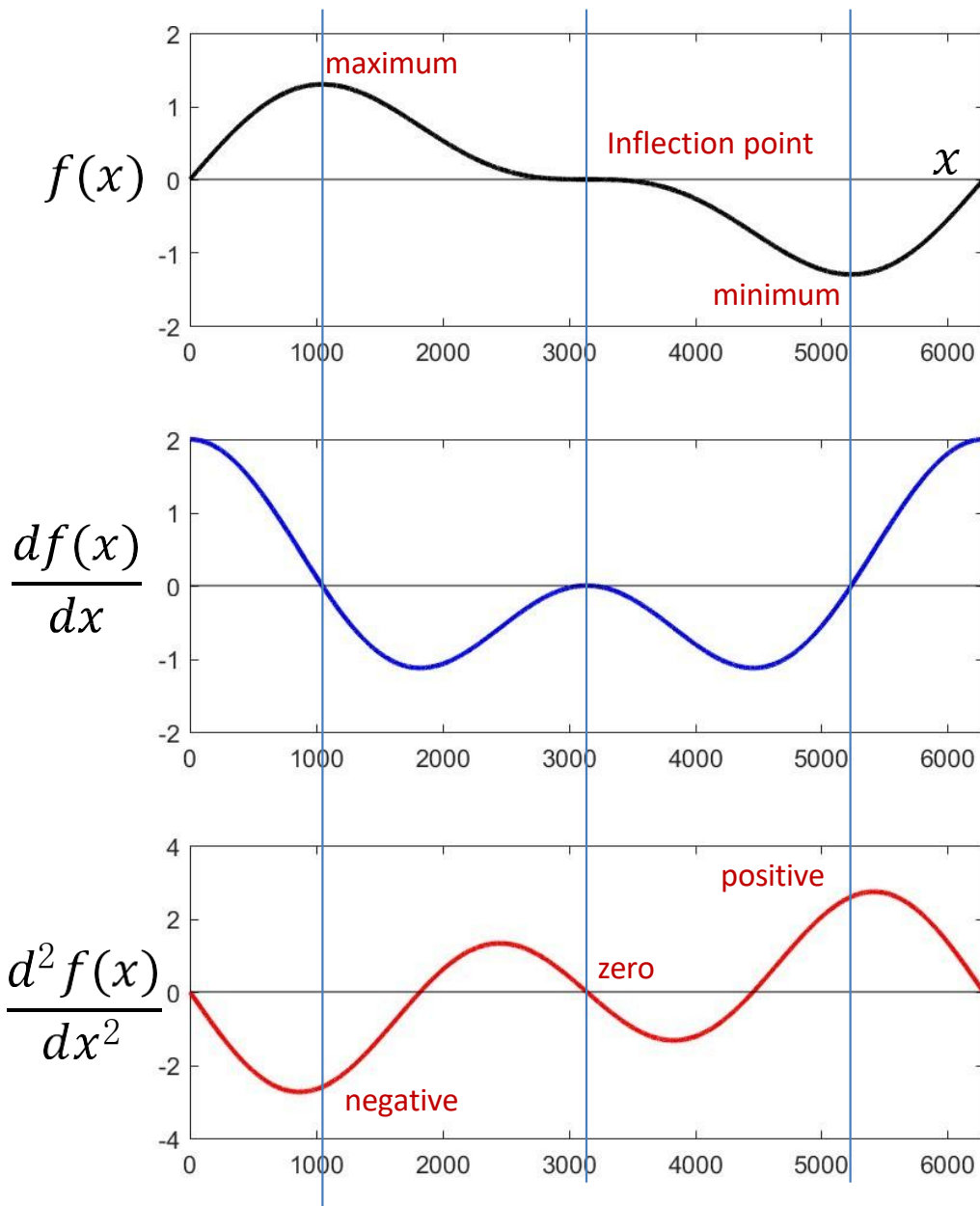
Critical points



Derivative is 0

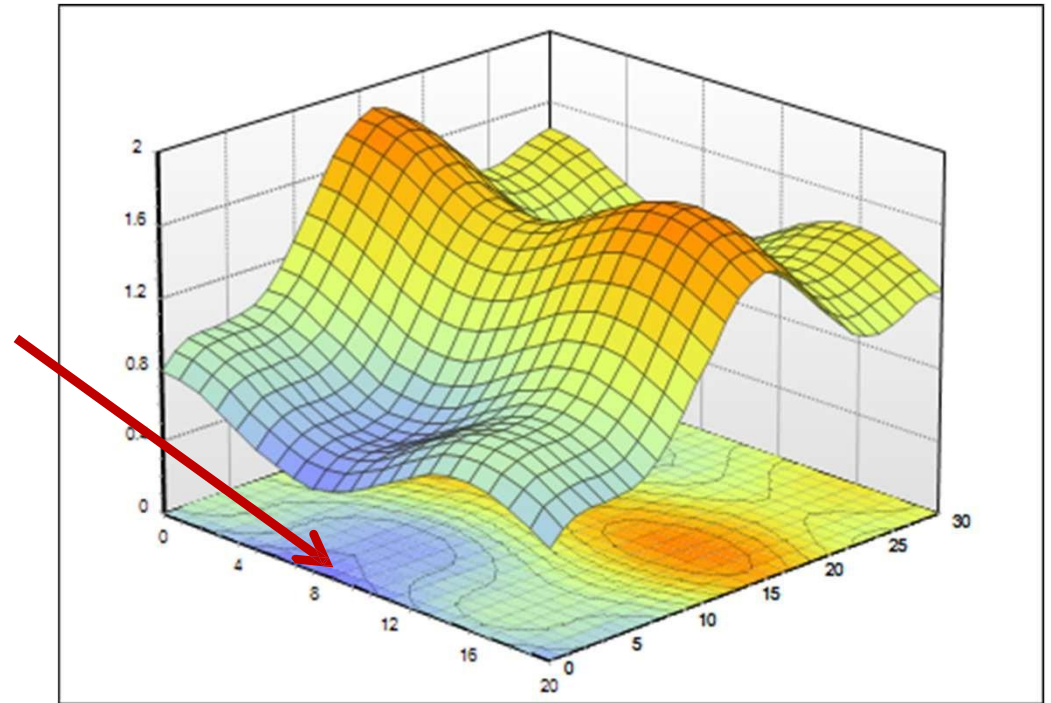
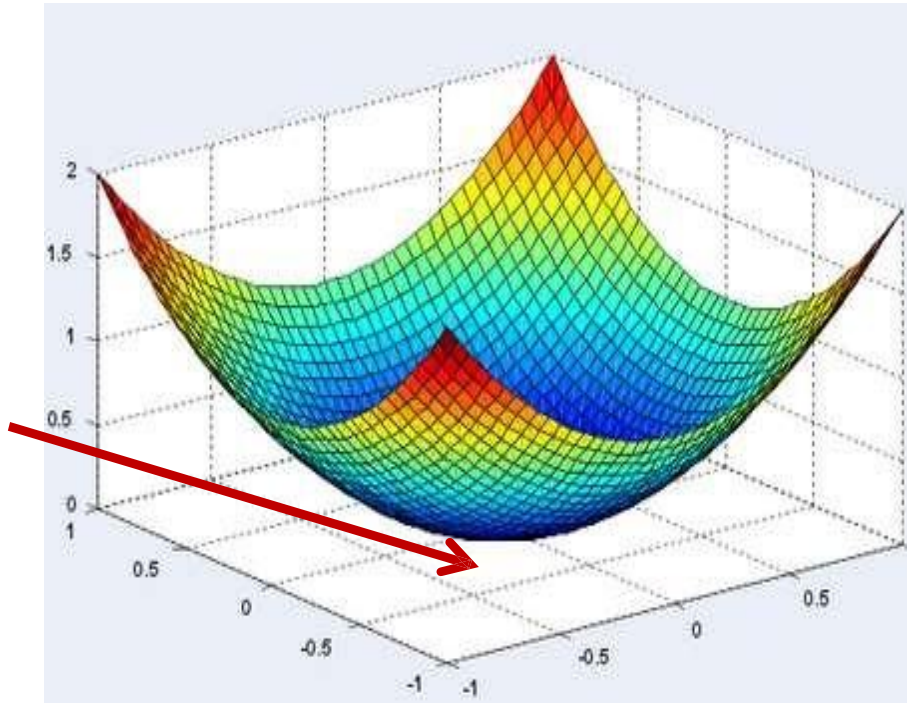
- All locations with zero derivative are *critical* points
 - These can be local maxima, local minima, or inflection points
- The *second* derivative is
 - Positive (or 0) at minima
 - Negative (or 0) at maxima
 - Zero at inflection points

A note on derivatives of functions of single variable



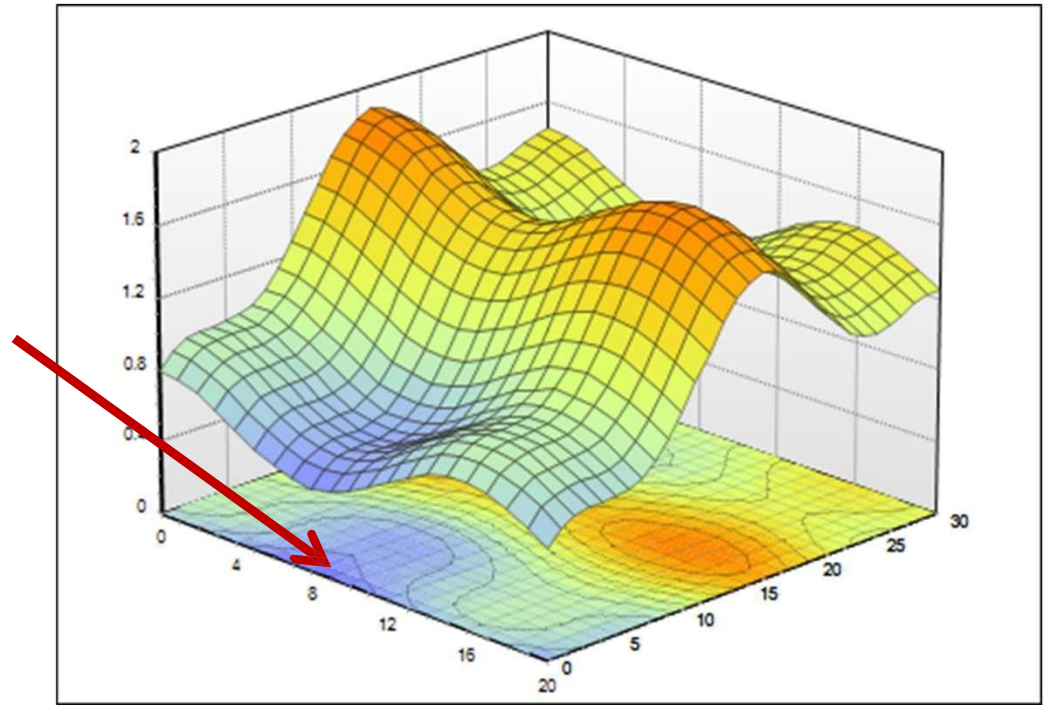
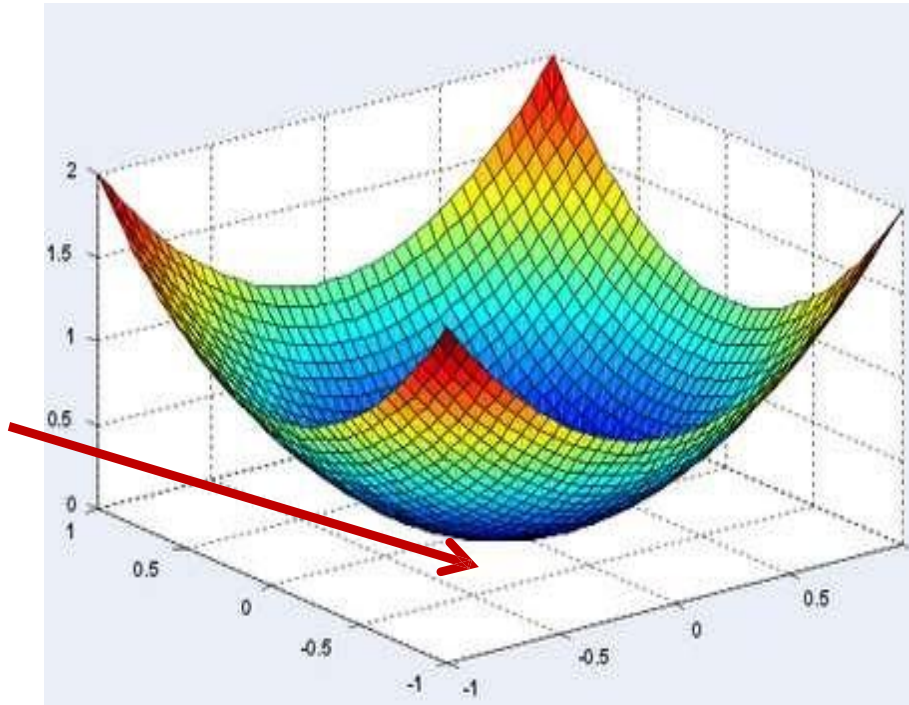
- All locations with zero derivative are *critical* points
 - These can be local maxima, local minima, or inflection points
- The *second* derivative is
 - ≥ 0 at minima
 - ≤ 0 at maxima
 - Zero at inflection points
- It's a little more complicated for functions of multiple variables..

What about functions of multiple variables?



- The optimum point is still “turning” point
 - Shifting in any direction will increase the value
 - For smooth functions, at the minimum/maximum, the gradient is 0
 - Really tiny shifts will not change the function value

Finding the minimum of a scalar function of a multivariate input



- The optimum point is a turning point – the gradient will be 0
- Find the location where the gradient is 0

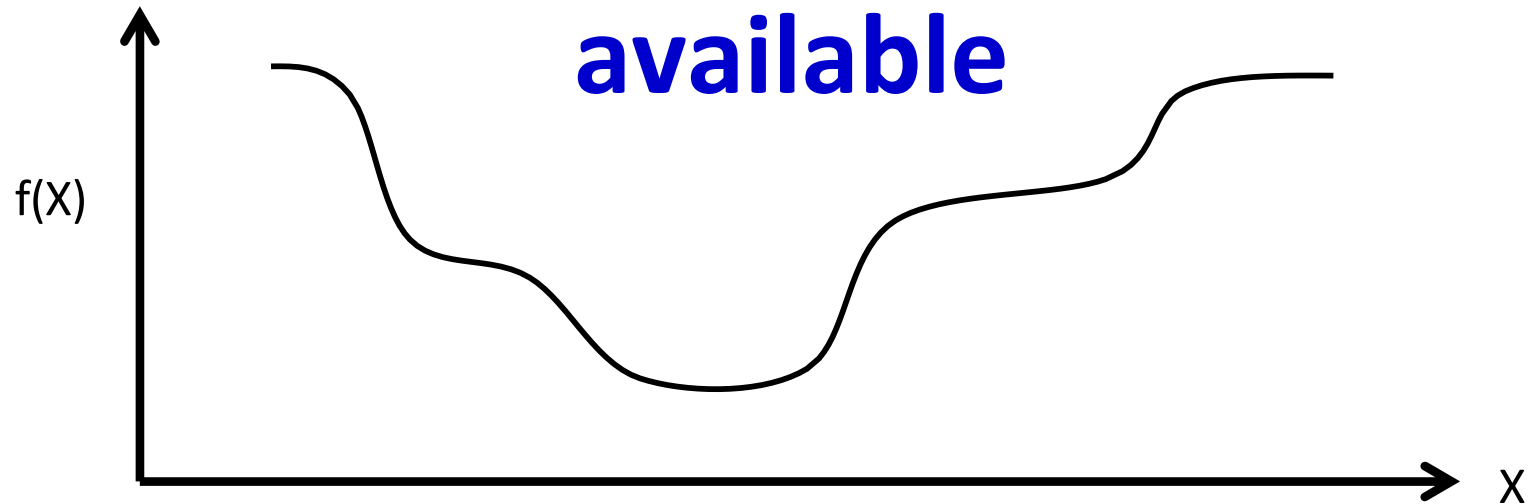
Unconstrained Minimization of function (Multivariate)

1. Solve for the X where the derivative (or gradient) equals to zero

$$A_X f(X) = 0$$

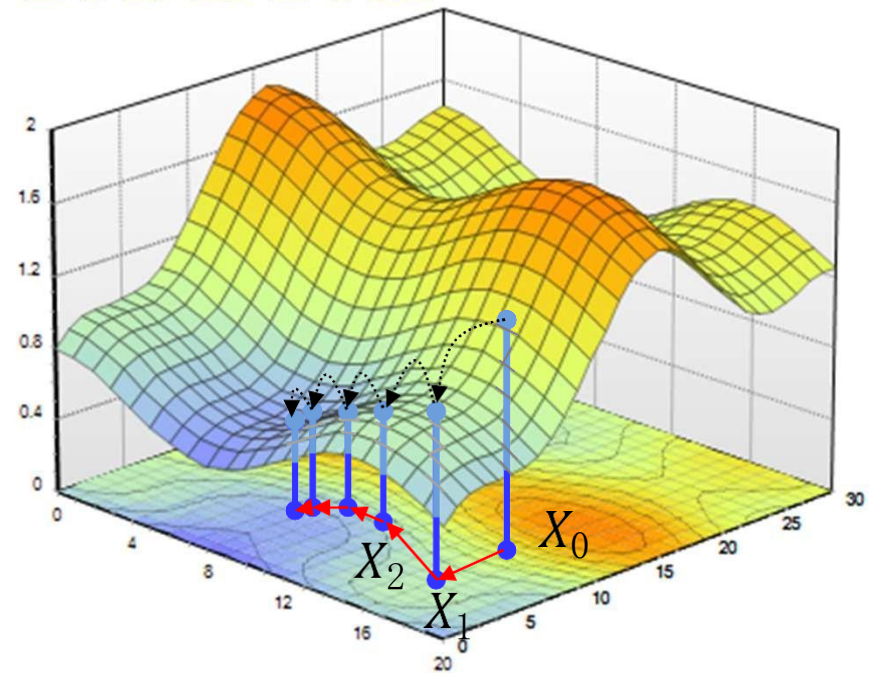
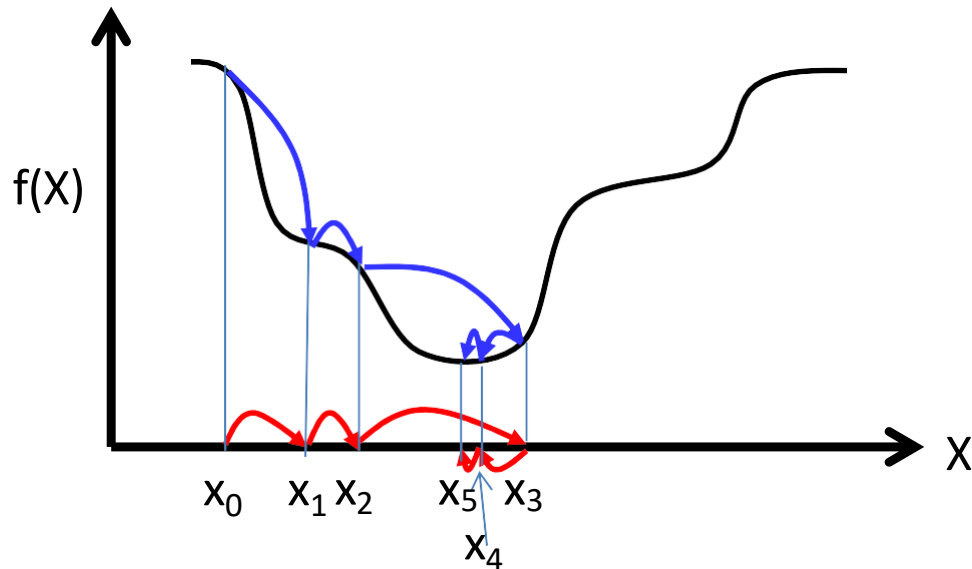
2. Compute the Hessian Matrix $\nabla_X^2 f(X)$ at the candidate solution and verify that
 - Hessian is positive definite (eigenvalues positive) -> to identify local minima
 - Hessian is negative definite (eigenvalues negative) -> to identify local maxima

Closed Form Solutions are not always available



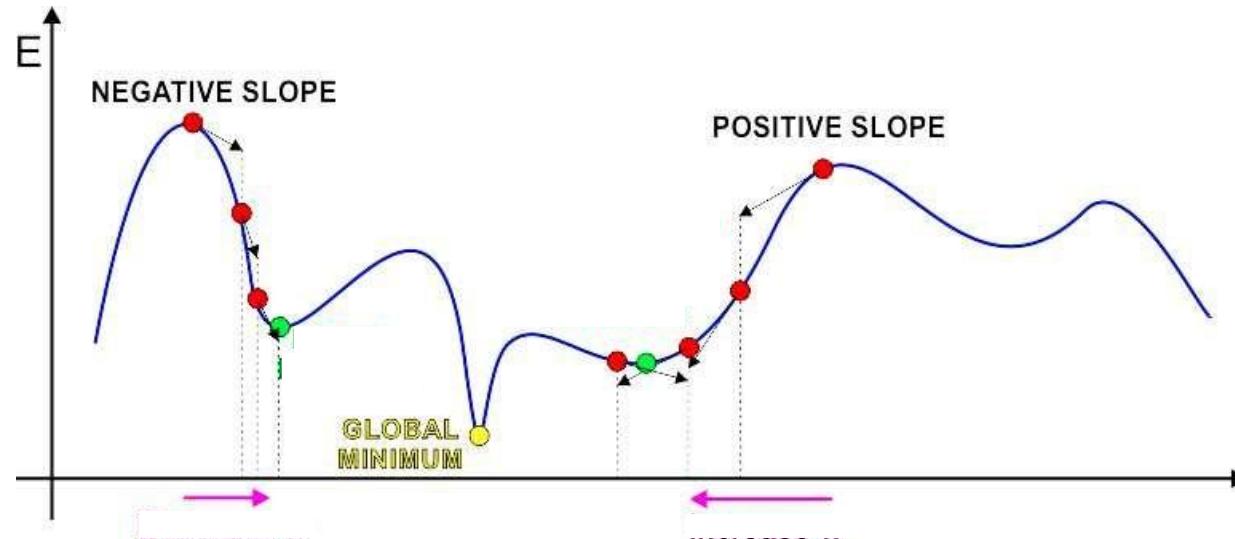
- Often it is not possible to simply solve $\nabla_x f(X) = 0$
 - The function to minimize/maximize may have an intractable form
- In these situations, iterative solutions are used
 - Begin with a “guess” for the optimal X and refine it iteratively until the correct value is obtained

Iterative solutions



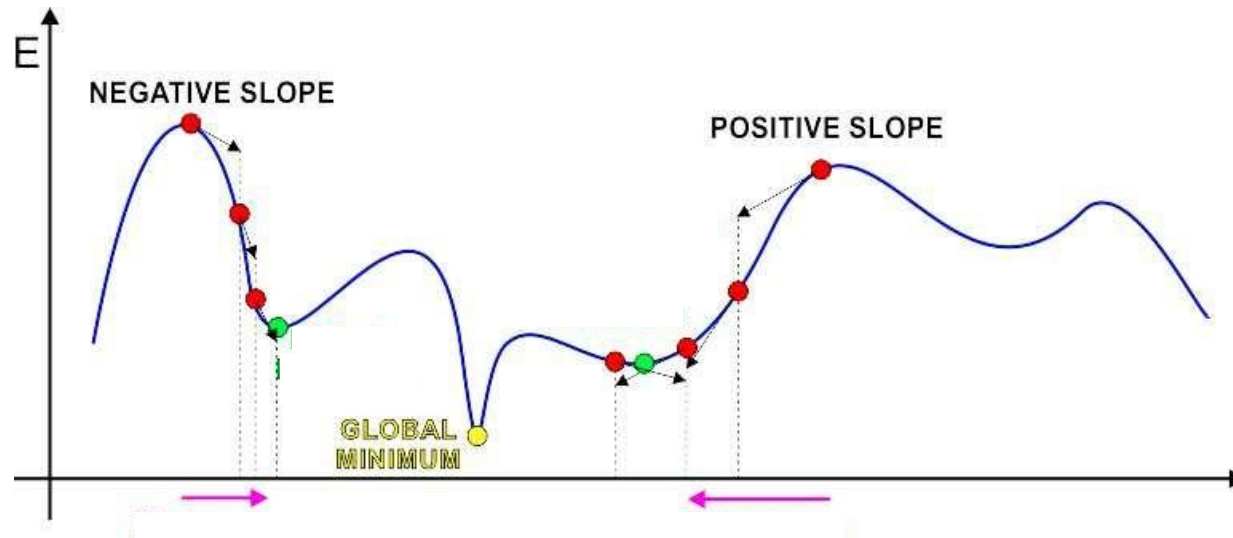
- Iterative solutions
 - Start from an initial guess X_0 for the optimal X
 - Update the guess towards a (hopefully) “better” value of $f(X)$
 - Stop when $f(X)$ no longer decreases
- Problems:
 - Which direction to step in
 - How big must the steps be

The Approach of Gradient Descent



- Iterative solution:
 - Start at some point
 - Find direction in which to shift this point to decrease error
 - This can be found from the derivative of the function
 - A negative derivative $\rightarrow$ moving right decreases error
 - A positive derivative $\rightarrow$ moving left decreases error
 - Shift point in this direction

The Approach of Gradient Descent



- Iterative solution: Trivial algorithm

- Initialize x^0

- While $f'(x^k) \neq 0$

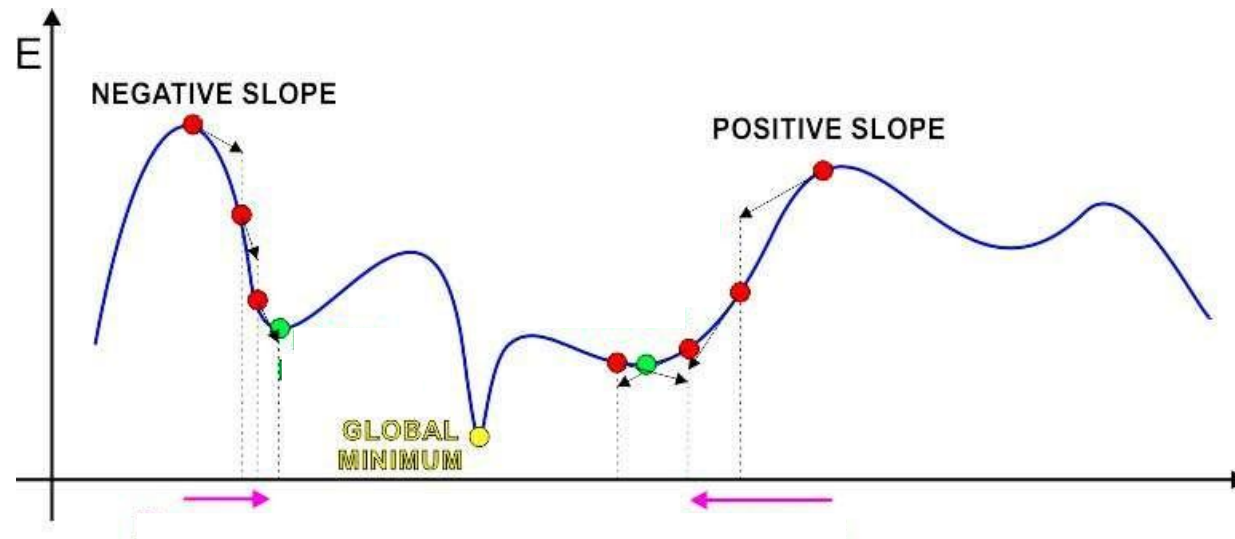
- If $\text{sign}(f'(x^k))$ is positive:

$$x^{k+1} = x^k - \text{step}$$

- Else

$$x^{k+1} = x^k + \text{step}$$

The Approach of Gradient Descent



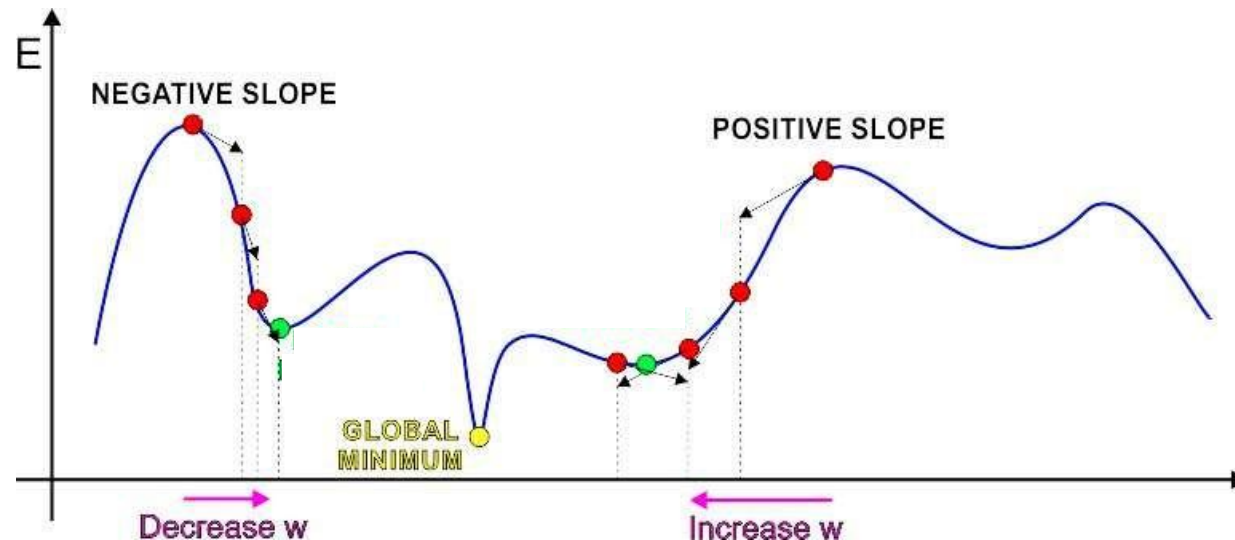
- Iterative solution: Trivial algorithm

- Initialize x^0
- While $f'(x^k) \neq 0$

$$x^{k+1} = x^k - \text{sign}(f'(x^k)) \cdot \text{step}$$

- Identical to previous algorithm

The Approach of Gradient Descent



- Iterative solution: Trivial algorithm

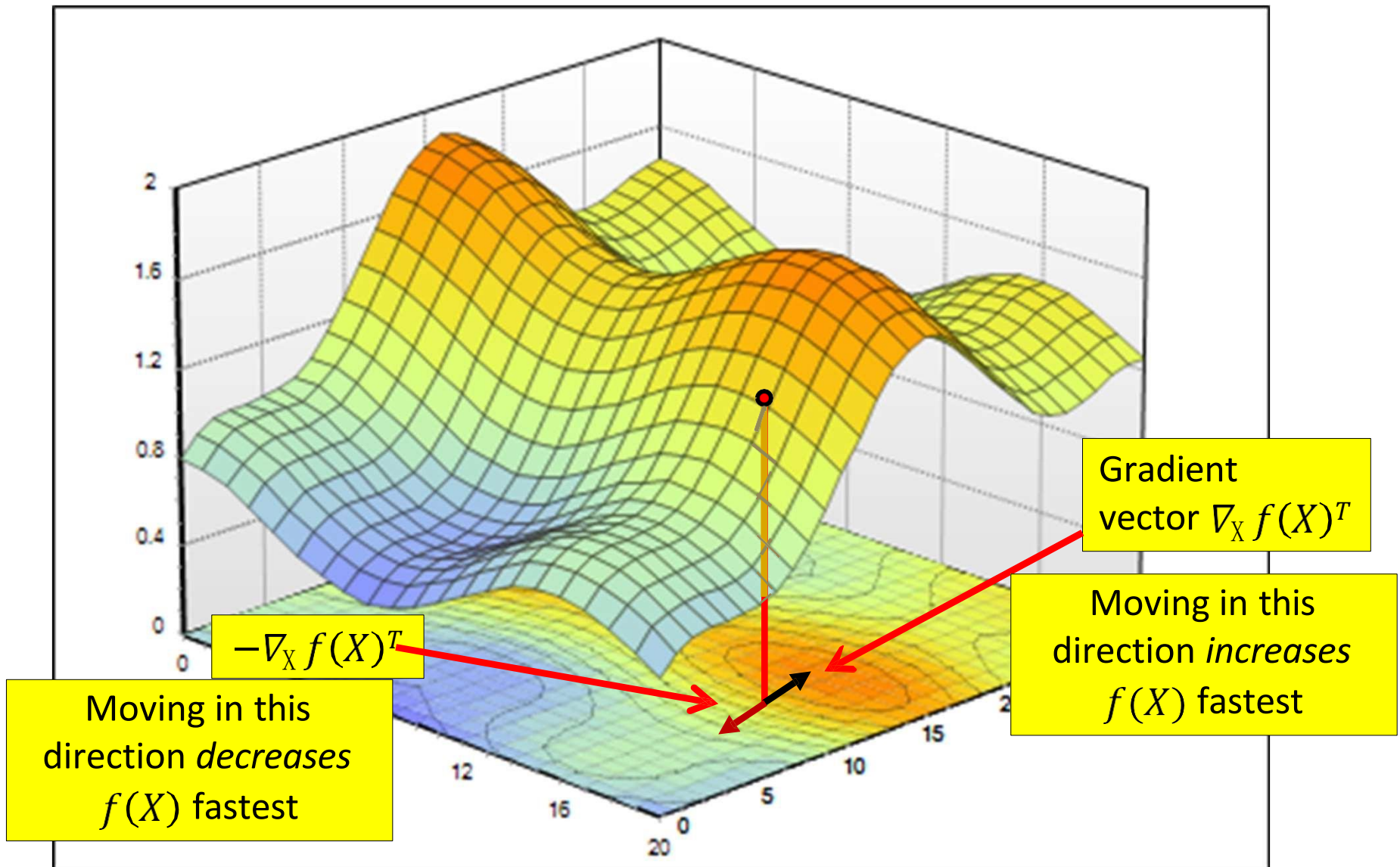
- Initialize x^0

- While $f'(x^k) \neq 0$

$$x^{k+1} = x^k - \eta^k f'(x^k)$$

- η^k is the “step size”

Gradients of multivariate functions



Gradient descent/ascent (**multivariate**)

- The gradient descent/ascent method to find the minimum or maximum of a function f iteratively
 - To find a *maximum* move *in the direction of the gradient*

$$x^{k+1} = x^k + \eta^k \nabla_x f(x^k)^T$$

- To find a *minimum* move *exactly opposite the direction of the gradient*

$$x^{k+1} = x^k - \eta^k \nabla_x f(x^k)^T$$

- Many solutions to choosing step size η^k

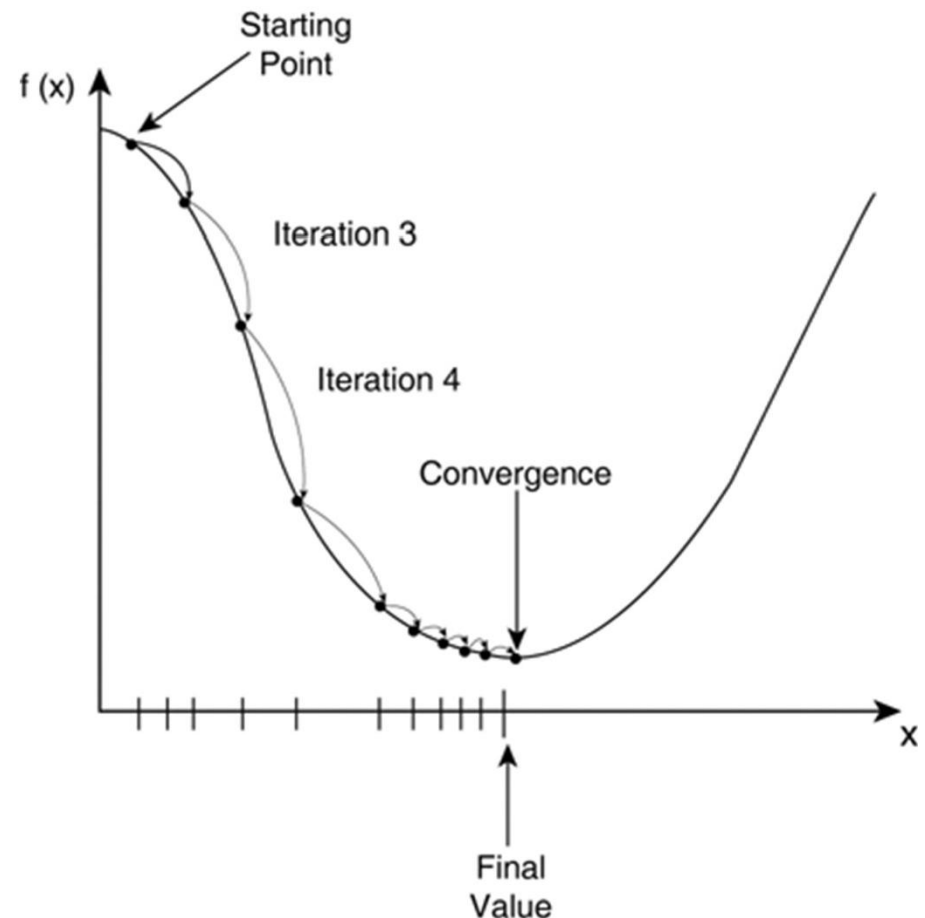
Gradient descent convergence criteria

- The gradient descent algorithm converges when one of the following criteria is satisfied

$$|f(x^{k+1}) - f(x^k)| < s_1$$

- Or

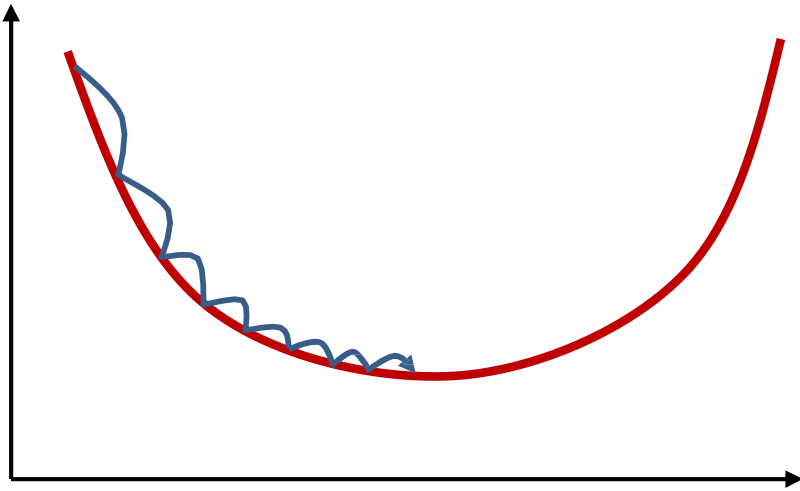
$$\|A_x f(x^k)\| < s_2$$



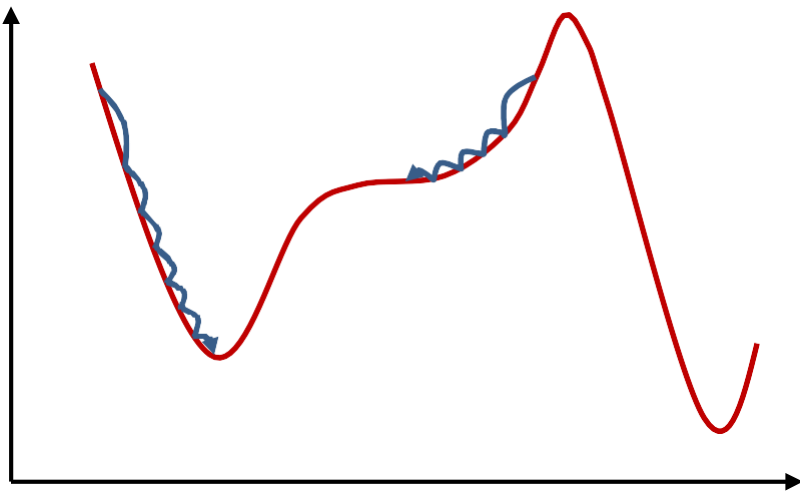
Overall Gradient Descent Algorithm

- Initialize:
 - x^0
 - $k = 0$
- do
 - $x^{k+1} = x^k - \eta^k \nabla_x f(x^k)^T$
 - $k = k + 1$
- while $|f(x^{k+1}) - f(x^k)| > \varepsilon$

Convergence of Gradient Descent



- For appropriate step size, for convex (bowl-shaped) functions gradient descent will always find the minimum.



- For non-convex functions it will find a local minimum or an inflection point

Problem Statement

- Given a training set of input-output pairs $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
- Minimize the following function

$$Loss(W) = \frac{1}{T} \sum_i div(f(X_i; W), d_i)$$

w.r.t. W

- This is problem of function minimization
 - An instance of optimization

Gradient Descent to train a network

- Initialize:

- W^0

- $k = 0$

do

- $W^{k+1} = W^k - \eta^k \nabla \text{Loss}(W^k)^T$

- $k = k + 1$

while $|\text{Loss}(W^k) - \text{Loss}(W^{k-1})| > \varepsilon$

Preliminaries

- Before we proceed: the problem setup

Problem Setup: Things to define

- Given a training set of input-output pairs $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
- Minimize the following function

$$Loss(W) = \frac{1}{T} \sum_i div(f(X_i; W), d_i)$$

w.r.t W

Problem Setup: Things to define

- Given a training set of input-output pairs $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$

- What are these input-output pairs?

$$Loss(W) = \frac{1}{T} \sum_i div(f(X_i; W), d_i)$$

Problem Setup: Things to define

- Given a training set of input-output pairs $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$

- What are these input-output pairs?

$$Loss(W) = \frac{1}{T} \sum_i div(f(X_i; W), d_i)$$

What is $f()$ and what are its parameters W ?

Problem Setup: Things to define

- Given a training set of input-output pairs $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$

- What are these input-output pairs?

$$Loss(W) = \frac{1}{T} \sum_i div(f(X_i; W), d_i)$$

What is the divergence $div()$?

What is $f()$ and what are its parameters W ?

Problem Setup: Things to define

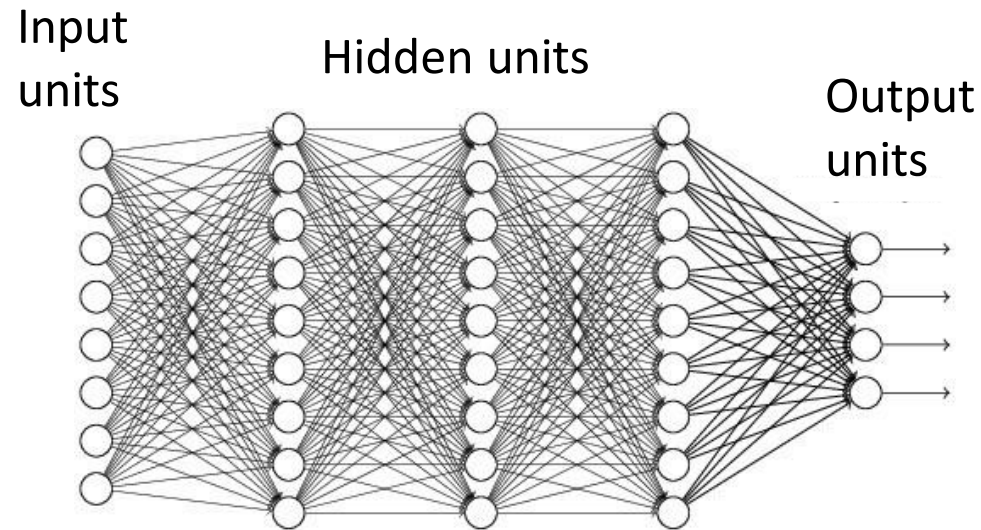
- Given a training set of input-output pairs $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
- Minimize the following function

$$Loss(W) = \frac{1}{T} \sum_i div(f(X_i; W), d_i)$$



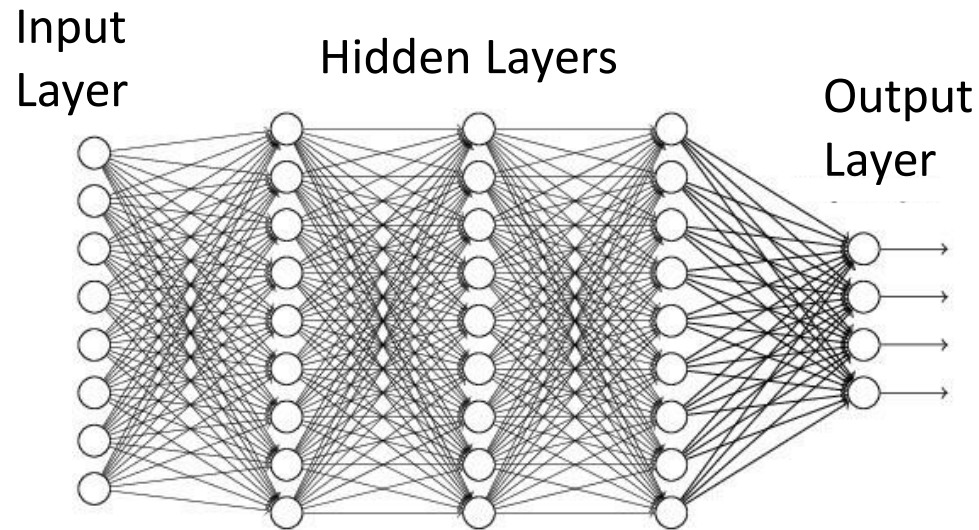
What is $f()$ and what are its parameters W ?

What is $f()$? Typical network



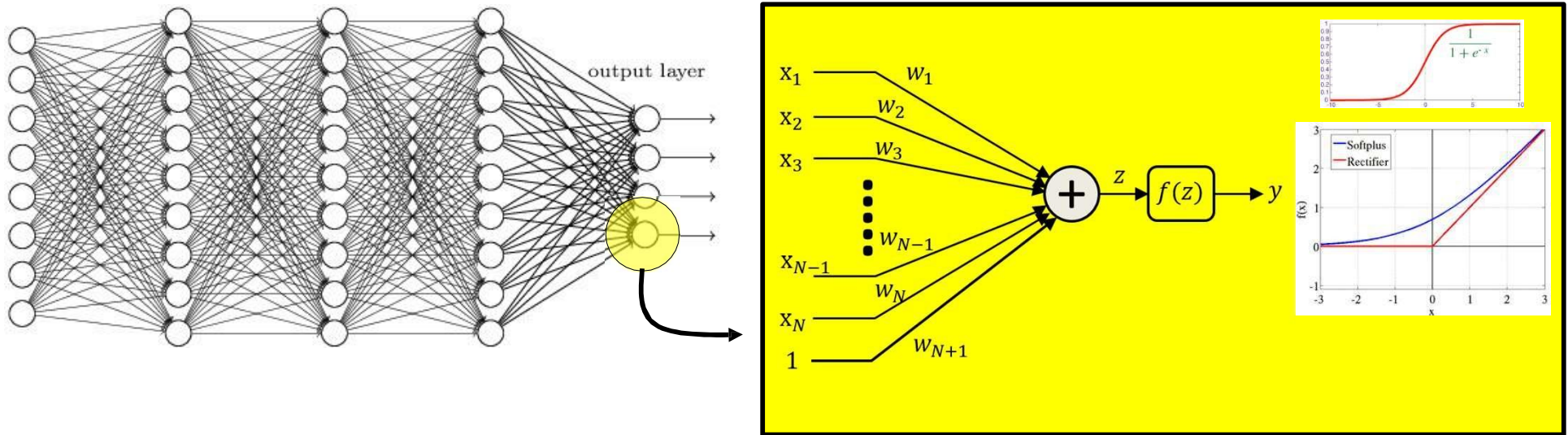
- Multi-layer perceptron
- *A directed* network with a set of inputs and outputs
 - No loops

Typical network



- We assume a “layered” network for simplicity
 - Each “layer” of neurons only gets inputs from the earlier layer(s) and outputs signals only to later layer(s)
 - We will refer to the inputs as the *input layer*
 - No neurons here – the “layer” simply refers to inputs
 - We refer to the outputs as the *output layer*
 - Intermediate layers are *“hidden” layers*

The individual neurons



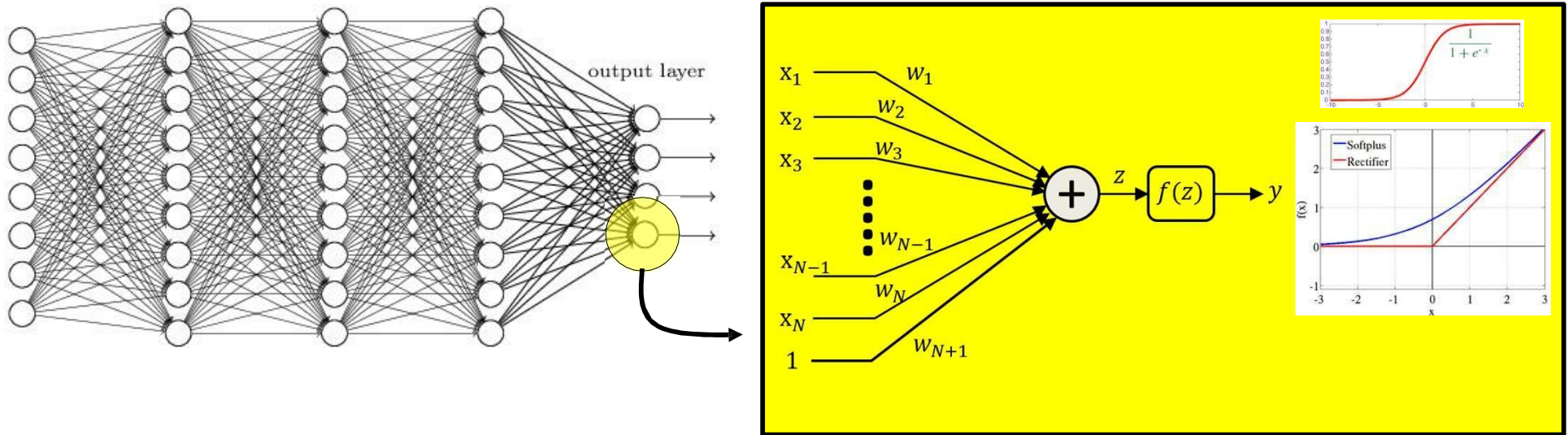
- Individual neurons operate on a set of inputs and produce a single output
 - **Standard setup:** A continuous activation function applied to an affine function of the inputs

$$y = f\left(\sum_i w_i x_i + b\right)$$

- More generally: *any* differentiable function

$$y = f(x_1, x_2, \dots, x_N; W)$$

The individual neurons



- Individual neurons operate on a set of inputs and produce a single output
 - **Standard setup:** A continuous activation function applied to an affine function of the inputs

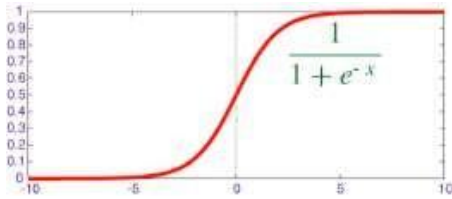
$$y = f\left(\sum_i w_i x_i + b\right)$$

- More generally: *any* differentiable function
$$y = f(x_1, x_2, \dots, x_N; W)$$

We will assume this unless otherwise specified

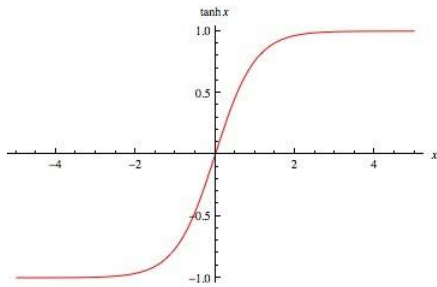
Parameters are weights w_i and bias b

Activations and their derivatives



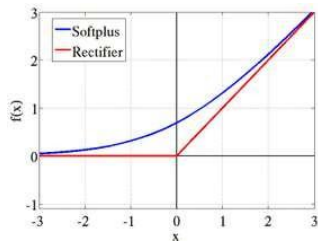
$$f(z) = \frac{1}{1 + \exp(-z)}$$

$$f'(z) = f(z)(1 - f(z))$$



$$f(z) = \tanh(z)$$

$$f'(z) = (1 - f^2(z))$$



$$f(z) = \begin{cases} z, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

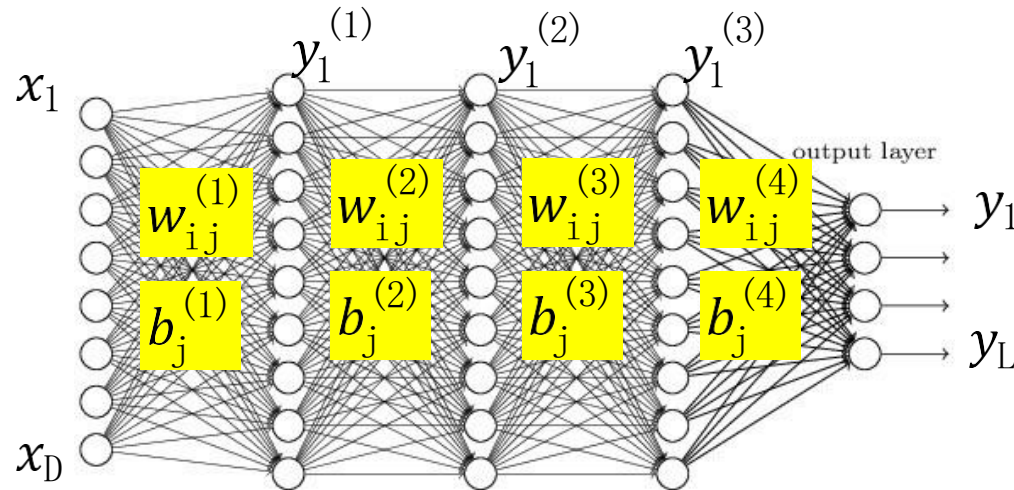
$$[*] \quad f'(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

$$f(z) = \log(1 + \exp(z))$$

$$f'(z) = \frac{1}{1 + \exp(-z)}$$

- Some popular activation functions and their derivatives

Notation



- The input layer is the 0th layer
- We will represent the output of the i -th perceptron of the k th layer as $y_i^{(k)}$
 - **Input to network:** $y_i^{(0)} = x_i$
 - **Output of network:** $y_i = y_i^{(N)}$
- We will represent the weight of the connection between the i -th unit of the $k-1$ th layer and the j th unit of the k -th layer as $w_{ij}^{(k)}$
 - The bias to the j th unit of the k -th layer is $b_j^{(k)}$

Problem Setup: Things to define

- Given a training set of input-output pairs $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
- Minimize the following function

$$Loss(W) = \frac{1}{T} \sum div(f(X_i; W), d_i)$$



What is $f()$ and what are its parameters W ?

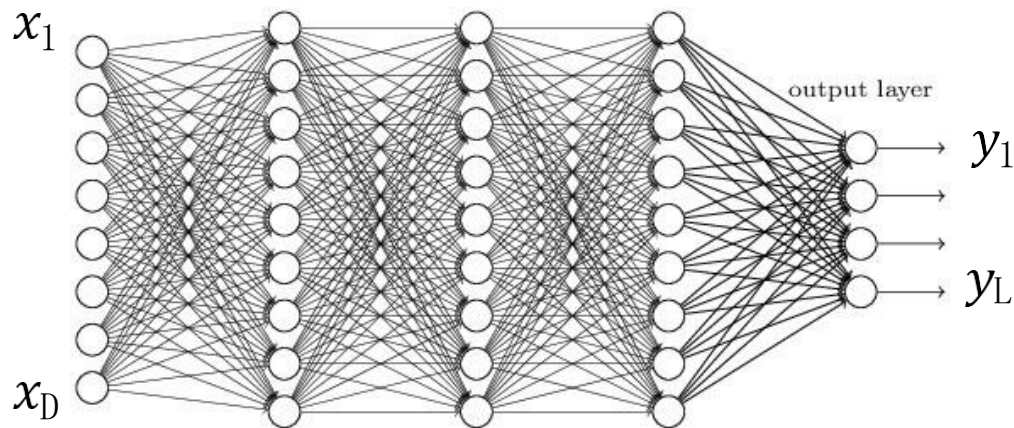
Problem Setup: Things to define

- Given a training set of input-output pairs $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$

- What are these input-output pairs?

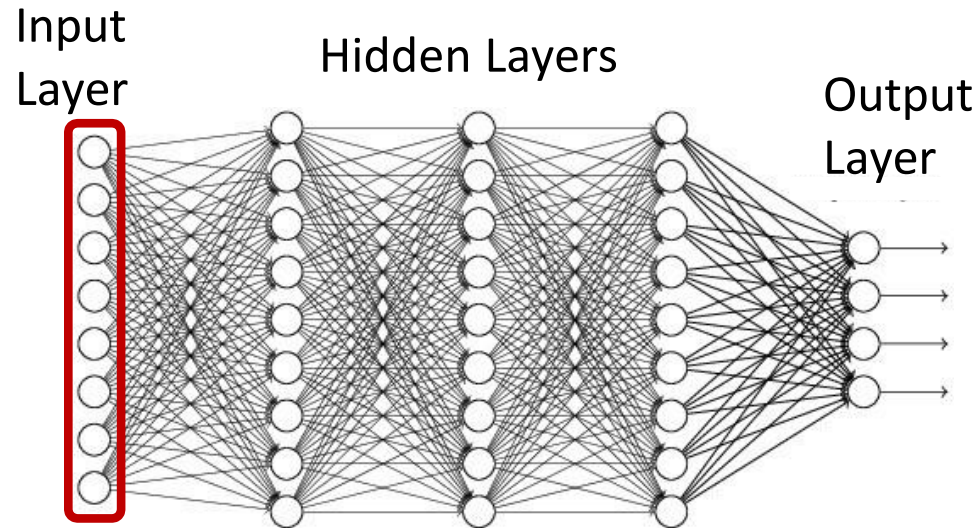
$$Loss(W) = \frac{1}{T} \sum_i div(f(X_i; W), d_i)$$

Input, target output, and actual output: Vector notation



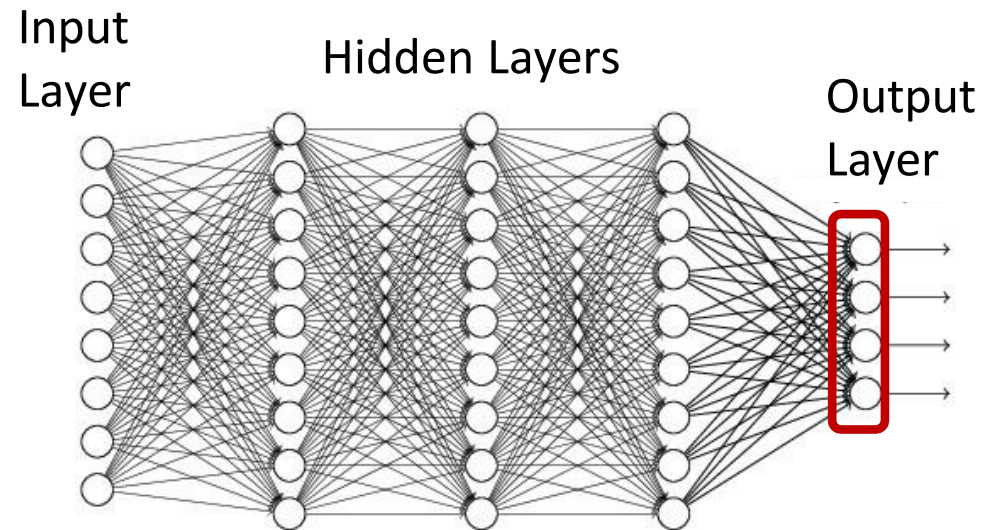
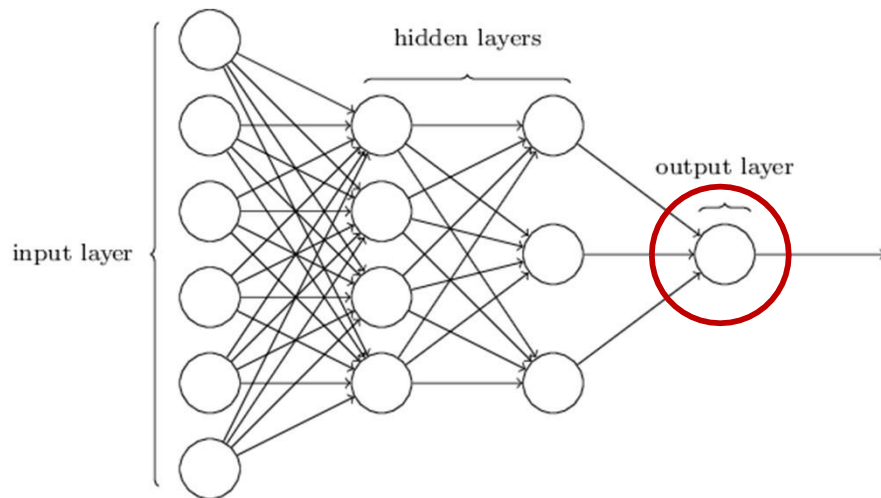
- Given a training set of input-output pairs $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
- $X_n = [x_{n1}, x_{n2}, \dots, x_{nD}]^T$ is the n th input vector
- $d_n = [d_{n1}, d_{n2}, \dots, d_{nL}]^T$ is the n th desired output
- $Y_n = [y_{n1}, y_{n2}, \dots, y_{nL}]^T$ is the n th vector of *actual* outputs of the network
 - Function of input X_n and network parameters
- We will sometimes drop the first subscript when referring to a *specific* instance

Representing the input



- Vectors of numbers
 - (or may even be just a scalar, if input layer is of size 1)
 - E.g. vector of pixel values
 - E.g. vector of speech features
 - E.g. real-valued vector representing text
 - We will see how this happens later in the course
 - Other real valued vectors

Representing the **output**



- If the desired *output* is **real-valued**, no special tricks are necessary
 - Scalar Output : single output neuron
 - $d = \text{scalar (real value)}$
 - Vector Output : as many output neurons as the dimension of the desired output
 - $d = [d_1 \ d_2 \ .. \ d_L]$ (vector of real values)

Training Neural Nets through Gradient Descent

Total training Loss:

$$Loss = \frac{1}{T} \sum_t Div(\mathbf{Y}_t, \mathbf{d}_t)$$

- Gradient descent algorithm:
- Initialize all weights and biases $\{w_{ij}^{(k)}\}$
 - Using the extended notation: the bias is also a weight
- Do:
 - For every layer k for all i, j , update:

$$\bullet w_{i,j}^{(k)} = w_{i,j}^{(k)} - \eta \frac{dLoss}{dw_{i,j}^{(k)}}$$

- Until *Loss* has converged

Assuming the bias is also represented as a weight

Training Neural Nets through Gradient Descent

Total training Loss:

$$Loss = \frac{1}{T} \sum_t Div(\mathbf{Y}_t, \mathbf{d}_t)$$

- Gradient descent algorithm:
- Initialize all weights and biases $\{w_{ij}^{(k)}\}$
 - Using the extended notation: the bias is also a weight
- Do:
 - For every layer k for all i, j , update:

$$\bullet w_{i,j}^{(k)} = w_{i,j}^{(k)} - \eta \frac{dLoss}{dw_{i,j}^{(k)}}$$

- Until *Loss* has converged

Assuming the bias is also represented as a weight

The derivative

Total training Loss:

$$Loss = \frac{1}{T} \sum_t Div(\mathbf{Y}_t, \mathbf{d}_t)$$

- Computing the derivative

Total derivative:

$$\frac{dLoss}{dw_{i,j}^{(k)}} = \frac{1}{T} \sum_t \frac{dDiv(\mathbf{Y}_t, \mathbf{d}_t)}{dw_{i,j}^{(k)}}$$

The derivative

Total training Loss:

$$Loss = \frac{1}{T} \sum_t Div(\mathbf{Y}_t, \mathbf{d}_t)$$

- Computing the derivative

Total derivative:

$$\frac{dLoss}{dw_{i,j}^{(k)}} = \frac{1}{T} \sum_t \frac{dDiv(\mathbf{Y}_t, \mathbf{d}_t)}{dw_{i,j}^{(k)}}$$

- So we must first figure out how to compute the derivative of divergences of individual training inputs

Calculus Refresher: Basic rules of calculus

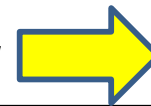
For any differentiable function

$y = f(x)$
with derivative

$$\frac{dy}{dx}$$

the following must hold for sufficiently small

Δx



$$\Delta y \approx \frac{dy}{dx} \Delta x$$

Calculus Refresher: Basic rules of calculus

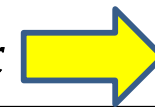
For any differentiable function

$y = f(x)$
with derivative

$$\frac{dy}{dx}$$

the following must hold for sufficiently small

Δx



$$\Delta y \approx \frac{dy}{dx} \Delta x$$

x



y

Introducing the
"influence" diagram:
x influences y

Calculus Refresher: Basic rules of calculus

For any differentiable function

$y = f(x)$
with derivative

$$\frac{dy}{dx}$$

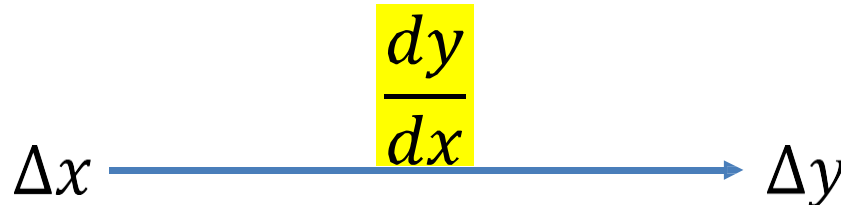
the following must hold for sufficiently small



$$\Delta y \approx \frac{dy}{dx} \Delta x$$



Introducing the
"influence" diagram:
 x influences y



The derivative graph:
The edge carries the
derivative.

Node and edge weights
multiply

Calculus Refresher: Basic rules of calculus

For any differentiable function

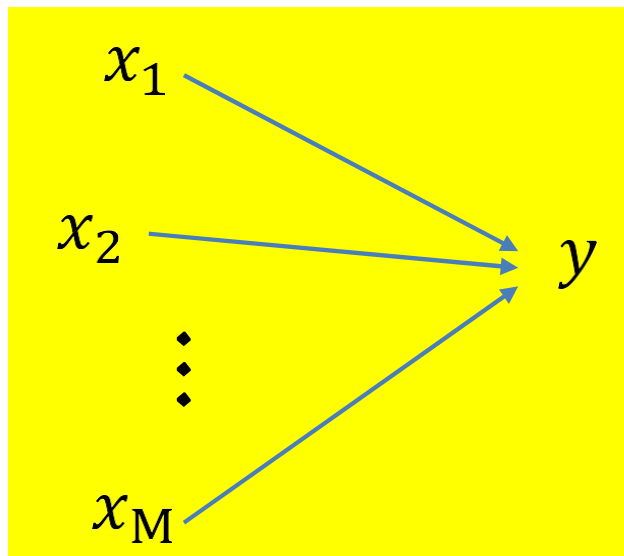
$$y = f(x_1, x_2, \dots, x_M)$$

What is the influence diagram relating $x_1, x_2, \dots, x_M$ and y ?

Calculus Refresher: Basic rules of calculus

For any differentiable function

$$y = f(x_1, x_2, \dots, x_M)$$



The derivative diagram?

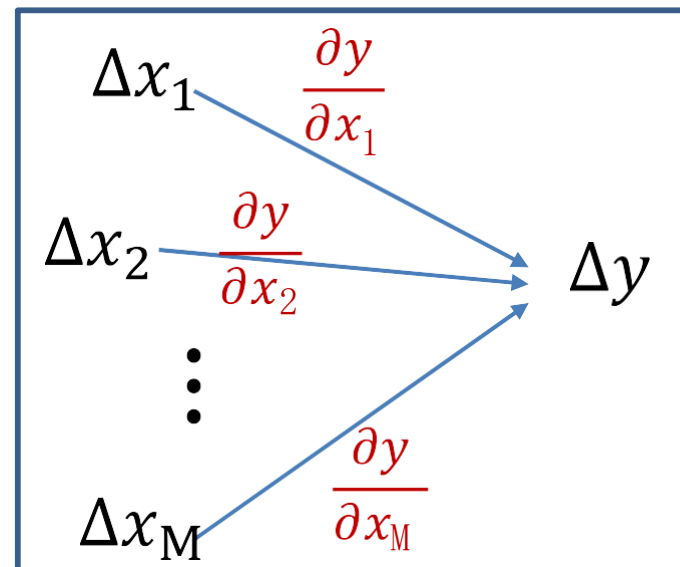
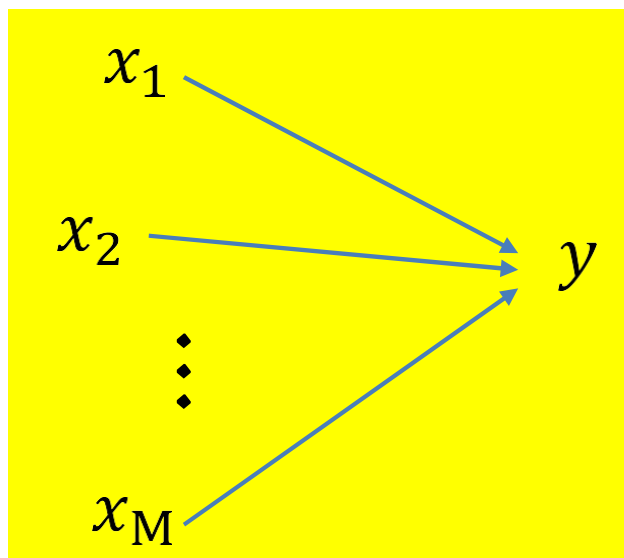
Calculus Refresher: Basic rules of calculus

For any differentiable function

$$y = f(x_1, x_2, \dots, x_M)$$

with partial derivatives

$$\frac{\partial y}{\partial x_1}, \frac{\partial y}{\partial x_2}, \dots, \frac{\partial y}{\partial x_M}$$



Calculus Refresher: Basic rules of calculus

For any differentiable function

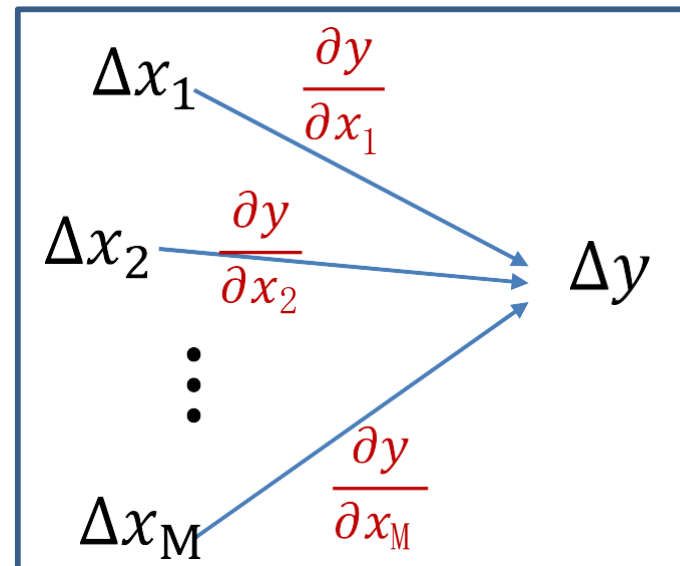
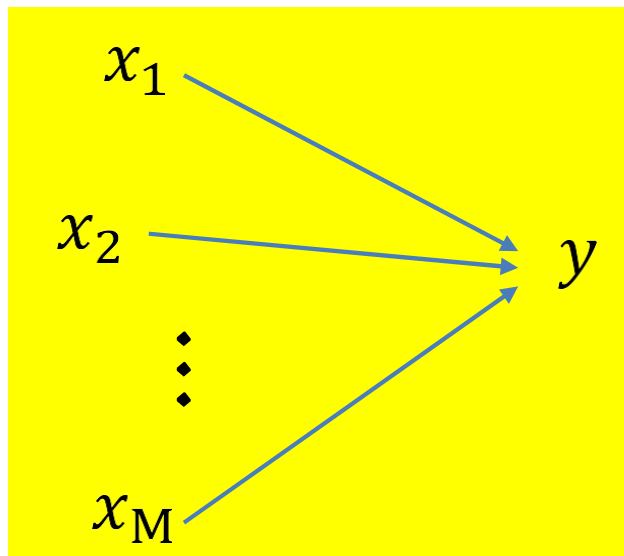
$$y = f(x_1, x_2, \dots, x_M)$$

with partial derivatives

$$\frac{\partial y}{\partial x_1}, \frac{\partial y}{\partial x_2}, \dots, \frac{\partial y}{\partial x_M}$$

the following must hold for sufficiently small $\Delta x_1, \Delta x_2, \dots, \Delta x_M$

$$\Delta y \approx \frac{\partial y}{\partial x_1} \Delta x_1 + \frac{\partial y}{\partial x_2} \Delta x_2 + \dots + \frac{\partial y}{\partial x_M} \Delta x_M$$



For any nested function $y = f(g(x))$

Calculus Refresher: Chain rule

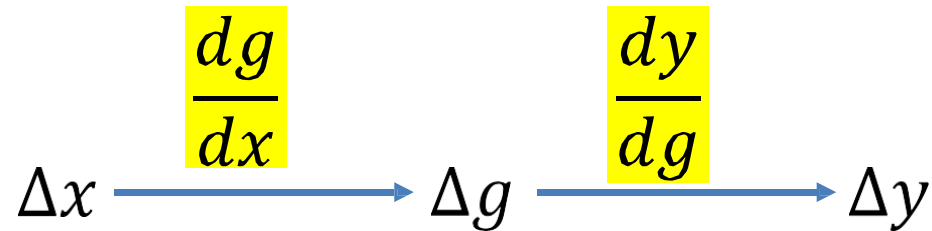
Calculus Refresher: Chain rule

For any nested function $y = f(g(x))$

$$\frac{dy}{dx} = \frac{dy}{dg(x)} \frac{dg(x)}{dx}$$



$$\Delta g = \frac{dg(x)}{dx} \Delta x$$



$$\Delta y = \frac{dy}{dg(x)} \Delta g$$

$$\Delta y = \frac{dy}{dg(x)} \frac{dg(x)}{dx} \Delta x$$

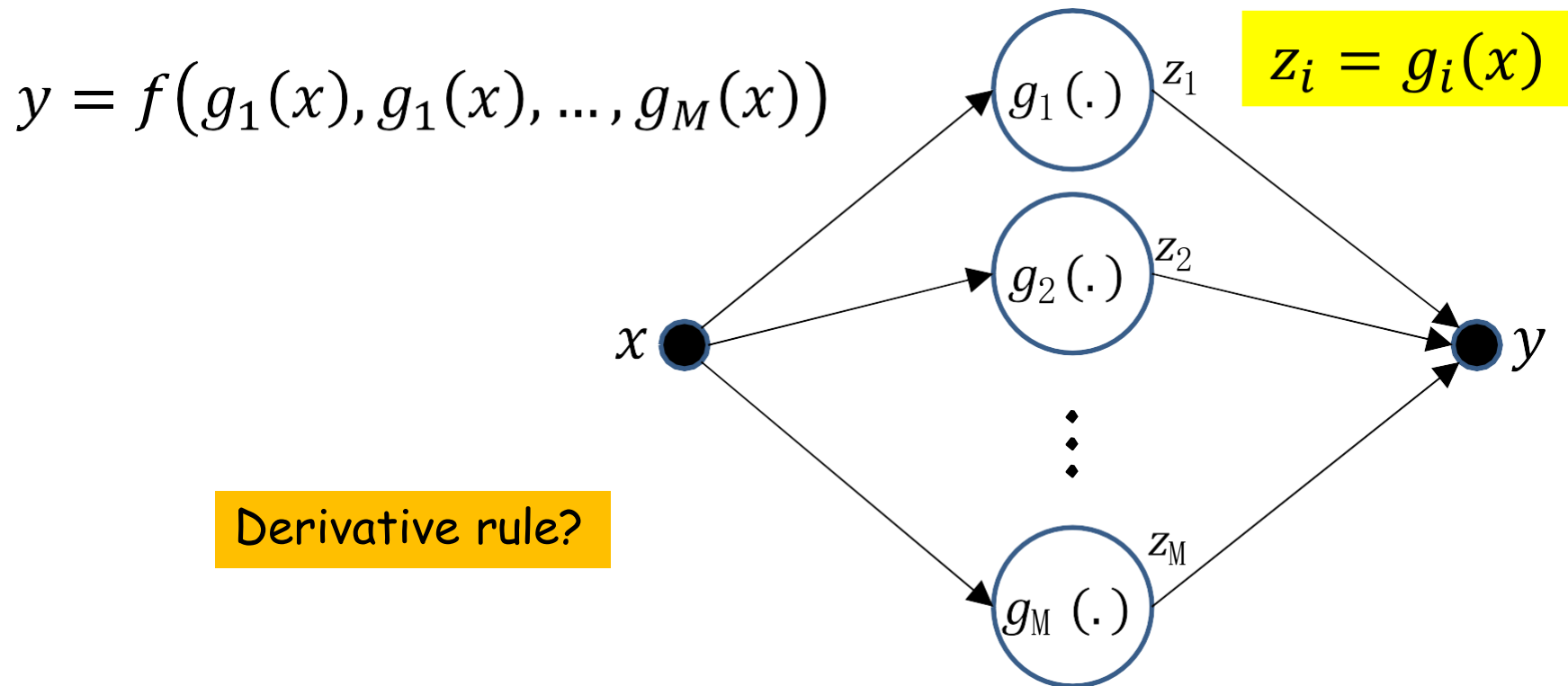
$$\Delta y = \frac{dy}{dx} \Delta x$$

Distributed Chain Rule: Influence Diagram

$$y = f(g_1(x), g_1(x), \dots, g_M(x))$$

Shorthand: $z_i = g_i(x)$

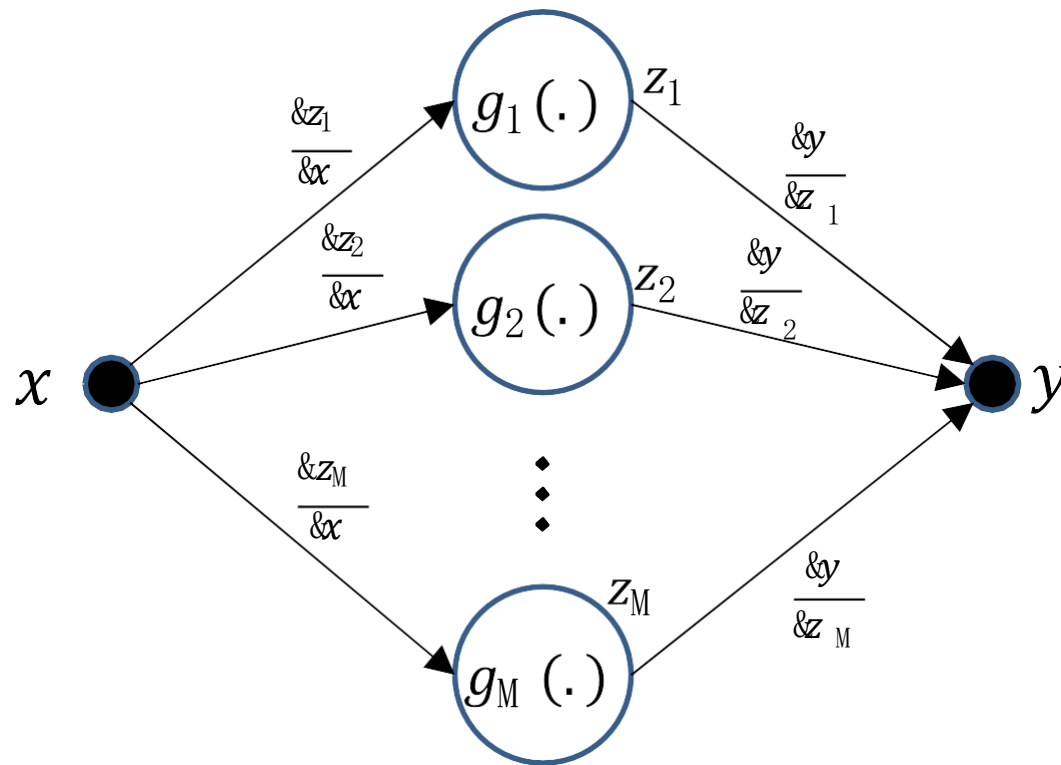
Distributed Chain Rule: Influence Diagram



- x affects y through each of $g_1 \dots g_M$

Distributed Chain Rule: Influence Diagram

$$y = f(g_1(x), g_1(x), \dots, g_M(x))$$

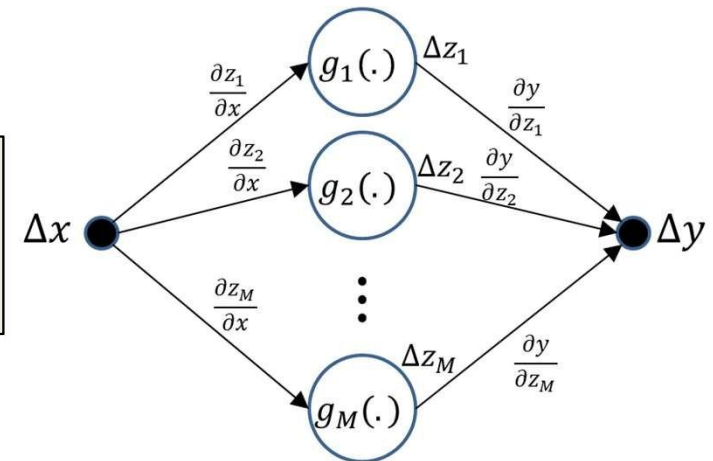
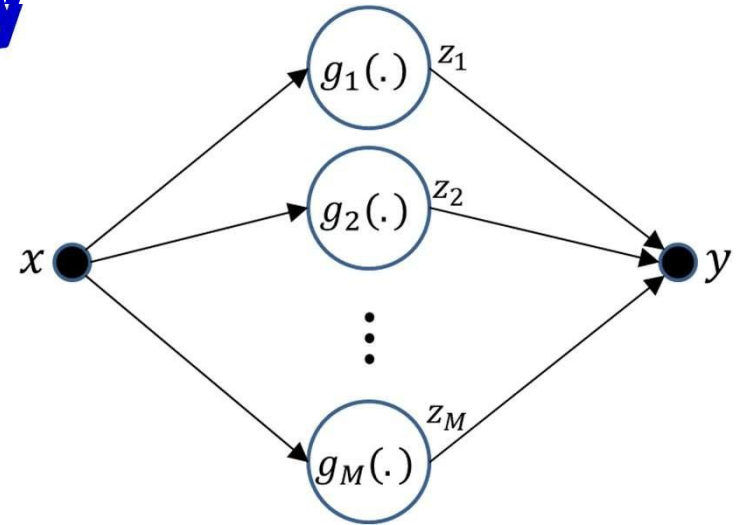


Calculus Refresher: Chain rule summary

For $y = f(z_1, z_2, \dots, z_M)$
where $z_i = g_i(x)$

$$\Delta y = \sum_i \frac{\partial y}{\partial z_i} \Delta z_i \quad \Delta z_i = \frac{dz_i}{dx} \Delta x$$

$$\frac{dy}{dx} = \frac{\partial y}{\partial z_1} \frac{dz_1}{dx} + \frac{\partial y}{\partial z_2} \frac{dz_2}{dx} + \dots + \frac{\partial y}{\partial z_M} \frac{dz_M}{dx}$$

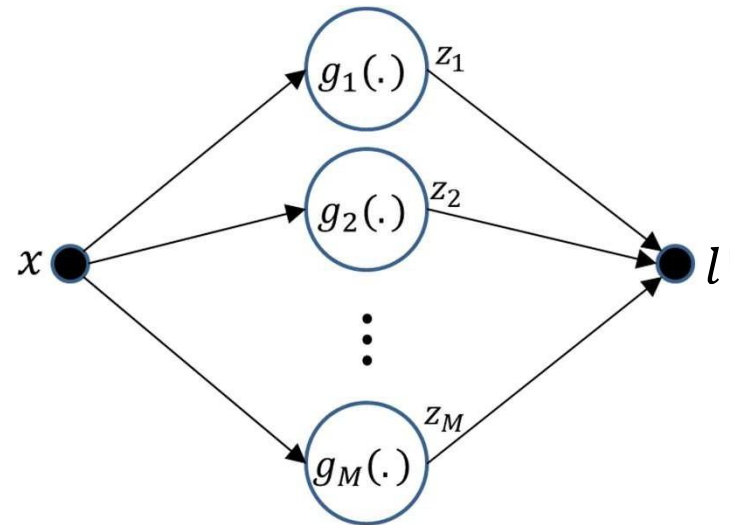


Calculus Refresher: Chain rule summary

For any nested function $l = f(y)$ where $y = g(z)$

$$\frac{dl}{dz} = \frac{dl}{dy} \frac{dy}{dz}$$

For $l = f(z_1, z_2, \dots, z_M)$
where $z_i = g_i(x)$

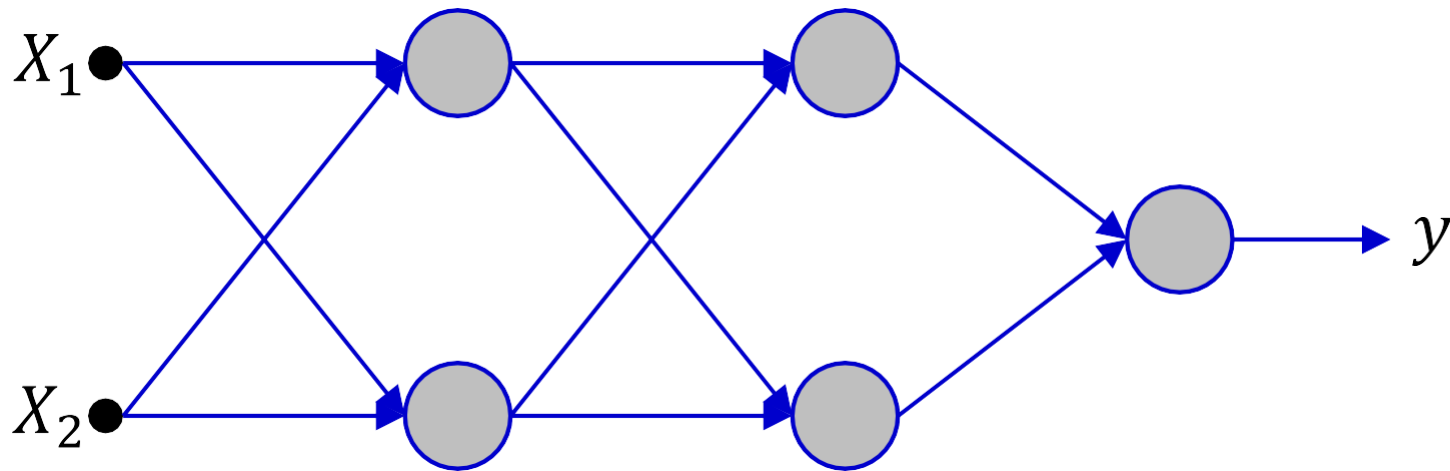


$$\frac{dl}{dx} = \frac{\partial l}{\partial z_1} \frac{dz_1}{dx} + \frac{\partial l}{\partial z_2} \frac{dz_2}{dx} + \dots + \frac{\partial l}{\partial z_M} \frac{dz_M}{dx}$$

Our problem for today

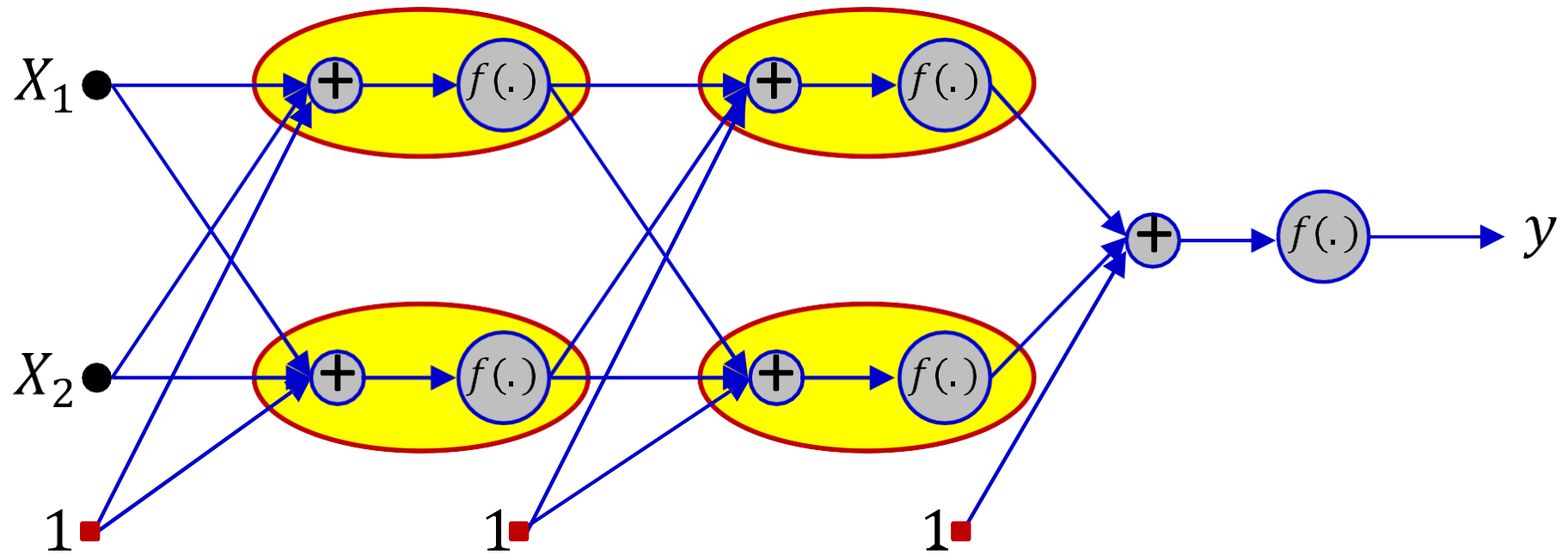
- How to compute $\frac{dDiv(\mathbf{Y}, d)}{dw_{i,j}^{(k)}}$ for a single data instance

A first closer look at the network



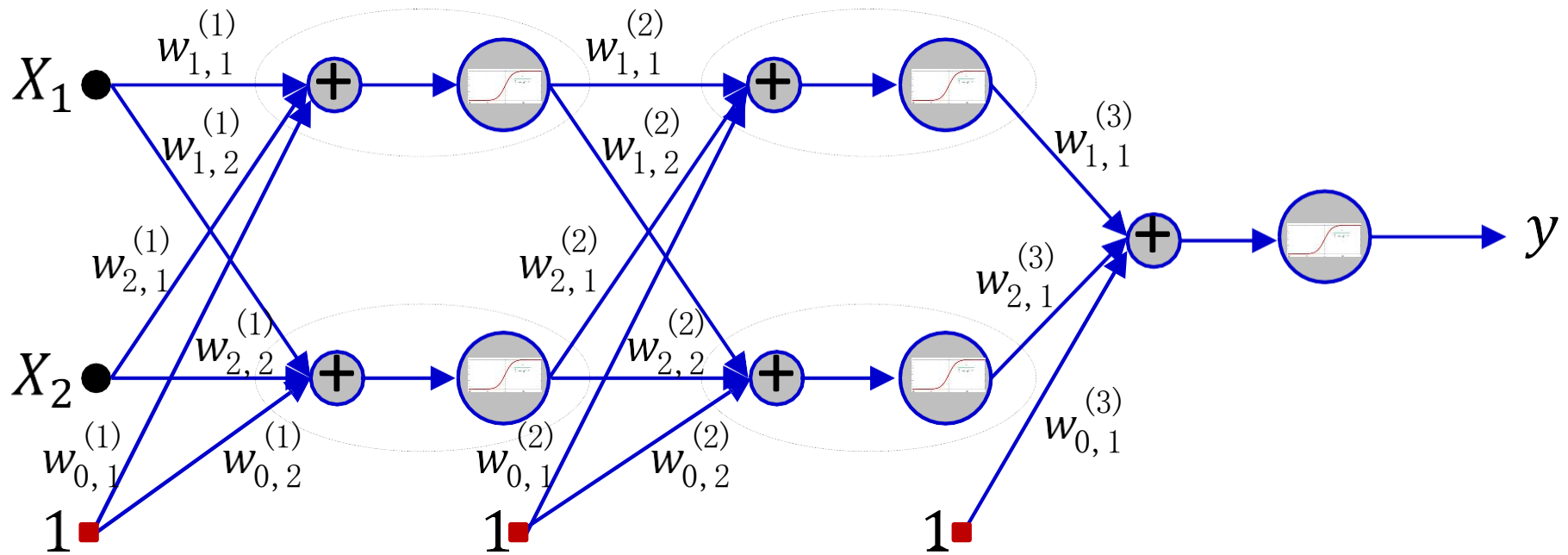
- Showing a tiny 2-input network for illustration
 - Actual network would have many more neurons and inputs

A first closer look at the network



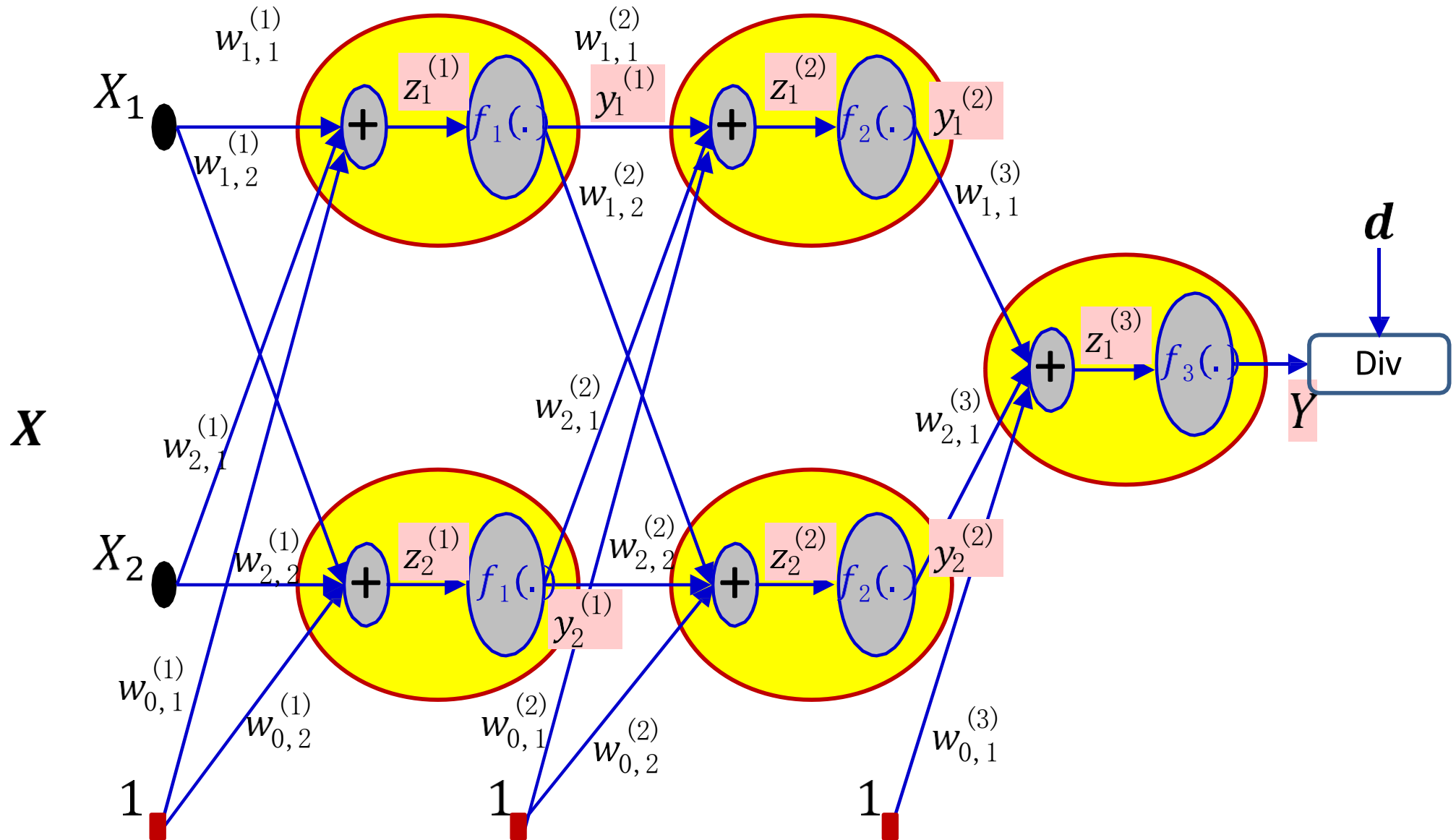
- Showing a tiny 2-input network for illustration
 - Actual network would have many more neurons and inputs
- Explicitly separating the affine function of inputs from the activation

A first closer look at the network

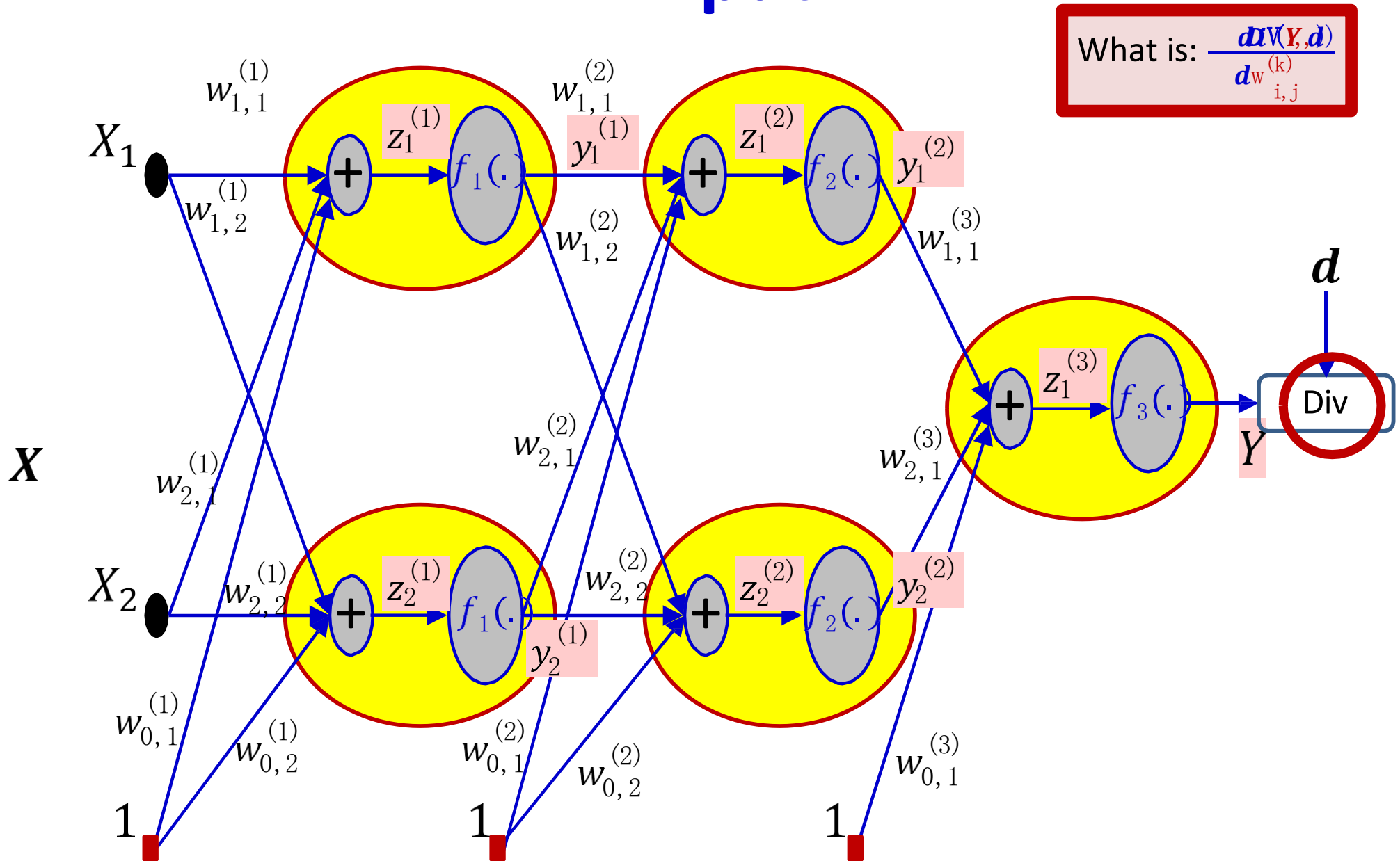


- Showing a tiny 2-input network for illustration
 - Actual network would have many more neurons and inputs
- Expanded **with all weights shown**
- Let's label the other variables too...

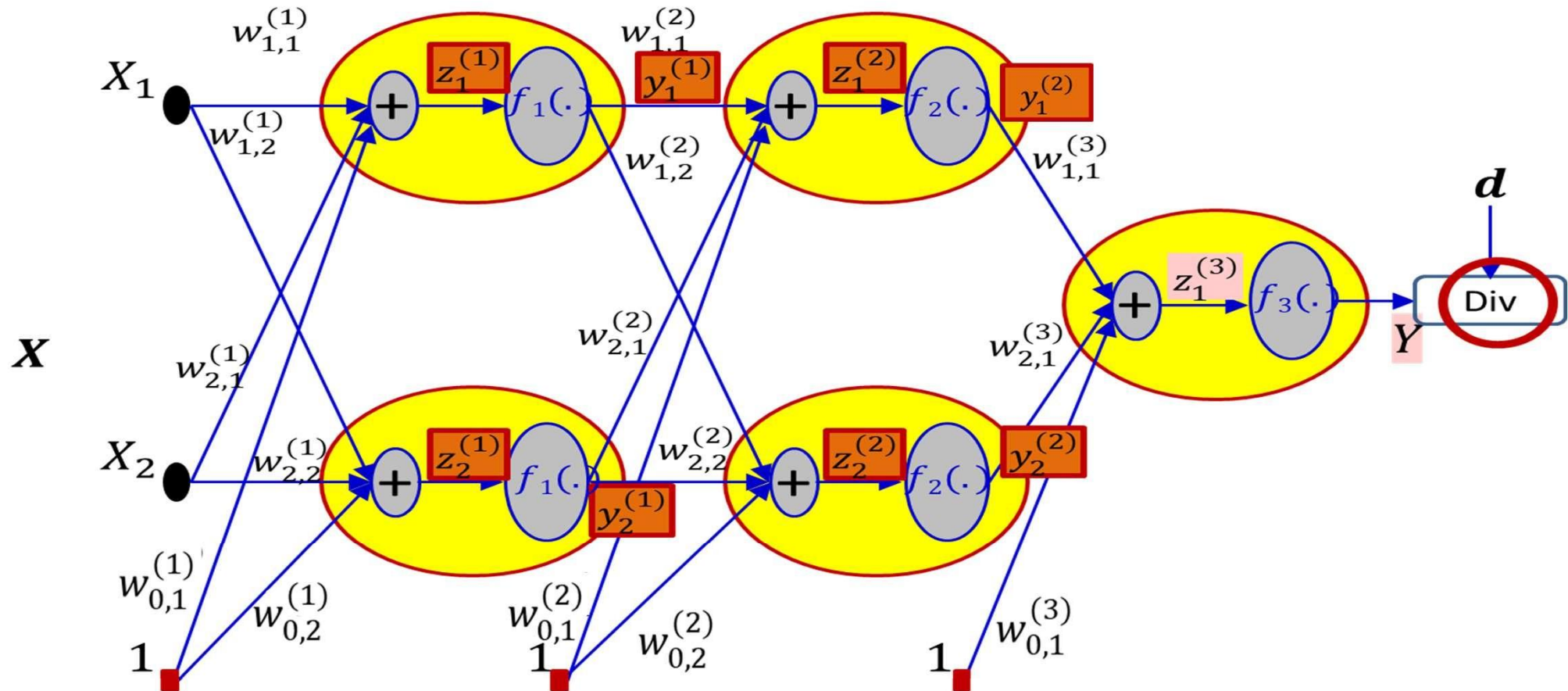
Computing the derivative for a *single* input



Computing the derivative for a *single* input



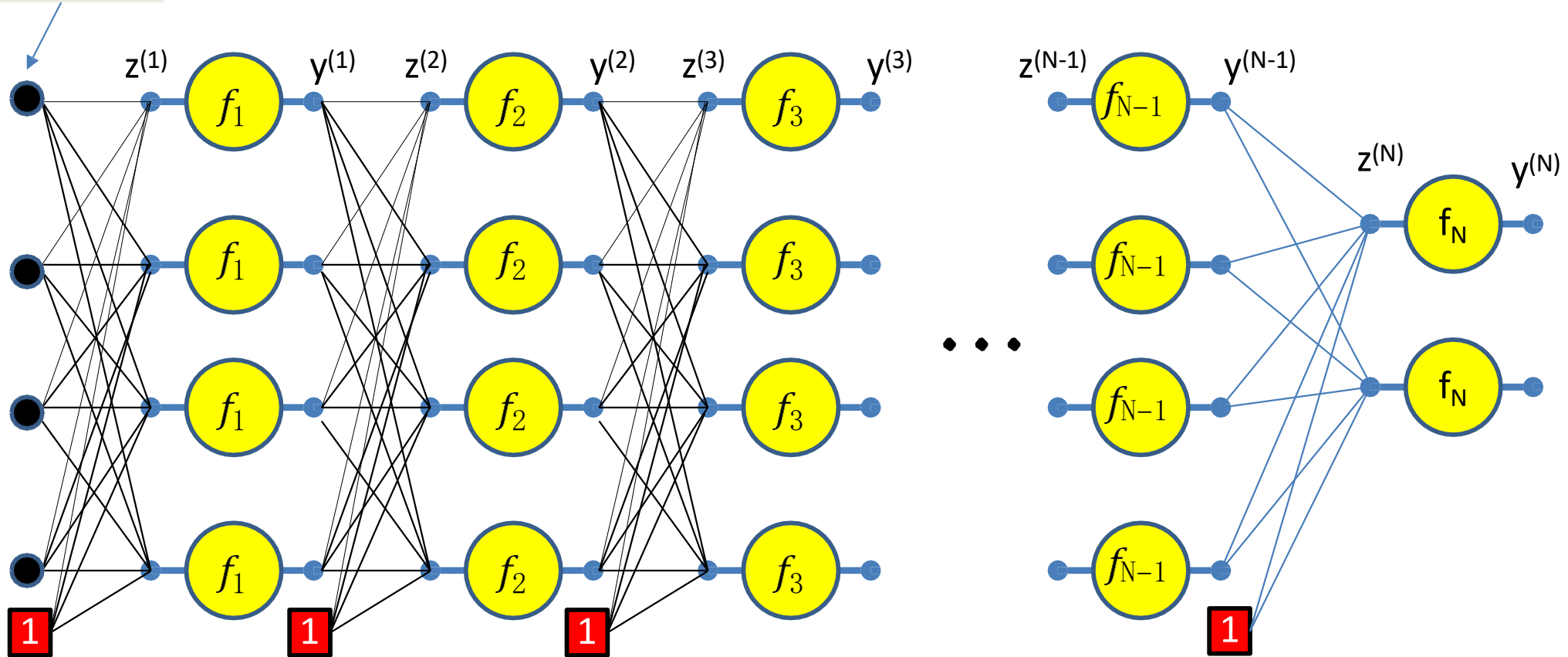
Computing the gradient



- Note: computation of the derivative $\frac{dDiv(Y,d)}{dw_{i,j}^{(k)}}$ requires intermediate and final output values of the network in response to the input

The “forward pass”

$$y^{(0)} = x$$

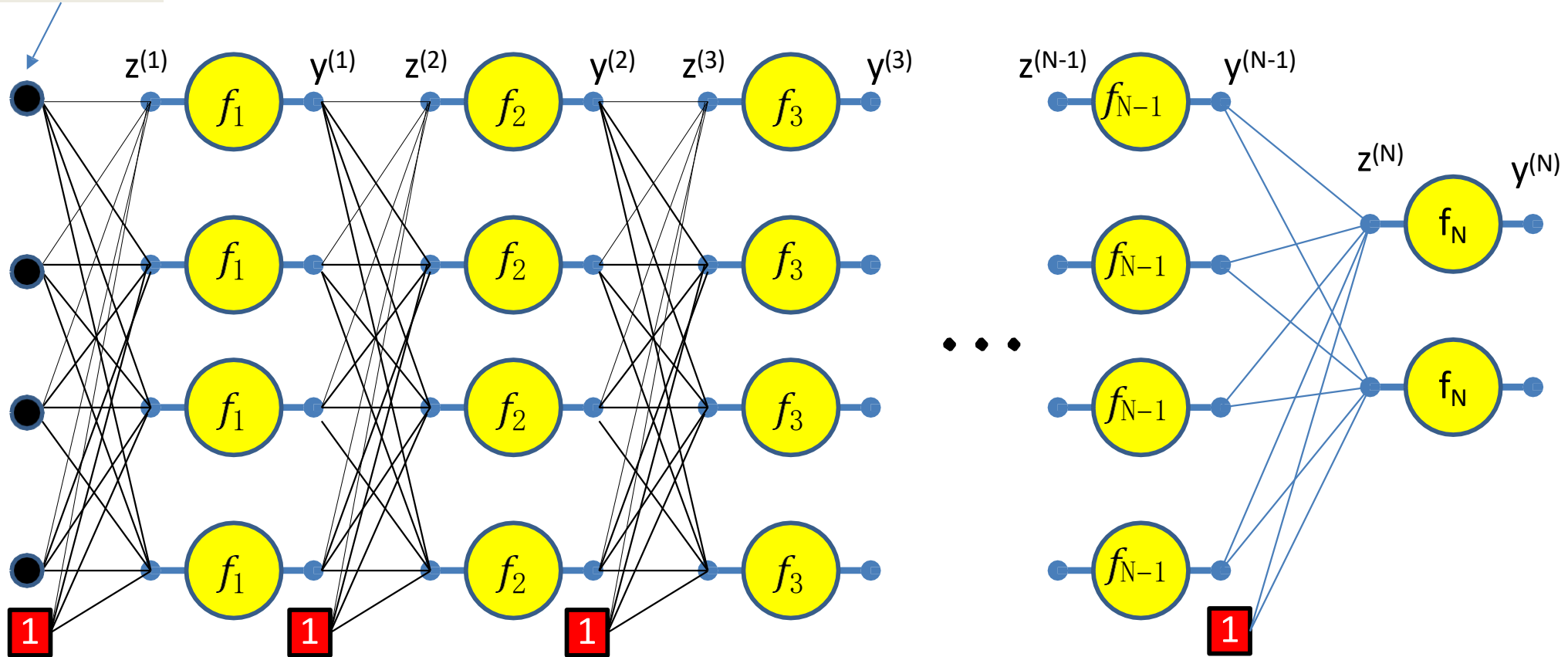


We will refer to the process of computing the output from an input as the *forward pass*

We will illustrate the forward pass in the following slides

The “forward pass”

$$y^{(0)} = x$$

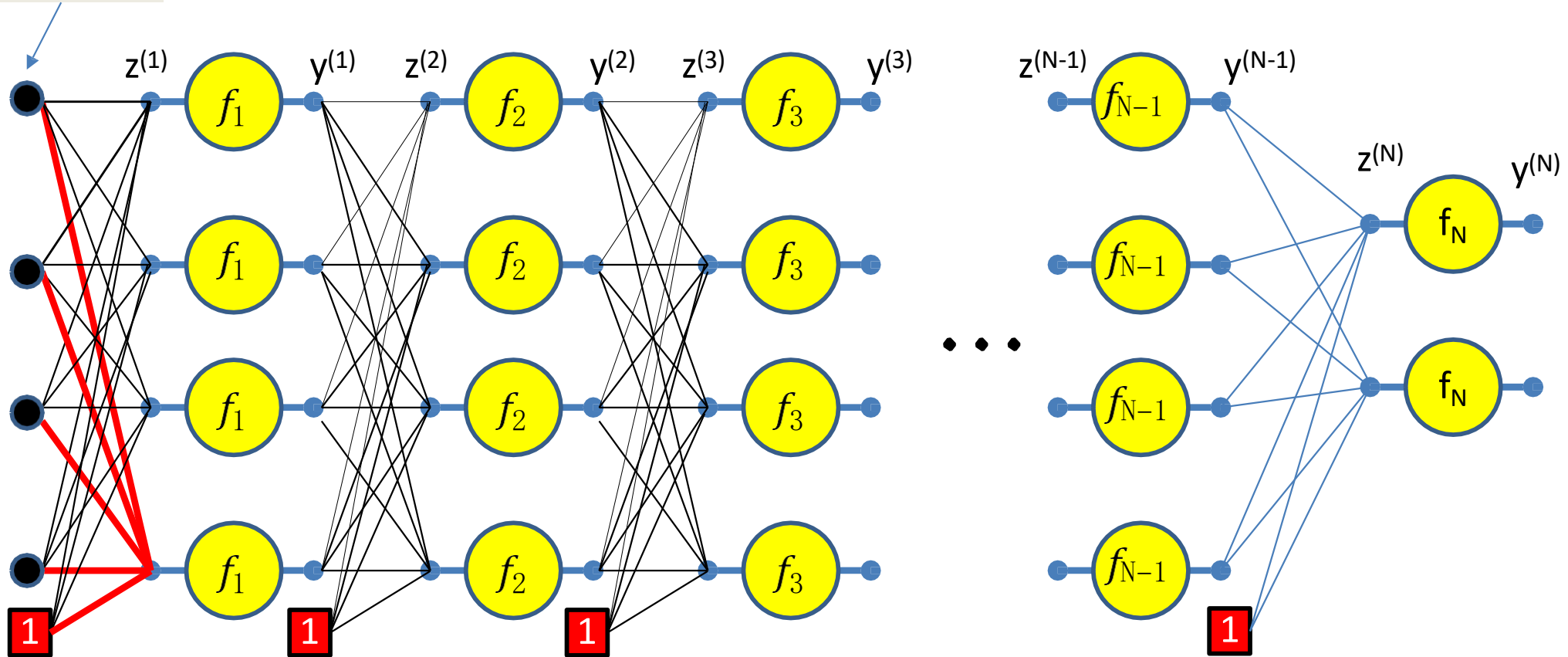


Setting $y_i^{(0)} = x_i$ for notational convenience

Assuming $w_{0j}^{(k)} = b_j^{(k)}$ and $y_0^{(k)} = 1$ -- assuming the bias is a weight and extending the output of every layer by a constant 1, to account for the biases

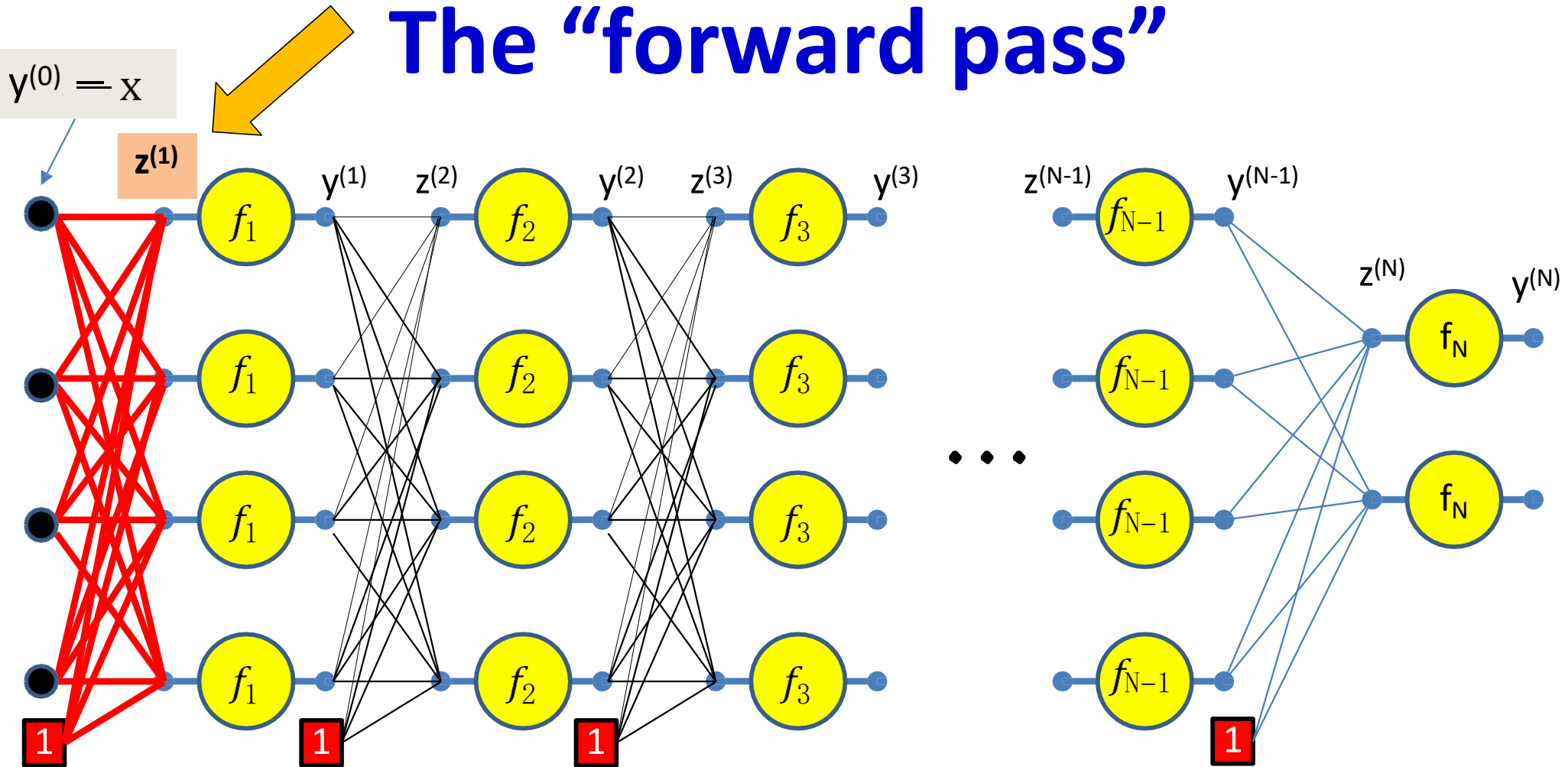
The “forward pass”

$$y^{(0)} = x$$

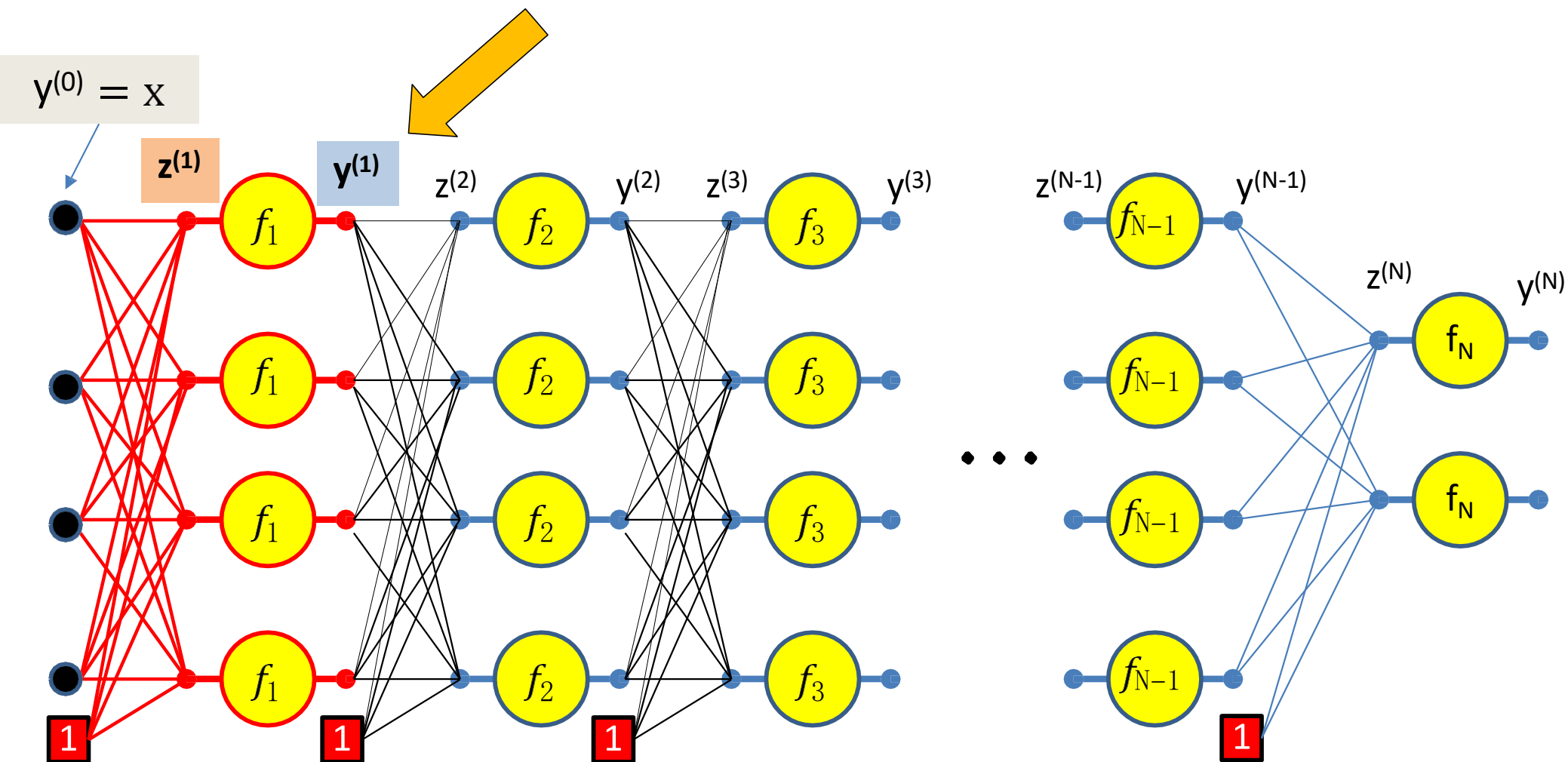


$$z_1^{(1)} = \sum_i w_{i1}^{(1)} y_i^{(0)}$$

The “forward pass”

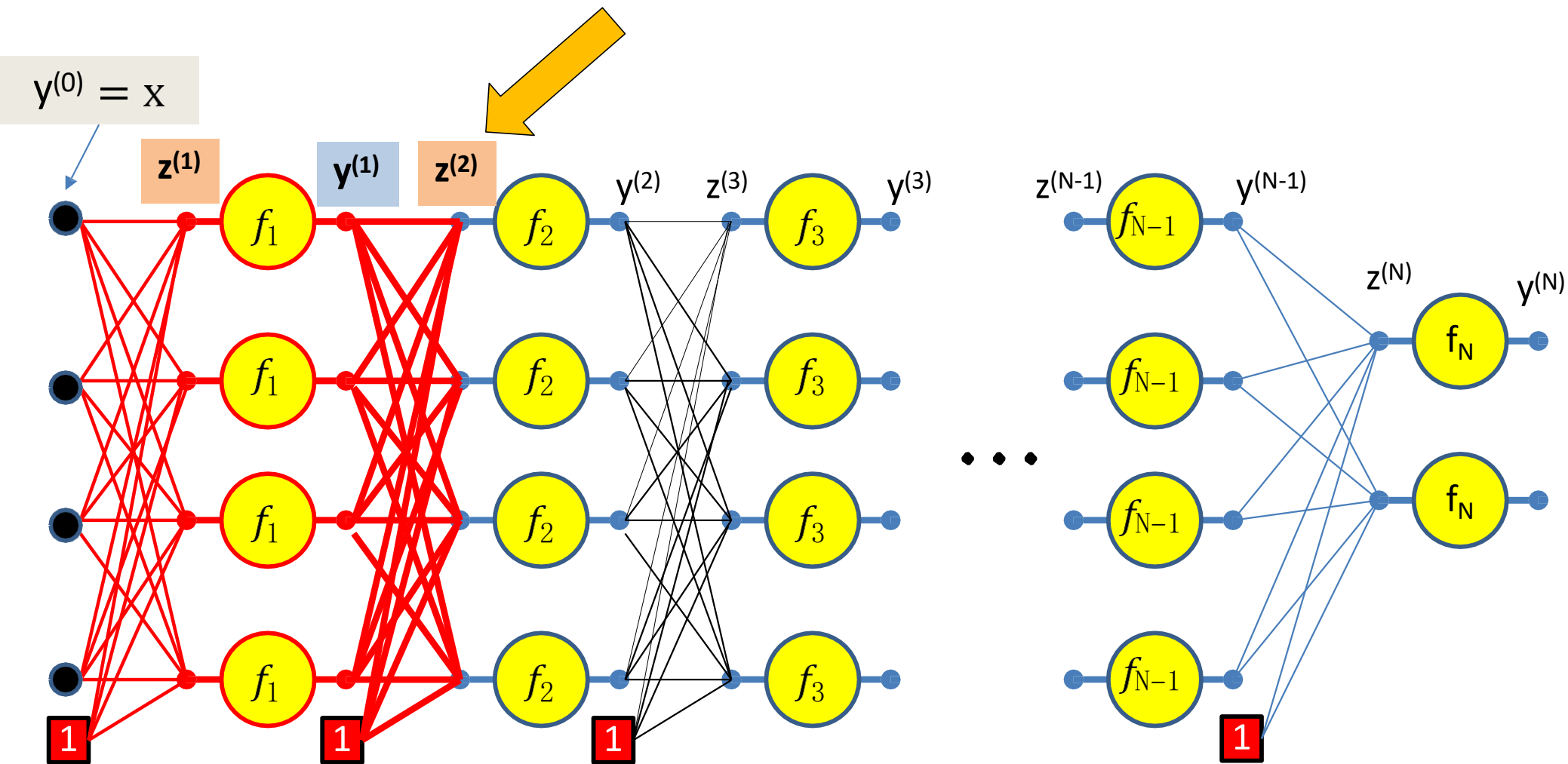


$$z_j^{(1)} = \sum_i w_{ij}^{(1)} y_i^{(0)}$$



$$z_j^{(1)} = \sum_i w_{ij}^{(1)} y_i^{(0)}$$

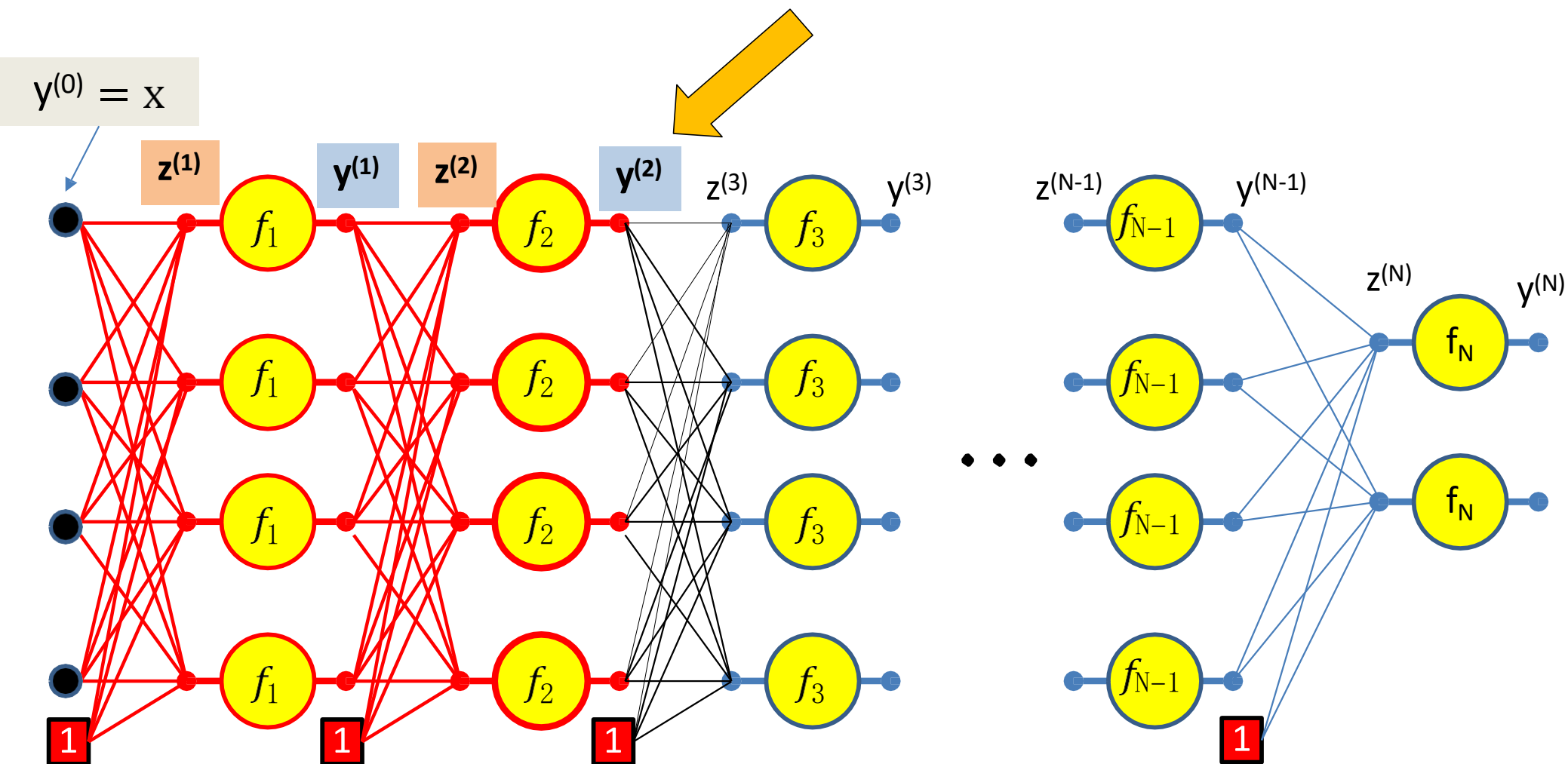
$$y_j^{(1)} = f_1(z_j^{(1)})$$



$$z_j^{(1)} = \sum_i w_{ij}^{(1)} y_i^{(0)}$$

$$y_j^{(1)} = f_1(z_j^{(1)})$$

$$z_j^{(2)} = \sum_i w_{ij}^{(2)} y_i^{(1)}$$

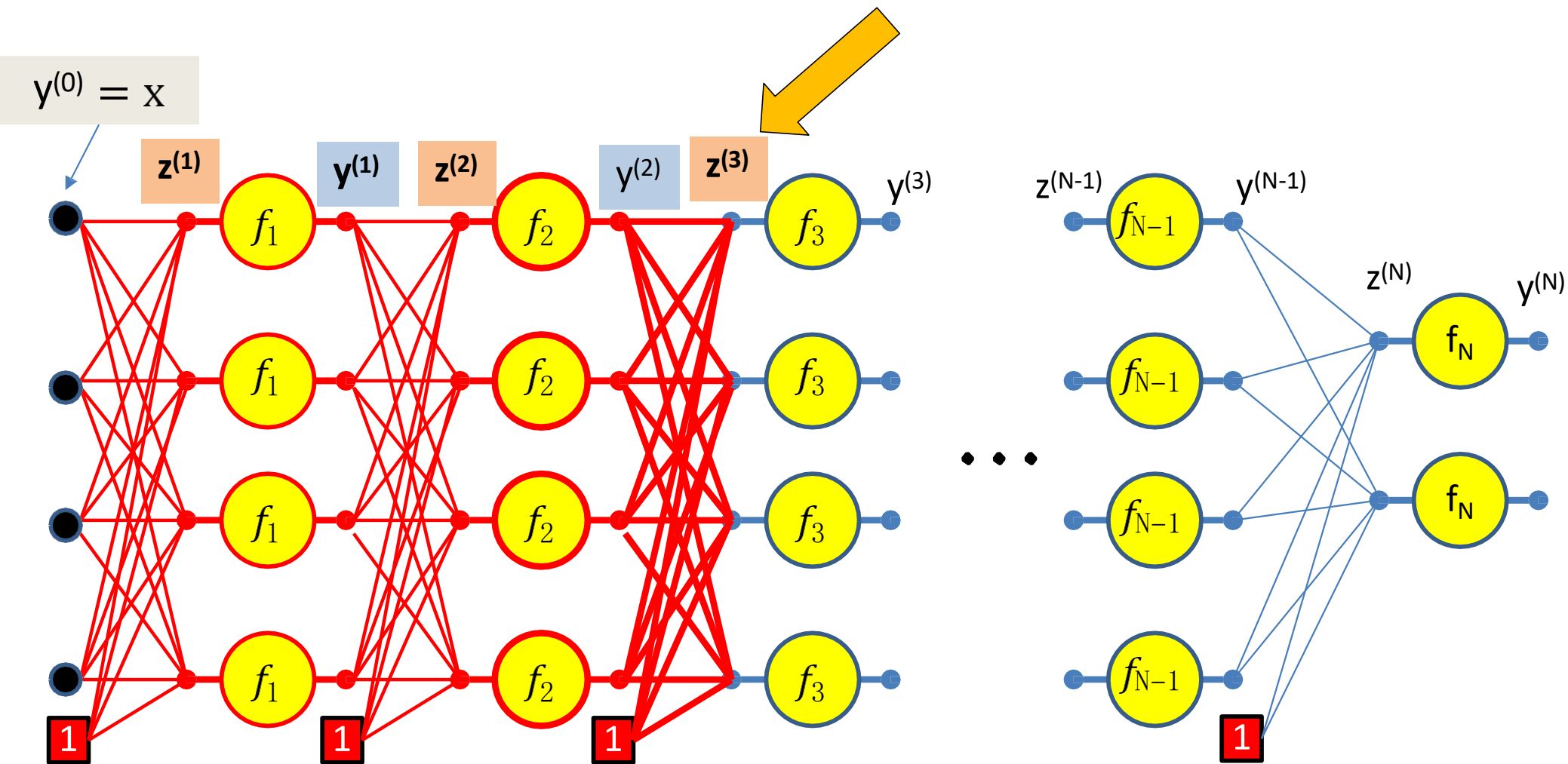


$$z_j^{(1)} = \sum_i w_{ij}^{(1)} y_i^{(0)}$$

$$y_j^{(1)} = f_1(z_j^{(1)})$$

$$z_j^{(2)} = \sum_i w_{ij}^{(2)} y_i^{(1)}$$

$$y_j^{(2)} = f_2(z_j^{(2)})$$



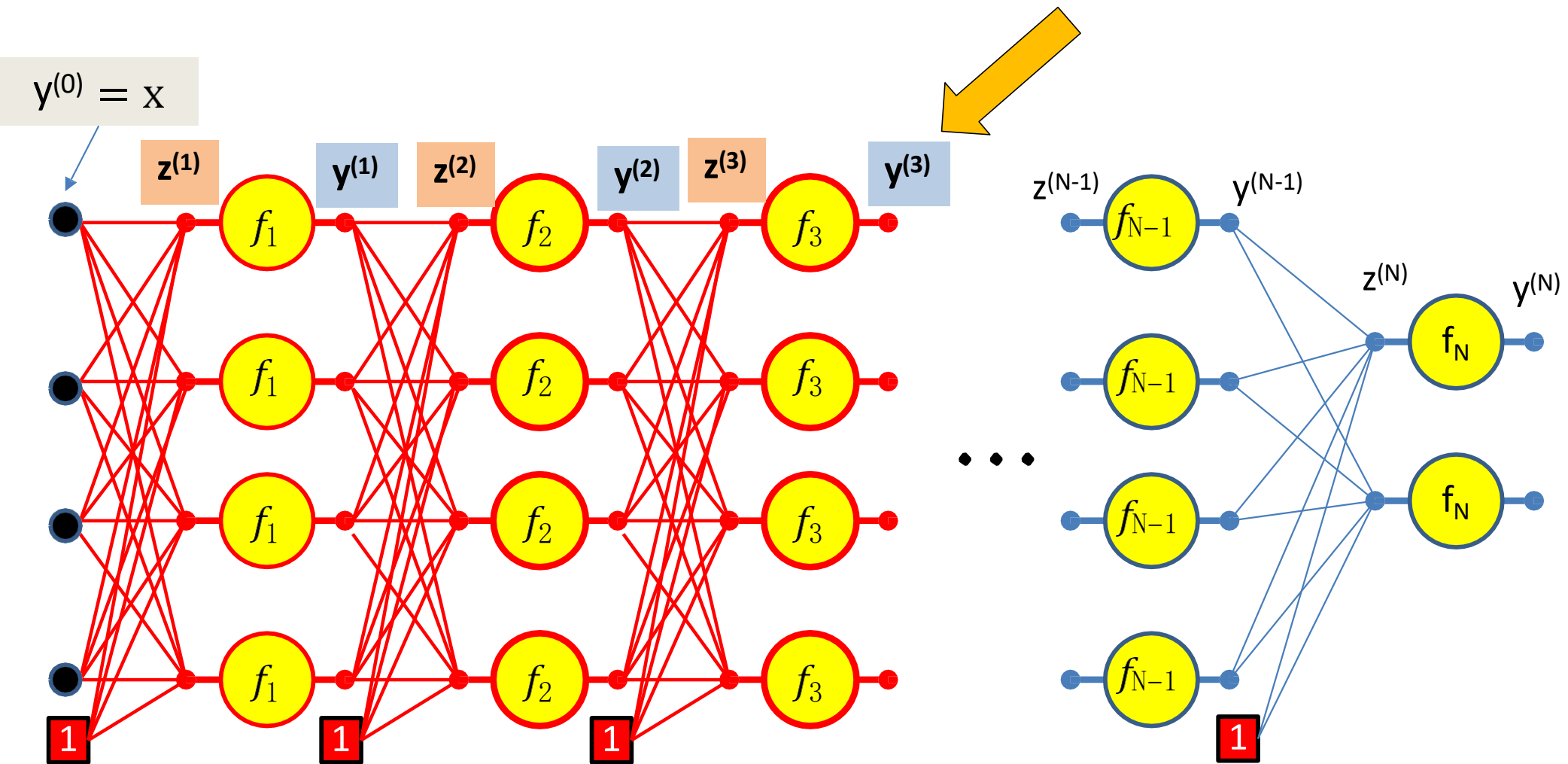
$$z_j^{(1)} = \sum_i w_{ij}^{(1)} y_i^{(0)}$$

$$y_j^{(1)} = f_1(z_j^{(1)})$$

$$z_j^{(2)} = \sum_i w_{ij}^{(2)} y_i^{(1)}$$

$$y_j^{(2)} = f_2(z_j^{(2)})$$

$$z_j^{(3)} = \sum_i w_{ij}^{(3)} y_i^{(2)}$$



$$z_j^{(1)} = \sum_i w_{ij}^{(1)} y_i^{(0)}$$

$$y_j^{(1)} = f_1(z_j^{(1)})$$

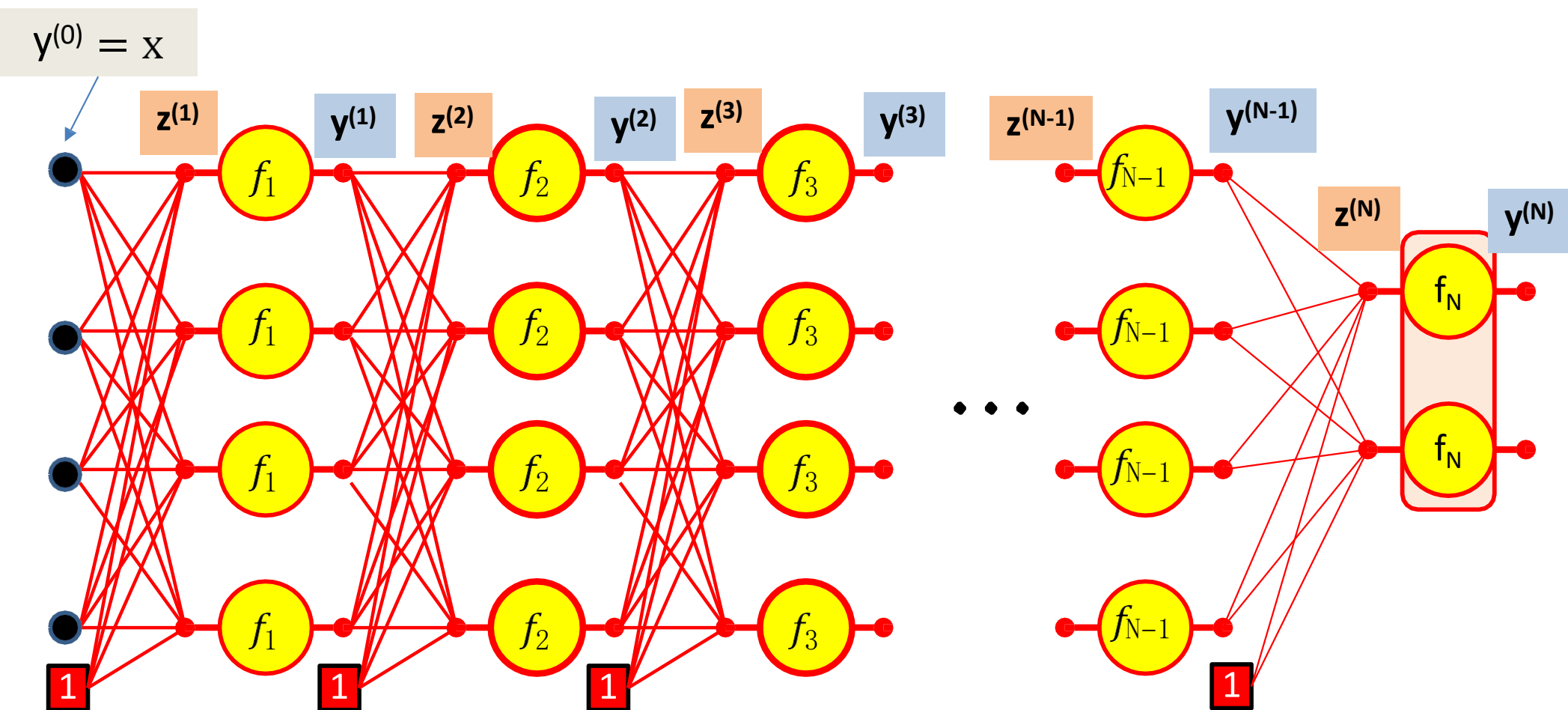
$$z_j^{(2)} = \sum_i w_{ij}^{(2)} y_i^{(1)}$$

$$y_j^{(2)} = f_2(z_j^{(2)})$$

$$z_j^{(3)} = \sum_i w_{ij}^{(3)} y_i^{(2)}$$

$$y_j^{(3)} = f_3(z_j^{(3)})$$

...

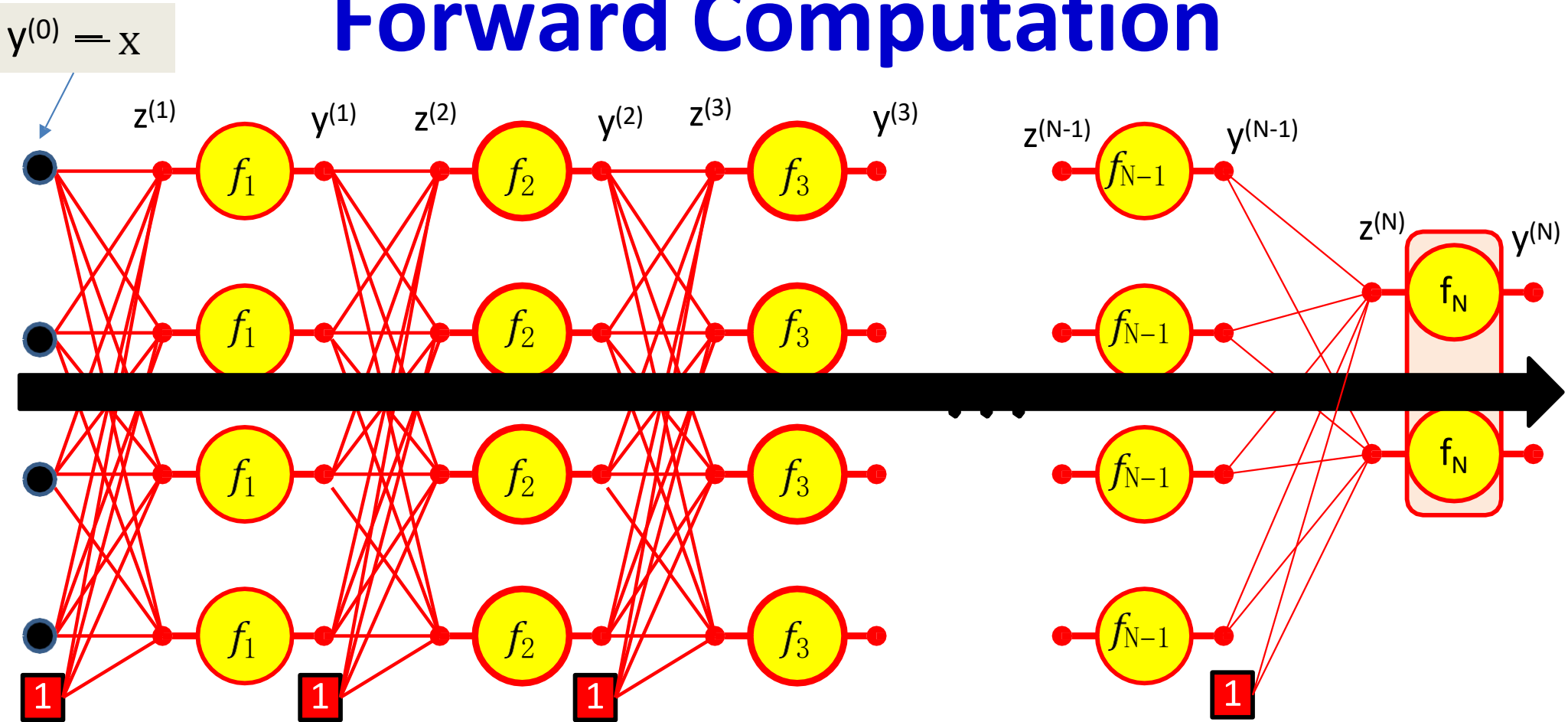


$$y_j^{(N-1)} = f_{N-1} \left(z_j^{(N-1)} \right)$$

$$z_j^{(N)} = \sum_i w_{ij}^{(N)} y_i^{(N-1)}$$

$$y^{(N)} = f_N \left(z^{(N)} \right)$$

Forward Computation



ITERATE FOR $k = 1:N$

for $j = 1:\text{layer-width}$

$$y_i^{(0)} = x_i$$

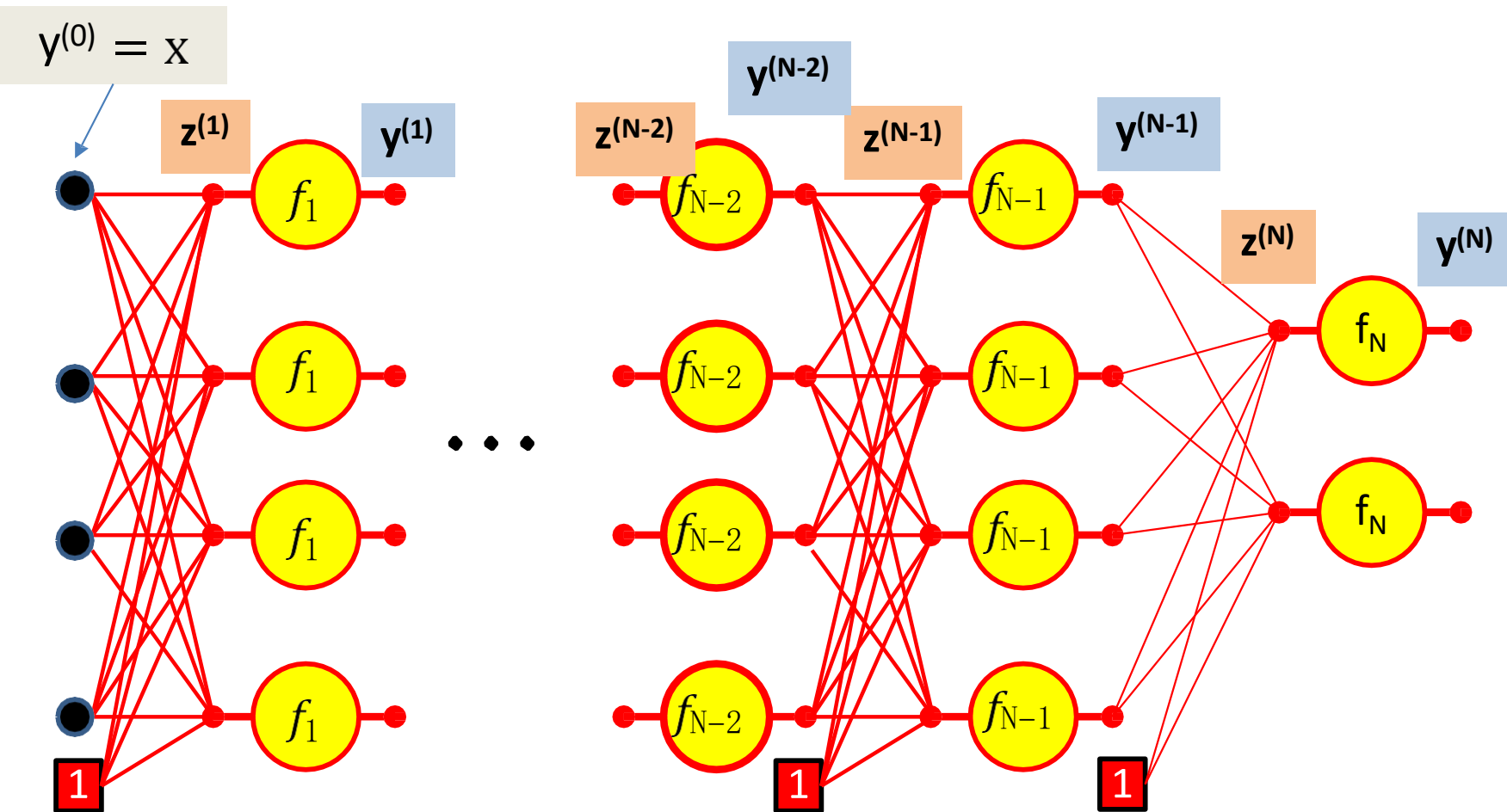
$$z_j^{(k)} = \sum_i w_{ij}^{(k)} y_i^{(k-1)}$$

$$y_j^{(k)} = f_k(z_j^{(k)})$$

Forward “Pass”

- Input: D dimensional vector $\mathbf{x} = [x_j, j = 1 \dots D]$
- Set:
 - $D_0 = D$, is the width of the 0th (input) layer
 - $y_j^{(0)} = x_j, j = 1 \dots D$; $y_0^{(k=1 \dots N)} = x_0 = 1$
- For layer $k = 1 \dots N$
 - For $j = 1 \dots D_k$ D_k is the size of the k th layer
 - $z_j^{(k)} = \sum_{i=0}^{D_{k-1}} w_{i,j}^{(k)} y_i^{(k-1)}$
 - $y_j^{(k)} = f_k(z_j^{(k)})$
- Output:
 - $Y = y_j^{(N)}, j = 1 \dots D_N$

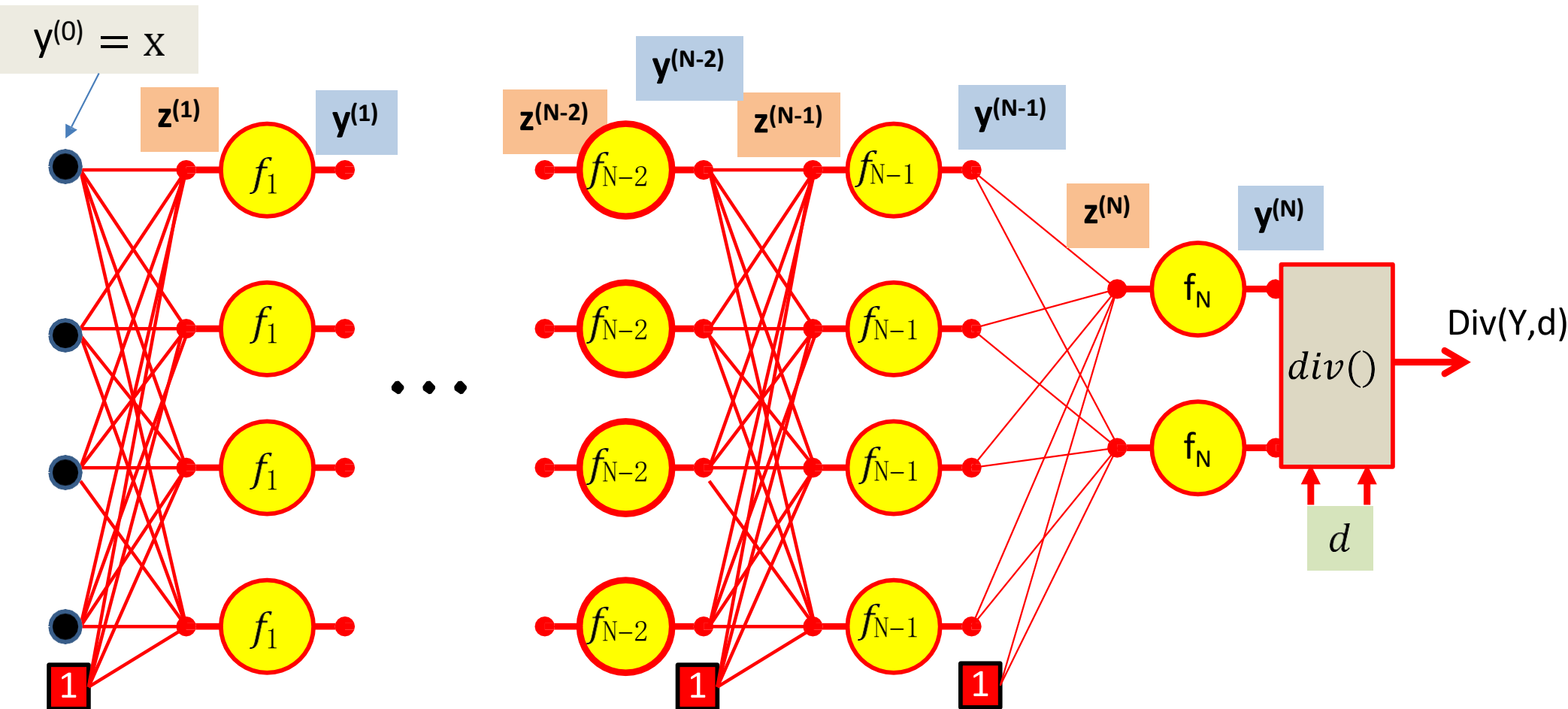
Computing derivatives



We have computed all these intermediate values in the forward computation

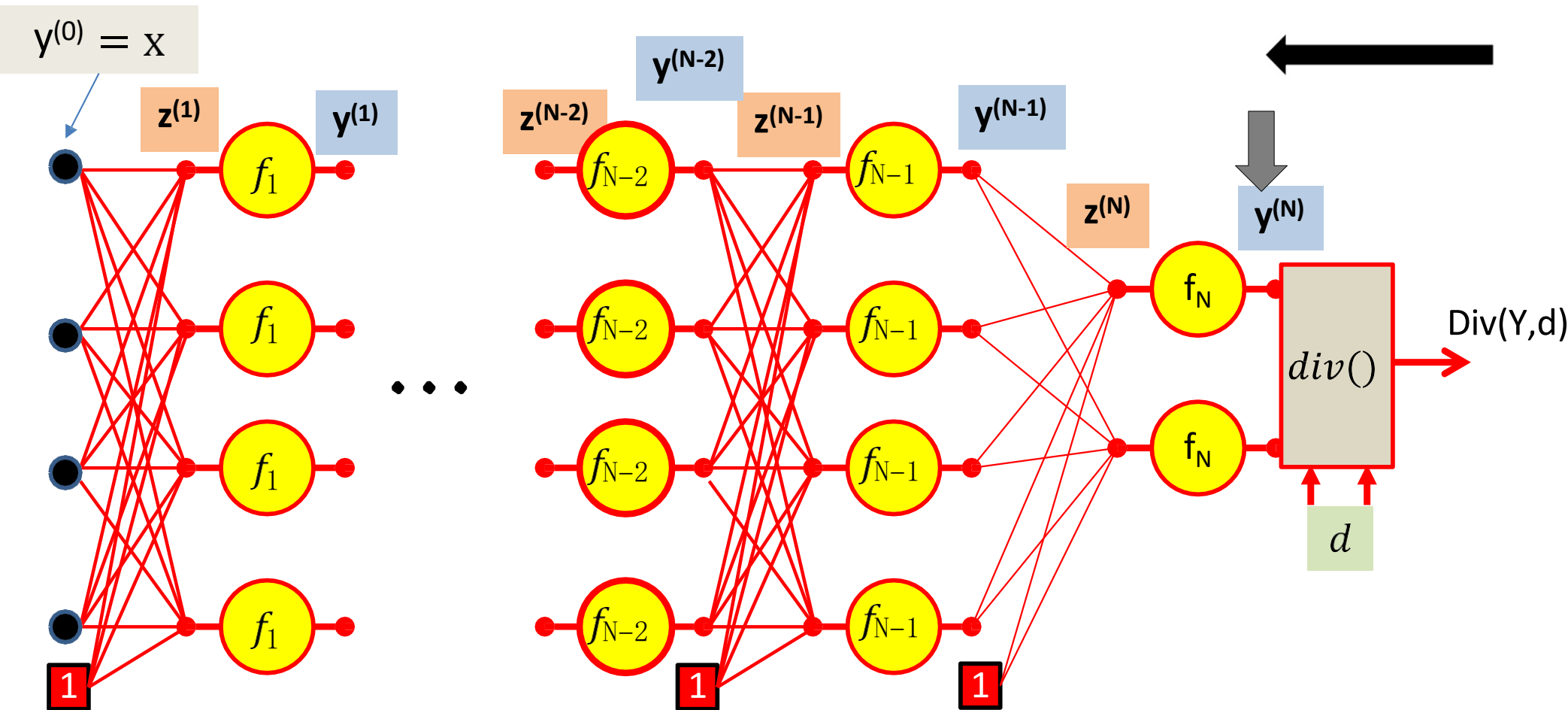
We must remember them - we will need them to compute the derivatives

Computing derivatives



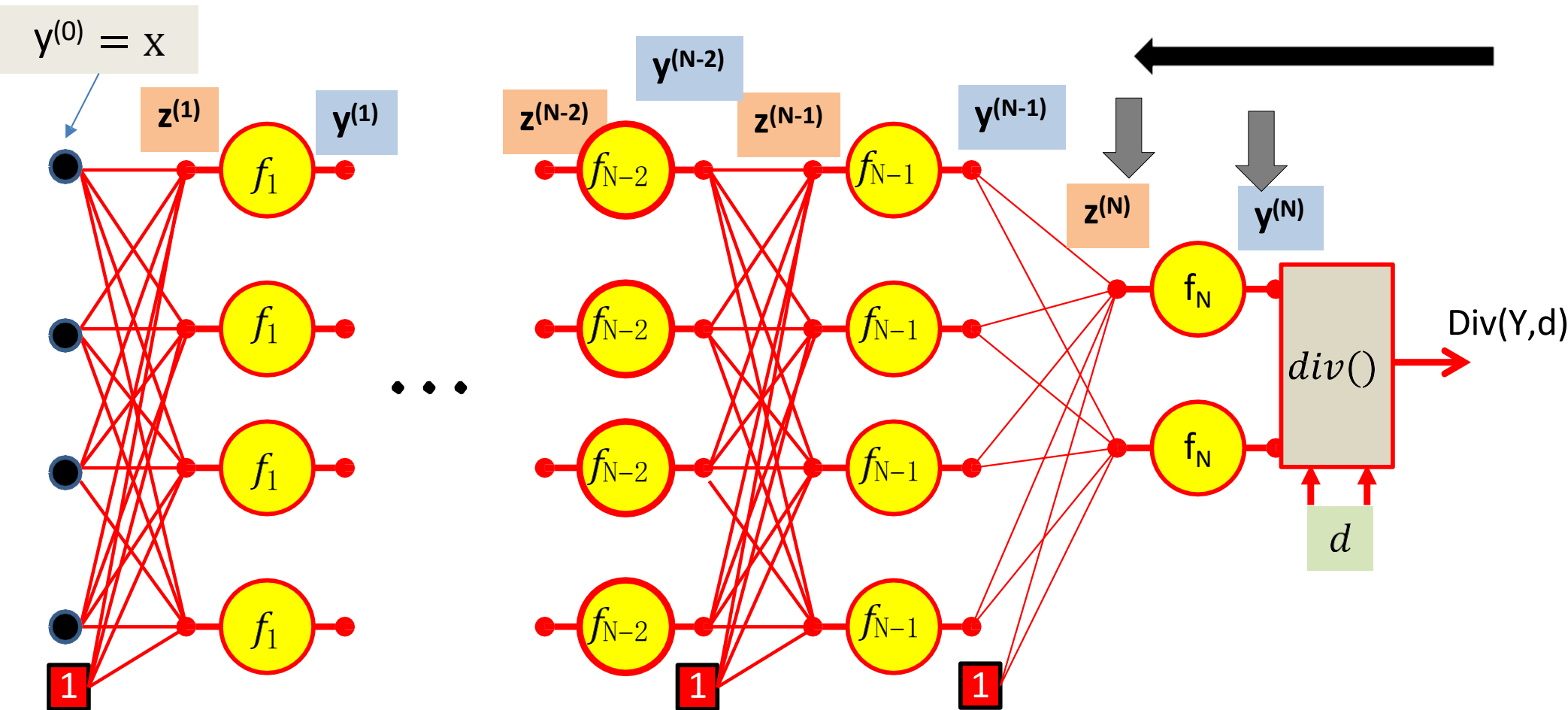
First, we compute the divergence between the output of the net $y = y^{(N)}$ and the desired output d

Computing derivatives



We then compute $\nabla_{F^{(N)}} div(.)$ the derivative of the divergence w.r.t. the final output of the network $y^{(N)}$

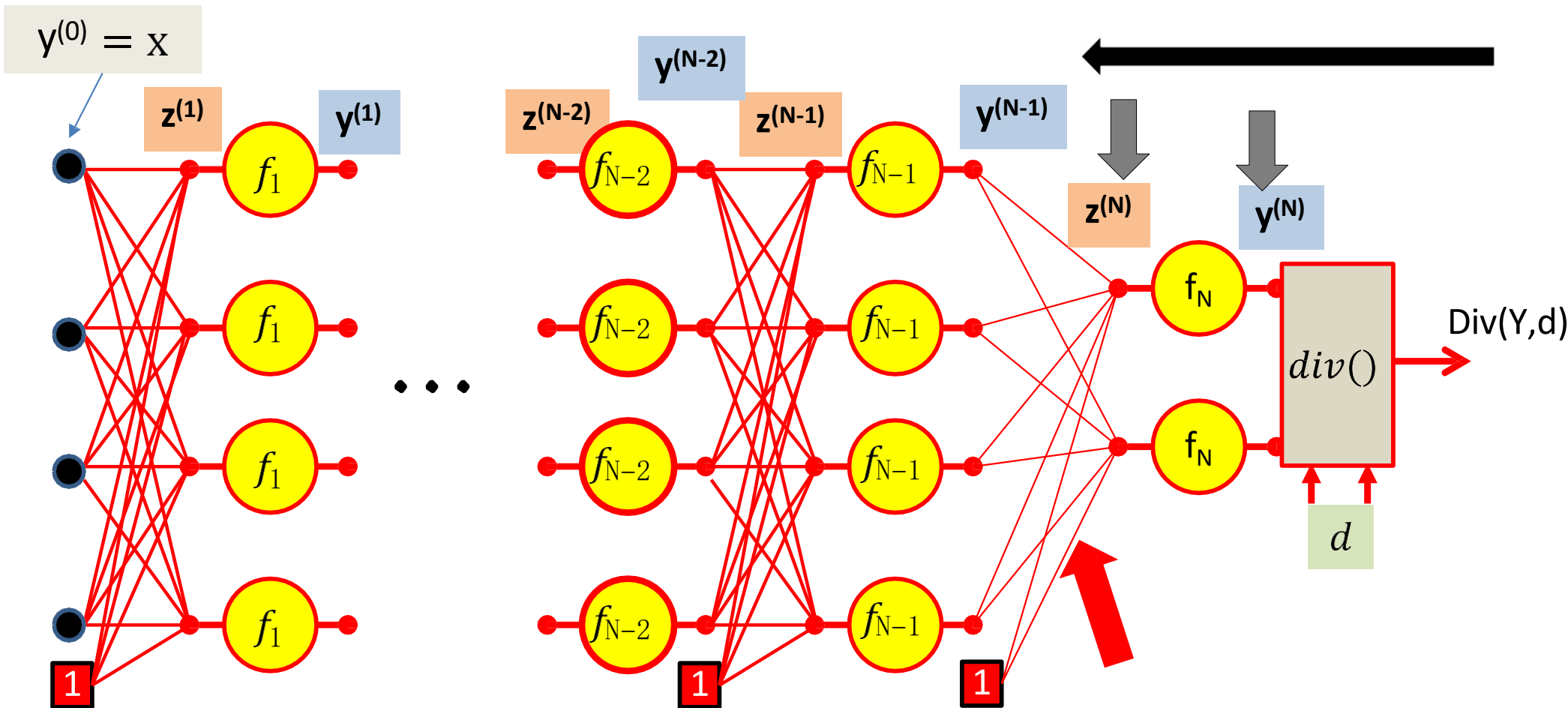
Computing derivatives



We then compute $\nabla_{F^{(N)}} div(.)$ the derivative of the divergence w.r.t. the final output of the network $y^{(N)}$

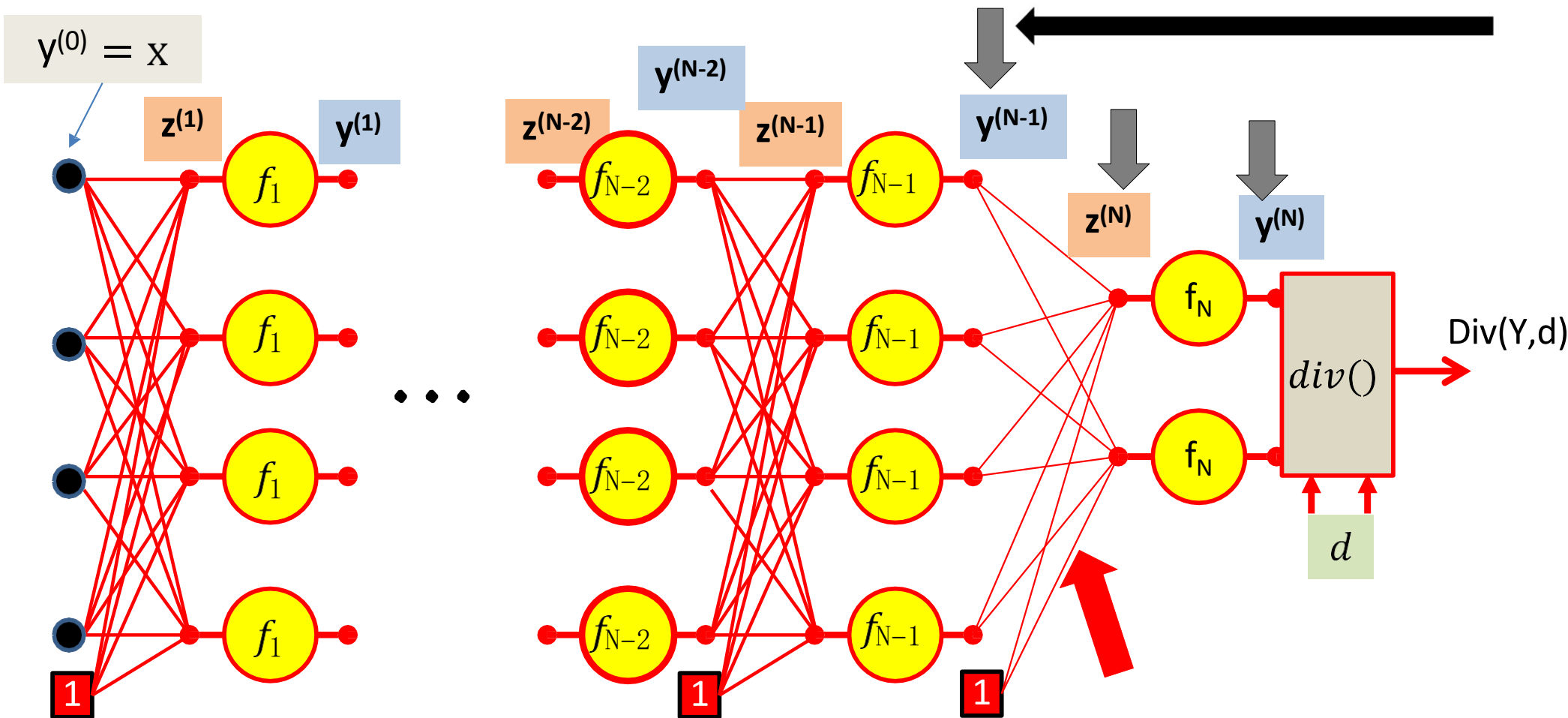
We then compute $\nabla_{z^{(N)}} div(.)$ the derivative of the divergence w.r.t. the *pre-activation* affine combination $z^{(N)}$ using the chain rule

Computing derivatives



Continuing on, we will compute $\nabla_{w^{(N)}} div(.)$ the derivative of the divergence with respect to the weights of the connections to the output layer

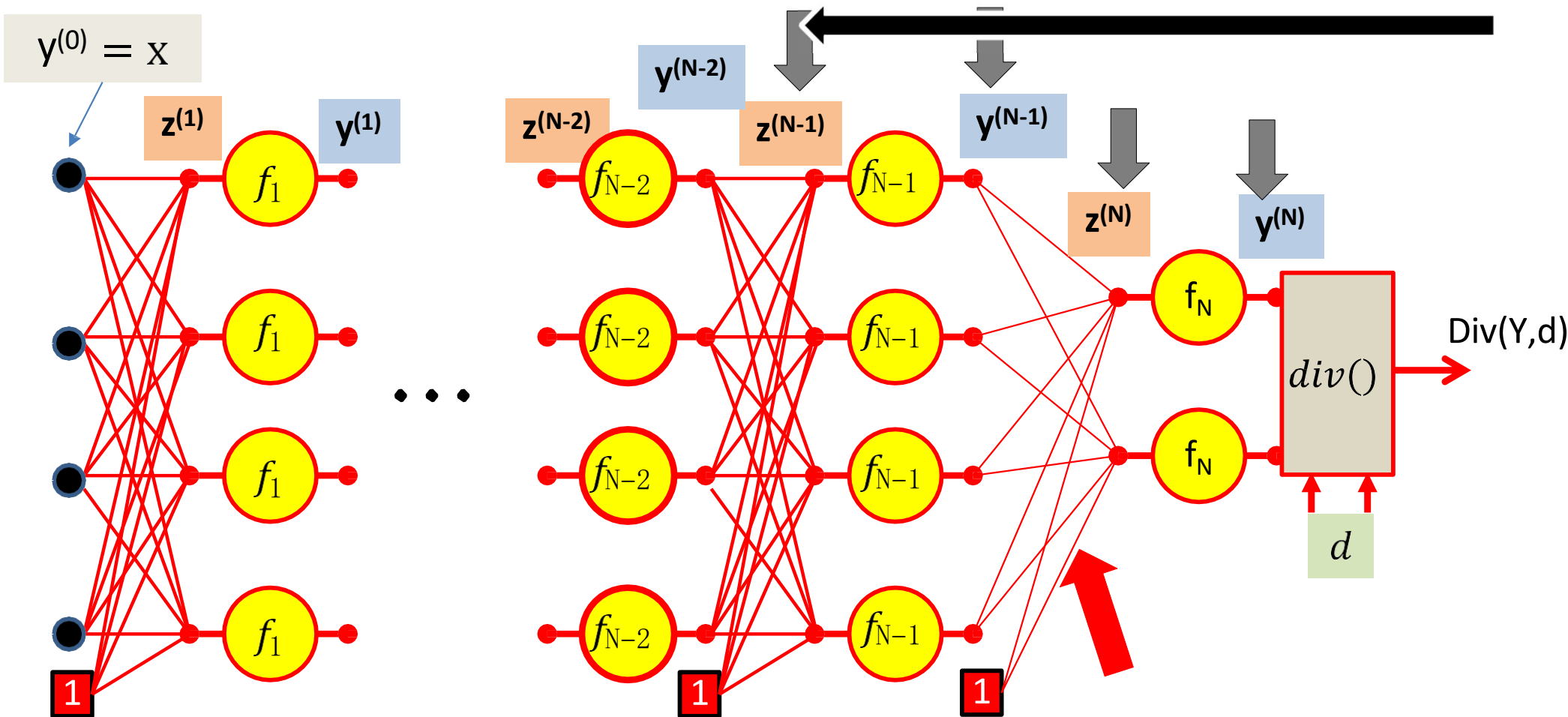
Computing derivatives



Continuing on, we will compute $\nabla_{W^{(N)}} div(.)$ the derivative of the divergence with respect to the weights of the connections to the output layer

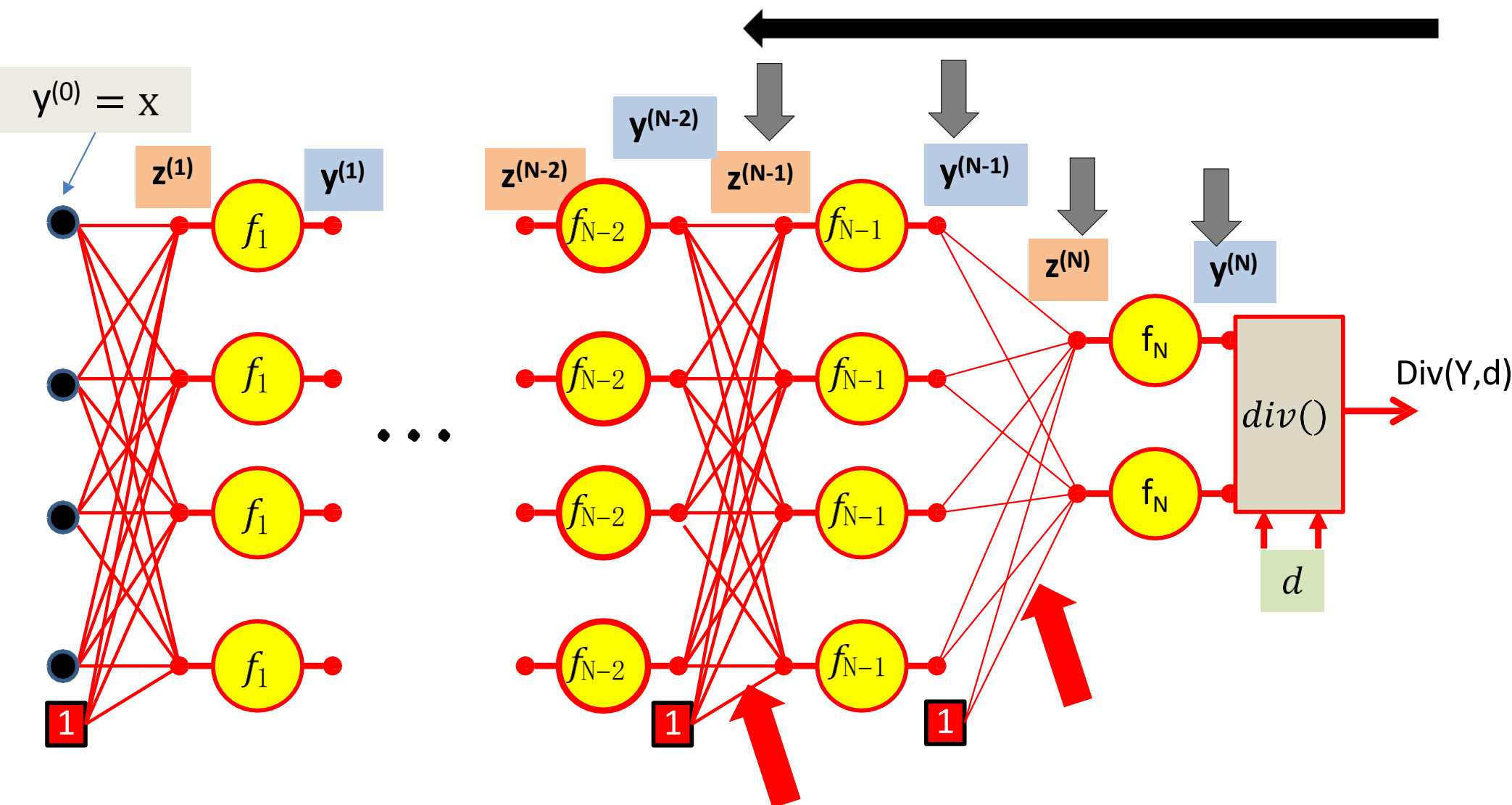
Then continue with the chain rule to compute $\nabla_{F^{(N-1)}} div(.)$ the derivative of the divergence w.r.t. the output of the N-1th layer

Computing derivatives



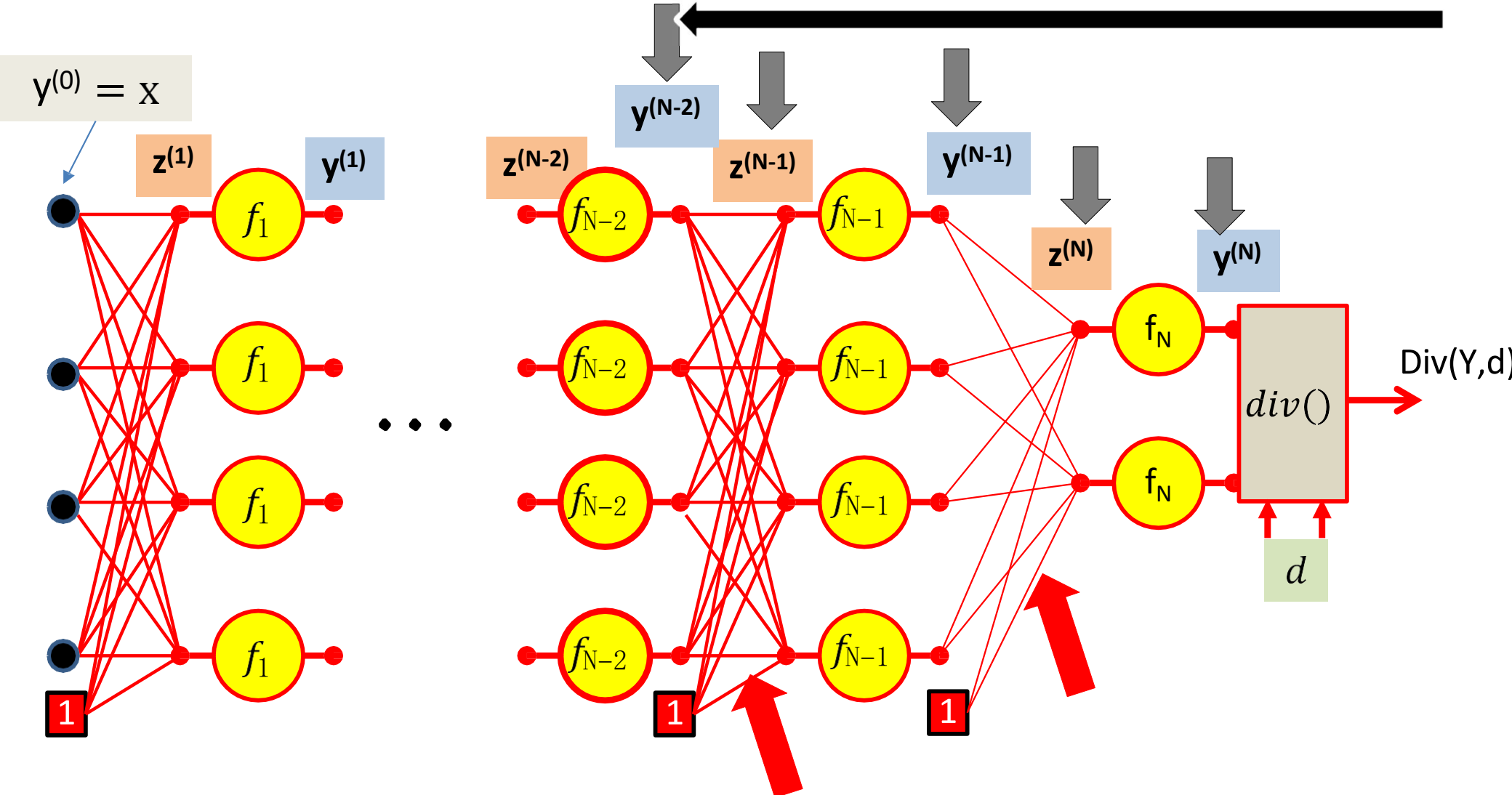
We continue our way backwards in the order shown

$$\nabla_{z^{(N-1)}} div(.)$$



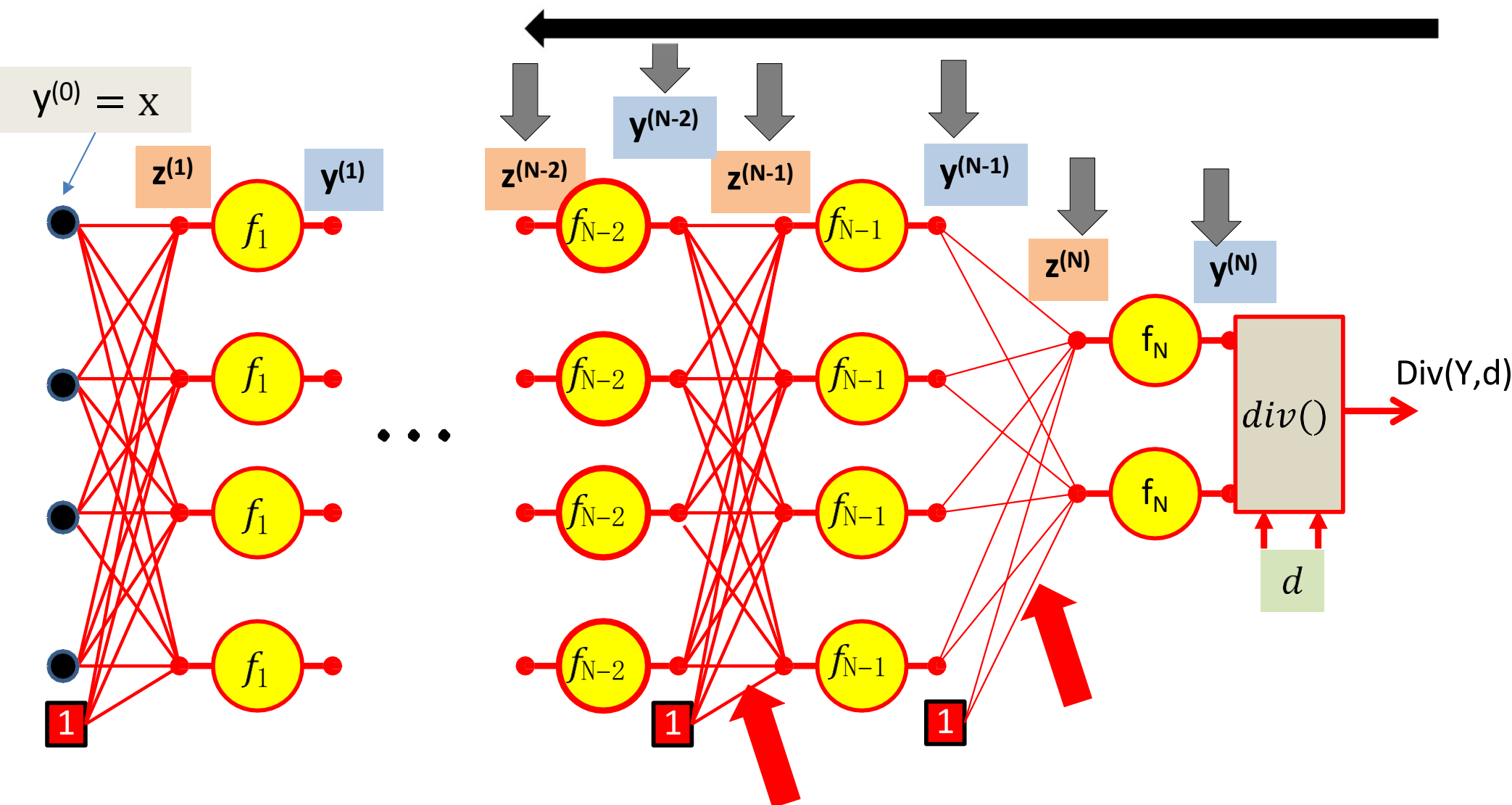
We continue our way backwards in the order shown

$$\nabla_{W^{(N-1)}} \text{div}(\cdot)$$



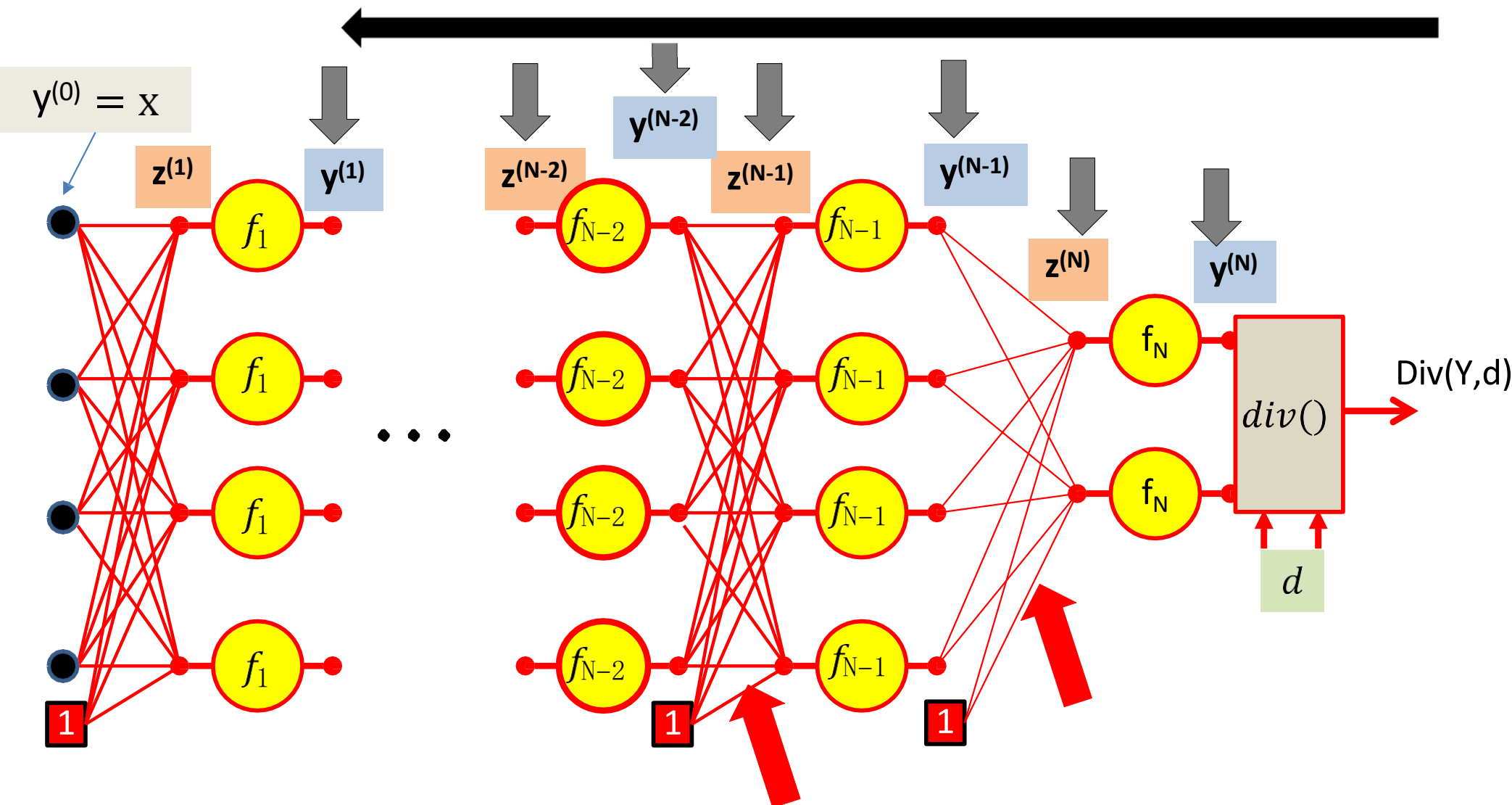
We continue our way backwards in the order shown

$$\nabla_{Y^{(N-2)}} div(.)$$



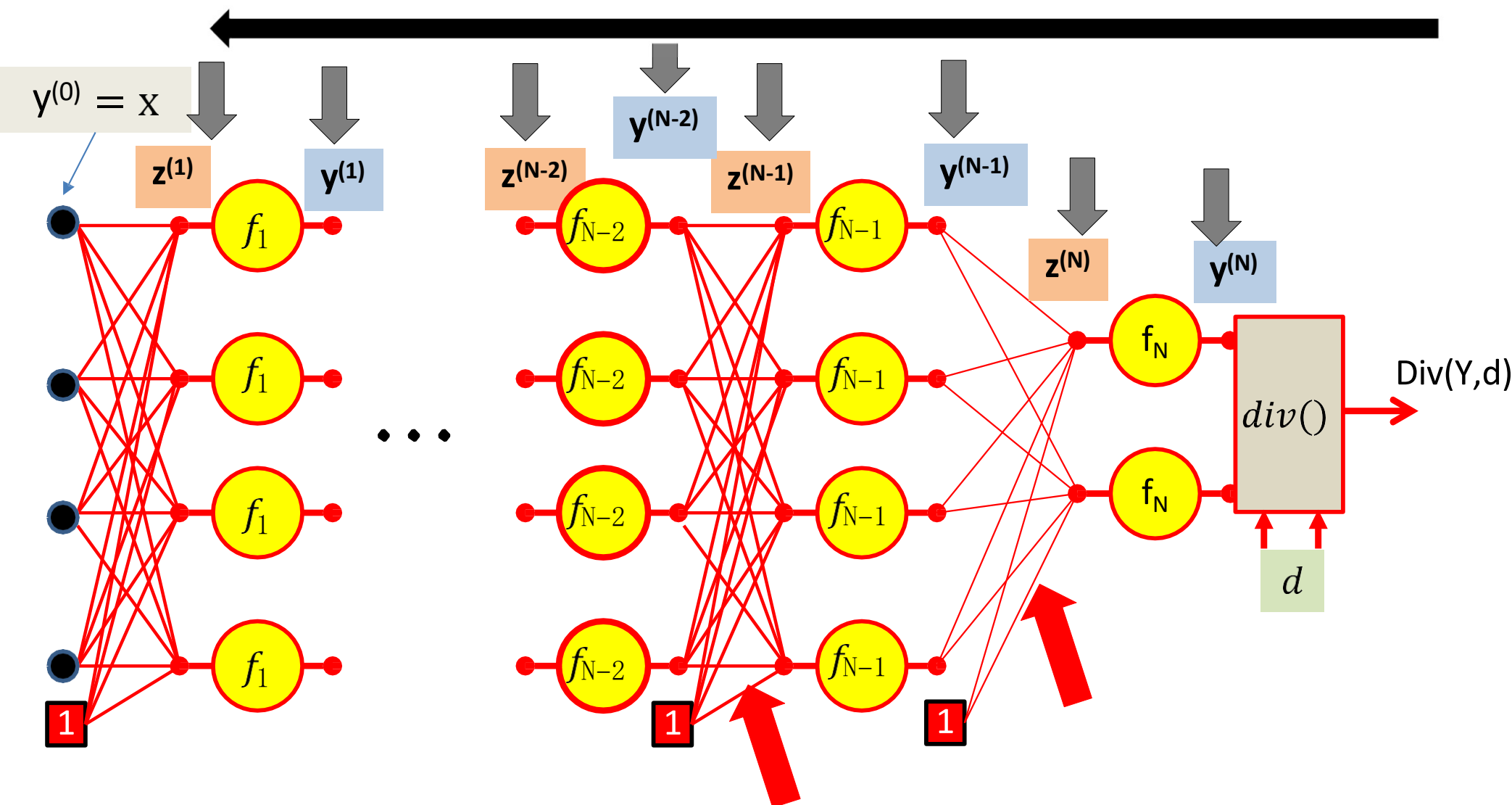
We continue our way backwards in the order shown

$$\nabla_{z^{(N-2)}} div(.)$$



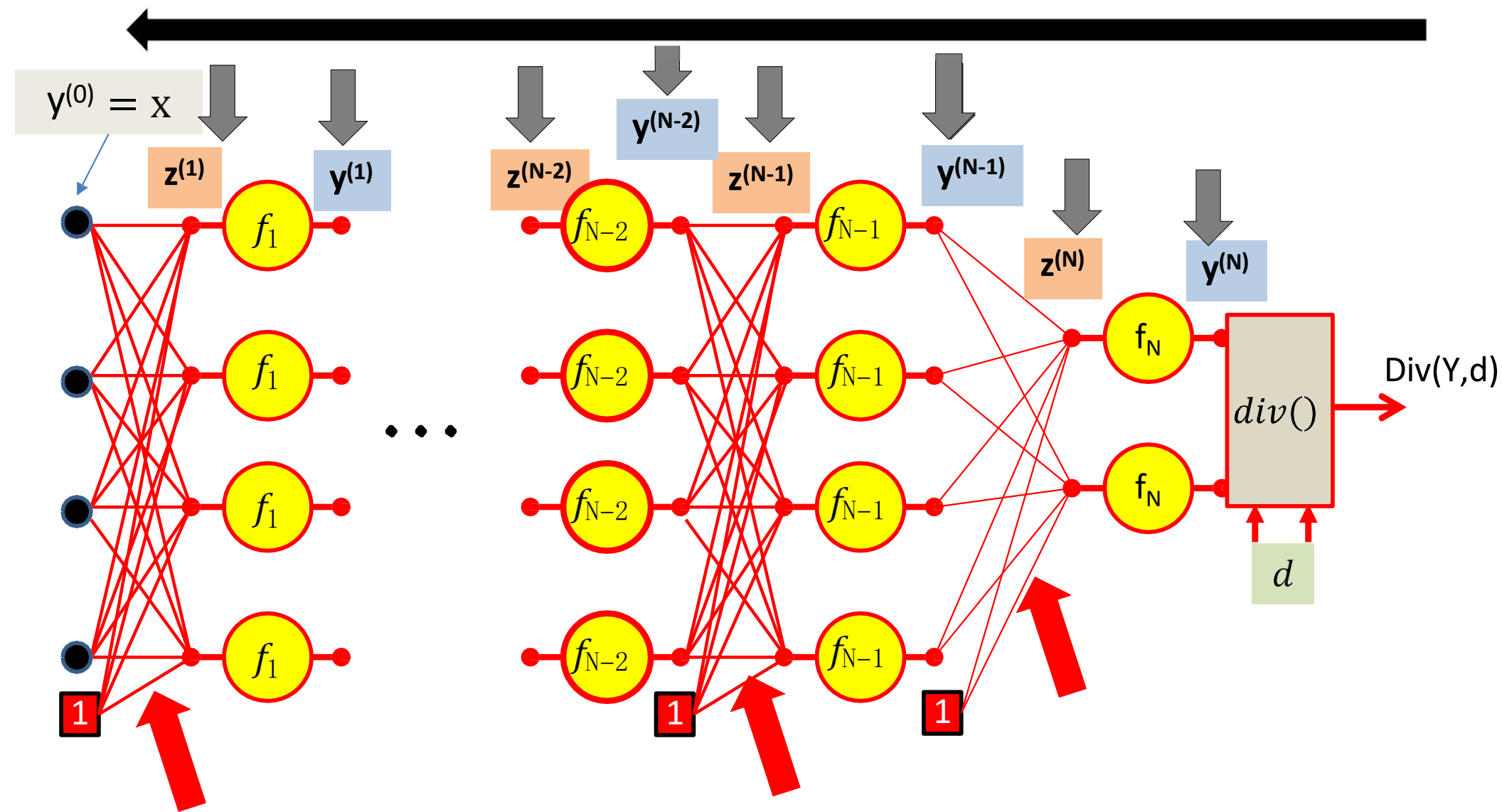
We continue our way backwards in the order shown

$$\nabla_{Y^{(1)}} \text{div}(\cdot)$$



We continue our way backwards in the order shown

$$\nabla_{z^{(1)}} \text{div}(\cdot)$$



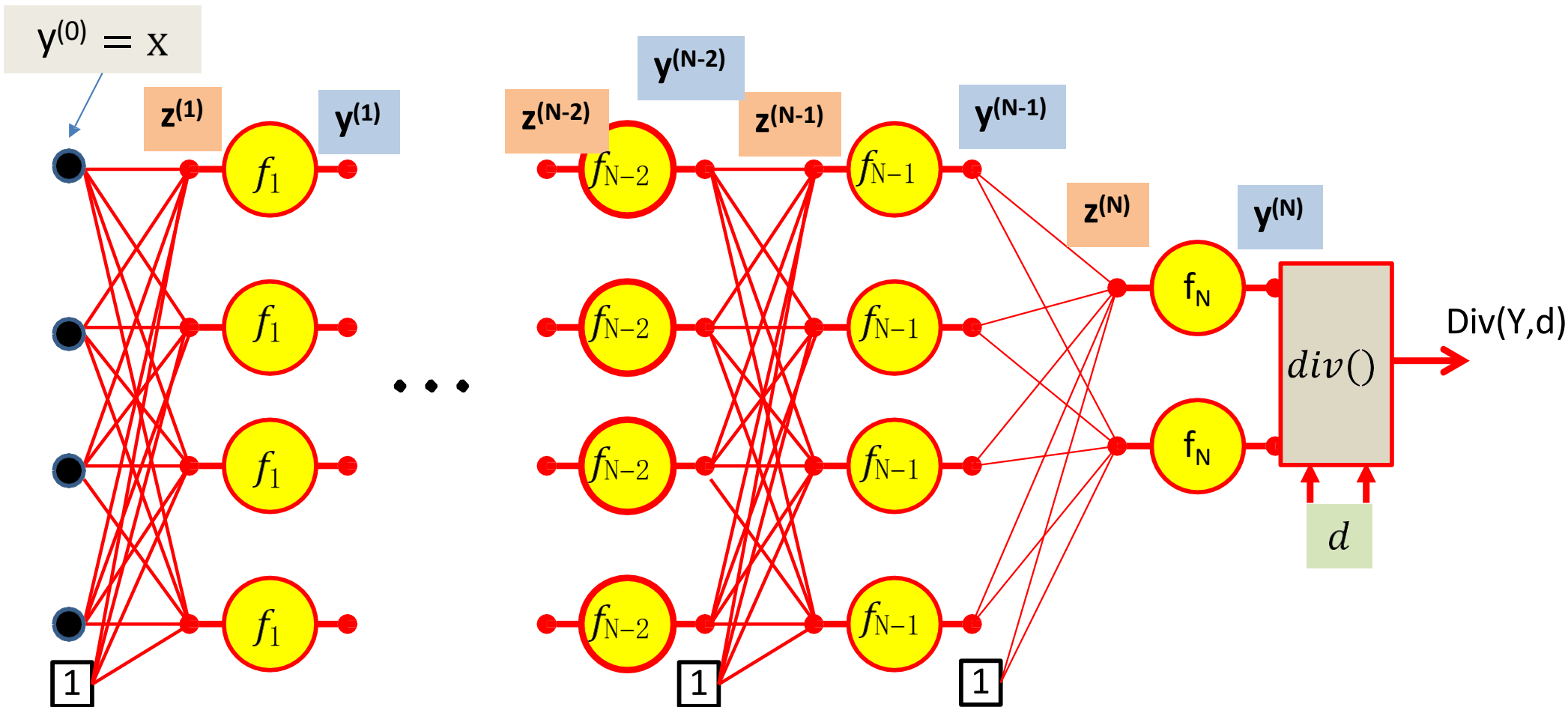
We continue our way backwards in the order shown

$$\nabla_{W^{(1)}} div(.)$$

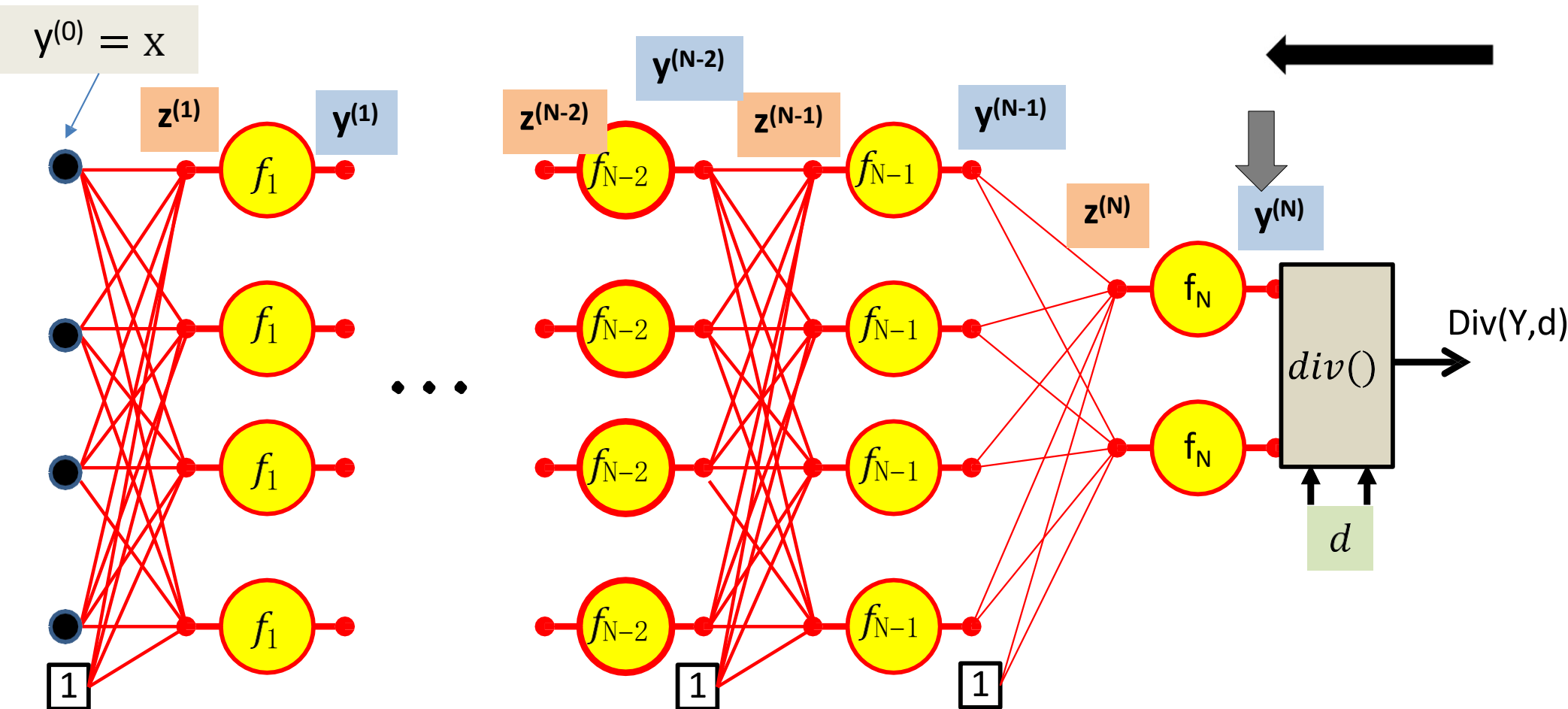
Backward Gradient Computation

- Let's actually see the math..

Computing derivatives



Computing derivatives



The derivative w.r.t the actual output of the final layer of the network is simply the derivative w.r.t to the output of the network

$$\frac{\partial Div(Y, d)}{\partial y_i^{(N)}} = \frac{\partial Div(Y, d)}{\partial y_i}$$

Calculus Refresher: Chain rule

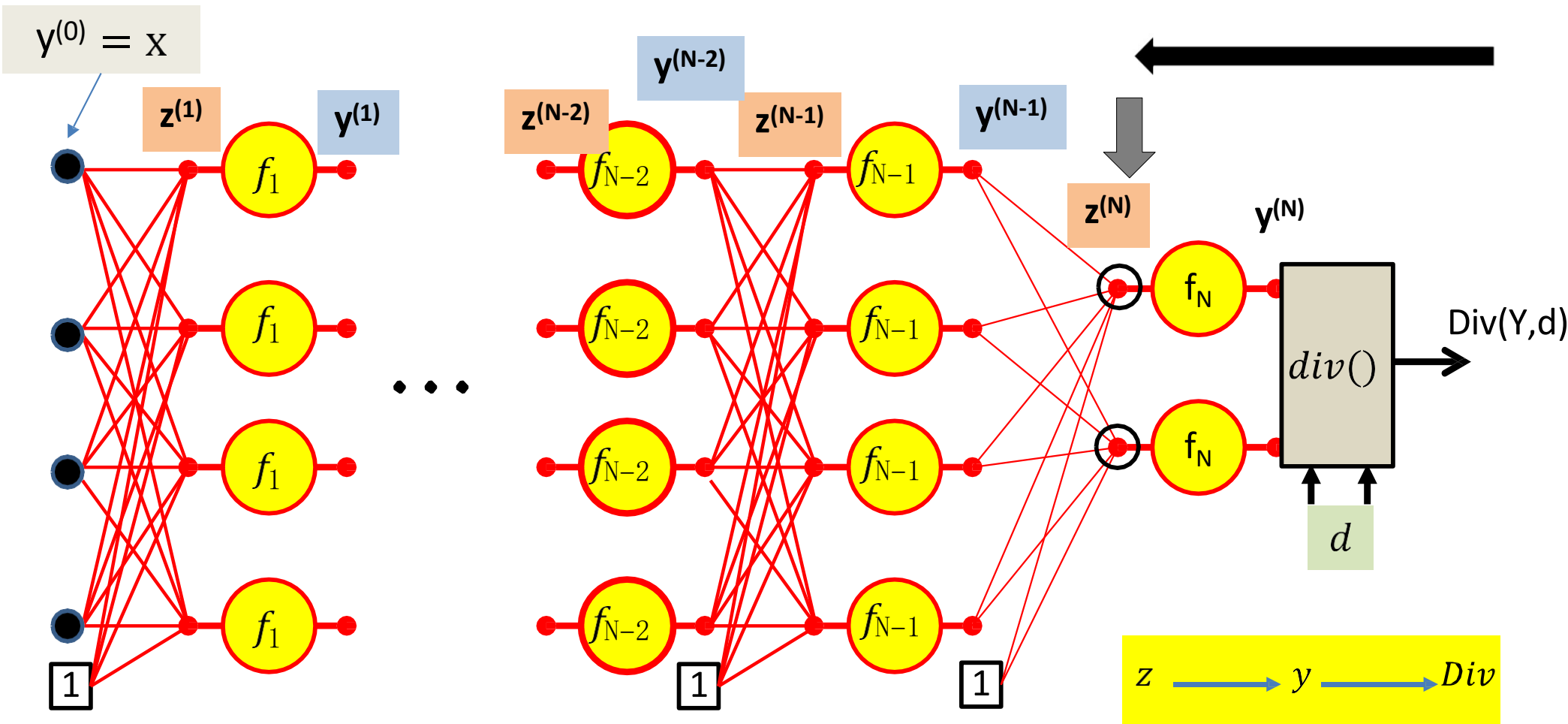
For any nested function $l = f(y)$ where $y = g(z)$

$$\frac{dl}{dz} = \frac{dl}{dy} \frac{dy}{dz}$$

$$\frac{dl}{dz} = \frac{dl}{dy} \frac{dy}{dz}$$

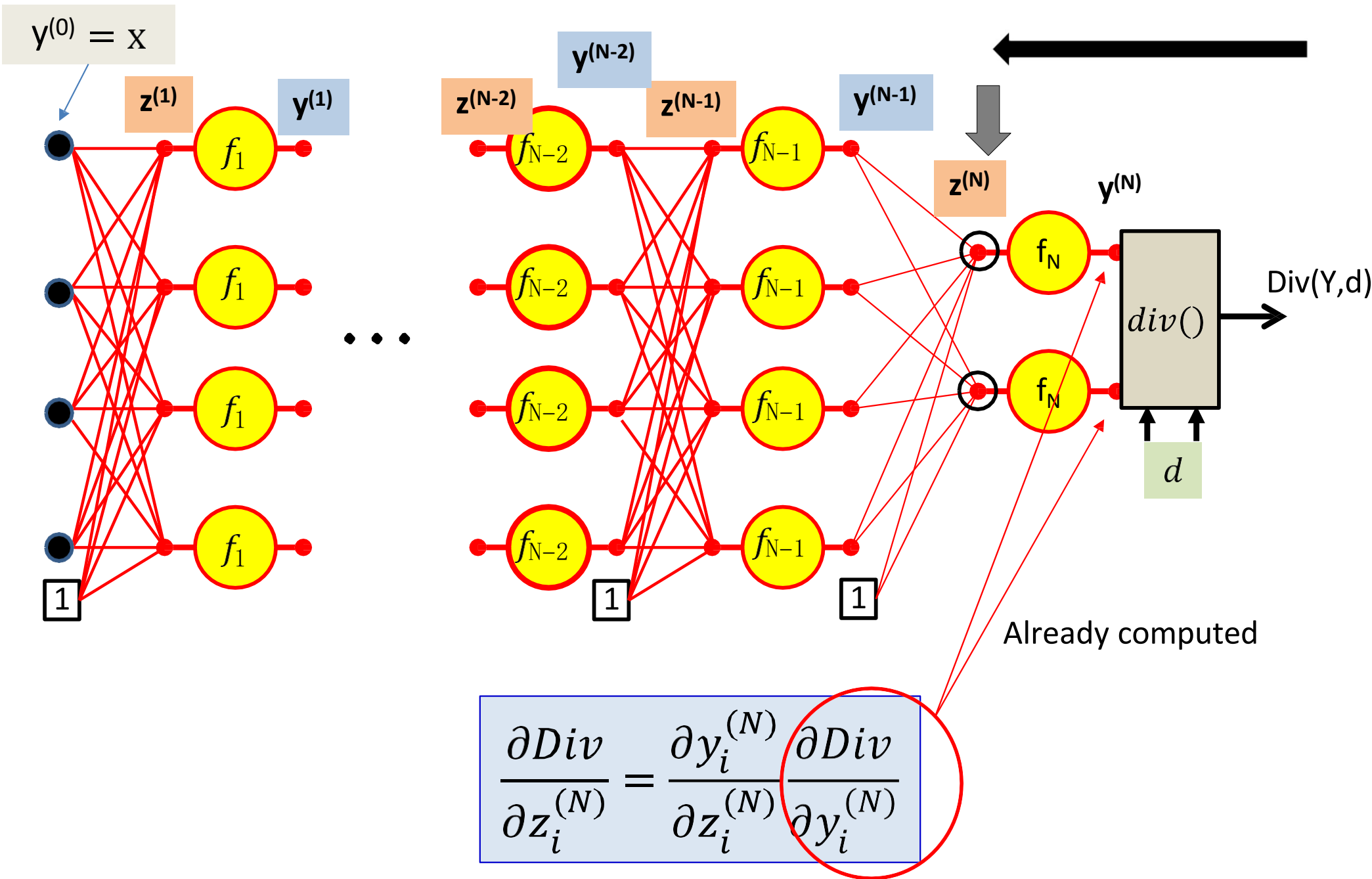


Computing derivatives

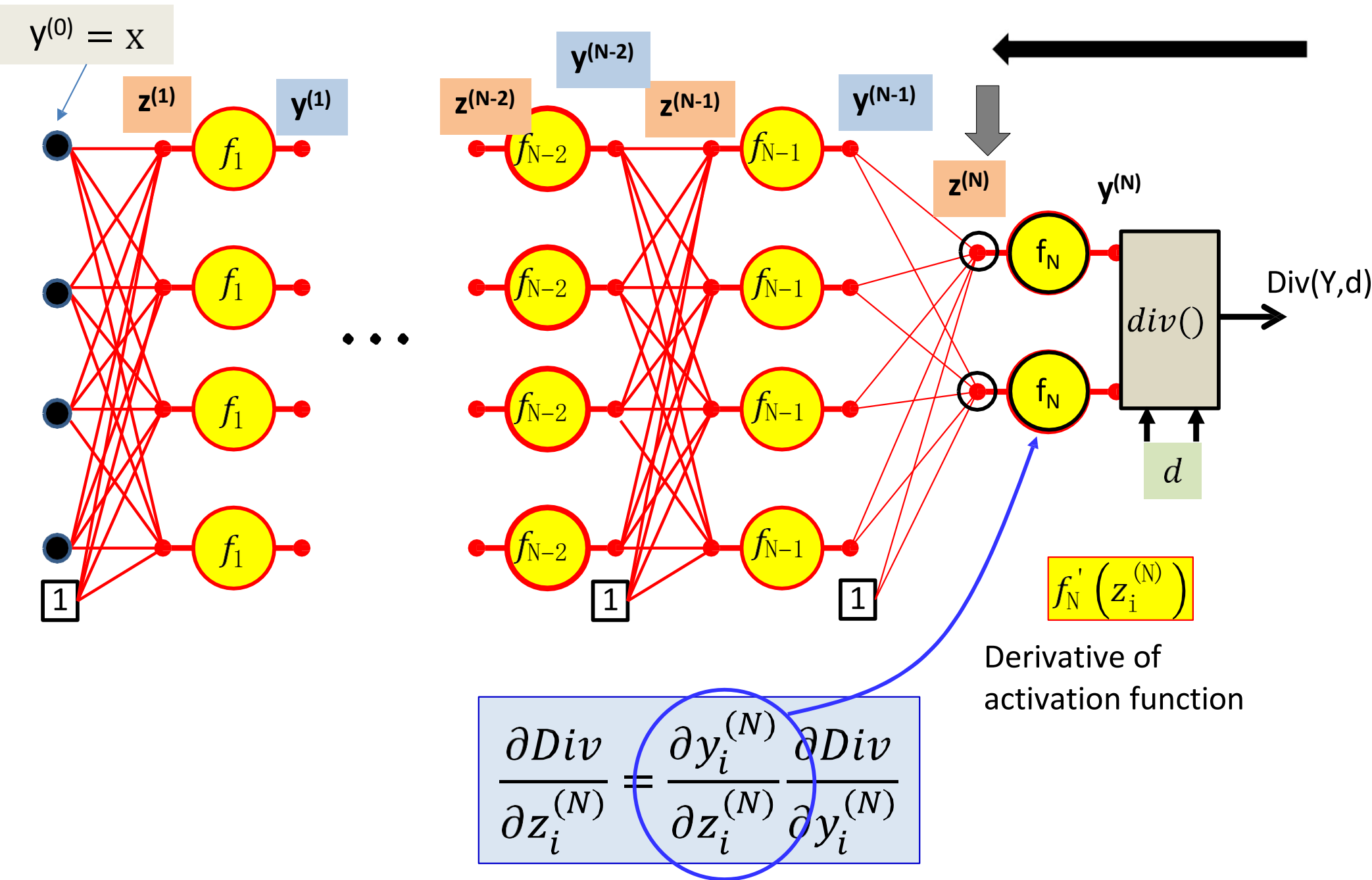


$$\frac{\partial Div}{\partial z_i^{(N)}} = \frac{\partial y_i^{(N)}}{\partial z_i^{(N)}} \frac{\partial Div}{\partial y_i^{(N)}}$$

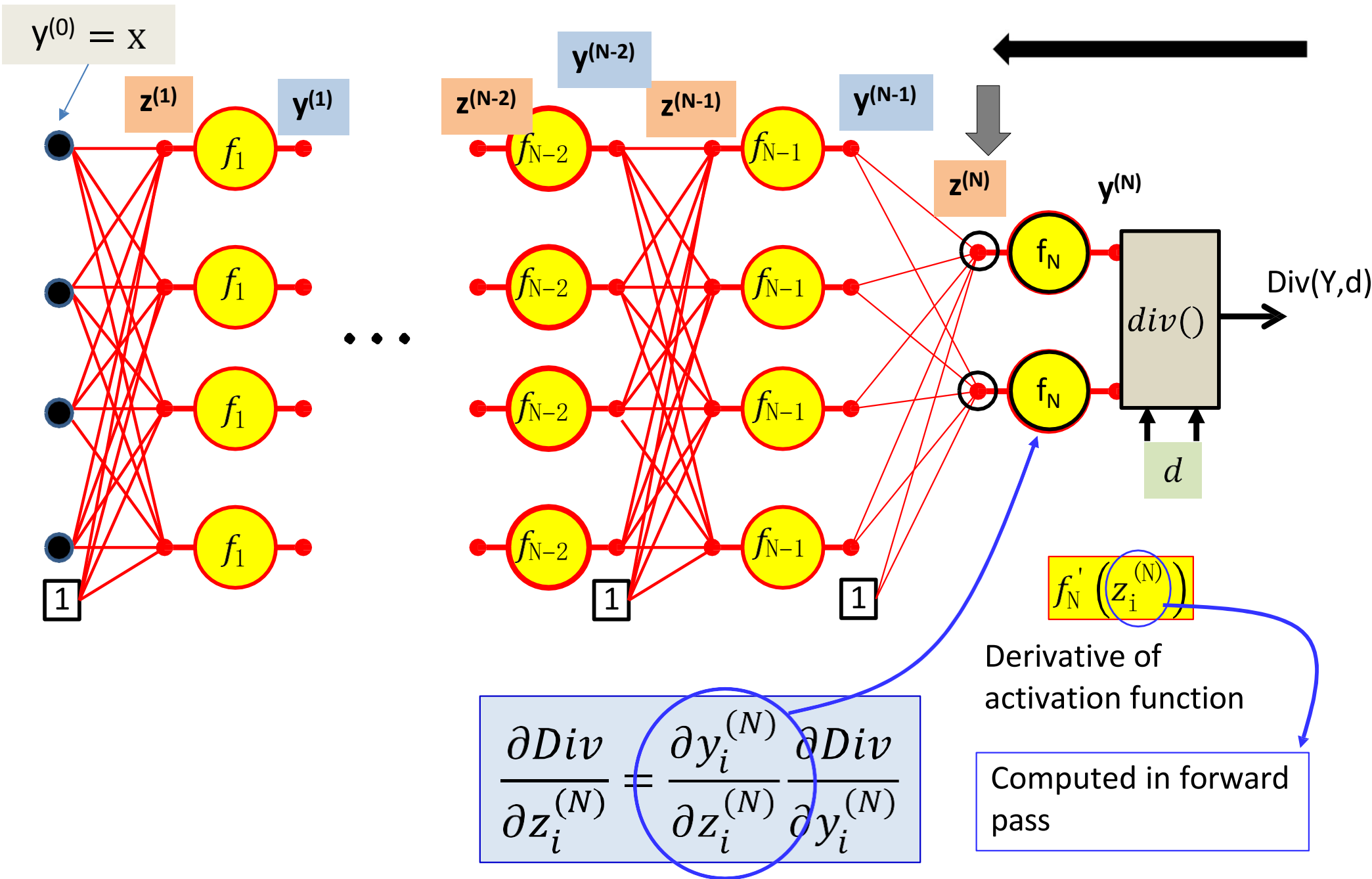
Computing derivatives



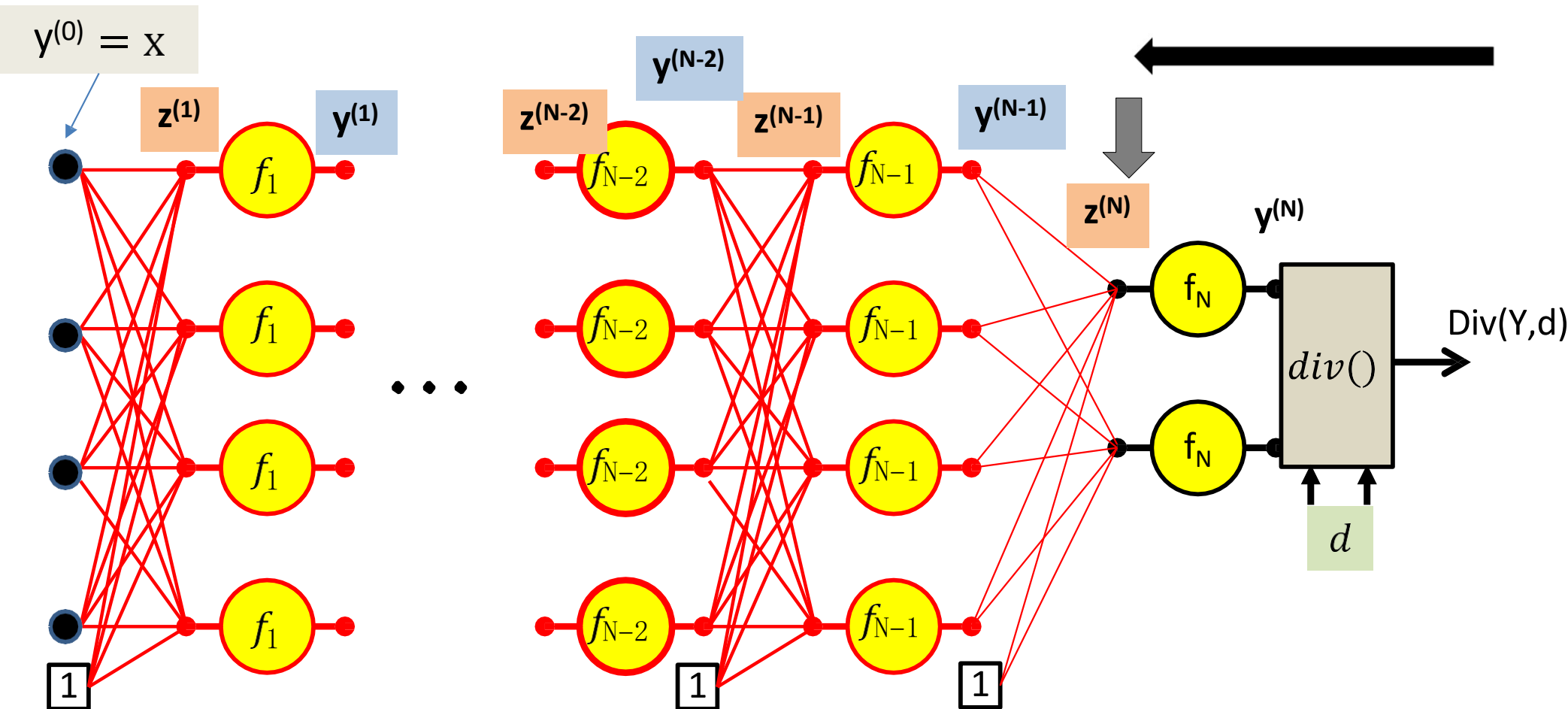
Computing derivatives



Computing derivatives

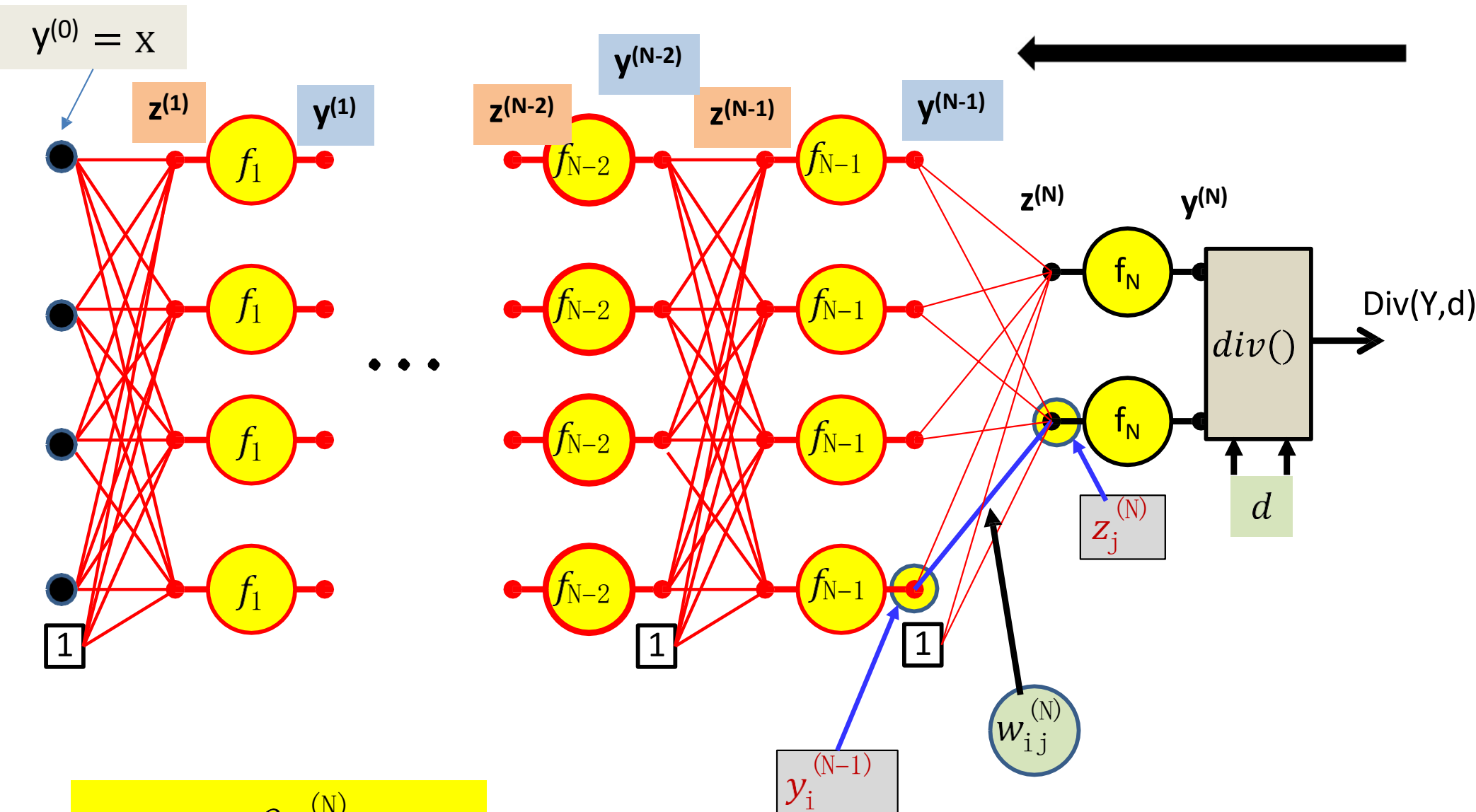


Computing derivatives



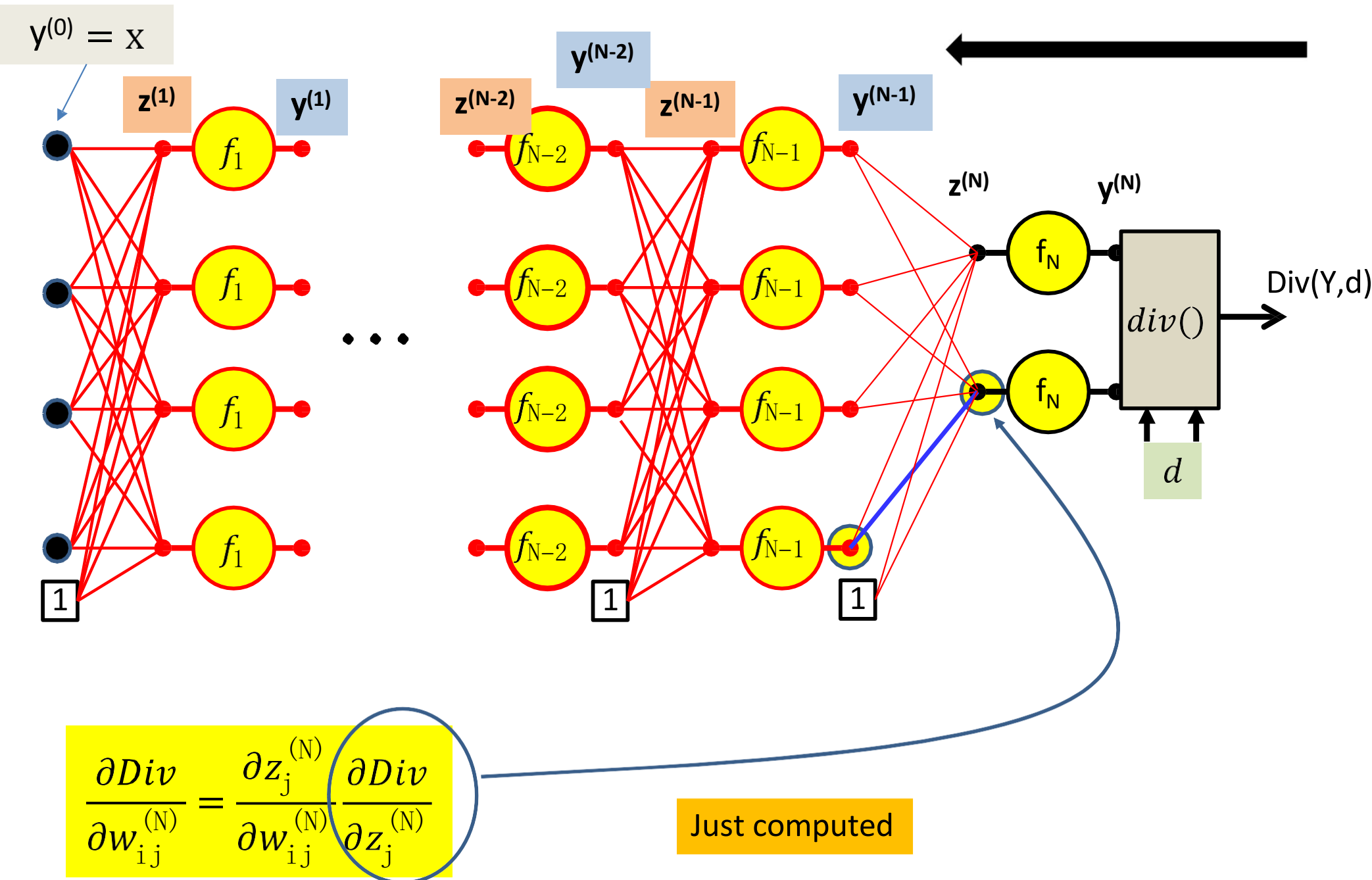
$$\frac{\partial Div}{\partial z_i^{(N)}} = f'_N \left(z_i^{(N)} \right) \frac{\partial Div}{\partial y_i^{(N)}}$$

Computing derivatives

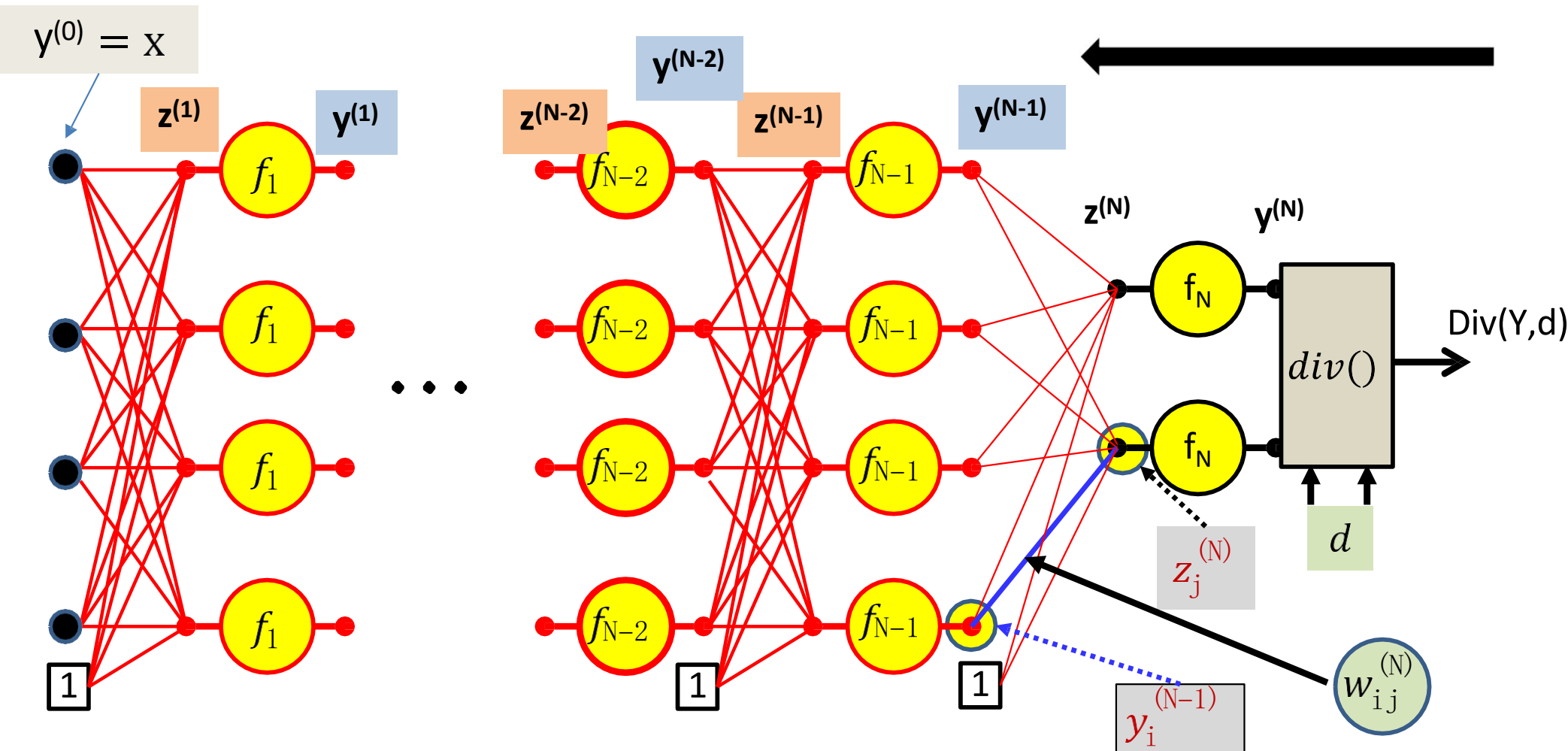


$$\frac{\partial Div}{\partial w_{ij}^{(N)}} = \frac{\partial z_j^{(N)}}{\partial w_{ij}^{(N)}} \frac{\partial Div}{\partial z_j^{(N)}}$$

Computing derivatives



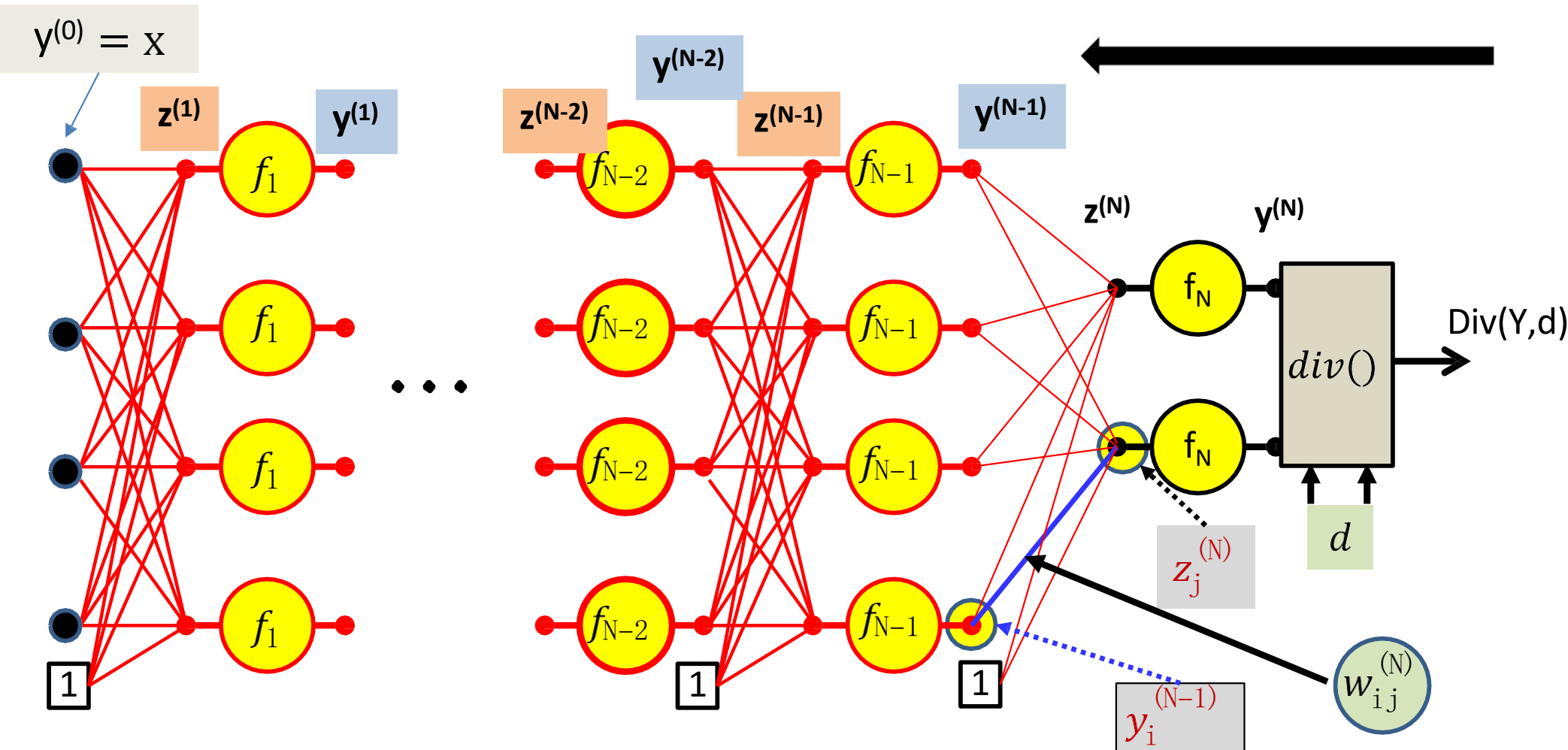
Computing derivatives



$$\frac{\partial Div}{\partial w_{ij}^{(N)}} = \frac{\partial z_j^{(N)}}{\partial w_{ij}^{(N)}} \frac{\partial Div}{\partial z_j^{(N)}}$$

Because $z_j^{(N)} = w_{ij}^{(N)} y_i^{(N-1)} + \text{other terms}$

Computing derivatives

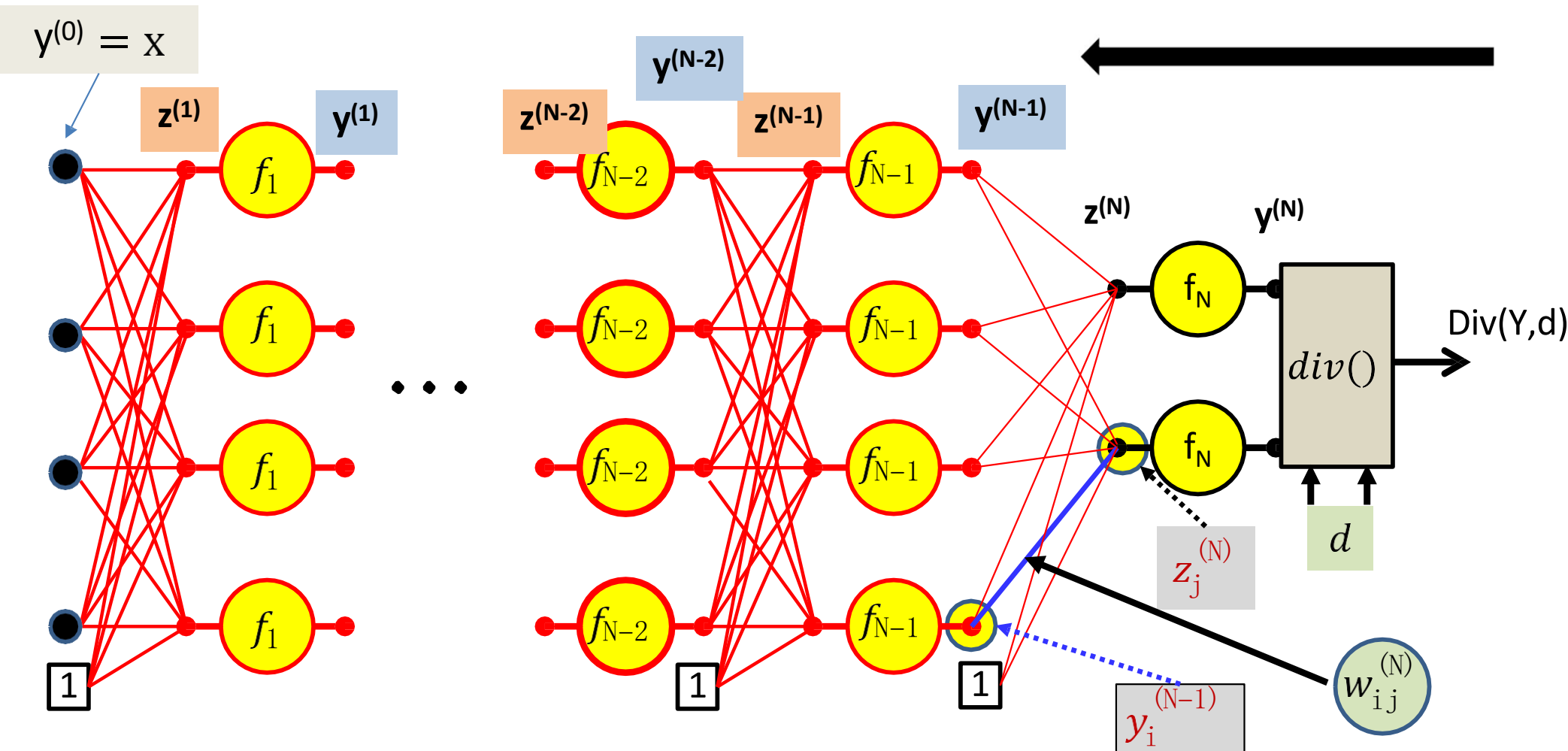


$$\frac{\partial \text{Div}}{\partial w_{ij}^{(N)}} = \frac{\partial z_j^{(N)}}{\partial w_{ij}^{(N)}} \frac{\partial \text{Div}}{\partial z_j^{(N)}}$$

Computed in forward pass

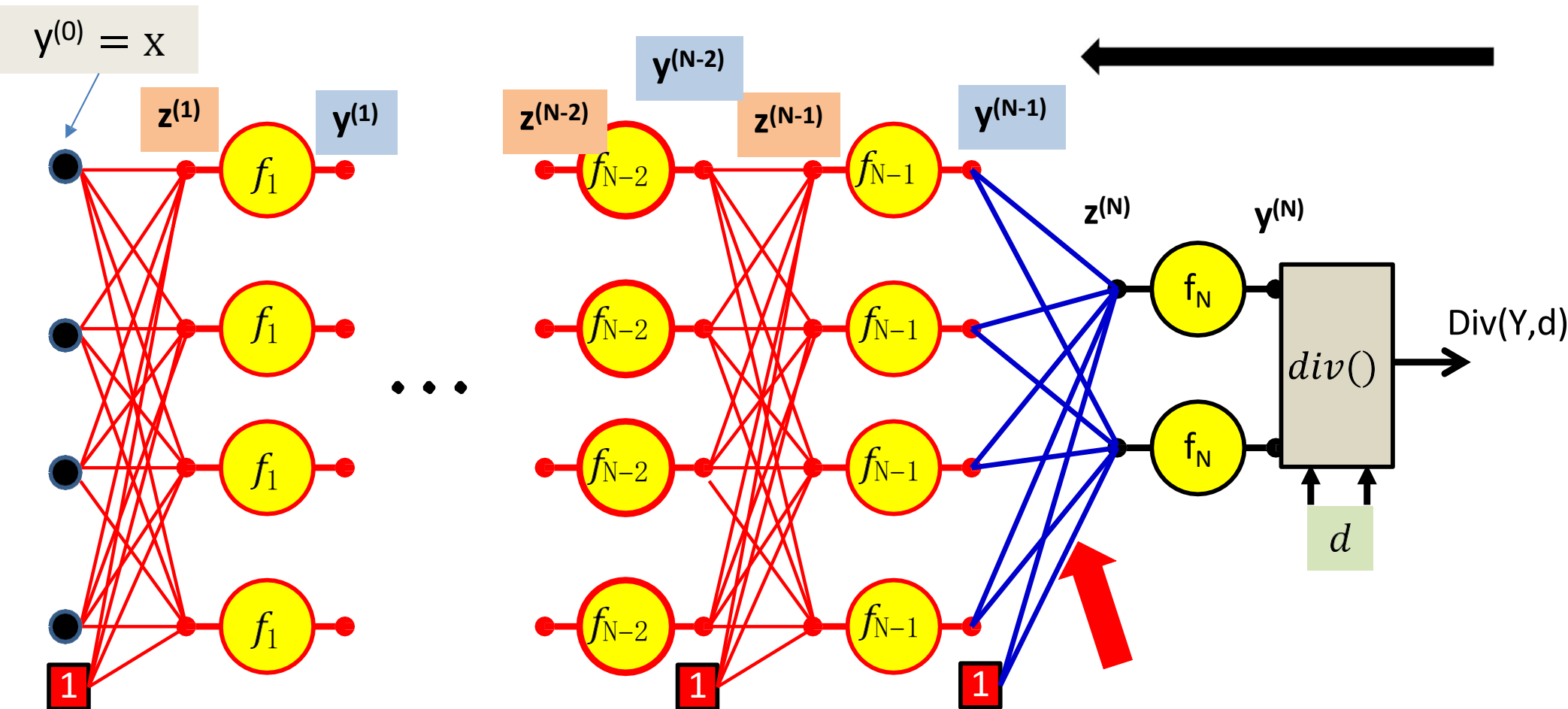
Because $z_j^{(N)} = w_{ij}^{(N)} y_i^{(N-1)} + \text{other terms}$

Computing derivatives



$$\frac{\partial Div}{\partial w_{ij}^{(N)}} = y_i^{(N-1)} \frac{\partial Div}{\partial z_j^{(N)}}$$

Computing derivatives

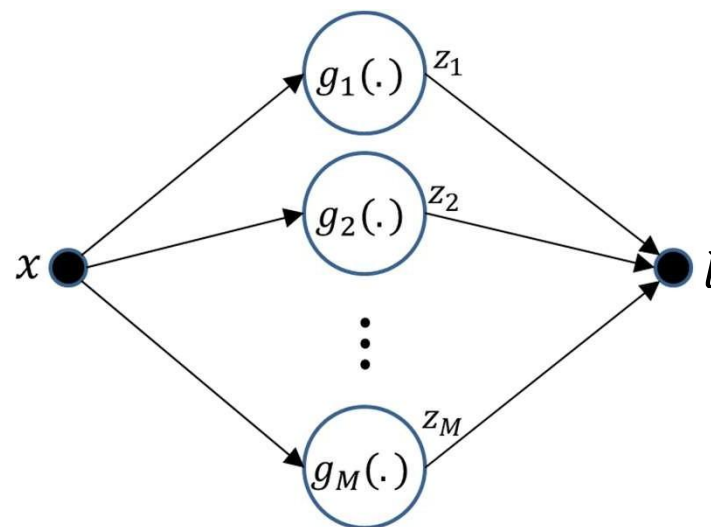


$$\frac{\partial Div}{\partial w_{ij}^{(N)}} = y_i^{(N-1)} \frac{\partial Div}{\partial z_j^{(N)}}$$

For the bias term $y_0^{(N-1)} = 1$

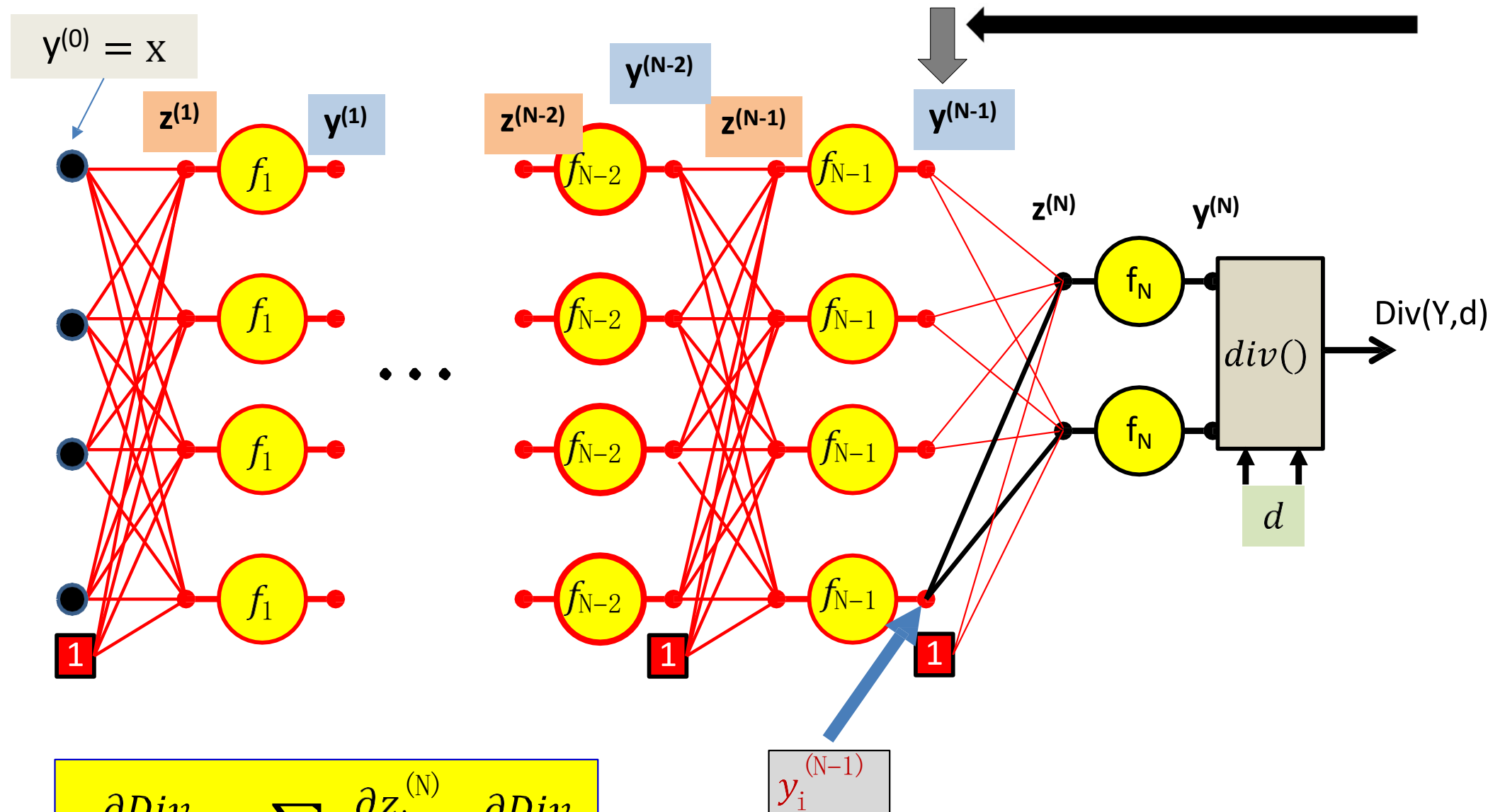
Calculus Refresher: Chain rule

For $l = f(z_1, z_2, \dots, z_M)$
where $z_i = g_i(x)$



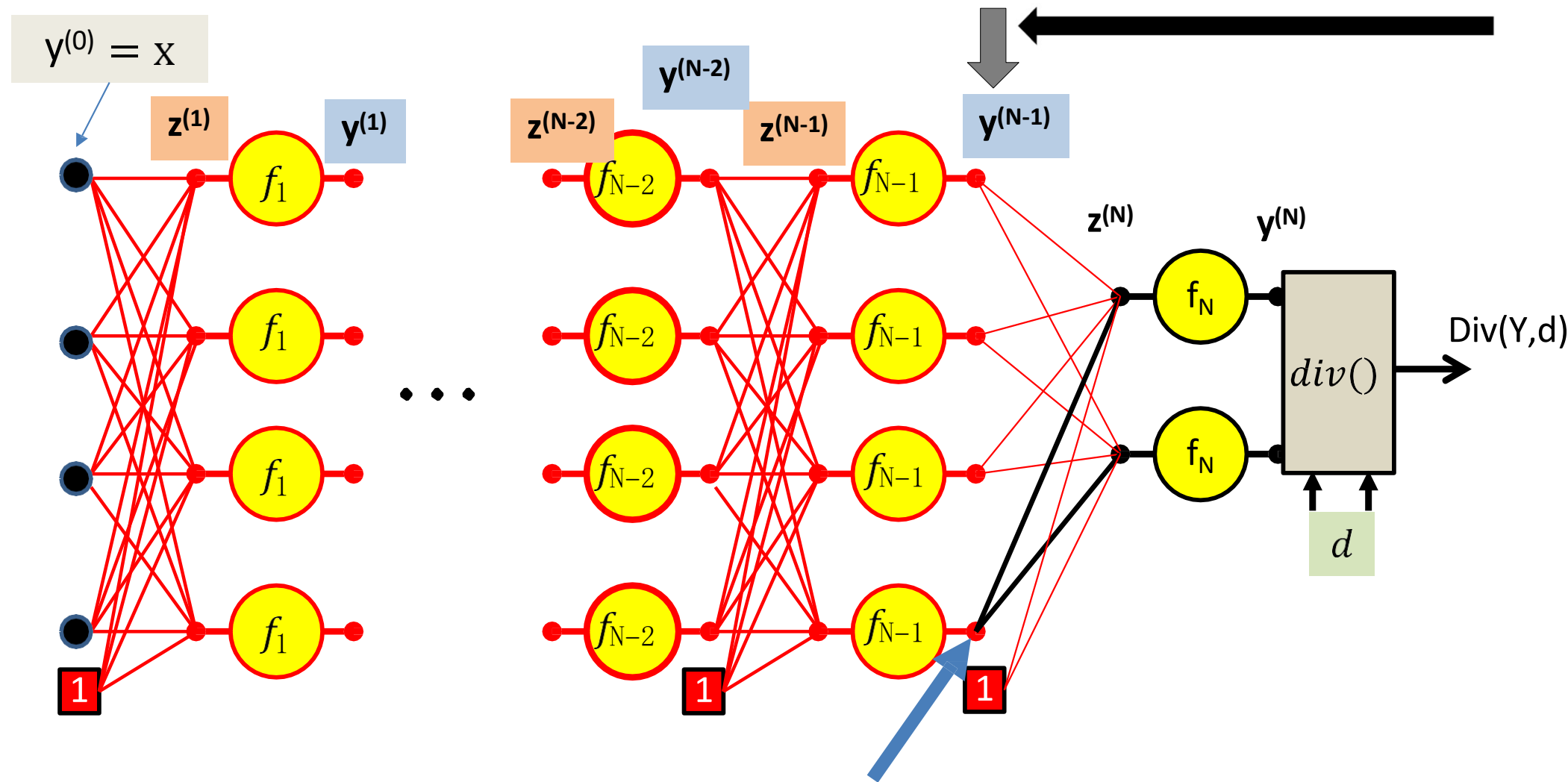
$$\frac{dl}{dx} = \frac{\partial l}{\partial z_1} \frac{dz_1}{dx} + \frac{\partial l}{\partial z_2} \frac{dz_2}{dx} + \dots + \frac{\partial l}{\partial z_M} \frac{dz_M}{dx}$$

Computing derivatives



$$\frac{\partial Div}{\partial y_i^{(N-1)}} = \sum_j \frac{\partial z_j^{(N)}}{\partial y_i^{(N-1)}} \frac{\partial Div}{\partial z_j^{(N)}}$$

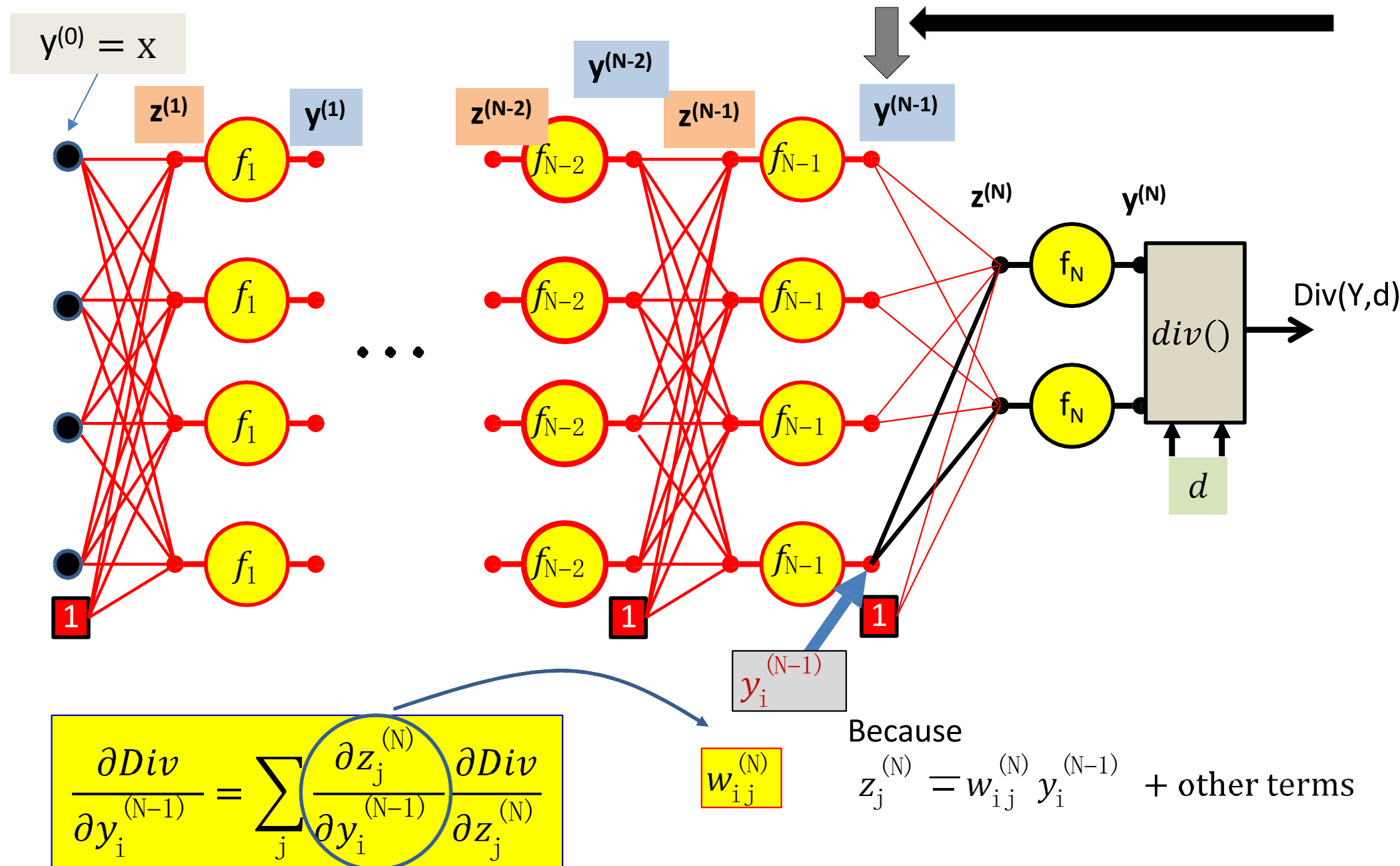
Computing derivatives



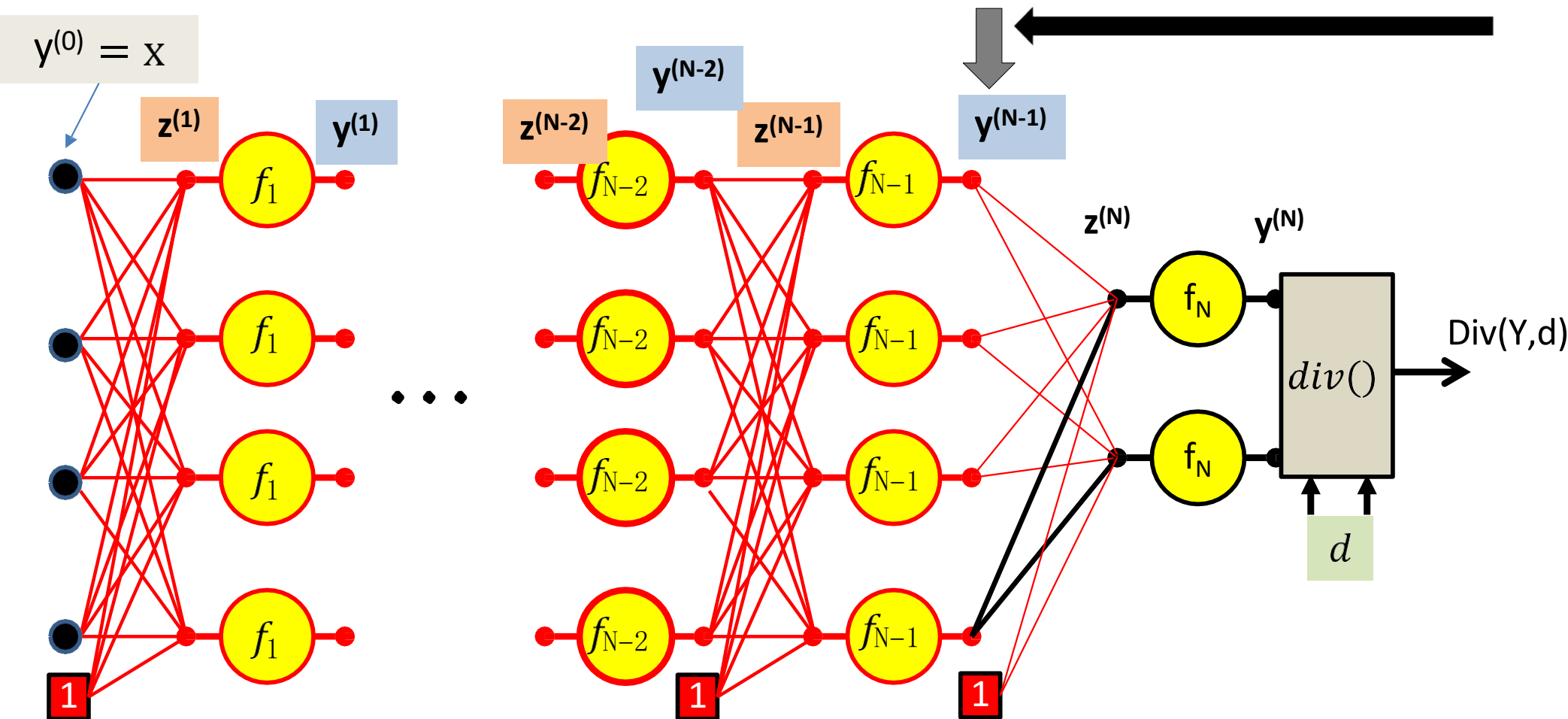
$$\frac{\partial Div}{\partial y_i^{(N-1)}} = \sum_j \frac{\partial z_j^{(N)}}{\partial y_i^{(N-1)}} \frac{\partial Div}{\partial z_j^{(N)}}$$

Already computed

Computing derivatives

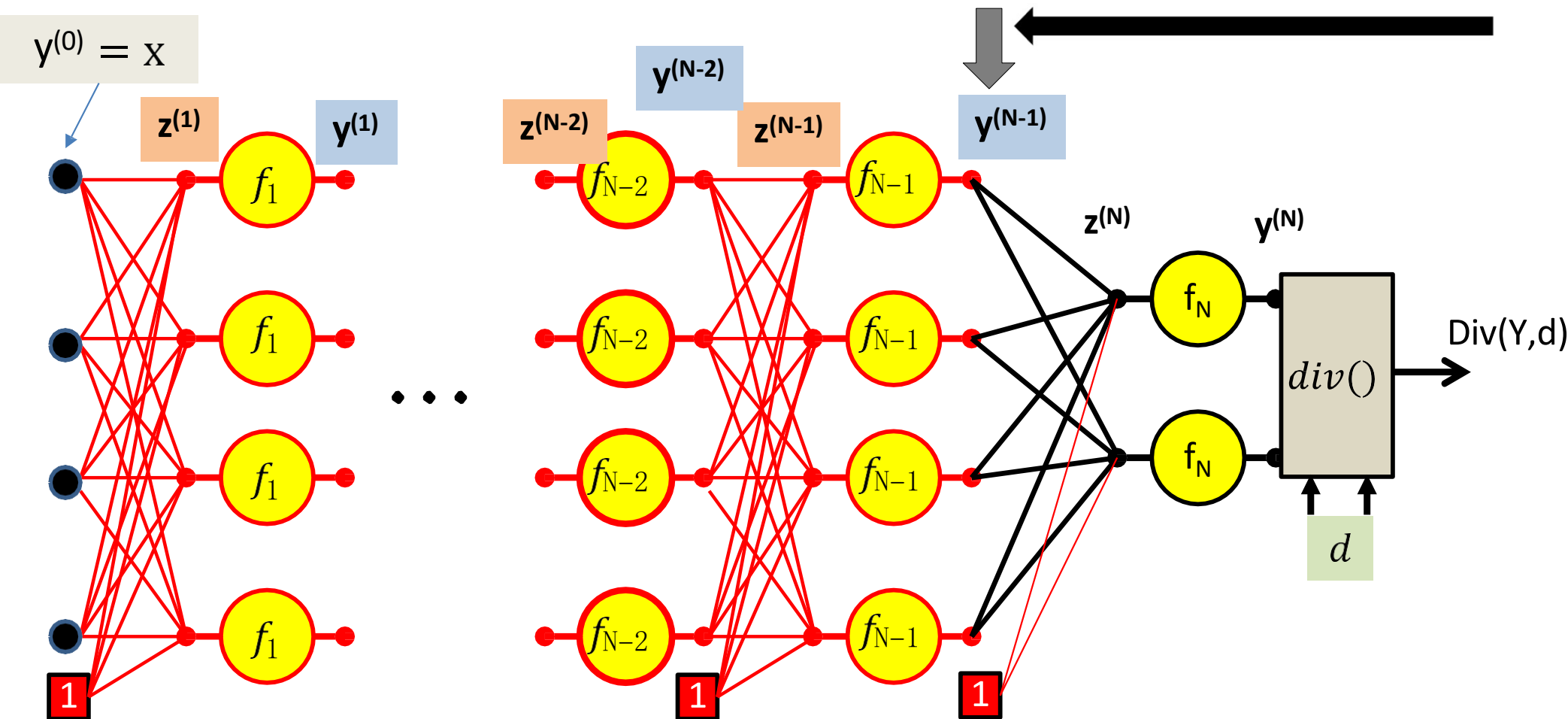


Computing derivatives



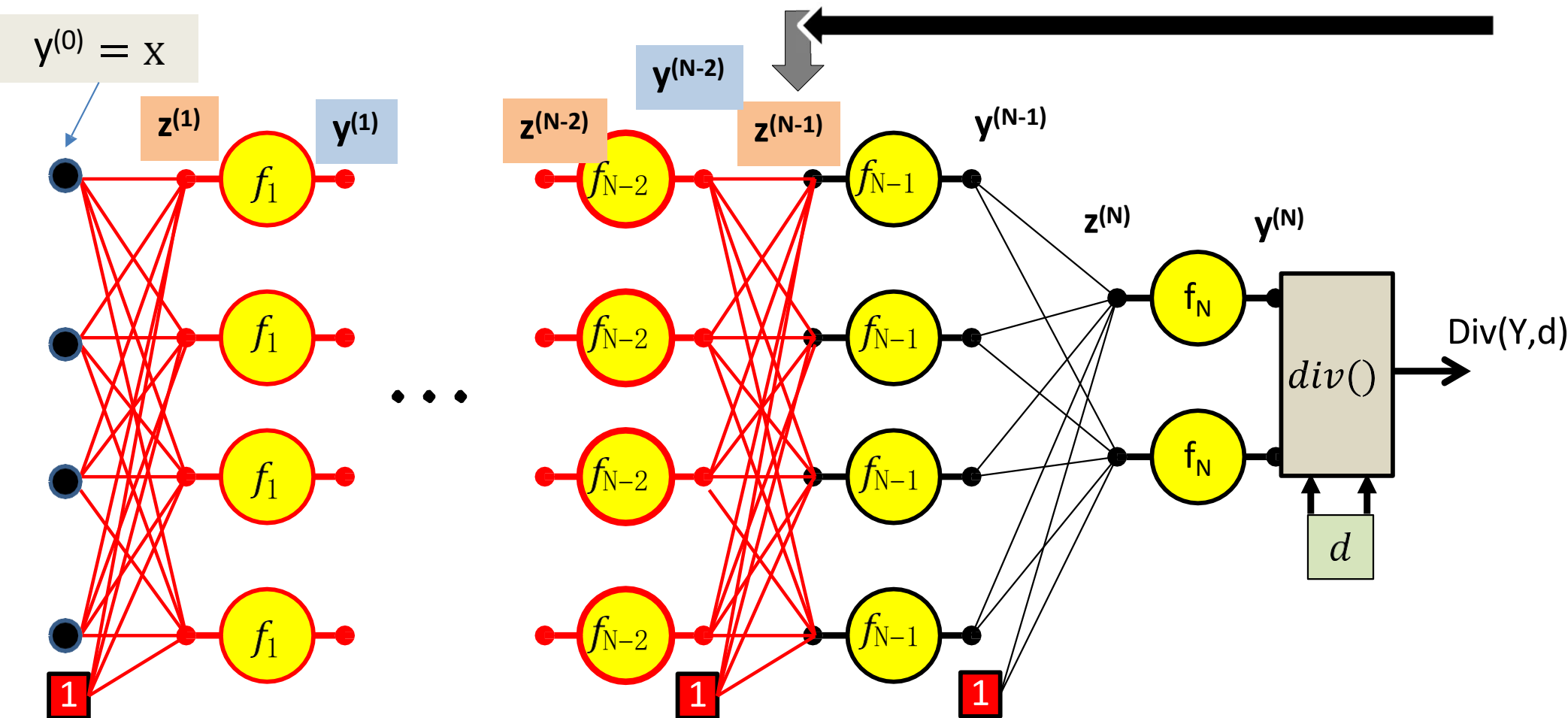
$$\frac{\partial Div}{\partial y_i^{(N-1)}} = \sum_j w_{ij}^{(N)} \frac{\partial Div}{\partial z_j^{(N)}}$$

Computing derivatives



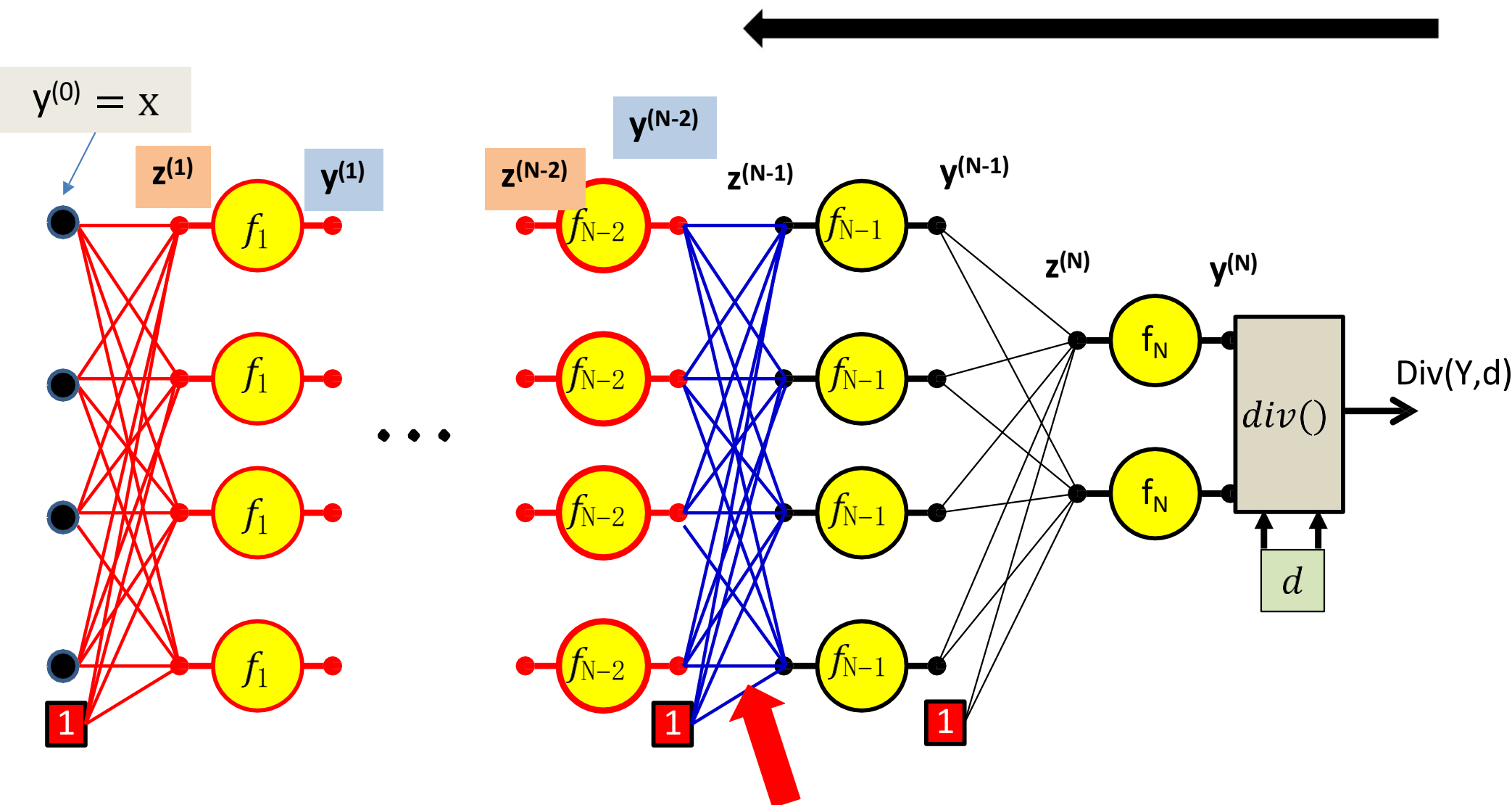
$$\frac{\partial Div}{\partial y_i^{(N-1)}} = \sum_j w_{ij}^{(N)} \frac{\partial Div}{\partial z_j^{(N)}}$$

Computing derivatives



We continue our way backwards in the order shown

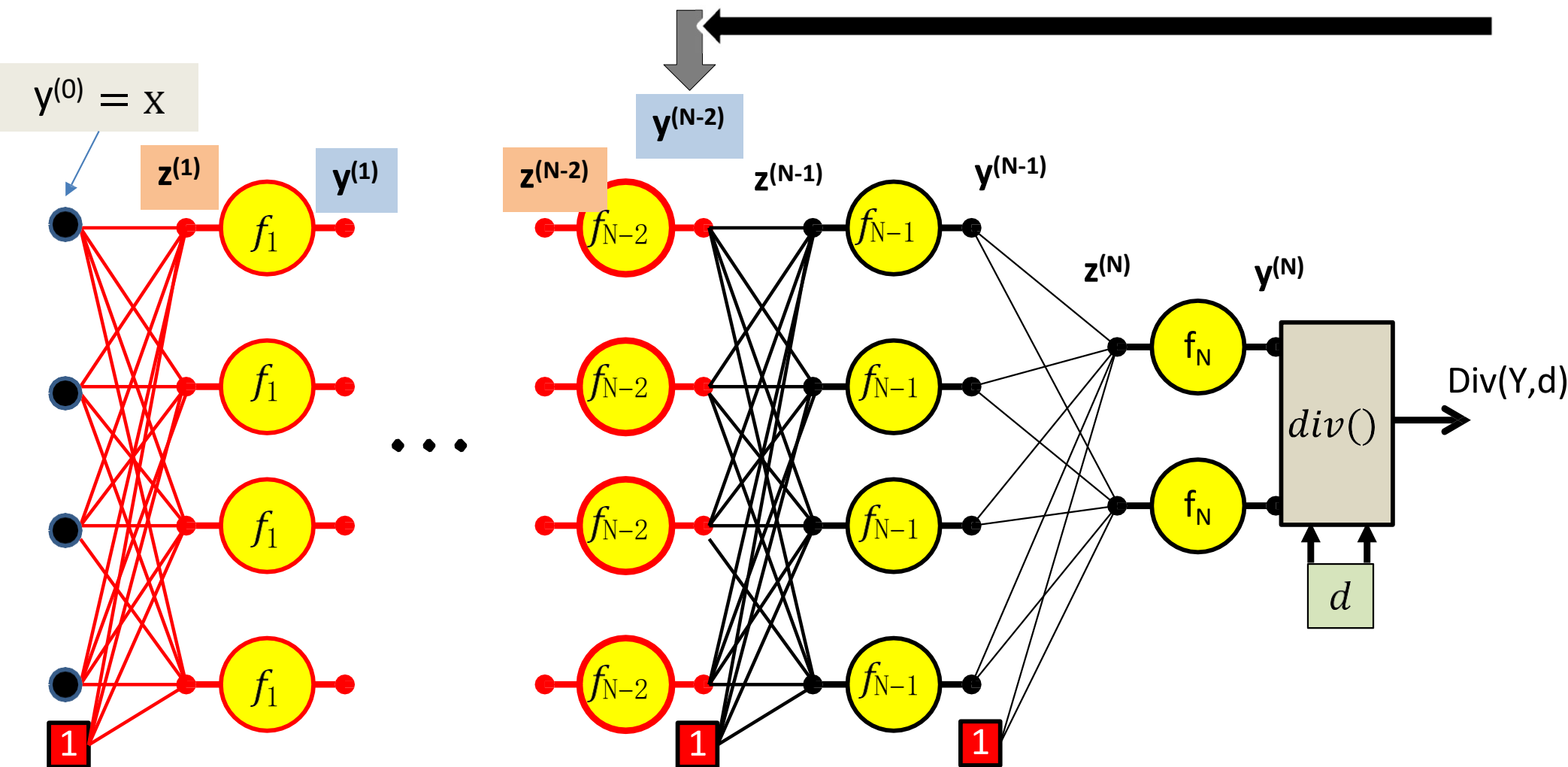
$$\frac{\partial Div}{\partial z_i^{(N-1)}} = f'_{N-1} \left(z_i^{(N-1)} \right) \frac{\partial Div}{\partial y_i^{(N-1)}}$$



We continue our way backwards in the order shown

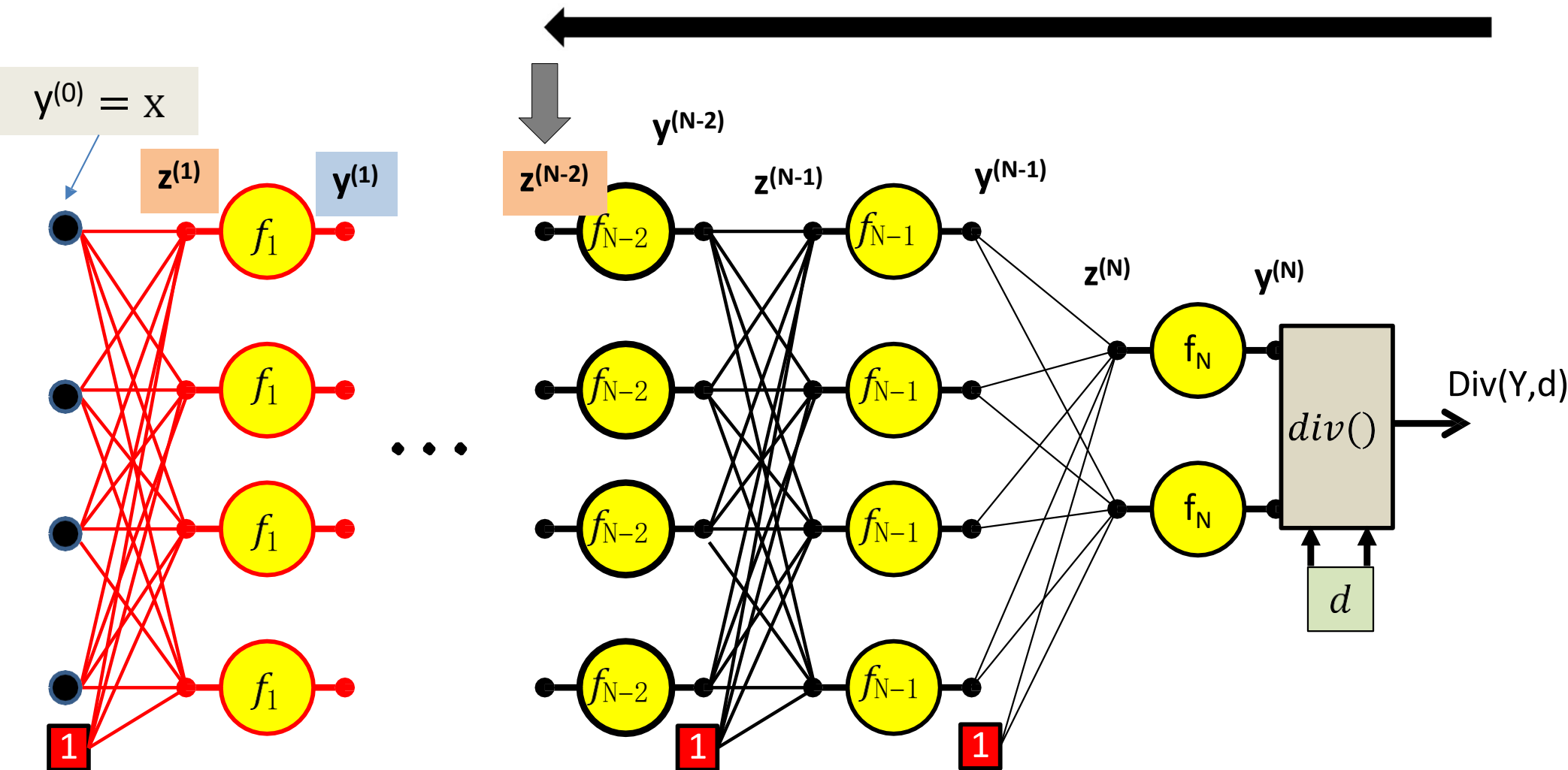
$$\frac{\partial Div}{\partial w_{ij}^{(N-1)}} = y_i^{(N-2)} \frac{\partial Div}{\partial z_j^{(N-1)}}$$

For the bias term $y_0^{(N-2)} = 1$



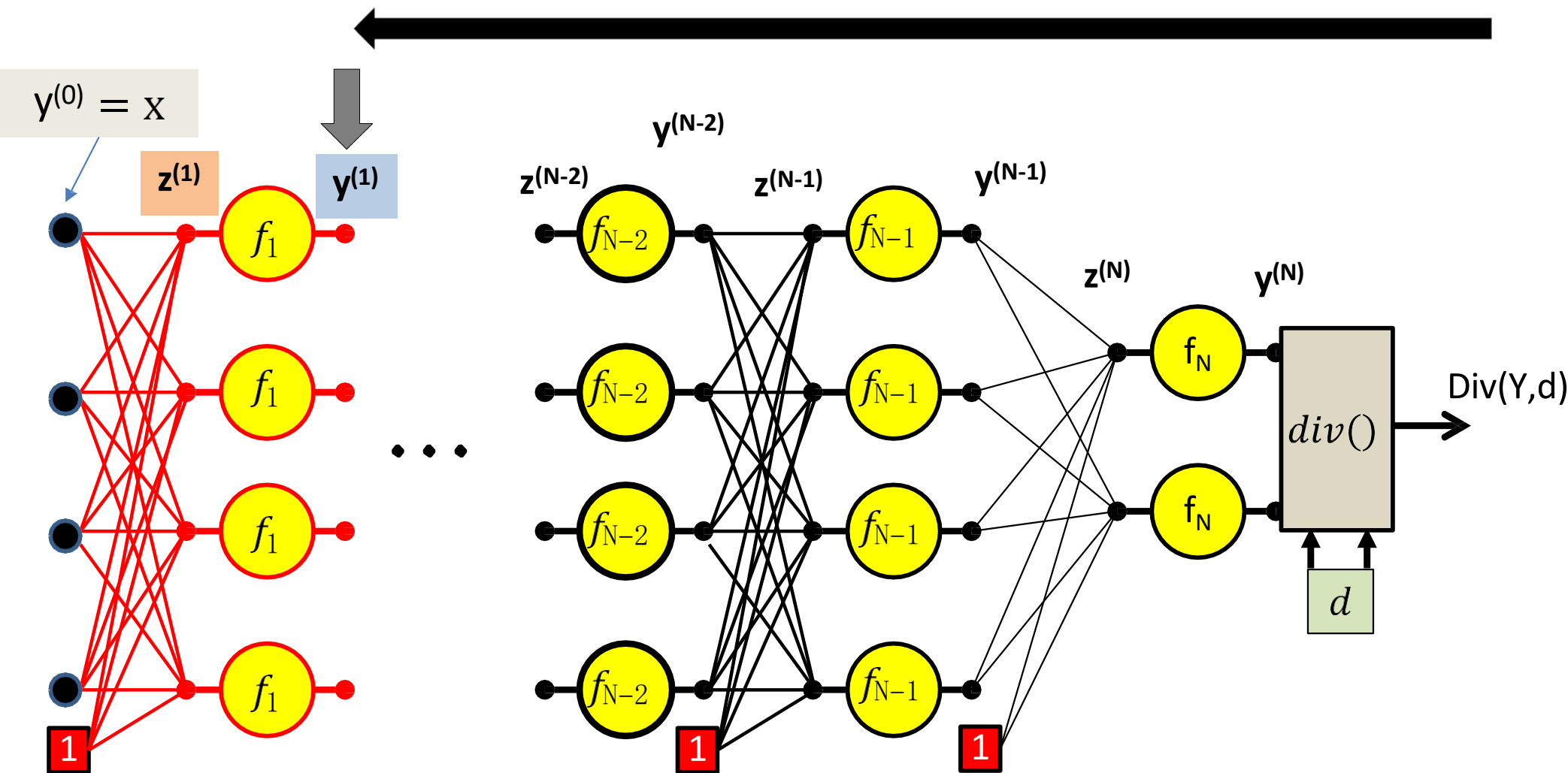
We continue our way backwards in the order shown

$$\frac{\partial \text{Div}}{\partial y_i^{(N-2)}} = \sum_j w_{ij}^{(N-1)} \frac{\partial \text{Div}}{\partial z_j^{(N-1)}}$$



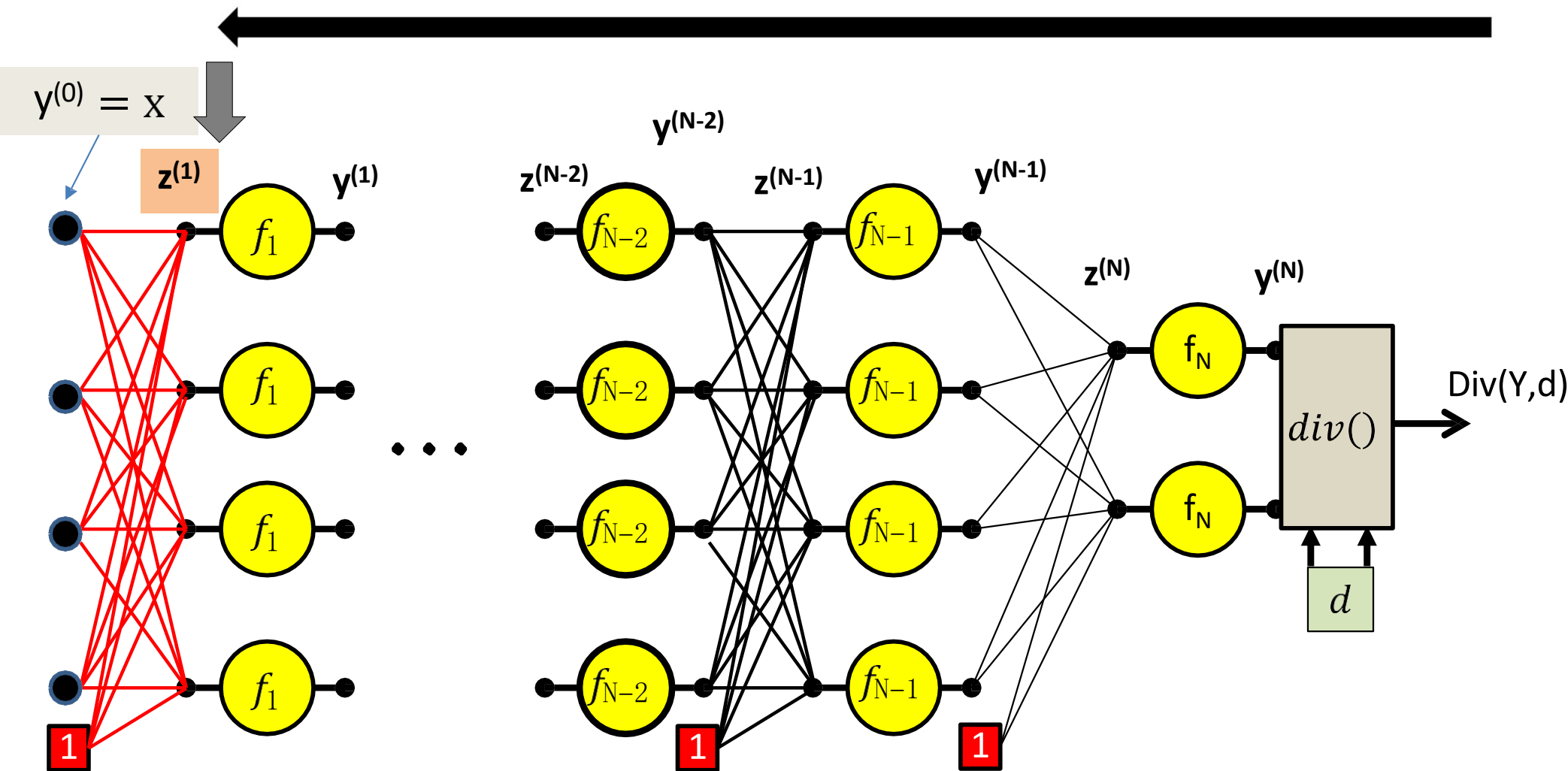
We continue our way backwards in the order shown

$$\frac{\partial Div}{\partial z_i^{(N-2)}} = f'_{N-2} \left(z_i^{(N-2)} \right) \frac{\partial Div}{\partial y_i^{(N-2)}}$$



We continue our way backwards in the order shown

$$\frac{\partial Div}{\partial y_1^{(1)}} = \sum_j w_{ij}^{(2)} \frac{\partial Div}{\partial z_j^{(2)}}$$

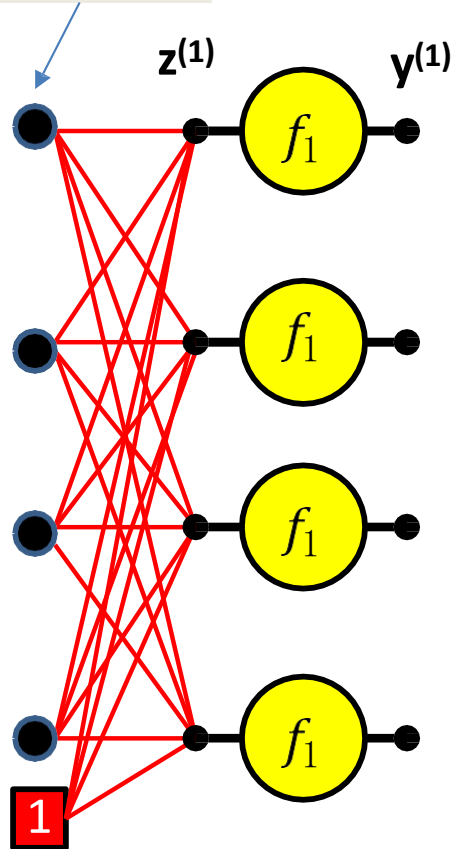


We continue our way backwards in the order shown

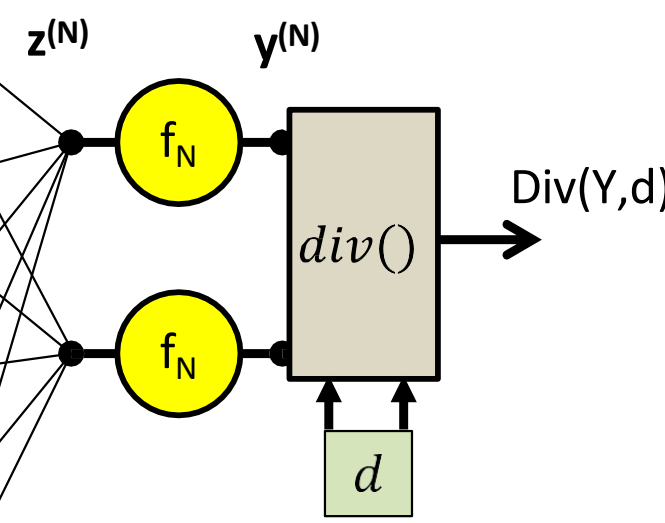
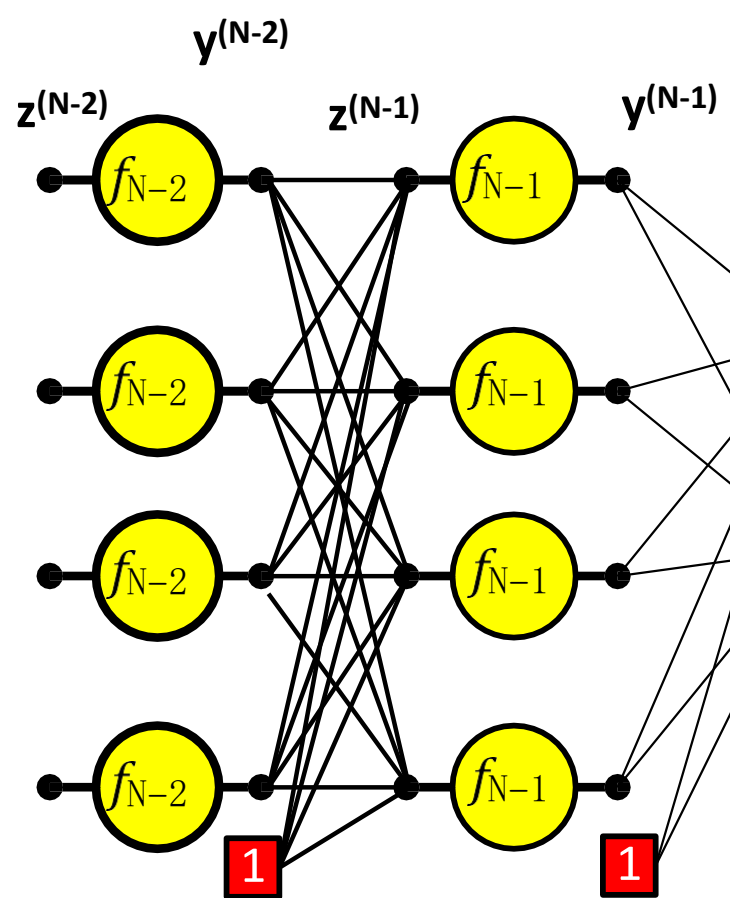
$$\frac{\partial Div}{\partial z_i^{(1)}} = f_1' \left(z_i^{(1)} \right) \frac{\partial Div}{\partial y_i^{(1)}}$$



$y^{(0)} = X$



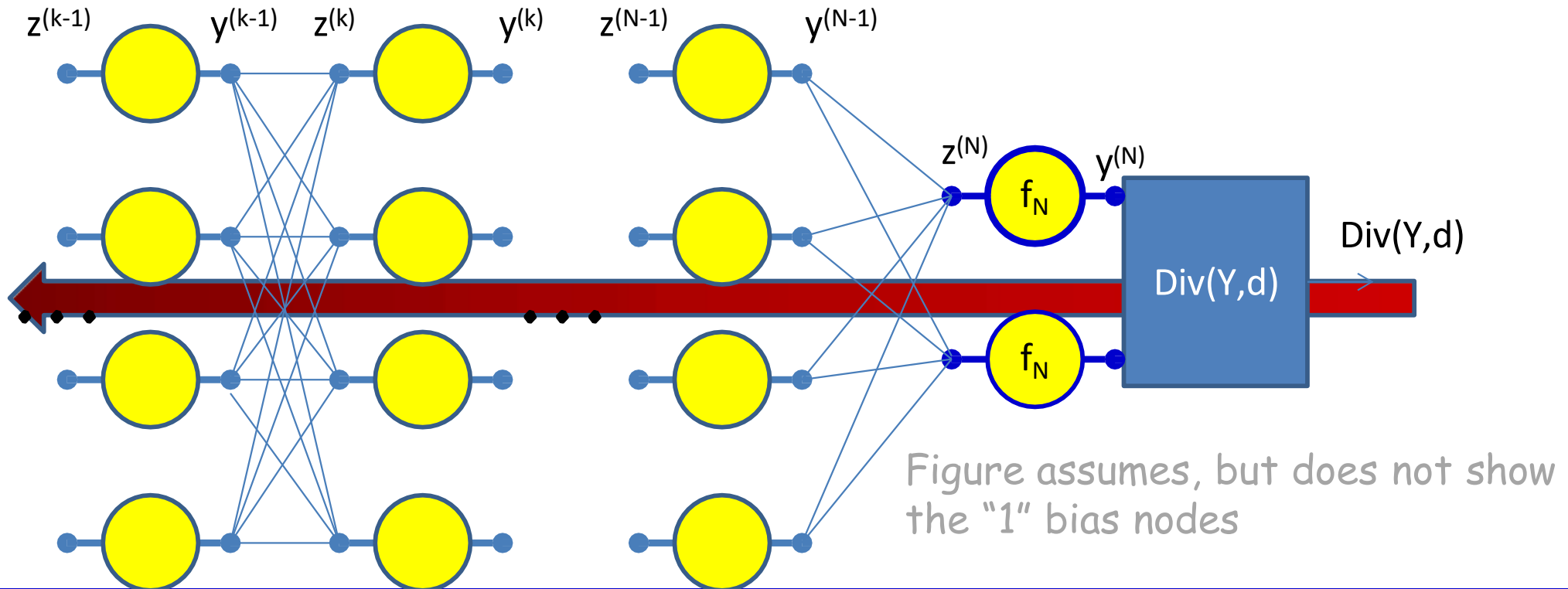
...



We continue our way backwards in the order shown

$$\frac{\partial Div}{\partial w_{ij}^{(1)}} = y_i^{(0)} \frac{\partial Div}{\partial z_j^{(1)}}$$

Gradients: Backward Computation



Initialize: Gradient w.r.t network output

$$\frac{\partial Div}{\partial y_i^{(N)}} = \frac{\partial Div(Y, d)}{\partial y_i}$$

$$\frac{\partial Div}{\partial z_i^{(N)}} = f'_k(z_i^{(N)}) \frac{\partial Div}{\partial y_i^{(N)}}$$

For $k = N - 1..0$

For $i = 1: \text{layer width}$

$$\frac{\partial Div}{\partial y_i^{(k)}} = \sum_j w_{ij}^{(k+1)} \frac{\partial Div}{\partial z_j^{(k+1)}}$$

$$\frac{\partial Div}{\partial z_i^{(k)}} = f'_k(z_i^{(k)}) \frac{\partial Div}{\partial y_i^{(k)}}$$

$$\forall j \frac{\partial Div}{\partial w_{ij}^{(k+1)}} = y_i^{(k)} \frac{\partial Div}{\partial z_j^{(k+1)}}$$

Backward Pass

- Output layer (N) :
 - For $i = 1 \dots D_N$
 - $\frac{\partial Div}{\partial y_i^{(N)}} = \frac{\partial Div(Y,d)}{\partial y_i}$ [This is the derivative of the divergence]
 - $\frac{\partial Div}{\partial z_i^{(N)}} = \frac{\partial Div}{\partial y_i^{(N)}} f'_N(z_i^{(N)})$
 - $\frac{\partial Div}{\partial w_{ij}^{(N)}} = y_i^{(N-1)} \frac{\partial Div}{\partial z_j^{(N)}} \text{ for } j = 0 \dots D_{N-1}$
- For layer $k = N - 1$ *downto* 1
 - For $i = 1 \dots D_k$
 - $\frac{\partial Div}{\partial y_i^{(k)}} = \sum_j w_{ij}^{(k+1)} \frac{\partial Div}{\partial z_j^{(k+1)}}$
 - $\frac{\partial Div}{\partial z_i^{(k)}} = \frac{\partial Div}{\partial y_i^{(k)}} f'_k(z_i^{(k)})$
 - $\frac{\partial Div}{\partial w_{ij}^{(k)}} = y_i^{(k-1)} \frac{\partial Div}{\partial z_j^{(k)}} \text{ for } j = 0 \dots D_{k-1}$

Backward Pass

- Output layer (N) :

- For $i = 1 \dots D_N$

- $\frac{\partial Div}{\partial y_i^{(N)}} = \frac{\partial Div(Y, d)}{\partial y_i}$

- $\frac{\partial Div}{\partial z_i^{(N)}} = \frac{\partial Div}{\partial y_i^{(N)}} f'_N(z_i^{(N)})$

- $\frac{\partial Div}{\partial w_{ij}^{(N)}} = y_i^{(N-1)} \frac{\partial Div}{\partial z_j^{(N)}} \text{ for } j = 0 \dots D_{N-1}$

Called "**Backpropagation**" because the derivative of the loss is propagated "backwards" through the network

- For layer $k = N - 1$ *downto* 1

Very analogous to the forward pass:

- For $i = 1 \dots D_k$

- $\frac{\partial Div}{\partial y_i^{(k)}} = \sum_j w_{ij}^{(k+1)} \frac{\partial Div}{\partial z_j^{(k+1)}}$

Backward weighted combination of next layer

- $\frac{\partial Div}{\partial z_i^{(k)}} = \frac{\partial Div}{\partial y_i^{(k)}} f'_k(z_i^{(k)})$

Backward equivalent of activation

- $\frac{\partial Div}{\partial w_{ij}^{(k)}} = y_i^{(k-1)} \frac{\partial Div}{\partial z_j^{(k)}} \text{ for } j = 0 \dots D_{k-1}$

Story so far

- Neural networks must be trained to minimize the average divergence between the output of the network and the desired output over a set of training instances, with respect to network parameters.
- Minimization is performed using gradient descent
- Gradients (derivatives) of the divergence (for any individual instance) w.r.t. network parameters can be computed using backpropagation
 - Which requires a “forward” pass of inference followed by a “backward” pass of gradient computation
- The computed gradients can be incorporated into gradient descent

Issues

- Convergence: How well does it learn
 - And how can we improve it
- How well will it generalize (outside training data)
- What does the output really mean?
- *Etc..*